



Empresa Operativa Chile

Crear, operar y controlar una empresa chilena a través del tiempo

Contabilidad y tributación explicables · Reglas versionadas por año · Evidencia obligatoria
Sin servidor · Sin cuentas · Sin telemetría · Los datos no salen del dispositivo

Android · APK

Windows · MSI/EXE

Navegador · PWA

CLI · Node 20+

● EMPRESA REAL

Tu contabilidad, con bitácora de auditoría de cada cambio.

● SANDBOX

Empresa ficticia para practicar. Nunca toca la empresa real.

Documentación de sistema completa

Documentación de sistema · Empresa Operativa Chile

Los 20 documentos de la serie

Versión del manifiesto 1.4.0 · commit 2b0e006

Análisis del 2026-08-27

Documento técnico generado desde docs/system-documentation/. La fuente es el Markdown del repositorio: si este PDF y el Markdown difieren, manda el Markdown. Esta aplicación no presenta ni paga nada ante el SII y no es asesoría tributaria.

Documentación de sistema — Empresa Operativa Chile

Documentación técnica completa del repositorio, escrita recorriendo el código fuente. Complementa —no reemplaza— la documentación de producto que ya vive en [docs/](#).

Qué es este sistema

Empresa Operativa Chile es una aplicación *local-first* que acompaña a una sociedad chilena (SpA) desde antes de existir hasta el cierre de cada mes y de cada año: constitución con evidencia obligatoria, registro de operaciones, IVA con remanente arrastrado, borrador del F29, obligaciones, capital y patrimonio, capital propio tributario, patente municipal, cierre inmutable y bitácora de auditoría *append-only*.

El mismo motor de cálculo y la misma interfaz corren en **navegador (PWA)**, **Android (APK vía Capacitor)** y **Windows (Tauri 2)**, más una **CLI** en Node. Los datos nunca salen del dispositivo: no hay servidor, no hay cuentas y no hay telemetría.

Propósito de esta documentación

Dejar el sistema **auditable y transferible**: que alguien que llega hoy pueda entender qué hace cada módulo, por qué está construido así, dónde están los riesgos y cómo se instala, prueba y publica — sin tener que preguntar a nadie.

Público al que se dirige

Perfil	Por dónde empezar
Persona técnica nueva en el proyecto	18 · Guía del nuevo desarrollador
Auditoría técnica o revisión de código	03 · Arquitectura → 11 · Seguridad → 15 · Riesgos
Dirección / decisión	17 · Resumen ejecutivo
Persona no técnica	01 · Visión general , sección final
Quien opera el sistema	13 · Despliegue y operación y 14 · Troubleshooting



Tabla de contenidos

#	Documento	Qué contiene	Estado
00	README (este archivo)	Portada, índice, convenciones, pendientes	✓
01	Visión general del sistema	Qué hace, para quién, límites; explicación para persona no técnica	✓
02	Instalación y ejecución	Requisitos, instalación, ejecución en las 4 superficies, carpetas que no están	✓
03	Arquitectura	Capas, decisiones, diagramas Mermaid, regla <code>node:*</code>	✓
04	Mapa del código	Inventario jerárquico con estado aparente de cada elemento	✓
05	Referencia técnica	Catálogo de funciones: firma, efectos, riesgo al modificar	✓
06	Explicación profunda del código	Flujo por flujo, módulo a módulo	✓
07	Persistencia	No hay base de datos: cuál es el mecanismo real y cómo se versionan las reglas	✓
08	Flujo de datos	Del teclado al disco y de vuelta; ciclo de las reglas tributarias	✓
09	APIs e integraciones	Se demuestra que no hay red; contratos de datos e interfaces de plataforma	✓
10	Configuración	Variables de entorno, manifiestos, CSP, capacidades Tauri	✓
11	Seguridad	Controles presentes y ausentes, superficie real, hallazgos	✓
12	Pruebas y calidad	13 suites, 153 pruebas, 6 workflows, gates propios	✓
13	Despliegue y operación	Pages, APK, instaladores Windows, release, generación de PDF	✓
14	Troubleshooting	Síntoma → causa → diagnóstico → solución → riesgo	✓
15	Riesgos y deuda técnica	Hallazgos clasificados. Informativo: no se corrigió nada	✓
16	Glosario	Términos técnicos y de dominio, para persona no técnica	✓

#	Documento	Qué contiene	Estado
1 7	Resumen ejecutivo	Para decisión, con esfuerzo cuantificado	✓
1 8	Guía del nuevo desarrollador	Itinerario de incorporación por días	✓
1 9	Matriz de trazabilidad	Funcionalidad → módulo → función → datos → prueba, en una fila	✓

 [assets/diagramas/](#) · los **19 diagramas Mermaid** rasterizados a SVG para el PDF, cacheados por hash de su contenido
  [pdf/](#) · **21 PDF · 535 páginas**: uno por documento más [documentacion-completa.pdf](#) (244 páginas), generados con [scripts/build-system-docs.mjs](#)

Qué documento es la fuente de verdad de cada tema

Este repositorio **ya tenía** documentación buena antes de esta serie. No se duplicó ni se movió nada. La regla es: cuando dos documentos hablan del mismo tema, manda el de la columna izquierda.

Tema	Fuente de verdad	Qué añade <code>system-documentation/</code>
Decisiones de arquitectura	<code>docs/ARCHITECTURE.md</code>	Diagramas Mermaid, capas, y el mapa corregido (ver 15)
Uso de la aplicación	<code>docs/MANUAL.md</code>	Nada: no se duplica el manual de usuario
Ruta para principiantes	<code>docs/EMPEZAR-AQUI.md</code> (generado)	Nada
Términos de dominio	<code>docs/GLOSSARY.md</code> (generado)	16 añade los términos técnicos que ese no cubre
Tasas, plazos y su fuente legal	<code>packages/chile-tax-rules/rules/2026.json</code> y <code>docs/SOURCES-2026.md</code>	07 y 08 documentan el mecanismo que las versiona
Rutina mensual / anual	<code>docs/RUNBOOK-MENSUAL.md</code> · <code>docs/RUNBOOK-ANUAL.md</code>	Nada
Capital propio tributario	<code>docs/tax/CAPITAL-PROPIO-TRIBUTARIO.md</code>	06 explica la implementación
Patente municipal	<code>docs/municipal/PATENTE-MUNICIPAL.md</code>	06 explica la implementación
Capital y patrimonio	<code>docs/accounting/CAPITAL-PATRIMONIO.md</code>	07 documenta el modelo de datos
Postura de seguridad	<code>SECURITY.md</code>	11 añade la revisión del código y los controles ausentes
Cómo contribuir	<code>CONTRIBUTING.md</code>	18 añade el itinerario de lectura
Historia de versiones	<code>CHANGELOG.md</code>	Nada
Qué viene	<code>docs/ROADMAP.md</code>	17 cuantifica el esfuerzo

Datos del análisis

Campo	Valor
Repositorio	empresa-operativa-chile
Rama analizada	main
Commit analizado	2b0e006ba02c406346f08a23c739c81b89c93b0a
Fecha del commit	2026-08-17
Versión del manifiesto	1.4.0 (package.json)
Fecha del análisis	2026-08-27
Archivos versionados	262 (git ls-files)
Archivos de código analizados	72 (.mjs , .js , .rs) — 13.251 líneas
Suites de prueba	13 · 153 pruebas, todas verdes
Workflows de CI	6
Dependencias de producción	0

Convenciones usadas

Marcador	Significado
(sin marcador)	Hecho verificado leyendo el repositorio, con archivo y símbolo citados
INFERENCIA	Conclusión razonada a partir del código, no afirmación literal del repositorio
REQUIERE VALIDACIÓN	Depende de una ejecución real, de un servicio externo o de una decisión humana
NO DOCUMENTADO EN EL REPOSITORIO	Existe en el código y nadie lo explica
NO IDENTIFICADO	Se buscó y no hay evidencia en ningún sentido

Otras reglas de escritura de esta serie:

- **Identificadores en su forma original.** `f29Basic` , `capitalEnterado` , `rules/2026.json` y `EMPRESA_OPERATIVA_DATA` se citan sin traducir, para que buscar el término en el repositorio encuentre exactamente lo que el documento dice.
- **Las cifras se contaron.** Ningún «unos cuantos» ni «aproximadamente»: cada número de estos documentos salió de un comando, y el comando está registrado en [12 · Pruebas y calidad](#).
- **Cada diagrama lleva texto debajo** que explica qué muestra **y qué no**. Ningún diagrama es necesario para entender el documento.
- **Cada documento se lee solo.** Hay enlaces al resto, pero no se supone que se hayan leído.

✓ Qué se verificó ejecutando

No es una lista de intenciones: cada fila se ejecutó y su salida está en [12 · Pruebas y calidad](#).

Comprobación	Resultado
Las 153 pruebas	✓ <code>pass 153 · fail 0</code> en 0,64 s
Los 5 gates de sincronía documental	✓ los cinco en verde
<code>validate-rules.mjs</code>	✓ reglas 2026 validadas
Build completo	✓ 49 archivos, build <code>630ed69fe514</code>
Build reproducible	✓ dos builds seguidos, <code>build-info.json</code> idéntico
La CLI (4 comandos, valores exactos)	✓
El servidor local sirve la app	✓
Los 3 vectores de <i>path traversal</i>	✓ los tres → 404
Método no permitido	✓ <code>POST /</code> → 405
Sin secretos, respaldos reales ni certificados versionados	✓ ninguna coincidencia
Ausencia de llamadas de red	✓ una sola, en <code>sw.js</code> , al propio origen
Los 401 enlaces relativos de esta serie y del README raíz	✓ 0 rotos
Las 9 anclas internas	✓ 0 rotas
Los 19 diagramas Mermaid compilan	✓
Ningún diagrama se imprime ilegible en A4	✓ tras partir el ER y reorientar cuatro
Maquetado al ancho exacto de una A4 (688 px)	✓ 0 tablas desbordadas , 0 imágenes fuera de caja, 0 imágenes rotas
Los 21 PDF se generan	✓ 535 páginas

Pendientes de validar

Lo que esta documentación **no pudo verificar**, y por qué. Se declara aquí en vez de esconderse.

Pendiente	Por qué no se pudo verificar	Cómo se validaría
Compilación del APK de Android	Requiere JDK 21, SDK de Android y Gradle; no se ejecutó en esta máquina	Ejecutar el workflow <code>android.yml</code> (<code>workflow_dispatch</code>) y revisar la salida de <code>verify-apk.mjs</code>
Compilación de los instaladores de Windows	Requiere toolchain de Rust estable y <code>@tauri-apps/cli</code> ; no se ejecutó	Ejecutar <code>desktop.yml</code> o <code>pnpm desktop:build</code> en una máquina con Rust
Pruebas del backend Rust (<code>cargo test</code>)	Sin toolchain de Rust en la máquina de análisis	<code>cargo test --release</code> en <code>apps/contador-desktop/src-tauri</code>
Publicación en GitHub Pages	No se disparó ningún despliegue	Revisar la última ejecución de <code>pages.yml</code> en el repositorio
Que el APK publicado sea el verificado	Depende del release, no del código	<code>sha256sum</code> del APK descargado contra <code>SHA256SUMS.txt</code> del release
Vigencia de las tasas tributarias 2026	Depende de las fuentes oficiales del SII y de BCN/LeyChile, no del repositorio	Reabrir cada <code>source</code> de <code>rules/2026.json</code> — el propio archivo advierte que BCN no respondió el 16-08-2026
Comportamiento real en WebView de Android antigua	No se probó en dispositivo	Instalar el APK en un Android 6–8 y comprobar <code>crypto.randomUUID</code> y el arranque
Que el espejo de disco de Windows repone correctamente tras un corte	No se ejecutó la app de escritorio	Cerrar la app a la fuerza durante una escritura y volver a abrirla
Contenido de los archivos binarios versionados (PNG, PDF, SVG)	Se verificó su existencia, tamaño y número de páginas, no su contenido visual completo	Inspección visual manual

Además, todo lo marcado **REQUIERE VALIDACIÓN** dentro de cada documento pertenece a esta lista.

Cómo regenerar los PDF

```
node scripts/build-system-docs.mjs          # todos + el consolidado
node scripts/build-system-docs.mjs --only 03 # sólo el documento 03, para iterar rápido
node scripts/build-system-docs.mjs --no-diagrams # sin rasterizar Mermaid, más rápido
```

Requisitos: Chrome o Edge (se autodetecta; si no, `CHROME_PATH`) y, para los diagramas, `@mermaid-js/mermaid-cli` global — `pnpm add -g @mermaid-js/mermaid-cli` . Si falta `mmdc` el generador no falla: deja los diagramas como bloques de código y lo avisa.

El generador lee la versión de `package.json` y el commit con `git rev-parse --short HEAD` : la portada de cada PDF nunca se escribe a mano. Además **calcula si algún diagrama sale ilegible** en A4 y lo dice con nombre y factor de reducción. El detalle está en [13 · Despliegue y operación](#).

⚠ Estos PDF **no se generan en CI**, a diferencia del glosario, la guía y la presentación: hay que regenerarlos a mano cuando cambie el Markdown. Está registrado como [H-17](#).

Recordatorio heredado del proyecto y que esta documentación repite: la aplicación ****no presenta ni paga nada ante el SII y no es asesoría tributaria****. Cuando la aplicación y el SII no coincidan, **manda el SII**.

01 · Visión general del sistema

En una frase

Una aplicación que se instala en tu propio dispositivo y lleva la vida completa de una SpA chilena —desde la decisión de crearla hasta el cierre de cada año— calculando IVA, PPM, honorarios, capital propio tributario y patente municipal, **exigiendo evidencia antes de dar algo por cumplido** y dejando registro de todo lo que cambió.

La conclusión, antes del recorrido

Cinco cosas que conviene saber antes de abrir un solo archivo:

1. **No hay servidor y no hay red.** Ni una llamada HTTP saliente en toda la aplicación. Lo que parece un servidor (`apps/empresa-operativa/server.mjs`) sólo sirve archivos estáticos en `127.0.0.1` . Los datos viven en `localStorage` y, en Windows, además en archivos JSON del disco.

1. **Hay un solo motor de cálculo, y viaja completo a las tres plataformas.** No existe una versión Android del F29 y otra de escritorio: es el mismo `packages/accounting-engine/index.mjs` . La regla que lo sostiene —nada que viaje al dispositivo puede importar `node:*` — la hace cumplir el build, que **aborta**, y una prueba.

1. **Las tasas no están en el código.** Viven en `packages/chile-tax-rules/rules/2026.json` , cada una con su `source` oficial y su `lastVerified` . Pedir un año sin archivo **lanza un error** en vez de calcular con la tasa del año anterior.

1. **El producto se niega a mentir.** No marca un trámite como hecho sin evidencia, no modifica un período cerrado, no borra líneas de la bitácora, no dice «todo en orden» si faltan respaldos, y no publica tasas municipales que nadie verificó.

1. **Cero dependencias de producción.** El `package.json` raíz no declara ninguna, y un workflow falla si alguien añade una. No hay bundler, no hay framework de UI y no hay ORM.

Qué problema resuelve

Llevar una empresa pequeña en Chile no falla por no saber sumar. Falla por perder el hilo: una factura sin respaldo, un remanente de IVA que no se arrastró, un trámite que se dio por hecho sin guardar el comprobante, un mes cerrado sin conciliar. El sistema ataca ese hilo, no la aritmética.

Su diferenciador real está en **distinguir magnitudes que otras herramientas funden en una sola**:

Magnitud	Qué es	Dónde se calcula
Capital social	Lo que dice el estatuto	<code>capital.mjs</code> → <code>capitalSocial</code>
Capital suscrito	Lo que el accionista se comprometió a aportar	<code>capital.mjs</code> → <code>capitalSuscrito</code>
Capital enterado	Lo que efectivamente entró	<code>workspace.mjs</code> → <code>capitalPosition()</code>
Patrimonio contable	Activos – pasivos, según la contabilidad	<code>workspace.mjs</code> → <code>estimatedBalance()</code>
Capital propio tributario (CPT)	Activo – pasivo exigible a valores tributarios	<code>accounting-engine/tax-equity.mjs</code>
Capital base de patente	El capital que el D.L. 3.063 manda usar ese año	<code>accounting-engine/municipal-patent.mjs</code>

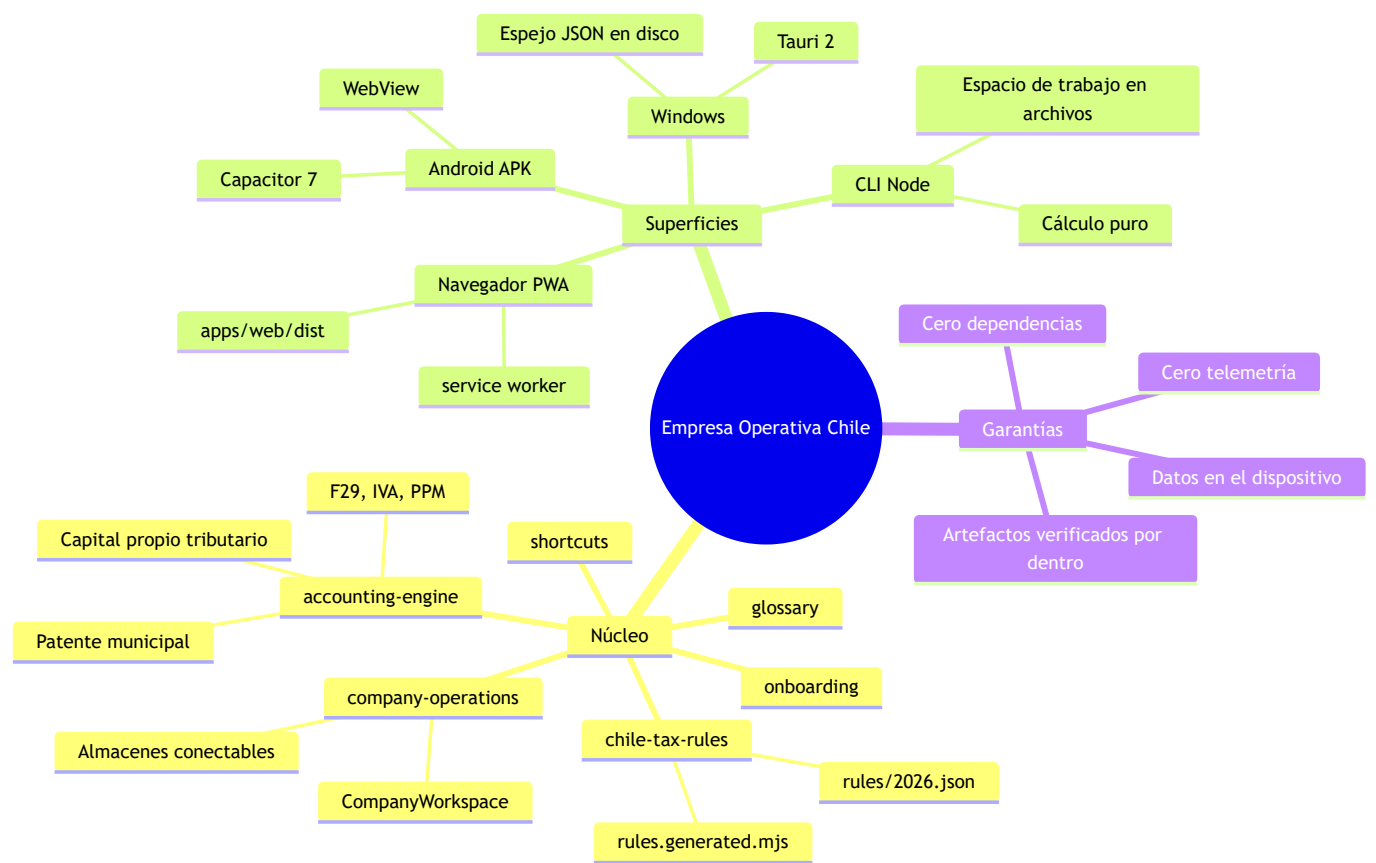
Confundir dos de estas cambia la cifra que se paga. El sistema las trata como seis números distintos, cada uno con su momento, su fuente legal y su evidencia.

Qué hace, funcionalidad por funcionalidad

Área	Qué resuelve	Vista
Empezar aquí	La ruta ordenada por <i>tiempo</i> (14 etapas, 5 fases) para quien nunca creó una empresa	<code>empezar.js</code>
Panel	Indicadores del mes, diagnóstico de salud y próximos vencimientos	<code>panel.js</code>
Operaciones	Ventas, compras, gastos, honorarios, aportes, préstamos, retiros, pagos de impuestos	<code>operaciones.js</code>
Impuestos	Borrador del F29 con el remanente arrastrado y los tres vencimientos del SII	<code>impuestos.js</code>
Obligaciones	Calendario de cumplimiento; una obligación sólo se cierra con comprobante	<code>obligaciones.js</code>
Cierre	Cierre mensual inmutable con lista de control declarada	<code>cierre.js</code>
Constitución	Los 9 trámites, cada uno con su autoridad, su evidencia y su enlace oficial	<code>constitucion.js</code>
Empresa	Ficha: razón social, RUT validado, régimen, giro, domicilio, comuna	<code>empresa.js</code>
Capital	Las seis magnitudes, el ledger patrimonial y el cierre anual	<code>capital.js</code>
Auditoría	Bitácora <i>append-only</i> de todo lo que cambió	<code>auditoria.js</code>
Datos	Exportación, importación, respaldos y CSV	<code>datos.js</code>
Academia	Explicaciones ancladas al mismo motor que calcula	<code>academia.js</code>
Glosario	54 términos, con «no confundir con»	<code>glosario.js</code>
Ayuda	Los manuales dentro de la app, sin conexión, más los 12 atajos	<code>ayuda.js</code>

Catorce vistas en total. Todas se registran en `NAV`, dentro de `apps/web/src/app.js`, y una prueba falla si alguna queda fuera del router.

Las cuatro superficies



Qué muestra el mapa mental y qué no. Muestra que hay un solo núcleo del que cuelgan cuatro superficies, y qué compone cada una. **No muestra** dirección de dependencia ni orden de ejecución: las cuatro superficies *importan* del núcleo y nunca al revés, y ese sentido único es precisamente lo que hace posible que el APK y el instalador de Windows calculen igual. Tampoco muestra los scripts de build, que son quienes copian el núcleo a cada superficie; eso está en [03 · Arquitectura](#).

Superficie	Cómo se construye	Dónde guarda
Navegador / PWA	<code>apps/web/dist</code> servido por Pages o por <code>server.mjs</code>	<code>localStorage</code> del navegador
Android	Capacitor 7 empaqueta <code>apps/web/dist</code> en <code>assets/public/</code> del APK	<code>localStorage</code> de la WebView (sandbox de la app)
Windows	Tauri 2 embebe <code>apps/web/dist</code> en el ejecutable	<code>localStorage</code> + espejo JSON en <code>app_data_dir</code>
CLI	Importa <code>packages/</code> directamente desde Node	Archivos JSON/NDJSON en <code>--datos</code> (por defecto <code>./.local-data</code>)

Qué NO hace (declarado por el propio sistema)

`packages/onboarding/index.mjs` expone `COVERAGE.notCovered`, con **diez** áreas explícitamente fuera de alcance. No es una lista escrita para esta documentación: la aplicación la muestra en pantalla y `docs/EMPEZAR-AQUI.md` se genera desde ella.

- Trabajadores y remuneraciones (sin liquidaciones, cotizaciones ni Libro de Remuneraciones)
- Comercio exterior (importaciones, exportaciones y sus regímenes de IVA)
- Existencias e inventario (la app está pensada para servicios)
- Activo fijo y depreciación
- Corrección monetaria del art. 41 de la LIR — **el CPT que calcula es nominal**
- Reorganizaciones empresariales (división, fusión, transformación, conversión)
- Registros empresariales completos (RAI, DDAN, REX, SAC) del Régimen General
- Término de giro
- Conexión con el SII o con municipalidades — **no existe y no se simula**
- Tasas de patente por comuna — no se publican porque no hay una tasa nacional

Y tres límites técnicos que `docs/ARCHITECTURE.md` declara y esta documentación confirma leyendo el código:

- el remanente de crédito fiscal se arrastra pero **no se reajusta** (art. 27 del D.L. 825);
- los vencimientos consideran sábado y domingo pero **no feriados legales** — el propio resultado

devuelve `checkHolidays: true`;

- el espejo de Windows y los respaldos **no están cifrados**.

Lo que está sólido y conviene no romper

Un informe que sólo enumera problemas da una imagen falsa. Estas propiedades son buenas y son frágiles: alguien las romperá si nadie dijo que importaban.

Propiedad	Cómo está protegida hoy
Un año sin reglas falla en vez de degradar	<code>loadRules()</code> lanza · prueba en <code>tests/rules.test.mjs</code>
Nada que viaje al dispositivo importa <code>node:*</code>	<code>build-web.mjs</code> aborta el build · prueba en <code>tests/webapp.test.mjs</code>
El módulo embebido de reglas nunca se desfasa del JSON	<code>build-rules.mjs --check</code> en CI · <code>deepEqual</code> en <code>tests/rules.test.mjs</code>
El glosario, la guía y los atajos no pueden contradecir al código	Se generan desde <code>packages/</code> y CI compara con <code>--check</code>
Un período cerrado es inmutable en las dos direcciones	<code>addTransaction</code> , <code>updateTransaction</code> y <code>deleteTransaction</code> lo comprueban · prueba dedicada
La bitácora no se puede borrar	No existe ninguna operación de borrado sobre <code>KEY.audit</code>
Real y sandbox no se pisan	Almacenes distintos con <i>namespace</i> distinto, no una bandera
El build es reproducible	CI construye dos veces y compara <code>build-info.json</code>
Un APK vacío no se publica	<code>verify-apk.mjs</code> abre el ZIP y cuenta lo que hay dentro
Toda acción de las acciones de CI está fijada a un SHA	Paso <code>workflows</code> de <code>ci.yml</code>



Para una persona no técnica

Sin jerga. Si algo de aquí no se entiende, es un defecto de este texto, no de quien lee.

Qué es, con una comparación

Imagina una **carpeta muy ordenada** para tu empresa. No una carpeta de cartón: una que además sabe sumar, que te avisa cuando falta un papel y que apunta en una libreta todo lo que has ido metiendo y sacando, con fecha y hora.

Eso es esta aplicación. Se instala en tu teléfono o en tu computador y hace tres cosas:

1. **Calcula.** Cuánto IVA debes este mes, cuánto retienes de una boleta de honorarios, cuánto te toca de patente municipal. Y cuando calcula, te enseña de dónde salió cada número.

1. **Exige el papel.** No te deja marcar «ya hice el inicio de actividades» si no anotas el número del comprobante. Parece una molestia; es el punto entero. La mayoría de los problemas con el SII no vienen de calcular mal, vienen de no encontrar el papel meses después.

1. **Recuerda.** Cada cambio queda apuntado en una libreta que la aplicación **no permite borrar**.

Si dentro de un año te preguntas «¿por qué agosto quedó así?», la respuesta está ahí.

Lo importante: tus datos son tuyos

No hay ninguna cuenta que crear, ninguna contraseña, ningún «iniciar sesión». La aplicación **no manda tus números a ninguna parte**: se quedan en el aparato donde la instalaste, igual que las fotos de tu teléfono.

Eso tiene una cara buena y una mala, y las dos son importantes:

- **Buena:** nadie más los ve. Ni el autor de la aplicación, ni una empresa, ni un servidor.
- **Mala:** **nadie los respalda por ti.** Si borras la aplicación o cambias de teléfono sin

exportar una copia, los datos se van y no hay forma de recuperarlos. La aplicación tiene un botón para exportar; conviene usarlo cada mes.

Los dos modos: practica antes de tocar lo real

La aplicación tiene dos espacios completamente separados:

- **SANDBOX** — una empresa inventada, con datos de mentira, para practicar. Ya viene llena.
- **EMPRESA REAL** — la tuya.

No hay ninguna forma de que algo pase de uno al otro. Están guardados en sitios distintos, no separados por una casilla que alguien pueda olvidar de marcar. Si estás aprendiendo, quédate en SANDBOX hasta que te sientas cómodo.

Lo que NO hace, dicho claro

Esto es lo más importante de esta sección, así que va en negrita:

- **No presenta ni paga nada ante el SII.** Prepara el borrador y te dice cuándo vence. Ir al SII,

presentar y pagar sigue siendo cosa tuya, en la página del SII.

- **No es un contador y no es asesoría tributaria.** Es una calculadora que explica sus cuentas.

Cuando la aplicación y el SII digan cosas distintas, **manda el SII, siempre.**

- **No sabe cuánto cobra tu municipalidad.** Cada municipalidad de Chile elige su propia tarifa de

patente dentro de un rango que fija la ley. La aplicación no se inventa la de tu comuna: te avisa de que está usando el mínimo legal como *suposición* y te pide que averigües la tuya.

- **No sirve si tienes trabajadores contratados.** No calcula sueldos ni cotizaciones. Para eso

necesitarás otra herramienta.

- **No sirve si vendes productos físicos con inventario.** Está pensada para servicios.

Cuándo te ayuda de verdad

Si eres una persona que creó (o va a crear) una SpA para prestar servicios —desarrollo, diseño, consultoría, asesoría—, sin empleados, y estás llevando las cuentas tú, esta aplicación está hecha exactamente para esa situación. Fuera de ahí, ayuda menos, y prefiere decírtelo a fingir que sí.

02 · Instalación y ejecución

La conclusión, primero

Para trabajar en el motor, la web y la CLI **no hace falta instalar nada**: no hay dependencias de producción y no hay bundler. Basta Node ≥ 20 y `node scripts/build-all.mjs`.

`pnpm install` sólo es necesario para dos cosas: empaquetar el **APK de Android** (Capacitor) y compilar los **instaladores de Windows** (Tauri). Ambas tienen su propio `package.json` y su propio lockfile dentro de `apps/`.

El gestor de paquetes de este repositorio es `pnpm`. Está declarado en `package.json` como `"packageManager": "pnpm@11.2.2"` y los workflows instalan esa versión exacta con `pnpm/action-setup`. Escribir `npm` es un error: `pnpm-lock.yaml` es el lockfile válido y `npm install` lo ignoraría.

Requisitos

Requisito	Versión	Para qué	Obligatorio
Node.js	≥ 20 (<code>engines.node</code> en <code>package.json</code>)	Todo: build, pruebas, CLI, servidor	✅ Sí
pnpm	11.2.2 (<code>packageManager</code>)	Sólo Android y Windows	Sólo para empaquetar
Git	cualquiera reciente	Clonar; el generador de PDF lee el commit	✅ Sí
Chrome o Edge	cualquiera	Capturas, manual, guía, presentación y los PDF de esta documentación	Sólo para regenerar documentos
JDK	21 (Temurin en CI)	Compilar el APK	Sólo Android
SDK de Android + Gradle	el que traiga <code>cap add android</code>	Compilar el APK	Sólo Android
Rust	estable, <code>rust-version = "1.77"</code> mínimo declarado en <code>Cargo.toml</code>	Compilar la app de Windows	Sólo Windows
<code>@mermaid-js/mermaid-cli</code> (<code>mmdc</code>)	global	Rasterizar los diagramas de esta documentación al PDF	Opcional — degrada con aviso

CI ejecuta la matriz en **Ubuntu con Node 20 y 22** y en **Windows con Node 22** (`ci.yml` , `matrix.exclude` descarta Windows + Node 20).

Instalación mínima

```
git clone https://github.com/vladimiracunadev-create/empresa-operativa-chile.git
cd empresa-operativa-chile
node scripts/build-all.mjs
```

Eso es todo. `build-all.mjs` ejecuta tres pasos **en orden**, y el orden importa:

1. `build-rules.mjs` → `packages/chile-tax-rules/rules.generated.mjs`
2. `build-icons.mjs` → `apps/web/src/icons/*.png`
3. `build-web.mjs` → `apps/web/dist/`

La web embebe las reglas y los iconos. Generar la web con reglas viejas produciría un APK que calcula con la tasa del año pasado **sin que nada se ponga en rojo** — de ahí que el orden esté fijado en el script y no en la memoria de quien lo ejecuta.

Las carpetas que NO están, y quién las crea

Ésta es la sorpresa número uno de quien clona el repositorio. `git ls-files` devuelve 262 archivos y **ninguno** de ellos está en `apps/web/dist`, `apps/android/www` ni `apps/android/android`.

Ruta ausente	Quién la genera	Consecuencia de no generarla
<code>apps/web/dist/</code>	<code>scripts/build-web.mjs</code>	El servidor local aborta con «No existe apps/web/dist»
<code>apps/android/www/</code>	<code>apps/android/copiar-www.mjs</code>	Capacitor empaquetaría un APK vacío
<code>apps/android/android/</code>	<code>pnpm exec cap add android</code>	No hay proyecto Gradle que compilar
<code>apps/contador-desktop/src-tauri/target/</code>	<code>cargo</code> / <code>tauri build</code>	No hay binario
<code>apps/contador-desktop/src-tauri/gen/</code>	<code>tauri build</code>	No existen los esquemas de capacidades
<code>.local-data/</code>	La CLI, al primer comando de espacio de trabajo	Ninguna: se crea sola
<code>artefactos/</code>	Los workflows de Android y Windows	Sólo existe en CI

`.gitignore` las excluye a todas y explica por qué en el propio archivo: «*Artefactos de build (se regeneran: node scripts/build-all.mjs)*».

Dos casos especiales: artefactos generados que SÍ se versionan.

- `packages/chile-tax-rules/rules.generated.mjs` está commiteado y marcado en `.gitattributes` con `linguist-generated=true` para que no ensucie las revisiones. La razón es que el navegador no tiene `node:fs` para leer el JSON, así que el módulo embebido tiene que estar disponible sin ejecutar ningún build previo. CI comprueba en cada push que no se haya desfasado (`build-rules.mjs --check`).
- `apps/web/src/icons/` está commiteado con sus 5 PNG y el SVG fuente, aunque `build-icons.mjs` los regenera. Es lo que permite que `pnpm cli` y las pruebas funcionen sobre un clon limpio sin ejecutar `build-icons.mjs` antes: la prueba «el manifiesto PWA apunta a iconos que el generador crea» comprueba que esos archivos existan en `apps/web/src`, no en `dist`.

Ejecutar cada superficie

1. Navegador (lo más rápido)

```
pnpm start      # build completo + servidor en http://127.0.0.1:4180
pnpm app        # sólo el servidor, si el build ya está hecho
```

El servidor (`apps/empresa-operativa/server.mjs`) escucha en `127.0.0.1:4180` . **No en `0.0.0.0`** : son datos contables de una empresa real y no tienen por qué quedar expuestos a la red local porque alguien levantó la app en un café. Se puede cambiar con las variables `HOST` y `PORT` , con las consecuencias que describe [10 · Configuración](#).

En Windows hay dos lanzadores que hacen lo mismo sin abrir una terminal:

```
apps/empresa-operativa/start-windows.bat → doble clic
apps/empresa-operativa/start-windows.ps1 → PowerShell
```

Ambos comprueban que Node exista, ejecutan `build-all.mjs` , abren el navegador y arrancan el servidor. Están marcados en `.gitattributes` como `text eol=crlf` porque `cmd.exe` los necesita así.

2. CLI

No necesita build ni instalación:

```
pnpm cli ayuda
pnpm cli venta --neto 100000
pnpm cli f29 --ventas-netas 1000000 --compras-netas 300000
pnpm cli rut 76.000.000-0
pnpm cli resumen --periodo 2026-08 --sandbox
```

Los comandos de **espacio de trabajo** escriben en disco. El directorio se elige, en este orden de precedencia: `-datos DIR` → variable `EMPRESA_OPERATIVA_DATA` → `./.local-data` .

3. Android (APK)

```
pnpm android:prepare      # build-all + copiar apps/web/dist a apps/android/www
cd apps/android
pnpm install --frozen-lockfile
pnpm exec cap add android  # crea apps/android/android/ (proyecto Gradle)
node copiar-www.mjs        # segunda pasada: ahora sí se aplican los iconos
pnpm exec cap sync android
cd android && ./gradlew assembleDebug
cd ../../.. && pnpm android:verify apps/android/android/app/build/outputs/apk/debug/app-debug.apk
```

La doble ejecución de `copiar-www.mjs` **no es un error**. La primera copia la web; la segunda, después de que `cap add android` haya creado el proyecto nativo, aplica los iconos de `apps/android/res-icons/` sobre `android/app/src/main/res/` . El propio script lo dice: «El proyecto Android todavía no existe: los iconos se aplicarán tras `cap add android` ». El workflow `android.yml` hace exactamente las mismas dos pasadas.

El APK resultante es **de depuración**, firmado con la clave de debug de Android. Sirve para instalar y usar; no para Google Play.

REQUIERE VALIDACIÓN — este flujo no se ejecutó durante el análisis: exige JDK 21, SDK de Android y Gradle. Se transcribe de `.github/workflows/android.yml` y de `apps/android/package.json`.

4. Windows (Tauri 2)

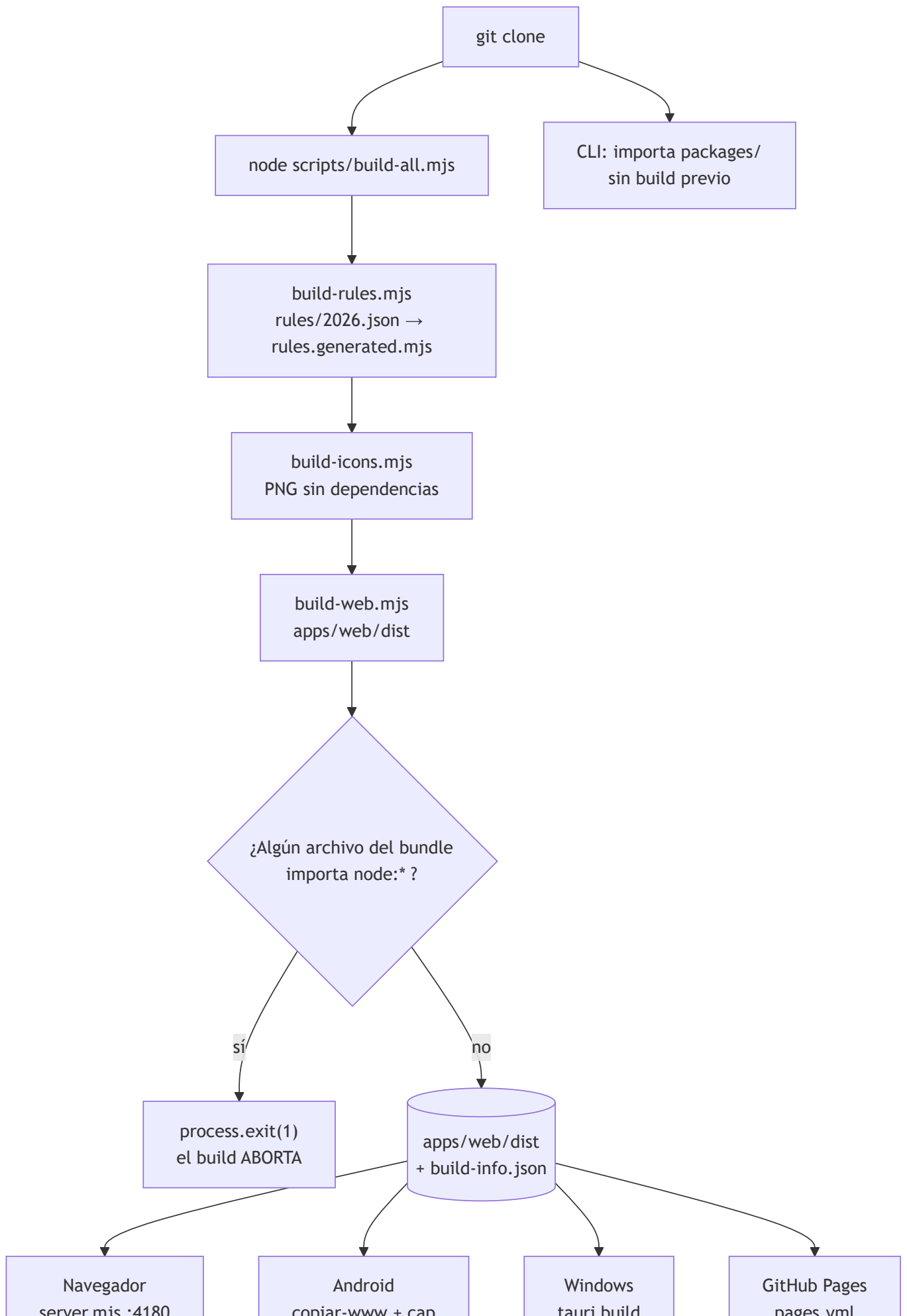
```
pnpm desktop:dev          # ventana de desarrollo
pnpm desktop:build        # build-all + tauri build → MSI, NSIS y portable
```

Produce tres formas de instalar, y cada una resuelve una situación distinta:

Artefacto	Para qué
<code>.msi</code>	Despliegue corporativo / instalación desatendida
<code>-setup.exe</code> (NSIS)	El camino normal para una persona
<code>-portable.exe</code>	Ejecutar sin instalar, desde un pendrive o un equipo ajeno

REQUIERE VALIDACIÓN — tampoco se ejecutó: exige la toolchain de Rust. Se transcribe de `.github/workflows/desktop.yml`, `apps/contador-desktop/package.json` y `tauri.conf.json`.

El diagrama de arranque



construye mjs

copy mjs sep

test mjs

release mjs

Qué muestra el diagrama y qué no. Muestra que `apps/web/dist` es el punto de convergencia: las cuatro formas de publicar el producto salen del mismo directorio, y la CLI es la única que se salta el build porque importa `packages/` directamente desde Node. **No muestra** la comprobación de reproducibilidad (CI construye dos veces y compara `build-info.json`), ni la verificación posterior del artefacto — `verify-apk.mjs` abre el APK, y el workflow de Windows cuenta la interfaz antes de sellarla—, ni que `build-web.mjs` también embarca los manuales y la presentación dentro del bundle.

Comprobar que la instalación quedó bien

```
pnpm test      # 153 pruebas con el runner nativo de Node
pnpm check     # el gate completo del repositorio
```

pnpm check es la comprobación que importa. Encadena ocho pasos y falla al primero que se rompa:

```
build-rules.mjs --check      reglas embebidas ≡ rules/*.json
build-glossary.mjs --check   docs/GLOSSARY.md ≡ packages/glossary
build-guide.mjs --check      docs/EMPEZAR-AQUI.md ≡ packages/onboarding
build-shortcuts.mjs --check  docs/ATAJOS-DE-TECLADO.md ≡ packages/shortcuts
build-presentation.mjs --check estructura de docs/presentacion.md
check-presentation.mjs       páginas reales de los PDF vs. lo anunciado
validate-rules.mjs           valores de las tasas del año en curso
node --test tests/*.test.mjs 153 pruebas
```

Salida real obtenida durante este análisis, en Windows 11 con Node v24.11.1 y pnpm 11.2.2:

```
Reglas embebidas sincronizadas (2026).
Glosario sincronizado (54 términos).
Guía sincronizada (14 etapas, 2 diagramas).
Atajos sincronizados (12).
Presentación válida: 8 diapositivas, 28 minutos.
Presentación: 8 diapositivas (713 KB), ≈28 min, pauta de 5 páginas (726 KB) – coincide con lo anunciado.
Reglas 2026 validadas; última verificación 2026-08-09
...
i tests 153
i pass 153
i fail 0
```

Todos los scripts de `package.json`

Script	Qué hace	Necesita
<code>pnpm build</code>	Build completo (reglas → iconos → web)	Node
<code>pnpm build:rules</code>	Sólo el módulo embebido de reglas	Node
<code>pnpm build:glossary</code>	Regenera <code>docs/GLOSSARY.md</code>	Node
<code>pnpm build:guide</code>	Regenera <code>EMPEZAR-AQUI</code> en <code>.md</code> , <code>.html</code> y <code>.pdf</code>	Node + Chrome
<code>pnpm build:icons</code>	Regenera los PNG de icono	Node
<code>pnpm build:web</code>	Sólo <code>apps/web/dist</code>	Node
<code>pnpm build:shortcuts</code>	Regenera <code>docs/ATAJOS-DE-TECLADO.md</code>	Node
<code>pnpm start</code>	Build + servidor local	Node
<code>pnpm app</code>	Sólo el servidor	Node + <code>dist</code> ya construido
<code>pnpm cli</code>	Ejecuta la CLI	Node
<code>pnpm test</code>	153 pruebas	Node
<code>pnpm check</code>	El gate completo (8 pasos)	Node
<code>pnpm desktop:dev</code>	Ventana Tauri de desarrollo	Rust + pnpm
<code>pnpm desktop:build</code>	Instaladores de Windows	Rust + pnpm
<code>pnpm android:prepare</code>	Build + copia a <code>apps/android/www</code>	Node
<code>pnpm android:verify</code>	Abre un APK y cuenta lo que lleva dentro	Node
<code>pnpm capturas</code>	Recaptura las pantallas del manual	Chrome + servidor levantado
<code>pnpm manual</code>	Regenera <code>MANUAL.pdf</code> y <code>MANUAL.html</code>	Chrome
<code>pnpm guia</code>	Alias de <code>build:guide</code>	Chrome
<code>pnpm docs</code>	Capturas + guía + manual + presentación	Chrome + servidor
<code>pnpm presentacion</code>	Regenera y verifica la presentación	Chrome

A esos se suma el generador de esta documentación, que **no** está en `package.json` para no mezclarlo con los procesos de release del producto:

```
node scripts/build-system-docs.mjs
```

Desinstalación y pérdida de datos

Un punto que conviene no descubrir tarde:

Superficie	Qué pasa al desinstalar
Navegador / PWA	Borrar los datos del sitio elimina <code>localStorage</code> . Pérdida definitiva
Android	Desinstalar la app borra su sandbox, incluida la WebView. Pérdida definitiva
Windows	El desinstalador quita el ejecutable; los JSON de <code>app_data_dir</code> sobreviven salvo que se borren a mano. INFERENCIA a partir de <code>lib.rs</code> , que escribe fuera del directorio de instalación · REQUIERE VALIDACIÓN con una desinstalación real
CLI	<code>.local-data/</code> es un directorio normal: sobrevive a todo

La única defensa es exportar desde la vista **Datos**. El propio texto de esa vista lo dice: *«Tus datos viven en este dispositivo y no se envían a ningún servidor. Eso también significa que el respaldo es tu responsabilidad»*.

03 · Arquitectura

Fuente de verdad de las decisiones: `docs/ARCHITECTURE.md`, que explica

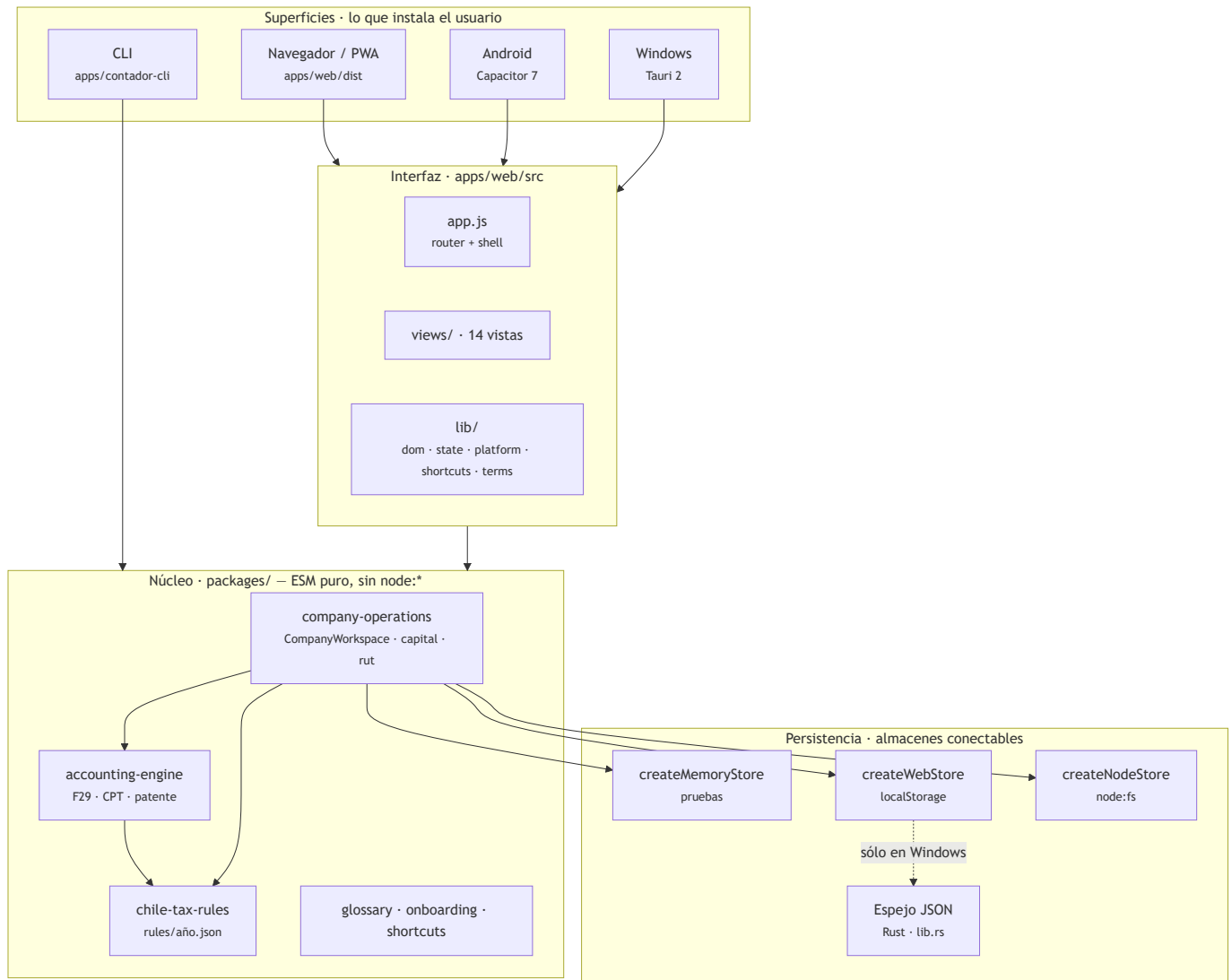
las siete decisiones de diseño con su problema, su decisión y su consecuencia. Este documento no las repite: añade los **diagramas**, el detalle de las capas y la verificación de cómo se hacen cumplir las reglas duras. Donde ambos hablen de lo mismo, manda `ARCHITECTURE.md` — con una excepción registrada en [15 · Riesgos](#): su mapa de directorios declara «10 vistas» y «50 pruebas», cifras que hoy son 14 y 153.

La conclusión, primero

La arquitectura entera se sostiene sobre una sola restricción: **un solo motor, tres plataformas**. Todo lo demás —la ausencia de bundler, los almacenes conectables, el re-render completo, las reglas como datos— es consecuencia de haber tomado esa decisión en serio y haberla convertido en un gate que aborta el build.

El sistema es **síncrono y de un solo hilo**. `CompanyWorkspace` no tiene un solo método `async`, y esa decisión propaga: `localStorage` es síncrono, el motor contable es síncrono, y por eso el espejo de Windows es una copia posterior en vez de la fuente. Cambiar eso obligaría a rehacer las catorce vistas.

Vista de capas



Qué muestra el diagrama y qué no. Muestra la dirección única de las dependencias: las superficies dependen de la interfaz, la interfaz del núcleo, el núcleo de la persistencia — y nunca al revés. Muestra también que la CLI se salta la interfaz por completo. **No muestra** los scripts de build, que son el mecanismo físico por el que el núcleo llega a cada superficie (eso está más abajo, en «El pipeline de build»), ni el sentido temporal: el espejo de Windows se escribe *después* de `localStorage`, con 180 ms de retardo, no en paralelo.

Una precisión importante que el diagrama simplifica: `node-store.mjs` **no viaja al navegador**. Es el único módulo de `company-operations` que importa `node:fs`, y por eso vive en un archivo aparte, al que sólo llega `index.mjs` (la entrada de Node). `apps/web` importa `workspace.mjs` y `store.mjs` directamente, saltándose `index.mjs`.

La regla dura: nada que viaje al dispositivo importa `node:*`

Es la regla más importante del repositorio y la única que se defiende en **tres** sitios distintos.

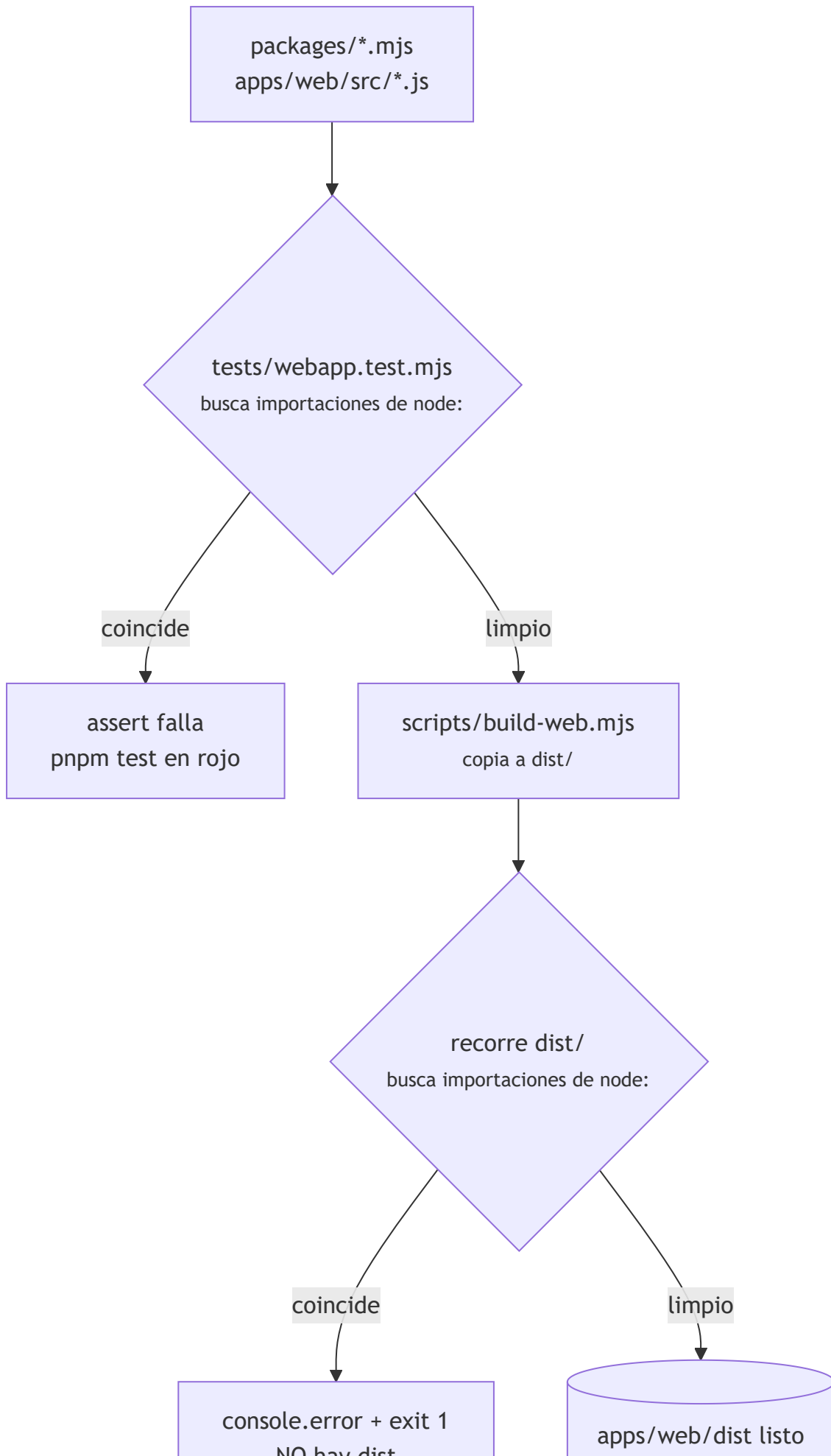
Por qué existe

Un `import fs from 'node:fs'` dentro de un módulo que acaba en el APK no produce un error de compilación. Produce una WebView que arranca, muestra una **pantalla en blanco** y deja todas las señales del build en verde. Es exactamente el tipo de fallo que sólo se detecta cuando alguien instala el APK.

Cómo se hace cumplir

Capa	Dónde	Qué comprueba	Cuándo actúa
1 · Build	<code>scripts/build-web.mjs</code> , líneas finales	Recorre <code>apps/web/dist</code> , filtra <code>.mjs</code> / <code>.js</code> y busca <code>/from\s+[']"node:/</code> . Si encuentra algo, imprime los archivos y <code>process.exit(1)</code>	En cada build, local y en CI
2 · Prueba	<code>tests/webapp.test.mjs</code> , test <code>ningún módulo que viaja al navegador importa node:*</code>	Mismo patrón sobre el código fuente : todos los <code>.js</code> de <code>apps/web/src</code> más los 13 módulos de <code>BROWSER_CORE</code>	En <code>pnpm test</code> y en CI
3 · Documento	<code>CONTRIBUTING.md</code> , sección «Tocar el motor operativo»	Lo dice y explica por qué	Al leer

La capa 1 mira el resultado; la capa 2 mira la fuente. Son complementarias: la 2 falla antes, la 1 atrapa cualquier módulo que se cuele al `dist` por una ruta que la 2 no conoce.



no hay que

para APK, Tauri y Pages

Qué muestra el diagrama y qué no. Muestra que hay dos puertas en serie y que ninguna de las dos produce un artefacto si la comprobación falla. **No muestra** el límite de la comprobación, que es real y está registrado en 15 · Riesgos: el patrón `/from\s+['"]node:/` sólo reconoce la forma `import ... from 'node:x'`. Un `import 'node:fs';` de efecto secundario, o un `await import('node:fs')` dinámico, **pasarían las dos puertas**.

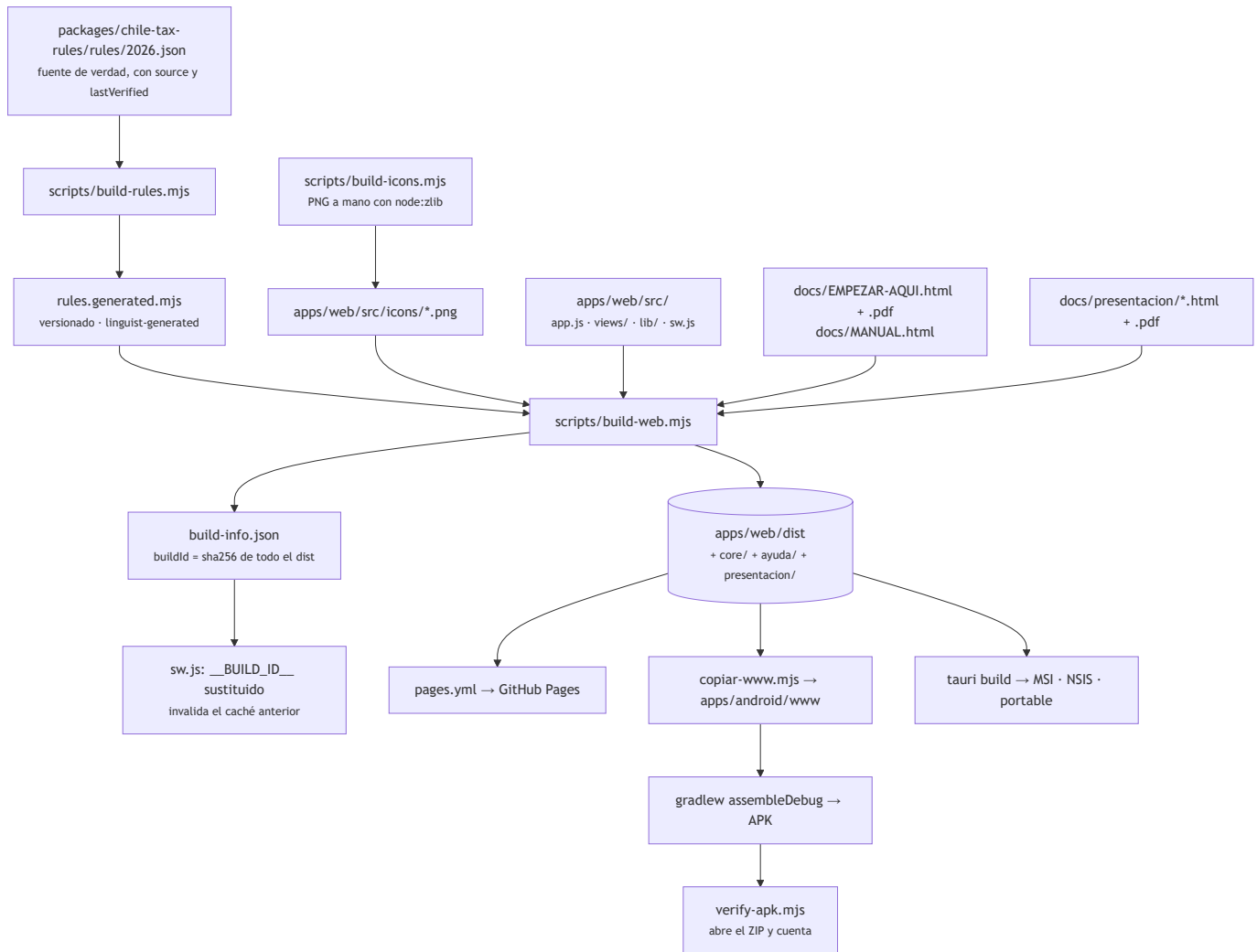
El otro lado de la regla

Lo que sí necesita `node:*` vive donde puede:

Archivo	Importa	Quién lo carga
<code>packages/company-operations/node-store.mjs</code>	<code>node:fs</code> , <code>node:path</code>	<code>index.mjs</code> (Node) — nunca el navegador
<code>packages/company-operations/index.mjs</code>	vía <code>node-store.mjs</code>	CLI, servidor, pruebas
<code>apps/empresa-operativa/server.mjs</code>	<code>node:http</code> , <code>node:fs</code> , <code>node:path</code> , <code>node:url</code>	Sólo el proceso servidor
<code>apps/contador-cli/src/index.mjs</code>	<code>node:fs</code> , <code>node:path</code>	Sólo la CLI
<code>scripts/**</code>	libremente	Sólo el build

La lista de lo que **sí** viaja está declarada dos veces, y ésta es una duplicación real registrada en 15 · Riesgos: `CORE` en `scripts/build-web.mjs` y `BROWSER_CORE` en `tests/webapp.test.mjs` . Hoy coinciden exactamente (13 módulos), pero nada comprueba que sigan coincidiendo.

El pipeline de build

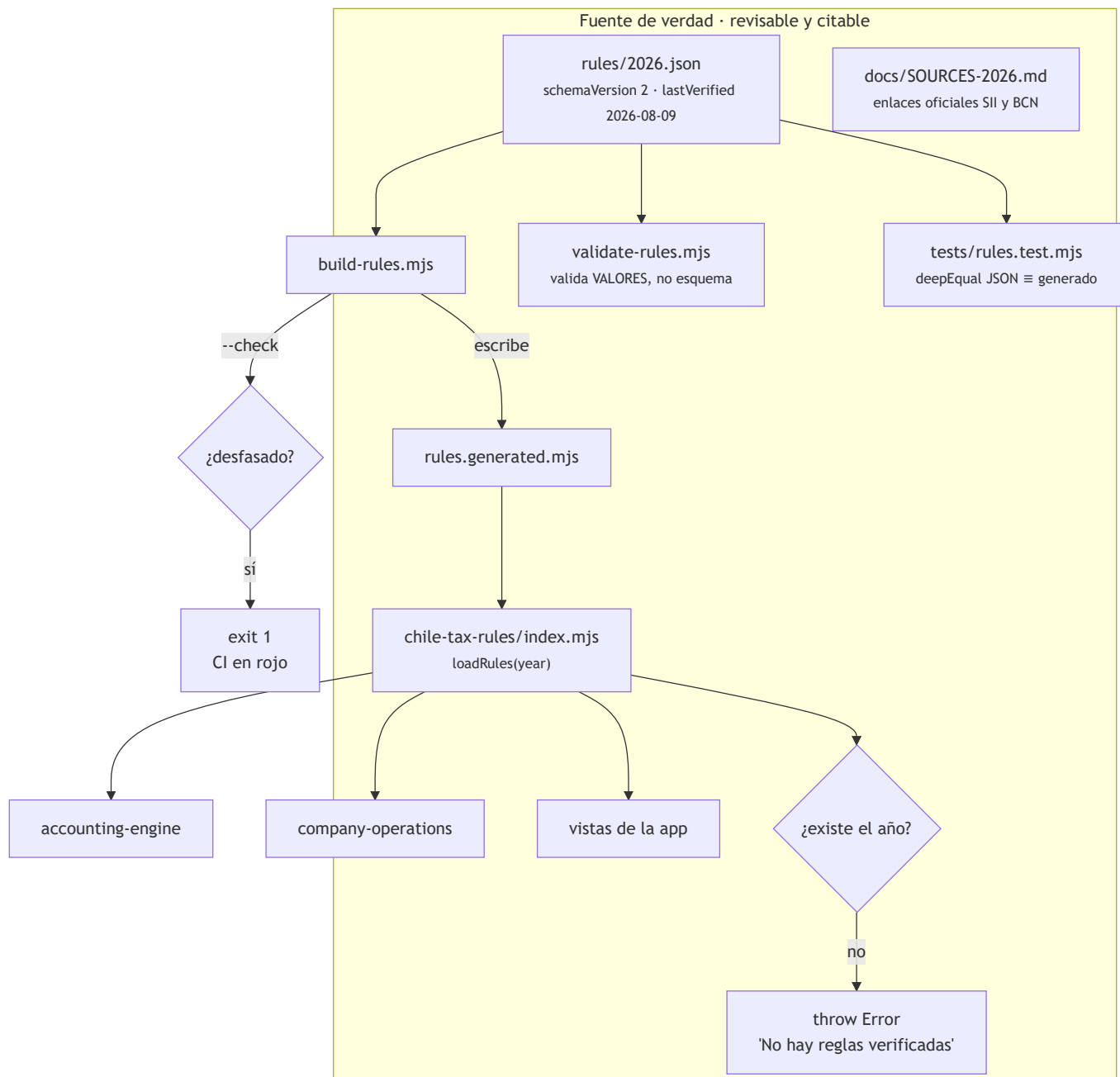


Qué muestra el diagrama y qué no. Muestra que todo converge en `apps/web/dist` y que ese directorio es literalmente lo que se publica en las tres plataformas: la demo pública nunca queda desfasada respecto de lo que la gente instala. Muestra también que los manuales y la presentación

viajan dentro del bundle — es contenido pesado embarcado a propósito, porque un manual que necesita internet no sirve justo cuando alguien lo necesita.

No muestra dos cosas. Primera: el `buildId` es un hash SHA-256 del contenido completo de `dist`, truncado a 12 caracteres, y CI construye dos veces para comprobar que **no cambia** — si cambiara, algo dependería del reloj o del orden del sistema de archivos, y el artefacto publicado no sería el que se probó. Segunda: el PDF grande del manual (7,7 MB) se queda **fuera** del bundle a propósito; sólo viajan `EMPEZAR-AQUI.html`, `EMPEZAR-AQUI.pdf` y `MANUAL.html`.

Las reglas tributarias como datos versionados



Qué muestra el diagrama y qué no. Muestra el ciclo completo de una tasa: se escribe en el JSON con su fuente, se embebe en un módulo ESM para que el navegador pueda leerla, y tres comprobaciones independientes vigilan que las dos copias no se separen. Muestra también el comportamiento clave: un año sin archivo **lanza**, no degrada.

No muestra la sutileza que hace que esto valga la pena: `validate-rules.mjs` no valida estructura (eso lo hace `build-rules.mjs`), valida **valores concretos** —que el IVA siga siendo 19 %, que la retención siga siendo 15,25 %, que el tope de patente sean 8.000 UTM y no 4.000—. Es la red que atrapa el error de dedo en un JSON que sigue siendo sintácticamente perfecto. El detalle está en [07 · Persistencia](#).

Almacenamiento conectable

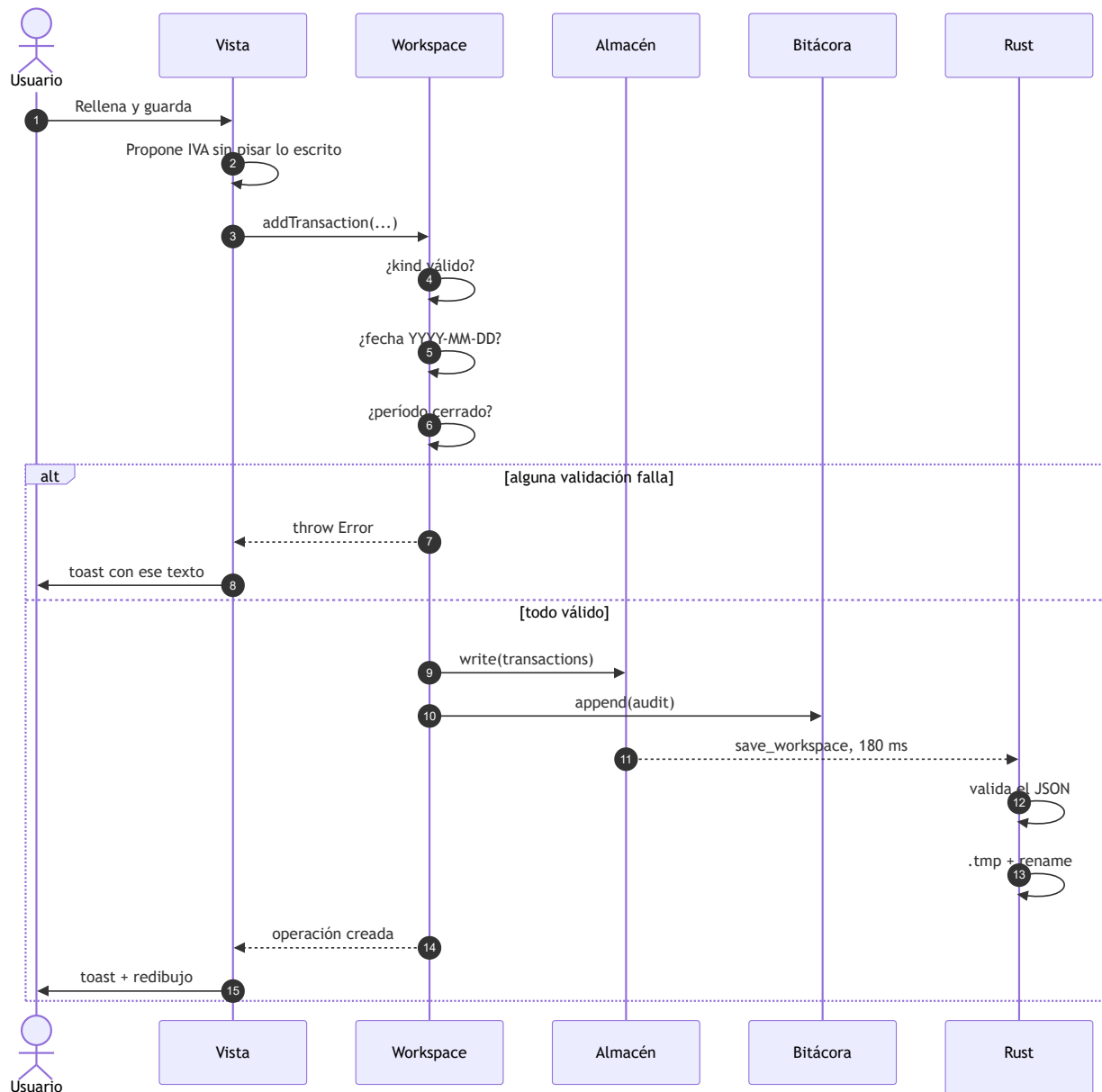
`CompanyWorkspace` no sabe dónde escribe. Habla un contrato de seis métodos, documentado en la cabecera de `packages/company-operations/store.mjs` :

```
read(key, fallback)      → valor JSON o fallback
write(key, value)         → persiste el valor (atómico si puede)
append(key, row)          → agrega una fila a un log append-only
readAll(key)              → filas del log, en orden
saveSnapshot(name, data) → guarda un respaldo y devuelve su ubicación
listSnapshots()           → nombres de respaldos existentes
```

Implementación	Archivo	Dónde escribe	Atomicidad
<code>createMemoryStore()</code>	<code>store.mjs</code>	<code>Map</code> en memoria, con clonado por JSON	N/A
<code>createWebStore({namespace})</code>	<code>store.mjs</code>	<code>localStorage</code> , prefijado por <code>namespace</code>	No
<code>createNodeStore({rootDir, mode})</code>	<code>node-store.mjs</code>	Archivos JSON + <code>audit.ndjson</code>	Sí: temporal + <code>rename</code>
<code>createPlatformStore(mode)</code>	<code>apps/web/src/lib/platformform.js</code>	Envuelve <code>createWebStore</code> y añade el espejo Tauri	El espejo sí (Rust); <code>localStorage</code> no

Consecuencia arquitectónica que conviene nombrar: no hay una sola rama `if (isBrowser)` dentro de la lógica de negocio, las pruebas corren en memoria sin tocar disco, y la separación EMPRESA REAL / SANDBOX se consigue **dando a cada modo su propio almacén** — no con una bandera que alguien pueda olvidar de comprobar. `createWebStore` exige `namespace` , y `state.js` lo construye como `empresa-operativa-chile:${mode}` .

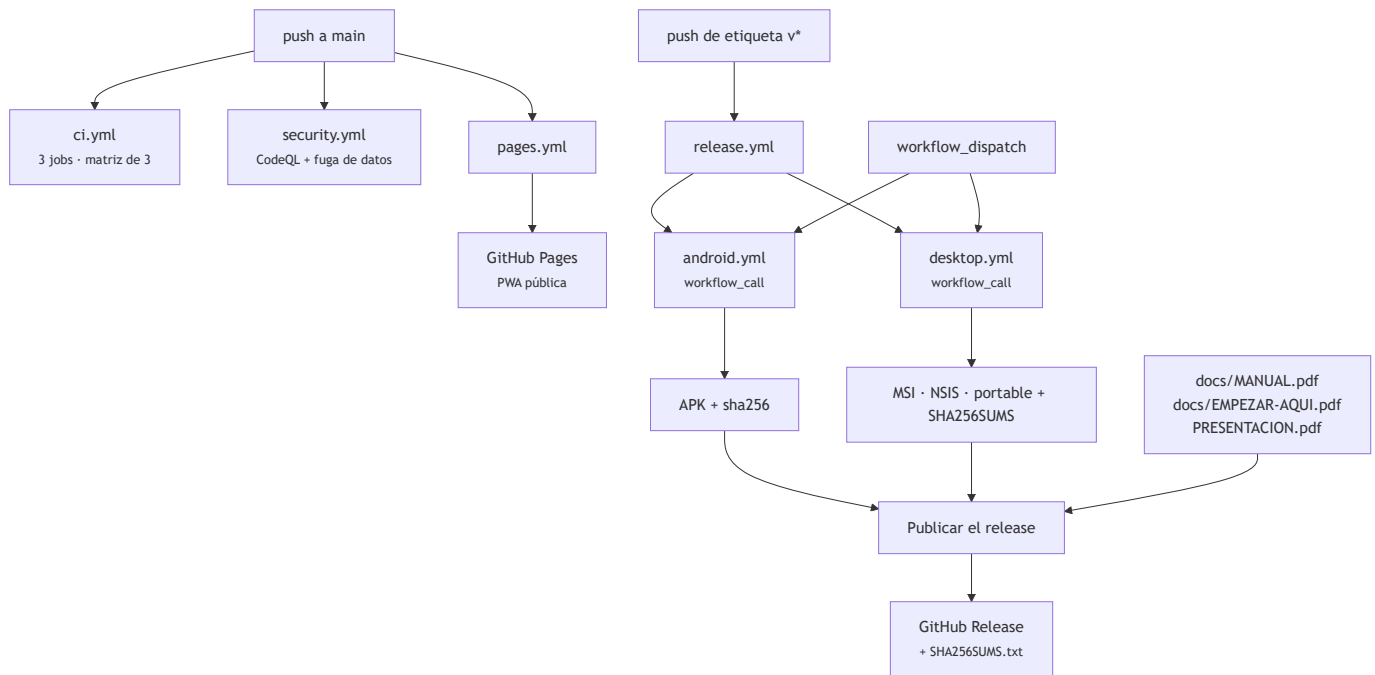
El ciclo de una operación, de principio a fin



Qué muestra la secuencia y qué no. Muestra dónde vive cada validación (todas en el motor, ninguna sólo en la vista) y que el mensaje de error **es** la explicación de la regla: el motor lanza textos escritos para una persona —«Un paso sólo puede marcarse como realizado si registras su evidencia»— y `attempt()` en `lib/dom.js` los muestra tal cual en vez de un «Error» genérico.

No muestra que el espejo de Rust es *best effort*: si la escritura a disco falla, la aplicación sigue funcionando con `localStorage` y sólo avisa por consola. Perder la sesión por un error de escritura sería peor que perder el espejo. Tampoco muestra el caso de Android y navegador, donde los pasos 12–14 simplemente no ocurren.

Despliegue



Qué muestra el diagrama y qué no. Muestra que `release.yml` **reutiliza** los workflows de Android y de Windows con `workflow_call` en vez de duplicar sus pasos: lo que se publica es exactamente lo que esos workflows verificaron. Muestra también que la documentación viaja con el release.

No muestra el detalle que hace que el release sea confiable: antes de publicar, `release.yml` comprueba con `test -f` que los seis artefactos obligatorios existan, y aborta si falta uno. Un release a medias es peor que un release que falla. Tampoco muestra que **etiquetar es una acción humana**: nada dispara `release.yml` automáticamente.

Decisiones que conviene entender antes de tocar nada

Resumidas desde `docs/ARCHITECTURE.md` y verificadas contra el código. El detalle del *porqué* está allí; aquí queda el *qué pasa si lo cambias*.

Decisión	Qué pasa si se revierte
Un motor, tres plataformas	El IVA de agosto podría dar distinto en el teléfono y en el escritorio
Almacenamiento conectable	Volverían los <code>if (isBrowser)</code> a la lógica de negocio; las pruebas necesitarían disco
Las reglas son datos	Una tasa vieja seguiría calculando con toda confianza el año en que deja de ser cierta
Sin bundler	El código depurado en producción dejaría de ser el del repositorio; entrarían dependencias
Re-render completo	Volvería la clase entera de errores de sincronización vista/datos
Se verifica el artefacto, no el build	Un APK vacío podría publicarse con todas las señales en verde
Integridad por encima de comodidad	«Cerrado» y «hecho» dejarían de significar algo

Y una decisión que no está en `ARCHITECTURE.md` y merece nombrarse: **la documentación se genera desde el código**. El glosario, la guía «Empezar aquí» y la tabla de atajos tienen una sola copia —en `packages/`— y los documentos son proyecciones que CI compara con `--check`. Un glosario escrito a mano en tres sitios se contradice en dos semanas; éste no puede.

04 · Mapa del código

Inventario jerárquico de los **262 archivos versionados**. Cada elemento lleva su ubicación, responsabilidad, dependencias, quién lo usa y su **estado aparente**.

Estados usados en este documento:

Estado	Significado
Activo	En uso, con consumidores identificados
Generado	Producido por un script; no se edita a mano
Legado	Se conserva por compatibilidad; hay un camino nuevo preferido
Duplicado	Existe otra copia del mismo conocimiento
Experimental	Presente pero sin consumidores en producción
No determinado	No se pudo establecer con evidencia

Ningún elemento del repositorio quedó marcado **Experimental**; los dos **Legado** y los dos **Duplicado** se justifican en su fila y se recogen en [15 · Riesgos](#).

Árbol general

```
empresa-operativa-chile/
├─ packages/           6 paquetes · el núcleo · ESM puro sin node:* (salvo node-store)
├─ apps/              5 aplicaciones · web, android, contador-desktop, empresa-operativa, contador-
cli
├─ scripts/           13 scripts + 4 librerías · build, generación de docs y verificación
├─ tests/             13 suites · 153 pruebas
├─ docs/              23 documentos Markdown + assets + presentación + esta serie
├─ curriculum/ labs/ cases/ material de aprendizaje: 16 partes, 16 laboratorios, 2 casos
├─ data/scenarios/    1 escenario JSON para la CLI
├─ .github/           6 workflows + 3 plantillas de issue + plantilla de PR
└─ raíz               package.json, .gitignore, .gitattributes, CHANGELOG, CONTRIBUTING,
SECURITY, CODE_OF_CONDUCT, LICENSE, README
```

1 · `packages/` — el núcleo

1.1 `packages/chile-tax-rules/` — reglas versionadas por año comercial

Elemento	Líneas	Responsabilidad	Depende de	Quién lo usa	Estado
<code>rules/2026.json</code>	138	Fuente de verdad de todas las tasas. IVA, honorarios, PPM, IDPC, UTM, patente municipal (rango, base, deducciones, prorrateo), CPT (dos métodos) y F29. Cada regla con <code>source</code> , <code>lastVerified</code> y <code>note</code>	—	<code>build-rules.mjs</code> , <code>validate-rules.mjs</code> , <code>tests/rules.test.mjs</code>	Activo
<code>rules.generated.mjs</code>	150	El mismo contenido embebido como ESM, para que el navegador lo lea sin <code>node:fs</code>	<code>rules/*.json</code>	<code>index.mjs</code>	Generado (versionado; <code>linguist-generated=true</code>)
<code>index.mjs</code>	47	<code>loadRules(year)</code> , <code>availableYears()</code> , <code>ruleProvenance(year, rule)</code> . Falla si el año no existe	<code>rules.generated.mjs</code>	Todo el motor, todas las vistas, la CLI	Activo
<code>municipalities.mjs</code>	114	Maestro de 15 comunas: identidad sin tasas. <code>findMunicipality</code> , <code>municipalityOptions</code> , <code>normalizeMunicipality</code> , <code>RATE_STATUS</code> , <code>UNVERIFIED_RATE_WARNING</code>	—	<code>municipal-patent.mjs</code> , <code>workspace.mjs</code> , <code>capital.js</code>	Activo

La decisión que define este paquete: el catálogo municipal trae `patentRate: null` y `status: 'UNVERIFIED'` para las 15 comunas. Publicar una tabla nacional de tasas sería fabricar treinta y tantas cifras que nadie verificó, y el usuario las leería como si fueran la boleta. La tasa la aporta el usuario con su fuente y su fecha, y queda en **su** espacio de trabajo.

1.2 packages/accounting-engine/ — el motor de cálculo

Elemento	Líneas	Responsabilidad	Depende de	Quién lo usa	Estado
<code>index.mjs</code>	305	Operaciones elementales (<code>saleFromNet</code> , <code>purchaseFromNet</code> , <code>honorariumFromGross</code> , <code>ppmFromSalesNet</code>), <code>f29Basic</code> , <code>f29DueDates</code> , <code>municipalPatent</code> , <code>idpcProPyme</code> , <code>journal</code> , <code>simulateScenario</code> , <code>clp</code>	<code>chile-tax-rules</code>	Vistas <code>panel</code> , <code>impuestos</code> , <code>obligaciones</code> , <code>academia</code> ; CLI; <code>workspace.mjs</code>	Activo
<code>tax-equity.mjs</code>	235	Capital propio tributario por dos métodos: art. 41 N.º 1 y simplificado 14 D N.º 3 (j). <code>calculateTaxEquity</code> , <code>allowsSimplifiedTaxEquity</code>	<code>chile-tax-rules</code>	<code>workspace.mjs</code> , vista <code>capital</code> , CLI	Activo
<code>municipal-patent.mjs</code>	229	Patente municipal con la bifurcación del art. 24: empresa nueva vs. en funcionamiento. <code>calculateMunicipalPatent</code>	<code>chile-tax-rules</code> , <code>municipalities.mjs</code>	<code>workspace.mjs</code> , vista <code>capital</code> , CLI	Activo

Nota sobre duplicidad funcional aparente. `index.mjs` exporta `municipalPatent()` y `municipal-patent.mjs` exporta `calculateMunicipalPatent()`. **No es un duplicado por descuido:** el propio código lo documenta. `municipalPatent()` es una estimación rápida que aplica una tasa a la cifra que le pasen —la usan la CLI y los laboratorios para enseñar el tope mínimo— y **no decide qué capital corresponde usar**. `calculateMunicipalPatent()` sí lo decide. Marcado **Activo** con propósito distinto, no duplicado.

1.3 packages/company-operations/ — el motor operativo

Elemento	Líneas	Responsabilidad	Depende de	Quién lo usa	Estado
<code>workspace.mjs</code>	1, 4, 2	El orquestador central. Clase <code>CompanyWorkspace</code> : ficha, capital, municipalidad, ejercicio anual, constitución, operaciones, obligaciones, períodos, respaldo y diagnóstico. Más <code>seedSandboxWorkspace()</code>	<code>capital.mjs</code> , <code>tax-equity.mjs</code> , <code>municipal-patent.mjs</code> , <code>municipalities.mjs</code> , <code>chile-tax-rules</code>	Todas las vistas vía <code>state.js</code> ; <code>index.mjs</code> ; CLI	Activo
<code>capital.mjs</code>	37, 1	Modelo societario: <code>EQUITY_MOVEMENT_KINDS</code> (9 tipos), <code>DATA_STATUS</code> (5), <code>DATA_ORIGIN</code> (5), <code>normalizeCapitalProfile</code> , <code>validateCapitalProfile</code> , <code>capitalPendingToPay</code> , <code>normalizeEquityMovement</code> , <code>summarizeEquityMovements</code>	—	<code>workspace.mjs</code> , vista <code>capital</code>	Activo
<code>store.mjs</code>	84	<code>createMemoryStore()</code> y <code>createWebStore({namespace})</code> . Define el contrato de seis métodos	—	<code>platform.js</code> , pruebas	Activo
<code>node-store.mjs</code>	87	Almacén de disco. Escritura atómica (temporal + <code>rename</code>) y bitácora NDJSON <i>append-only</i>	<code>node:fs</code> , <code>node:path</code>	<code>index.mjs</code> (Node) — nunca el navegador	Activo
<code>rut.mjs</code>	43	RUT chileno: <code>cleanRut</code> , <code>rutCheckDigit</code> (módulo 11), <code>validateRut</code>	—	Vista <code>empresa</code> , CLI	Activo
<code>index.mjs</code>	49	Entrada de Node: <code>CompanyWorkspace</code> extendida con <code>rootDir</code> , <code>seedSandbox(rootDir)</code> , <code>file(name)</code> , <code>dir</code>	<code>workspace.mjs</code> , <code>node-store.mjs</code>	CLI, pruebas	Activo

1.4 Paquetes de contenido

Elemento	Líneas	Responsabilidad	Consumidores	Estado
<code>glossary/index.mjs</code>	654	54 términos en 6 categorías, con <code>notToConfuseWith</code> , <code>related</code> , <code>legalReference</code> , <code>term()</code> , <code>searchTerms()</code> , <code>termsByCategory()</code> , <code>danglingReferences()</code>	Vista <code>glosario</code> , ayudas contextuales (<code>terms.js</code>), buscador <code>Ctrl+K</code> , <code>build-glossary.mjs</code> → <code>docs/GLOSSARY.md</code>	Activo
<code>onboarding/index.mjs</code>	624	14 etapas en 5 fases , <code>COVERAGE</code> (9 cubiertas / 10 no cubiertas), 12 <code>FIRST_QUESTIONS</code> . <code>stage()</code> , <code>stagesByPhase()</code> , <code>danglingReferences()</code>	Vista <code>empezar</code> , <code>build-guide.mjs</code> → <code>EMPEZAR-AQUI.{md,html,pdf}</code> , <code>lib/diagrams.mjs</code> , <code>capture-screenshots.mjs</code>	Activo
<code>shortcuts/index.mjs</code>	186	12 atajos en 3 grupos. <code>resolveShortcut(event, {typing})</code> , <code>shortcutsByGroup()</code> , <code>keysFor()</code>	<code>lib/shortcuts.js</code> , <code>build-shortcuts.mjs</code> → <code>docs/ATAJOS-DE-TECLADO.md</code>	Activo

Los tres comparten el mismo patrón —**datos en el código, documento generado, sincronía comprobada en CI**— y es la razón de que la documentación de este repositorio no pueda contradecir al producto.

2 · `apps/` — las superficies

2.1 `apps/web/src/` — la única interfaz del producto

Elemento	Líneas	Responsabilidad	Estado
<code>app.js</code>	249	Router, shell, barra lateral, cambio de modo/tema/período, arranque, instalación de atajos, registro del service worker. <code>APP_VERSION</code>	Activo
<code>index.html</code>	32	Documento base	Activo
<code>app.css</code>	785	Hoja de estilos completa, con tema claro y oscuro	Activo
<code>sw.js</code>	62	Service worker: caché de la propia app. No intercepta datos del usuario. <code>CACHE</code> con <code>__BUILD_ID__</code> sustituido en el build	Activo
<code>manifest.webmanifest</code>	24	Manifiesto PWA: 4 iconos y 2 accesos directos (<code>#operaciones</code> , <code>#impuestos</code>)	Activo
<code>lib/dom.js</code>	183	<code>html</code> (escapa toda interpolación por defecto), <code>raw</code> , <code>esc</code> , formateo CLP/fecha/período, <code>toast</code> , <code>attempt</code> , <code>openModal</code> , <code>confirmAction</code>	Activo
<code>lib/state.js</code>	66	Estado global, dos <code>CompanyWorkspace</code> vivos (real y sandbox), preferencias en <code>localStorage</code> , <code>selectablePeriods()</code>	Activo
<code>lib/platform.js</code>	150	Detección de plataforma, <code>createPlatformStore</code> , <code>hydrateFromDisk</code> , <code>saveTextFile</code> , <code>pickTextFile</code> , <code>openExternal</code>	Activo
<code>lib/shortcuts.js</code>	225	Oyente global único, buscador de comandos (<code>Ctrl+K</code>), diálogo de ayuda	Activo
<code>lib/terms.js</code>	144	Ayudas contextuales: <code>termChip</code> , <code>termWord</code> , <code>mountTerms</code> , globo flotante colgado del <code>body</code>	Activo
<code>icons/icon.svg</code>	—	Icono fuente; los PNG los genera <code>build-icons.mjs</code>	Activo

Las 14 vistas. Cada una exporta `{ id, label, title, icon, render() }` y opcionalmente `mount(root, rerender)` y `badge()`. Una prueba verifica los cinco campos en las catorce.

Vista	Líneas	Qué resuelve	Estado
<code>empezar.js</code>	267	Las 14 etapas ordenadas por <i>tiempo</i> , con el estado real de los trámites	Activo
<code>panel.js</code>	218	Indicadores del período, <code>healthCheck()</code> , próximos vencimientos	Activo
<code>operaciones.js</code>	273	Alta, edición, borrado y filtrado de operaciones; propone el IVA sin pisar lo escrito	Activo
<code>impuestos.js</code>	184	Borrador F29 con remanente arrastrado; lista de control persistida en el período	Activo
<code>obligaciones.js</code>	155	Calendario de cumplimiento con comprobante obligatorio	Activo
<code>cierre.js</code>	146	Cierre mensual con lista de 5 puntos declarada	Activo
<code>constitucion.js</code>	102	Los 9 trámites con autoridad, evidencia y enlace oficial	Activo
<code>empresa.js</code>	190	Ficha de la empresa; valida el RUT con <code>validateRut</code>	Activo
<code>capital.js</code>	694	La vista más grande: seis magnitudes, ledger patrimonial, CPT, patente y cierre anual	Activo
<code>auditoria.js</code>	92	Bitácora, con buscador que conserva el cursor	Activo
<code>datos.js</code>	179	Exportar, importar, respaldos, CSV	Activo
<code>academia.js</code>	254	Explicaciones que usan el mismo motor que calcula	Activo
<code>glosario.js</code>	95	Los 54 términos, buscables	Activo
<code>ayuda.js</code>	187	Los manuales en un marco del mismo origen; atajos	Activo

2.2 `apps/empresa-operativa/` — servidor local estático

Elemento	Líneas	Responsabilidad	Estado
<code>server.mjs</code>	101	Sirve <code>apps/web/dist</code> en <code>127.0.0.1:4180</code> . <code>resolveSafe()</code> bloquea el escape de directorio; cabeceras CSP, <code>nosniff</code> , <code>no-referrer</code>	Activo
<code>start-windows.bat</code>	30	Lanzador de doble clic. CRLF forzado por <code>.gitattributes</code>	Activo
<code>start-windows.ps1</code>	20	Lanzador PowerShell equivalente	Activo

2.3 apps/android/ — envoltorio Capacitor 7

Elemento	Responsabilidad	Estado
<code>capacitor.config.json</code>	<code>appId: cl.vladimiracuna.empresaoperativa</code> , <code>webDir: www</code> , <code>allowMixedContent: false</code> , <code>webContentsDebuggingEnabled: false</code> , <code>CapacitorHttp</code> desactivado	Activo
<code>copiar-www.mjs</code> (74)	Copia <code>dist</code> a <code>www</code> y cuenta vistas y núcleo antes de seguir; aplica iconos si el proyecto Android existe	Activo
<code>package.json</code> (21)	4 dependencias Capacitor + CLI. Versión <code>1.4.0</code> , sincronizada por una prueba	Activo
<code>res-icons/</code>	15 PNG + 1 XML de icono adaptativo, en 6 densidades (<code>mipmap-mdpi</code> ... <code>mipmap-xxxhdpi</code>)	Activo
<code>pnpm-lock.yaml</code> (792)	Lockfile válido	Activo
<code>package-lock.json</code> (1.186)	Lockfile de npm	Duplicado — ver nota

Nota sobre `apps/android/package-lock.json` . El repositorio declara `pnpm@11.2.2` como gestor y los workflows ejecutan `pnpm install --frozen-lockfile`, que lee `pnpm-lock.yaml` . El `package-lock.json` no lo consume nada en el repositorio: es un residuo de una instalación con npm. Registrado en [15 · Riesgos](#); **no se eliminó**, porque borrarlo es un cambio funcional.

2.4 apps/contador-desktop/ — shell Tauri 2 (Windows)

Elemento	Líneas	Responsabilidad	Estado
<code>src-tauri/src/lib.rs</code>	135	Cuatro comandos: <code>load_workspace</code> , <code>save_workspace</code> , <code>export_file</code> , <code>app_info</code> . <code>safe_mode()</code> valida el modo. Dos pruebas Rust	Activo
<code>src-tauri/src/main.rs</code>	6	Punto de entrada; sin consola en release	Activo
<code>src-tauri/build.rs</code>	1	<code>tauri_build::build()</code>	Activo
<code>src-tauri/Cargo.toml</code>	31	<code>tauri 2</code> , <code>serde</code> , <code>serde_json</code> . Sin base de datos embebida — el comentario explica por qué se quitó <code>rusqlite</code>	Activo
<code>src-tauri/tauri.conf.json</code>	49	Ventana, CSP, bundle MSI + NSIS, <code>identifier</code> igual al <code>appId</code> de Android	Activo
<code>src-tauri/capabilities/default.json</code>	7	<code>permissions: ["core:default"]</code> y nada más. Sin red, sin shell, sin FS genérico	Activo
<code>package.json</code>	15	Sólo <code>@tauri-apps/cli</code> . <code>version: "1.0.0"</code>	Legado — ver nota

Nota sobre la versión. `apps/contador-desktop/package.json` declara `1.0.0`, mientras `package.json` raíz, `tauri.conf.json`, `Cargo.toml`, `app.js` y `apps/android/package.json` declaran `1.4.0`. La prueba «la versión de la app coincide en package.json, Tauri y Cargo» comprueba cinco de esos seis sitios y **no** incluye éste. Como ese `package.json` sólo sirve para invocar el CLI de Tauri —que toma la versión de `tauri.conf.json`— el desfase no afecta al binario producido. Registrado en [15 · Riesgos](#).

2.5 apps/contador-cli/ — línea de comandos

Elemento	Líneas	Responsabilidad	Estado
<code>src/index.mjs</code>	323	23 comandos en dos familias: cálculo puro , 15 contando <code>ayuda</code> (no tocan disco), y espacio de trabajo , 8 (archivos en <code>--datos</code>)	Activo

3 · scripts/ — build, generación y verificación

Script	Líneas	Responsabilidad	-- check	Quién lo ejecuta	Estado
<code>build-all.mjs</code>	29	Orquesta reglas → iconos → web	—	<code>pnpm build</code> , <code>pnpm start</code> , CI, todos los workflows	Activo
<code>build-rules.mjs</code>	65	<code>rules/*.json</code> → <code>rules.generated.mjs</code>	✓	<code>pnpm check</code> , CI	Activo
<code>build-icons.mjs</code>	335	PNG a mano con <code>node:zlib</code> , sin dependencias	—	<code>build-all</code>	Activo
<code>build-web.mjs</code>	128	Compone <code>apps/web/dist</code> , embarca manuales y presentación, **aborta si hay <code>node:*</code> ** , calcula <code>buildId</code>	—	<code>build-all</code>	Activo
<code>build-glossary.mjs</code>	90	<code>packages/glossary</code> → <code>docs/GLOSSARY.md</code>	✓	<code>pnpm check</code> , CI	Activo
<code>build-guide.mjs</code>	407	<code>packages/onboarding</code> → <code>EMPEZAR-AQUI.{md,html,pdf}</code> + 2 diagramas SVG	✓ (sólo el <code>.md</code>)	<code>pnpm check</code> , CI, <code>pnpm docs</code>	Activo
<code>build-shortcuts.mjs</code>	86	<code>packages/shortcuts</code> → <code>docs/ATAJOS-DE-TECLADO.md</code>	✓	<code>pnpm check</code> , CI	Activo
<code>build-manual.mjs</code>	115	<code>docs/MANUAL.md</code> → <code>MANUAL.pdf</code> (densidad 2) y <code>MANUAL.html</code> (densidad 1)	—	<code>pnpm manual</code> , <code>pnpm docs</code>	Activo
<code>build-presentacion.mjs</code>	317	<code>docs/presentacion.md</code> → 4 artefactos: diapositivas y pauta, en HTML y PDF	✓	<code>pnpm check</code> , CI	Activo
<code>check-presentacion.mjs</code>	121	Abre los PDF, cuenta páginas y compara con lo que anuncian 5 archivos del repositorio	—	<code>pnpm check</code> , CI	Activo
<code>validate-rules.mjs</code>	48	Valida los valores de las tasas del año en curso	—	<code>pnpm check</code> , CI	Activo
<code>verify-apk.mjs</code>	124	Abre el APK como ZIP leyendo el directorio central a mano; 12 comprobaciones	—	<code>android.yml</code> , <code>pnpm android:verify</code>	Activo
<code>capture-screenshots.mjs</code>	115	Recaptura las pantallas con Chrome headless, en dos densidades	—	<code>pnpm capturas</code> , <code>pnpm docs</code>	Activo

Script	Líneas	Responsabilidad	-- check	Quién lo ejecuta	Estado
<code>build-system-docs.mjs</code>	—	Nuevo en esta serie. Genera los PDF de <code>docs/system-documentation/</code>	<code>--only</code>	Manual	Activo

`scripts/lib/` — librerías compartidas

Elemento	Líneas	Responsabilidad	Usado por	Estado
<code>chrome.mjs</code>	101	<code>findBrowser()</code> (Chrome o Edge), <code>screenshot()</code> , <code>printPdf()</code> . Perfil desechable para que las capturas sean reproducibles	<code>build-manual</code> , <code>build-guide</code> , <code>build-presentation</code> , <code>capture-screenshots</code> , <code>build-system-docs</code>	Activo
<code>markdown.mjs</code>	223	Conversor Markdown → HTML suficiente para este repositorio; <code>embedFrom()</code> embebe imágenes como data URI	Los mismos	Activo
<code>print-style.mjs</code>	174	<code>PRINT_CSS</code> (A4) y <code>SCREEN_CSS</code> (claro/oscuro), compartidos para que manual y guía parezcan del mismo producto	<code>build-manual</code> , <code>build-guide</code> , <code>build-system-docs</code>	Activo
<code>diagrams.mjs</code>	229	Genera 2 diagramas SVG desde los datos de <code>packages/onboarding</code>	<code>build-guide</code>	Activo

4 • tests/ — 13 suites, 153 pruebas

Suite	Líneas	Pruebas	Qué protege
<code>accounting.test.mjs</code>	9	6	Aritmética básica: IVA, retención, F29, patente mínima, IDPC, asiento de retiro
<code>f29.test.mjs</code>	66	8	IVA declarado vs. derivado, remanente, vencimientos y traslado de fin de semana
<code>rules.test.mjs</code>	44	4	JSON ≡ módulo generado; toda regla con fuente y fecha; año faltante lanza
<code>rut.test.mjs</code>	39	6	Dígito verificador módulo 11 y formatos aceptados
<code>capital.test.mjs</code>	178	19	Migración del campo <code>capital</code> antiguo, validaciones societarias, agregación de movimientos
<code>tax-equity.test.mjs</code>	177	17	Los dos métodos de CPT, el piso en cero, el rechazo del método a quien no califica
<code>municipal-patent.test.mjs</code>	211	23	La bifurcación del art. 24, topes, deducciones, prorrateo, UTM y tasa no verificada
<code>workspace.test.mjs</code>	146	14	Inmutabilidad del período, remanente entre períodos, evidencia obligatoria, respaldo
<code>company-operations.test.mjs</code>	34	4	Aislamiento real/sandbox sobre disco real
<code>glossary.test.mjs</code>	93	11	Referencias no rotas, unicidad de ids, categorías declaradas
<code>onboarding.test.mjs</code>	189	17	Etapas, fases, referencias a vistas y a trámites, cobertura declarada
<code>ayuda.test.mjs</code>	160	16	Atajos únicos, comportamiento al escribir, contenido de la vista de ayuda
<code>webapp.test.mjs</code>	115	8	**La regla <code>node:*</code> **, router completo, versión sincronizada, manifiesto PWA

Recuento verificado con `node --test tests/*.test.mjs`: `tests 153 · pass 153 · fail 0`.

5 · `.github/` — automatización

Elemento	Líneas	Disparadores	Estado
<code>workflows/ci.yml</code>	189	<code>push</code> / <code>PR</code> a main, manual	Activo
<code>workflows/security.yml</code>	95	<code>push</code> / <code>PR</code> a main, cron lunes 06:00 UTC, manual	Activo
<code>workflows/pages.yml</code>	58	<code>push</code> a main, manual	Activo
<code>workflows/android.yml</code>	99	Manual y <code>workflow_call</code>	Activo
<code>workflows/desktop.yml</code>	152	Manual y <code>workflow_call</code>	Activo
<code>workflows/release.yml</code>	121	Etiqueta <code>v*</code> , manual	Activo
<code>ISSUE_TEMPLATE/error.yml</code>	67	Reporte de error	Activo
<code>ISSUE_TEMPLATE/mejora.yml</code>	51	Propuesta de mejora	Activo
<code>ISSUE_TEMPLATE/regla-tributaria.yml</code>	69	Cambio de regla tributaria — el formulario exige fuente oficial	Activo
<code>ISSUE_TEMPLATE/config.yml</code>	11	Configuración del selector	Activo
<code>pull_request_template.md</code>	—	Plantilla de PR	Activo

6 · docs/ — documentación previa a esta serie

23 documentos Markdown. Los cuatro **generados** no se editan a mano; CI falla si se desvían.

Documento	Origen	Estado
GLOSSARY.md (588)	packages/glossary/index.mjs	Generado
EMPEZAR-AQUI.md (801) · .html (578) · .pdf	packages/onboarding/index.mjs	Generado
ATAJOS-DE-TECLADO.md (65)	packages/shortcuts/index.mjs	Generado
presentacion/*.html y *.pdf	docs/presentacion.md	Generado
MANUAL.html (581) · MANUAL.pdf	docs/MANUAL.md	Generado
ARCHITECTURE.md (168)	Escrito a mano	Activo (con drift — ver 15)
MANUAL.md (975)	Escrito a mano	Activo
SOURCES-2026.md (81)	Escrito a mano	Activo
README.md (63)	Escrito a mano	Activo (con drift)
ROADMAP.md (84)	Escrito a mano	Activo (con drift)
RUNBOOK-MENSUAL.md (43) · RUNBOOK-ANUAL.md (24)	Escrito a mano	Activo
DECISION-TREE.md (27) · ACCOUNTING-POLICIES.md (12)	Escrito a mano	Activo
product/ (6 documentos)	Escrito a mano	Activo
accounting/CAPITAL-PATRIMONIO.md · tax/CAPITAL-PROPIO-TRIBUTARIO.md · municipal/PATENTE-MUNICIPAL.md · guides/OFICINA-VIRTUAL.md	Escrito a mano	Activo
assets/capturas/ (19 PNG) · assets/compacto/ (16 PNG)	capture-screenshots.mjs	Generado
assets/diagramas/ (5 SVG)	2 por lib/diagrams.mjs, 3 a mano	Mixto

7 · Material de aprendizaje

Directorio	Contenido	Consumidores	Estado
curriculum/	curriculum.yaml (244 líneas) + 16 partes con su README	Lectura humana	Activo
labs/	16 laboratorios, cada uno con README.md y solution.md	Lectura humana	Activo
cases/	2 casos integrales	Lectura humana	Activo
data/scenarios/	1 escenario JSON para pnpm cli escenario	CLI	Activo
academy/README.md	Índice del material	Lectura humana	Activo

Hallazgo verificado. Ninguna prueba y ningún script del repositorio lee curriculum/ , labs/ , cases/ ni academy/ . El único lugar del código que los menciona es apps/web/src/views/academia.js:238 , y es una cadena de texto que los nombra al lector — no una lectura de archivo. data/scenarios/*.json sólo lo consume la CLI si el usuario le pasa la ruta a mano; ninguna prueba lo ejecuta. Comprobado con

```
grep -rn "labs/\\|curriculum/\\|cases/\\|academy/\\|data/scenarios" scripts/ tests/ packages/ apps/ .
```

Es material de lectura, no material verificado: si la solución de un laboratorio deja de coincidir con lo que el motor calcula, nada lo detecta. Registrado en

15 · Riesgos.

8 · Raíz del repositorio

Archivo	Responsabilidad	Estado
package.json	Versión 1.4.0 , type: module , engines.node >= 20 , packageManager: pnpm@11.2.2 , 21 scripts, cero dependencies	Activo
pnpm-lock.yaml	Lockfile raíz	Activo
.gitignore	Excluye node_modules , artefactos de build y datos operativos reales	Activo
.gitattributes	eol=lf global, CRLF para .bat / .ps1 , binarios sin conversión, linguist-generated y linguist-documentation	Activo
README.md (490)	Portada del proyecto	Activo (con drift)
CHANGELOG.md (367)	Keep a Changelog + SemVer	Activo
CONTRIBUTING.md	Reglas para cambiar una tasa; el documento más importante para un colaborador nuevo	Activo
SECURITY.md	Modelo de amenazas y qué no subir nunca	Activo
CODE_OF_CONDUCT.md · LICENSE (MIT)	—	Activo

05 · Referencia técnica

Catálogo de consulta. Para cada función relevante: **firma, propósito, parámetros, retorno, excepciones, efectos secundarios, quién la llama, a quién llama y riesgo al modificarla.**

El riesgo se gradúa así:

Riesgo	Significado
 Crítico	Un error aquí produce una cifra plausible y equivocada, o pierde datos, sin señal visible
 Alto	Rompe una garantía del producto (inmutabilidad, aislamiento, evidencia)
 Medio	Rompe una vista o un flujo, con error visible
 Bajo	Presentación o utilidad; el fallo se ve de inmediato

1 • packages/chile-tax-rules/index.mjs

loadRules(year = 2026)

Campo	Valor	
Propósito	Devolver el objeto completo de reglas de un año comercial	
Parámetros	`year: number`	string — año comercial
Retorno	El objeto tal cual está en <code>rules/<año>.json</code> , embebido en <code>rules.generated.mjs</code>	
Excepciones	<code>Error("No hay reglas verificadas para el año comercial <año>. Disponibles: ...")</code>	
Efectos secundarios	Ninguno. Función pura sobre un objeto congelado en el módulo	
La llaman	<code>accounting-engine/index.mjs</code> , <code>tax-equity.mjs</code> , <code>municipal-patent.mjs</code> , <code>workspace.mjs</code> , <code>validate-rules.mjs</code> , la CLI, las vistas <code>empresa</code> y <code>academia</code>	
Llama a	Nada	
Riesgo al modificar	 Crítico. Es la única puerta a las tasas. Hacer que degrade a otro año cuando falta el pedido es explícitamente el peor fallo posible del sistema, y <code>CONTRIBUTING.md</code> lo prohíbe por escrito. Una prueba (<code>tests/rules.test.mjs</code>) verifica que <code>loadRules(1999)</code> lanza	

availableYears()

Devuelve una **copia** de `AVAILABLE_YEARS` (hoy `[2026]`). El `[...AVAILABLE_YEARS]` no es decorativo: impide que un consumidor mute el arreglo del módulo.  **Bajo.**

ruleProvenance(year, ruleName)

Devuelve `{ rule, year, source, lastVerified, note }`. Lanza `Error("Regla desconocida: ...")` si el nombre no existe o no es un objeto. La usa `tests/rules.test.mjs` para exigir que las siete reglas trazables (`iva`, `honorarios`, `ppmProPyme`, `idpcProPyme`, `utm`, `municipalPatent`, `f29`) citen fuente y fecha.  **Medio** — si deja de lanzar, la prueba de trazabilidad se vuelve inútil.

2 · packages/accounting-engine/index.mjs

clp(n)

`Math.round(Number(n || 0))`. El peso chileno no usa decimales, y **todo** monto del sistema pasa por aquí.  **Alto**: cambiar el redondeo cambia todas las cifras del producto a la vez.

saleFromNet(net, year = 2026)

Campo	Valor
Retorno	<code>{ net, iva, total, rate }</code>
Excepciones	<code>Error("net debe ser un número >= 0")</code> vía <code>assertNonNegative</code>
Efectos	Ninguno
La llaman	<code>f29Basic</code> , <code>journal</code> , CLI (<code>venta</code>), pruebas
Llama a	<code>loadRules</code> , <code>clp</code>
Riesgo	 Alto — el IVA de todas las ventas derivadas sale de aquí

purchaseFromNet(net, year = 2026)

Igual que la anterior más `{ kind: 'purchase', note }`. La `note` es contenido, no adorno: recuerda que el IVA sólo es crédito fiscal si cumple los requisitos legales y está bien respaldado.  **Alto**.

honorariumFromGross(gross, year = 2026)

Retorna `{ gross, retention, liquid, rate }` aplicando `r.honorarios.retentionRate` (15,25 % en 2026).  **Alto** — la retención es dinero que la empresa debe enterar.

ppmFromSalesNet(salesNet, year = 2026, rate)

`rate` opcional; si falta, usa `r.ppmProPyme.initialYearRate`. Retorna `{ base, rate, ppm }`.  **Medio**.

f29Basic(input, year = 2026) **Crítico**

La función más consultada del motor.

```
f29Basic({
  salesNet = 0, purchasesNet = 0, previousVatCredit = 0, honorariaGross = 0,
  debitVat, creditVat, ppmRate
}, year = 2026)
```

Campo	Valor
Propósito	Producir el borrador de control del F29 de un período
Dos modos	Derivado: sólo netos → el IVA se calcula con la tasa general. Documentos: si llega <code>debitVat</code> o <code>creditVat</code> , se usan los montos reales
Retorno	<code>{ origin, debitVat, currentCreditVat, previousVatCredit, availableCreditVat, vatPayable, nextVatCredit, ppm, honorariaWithholding, estimatedF29Payment, limitations[] }</code>
Excepciones	<code>assertNonNegative</code> sobre los seis montos
Efectos	Ninguno
La llaman	Vistas <code>panel</code> e <code>impuestos</code> , <code>simulateScenario</code> , CLI (<code>f29</code> , <code>resumen</code>)
Llama a	<code>saleFromNet</code> , <code>purchaseFromNet</code> , <code>ppmFromSalesNet</code> , <code>honorariumFromGross</code>

Lo no obvio, y es lo importante: `origin` distingue los dos modos, y el arreglo `limitations`

crece en modo derivado con un aviso extra. La diferencia no es cosmética: una factura exenta, una nota de crédito o un redondeo del emisor hacen que el IVA real no sea $\text{neto} \times 19\%$. Si el motor recalculara sobre los netos cuando ya tiene los montos declarados, le cobraría al usuario un IVA que legalmente no debe.

`vatPayable = max(0, debit - (credit + previousVatCredit))` y `nextVatCredit = max(0, (credit + previousVatCredit) - debit)`. Las dos ramas del mismo `max`, y por eso nunca pueden ser ambas positivas.

Riesgo: ■ **Crítico.** Es la cifra que el usuario compara con la propuesta del SII. Cualquier cambio necesita prueba en `tests/f29.test.mjs`.

`f29DueDates(period, year = 2026)`

Campo	Valor
Parámetros	<code>period: 'YYYY-MM'</code>
Retorno	<code>{ period, general, internetWithPayment, internetWithoutPayment, checkHolidays: true, source }</code> — cada fecha es <code>{ date, shiftedFromWeekend }</code>
Excepciones	<code>Error("period debe tener formato YYYY-MM")</code>
La llaman	Vistas <code>panel</code> y <code>obligaciones</code> , CLI (<code>vencimientos</code>)

Lo no obvio: devuelve **tres** fechas y no una, porque el plazo del SII depende de si se presenta por internet y de si el formulario resulta con pago. Y `checkHolidays: true` viaja siempre: el cálculo desplaza sábados y

domingos pero **no conoce los feriados legales chilenos**. Ese campo es la declaración honesta de la limitación, no una bandera de configuración.

Riesgo:  **Alto** — una fecha mal calculada produce una multa.

`municipalPatent({ capital, rate, utm }, year = 2026)`

Estimación **rápida** sobre una cifra de capital **dada**. Aplica la tasa, respeta el mínimo (1 UTM) y el máximo (8.000 UTM), y lanza si la tasa está fuera del rango legal. **No decide qué capital corresponde usar**: para eso está `calculateMunicipalPatent`. Se conserva porque la CLI y los laboratorios la usan para enseñar el tope mínimo.  **Medio**.

`idpcProPyme({ incomeReceived, expensesPaid, adjustments }, year = 2026)`

`base = max(0, ingresos - gastos + ajustes)`, `estimatedIdpc = base × rate`.  **Medio** — declarado como referencia educativa en la propia regla.

`journal(operation, year = 2026)`  **Alto**

Devuelve el arreglo de asientos explicados de una operación. Soporta **nueve** tipos y **lanza** `Error("Tipo no soportado: ...")` para cualquier otro.

type	Asiento resultante
<code>capital_contribution</code>	Debe Banco / Haber Capital
<code>shareholder_loan</code>	Debe Banco / Haber Cuenta por pagar al accionista
<code>shareholder_loan_repayment</code>	Debe Cuenta por pagar / Haber Banco
<code>sale</code>	Dos líneas: ingreso neto + IVA débito
<code>purchase</code> / <code>expense</code>	Dos líneas: neto + IVA crédito
<code>honorarium</code>	Dos líneas: líquido pagado + retención por enterar
<code>owner_withdrawal</code>	Debe Cuenta accionista / retiros — no es gasto
<code>tax_payment</code>	Debe Impuestos por pagar / Haber Banco

Lo no obvio: que `capital_contribution` y `shareholder_loan` sean tipos distintos es una decisión de producto, no de implementación. Los dos entran por el mismo banco y son opuestos contablemente —uno acredita patrimonio, el otro pasivo exigible— y por eso la aplicación **obliga a elegir la naturaleza del depósito** en vez de suponer que todo lo que pone el dueño es capital.

`simulateScenario(scenario, year = 2026)`

Recorre `scenario.operations`, acumula totales por tipo, genera los asientos y devuelve `{ profile, totals, f29, entries }`. Sólo la usan la CLI (`escenario`) y las pruebas.

Lo no obvio: `totals.capital` y `totals.shareholderLoans` se cuentan por separado **a propósito** — son dos entradas de dinero del mismo dueño con efectos patrimoniales opuestos.  **Medio**.

3 · `packages/accounting-engine/tax-equity.mjs`

`allowsSimplifiedTaxEquity(taxRegime)` ■ Crítico

Devuelve `true` sólo si el régimen coincide con `/pro\s*pyme\s*general/i` o `/14\s*D\s*N?\.\??\s*3/i`, y `false` de forma explícita si el texto contiene «transparente». Ese corte previo importa: sin él, «Pro Pyme Transparente» pasaría el primer patrón.

Riesgo: ■ Crítico. Aplicar la fórmula simplificada a quien no califica da una cifra plausible y equivocada, que es el peor resultado posible. `tests/tax-equity.test.mjs` lo cubre.

`calculateTaxEquity(input)` ■ Crítico

```
calculateTaxEquity({
  fiscalYear, taxRegime = '', assets, liabilities, nonEffectiveValues = 0,
  equityMovements = {}, taxAdjustments = {}, openingTaxEquity = null,
  operations = {}, method, rulesYear
})
```

Campo	Valor
Retorno	Objeto con <code>calculatedCPT</code> , <code>breakdown[]</code> , <code>calculationMethod</code> , <code>formula</code> , <code>legalBasis</code> , <code>rulesYear</code> , <code>status: 'ESTIMADO'</code> , <code>assumptions[]</code> , <code>warnings[]</code> , <code>evidence[]</code>
Excepciones	Falta <code>fiscalYear</code> entero · método desconocido · el año sin <code>taxEquity</code> en sus reglas
Efectos	Ninguno
La llaman	<code>workspace.taxEquityFor()</code> , CLI (<code>cpt</code>)
Llama a	<code>loadRules</code> , <code>allowsSimplifiedTaxEquity</code> , y las privadas <code>general()</code> / <code>simplified()</code>

Las dos reglas de diseño, citadas del propio módulo: (1) *nunca devuelve sólo un número* —un CPT sin explicación no sirve ni para declarar ni para discutirlo con un contador—; (2) *nunca aplica la fórmula simplificada a quien no califica*, y si alguien la fuerza con `method`, añade una advertencia explícita en `warnings`.

Riesgo: ■ Crítico. El CPT determina la base de la patente municipal del año siguiente, y esa cadena es la lección central del producto.

`general({...})` — privada, método art. 41 N.º 1

`CPT = activos - valoresSinInversiónEfectiva - pasivosExigibles + ajustesPositivos - ajustesNegativos`

Si faltan `assets` o `liabilities`, **no falla**: calcula y añade a `warnings` que la cifra es referencial, y marca `complete: false`. ■ Crítico.

`simplified({...})` — privada, método art. 14 D) N.º 3 (j)

```
bruto = capitalAportado + basesImponibles + participaciones + ajustesPositivos  
      - disminuciones - pérdidas - art21Pagado - retiros - ajustesNegativos  
calculado = max(0, bruto)
```

Lo no obvio y jurídicamente relevante: cuando `bruto < 0`, la letra (j) manda considerar \$0. El módulo lo aplica, expone `rawCPT` con el negativo real, marca `flooredAtZero: true` y añade un `warning` con la cifra bruta. Perder cualquiera de esas tres señales convertiría un dato en una afirmación falsa. ■ **Crítico.**

4 · packages/accounting-engine/municipal-patent.mjs

calculateMunicipalPatent(input) Crítico

Campo	Valor	
Parámetro decisivo	`businessStage: 'NEW_BUSINESS' \	'ESTABLISHED_BUSINESS'
Retorno	{ period, businessStage, baseCapital, baseOrigin, rate, rateStatus, rateSource, legalRateRange, rawPatent, minimumPatent, maximumPatent, annualPatent, semesterAmount, cappedBy, utm, utmPeriod, municipality, legalBasis, source, rulesYear, status, breakdown[], assumptions[], warnings[] }	
Excepciones	businessStage inválido · year no entero · tasa no numérica · tasa fuera del rango legal · UTM ≤ 0 · el año no trae ninguna UTM verificada	
La llaman	workspace.municipalPatentFor(), vista capital, CLI (patente-municipal)	
Llama a	loadRules, resolveRate, resolveUtm	

La bifurcación del art. 24, que es la razón de existir del módulo:

```
NEW_BUSINESS      → base = initialOwnCapital (capital propio inicial DECLARADO)
ESTABLISHED_BUSINESS → base = taxEquity (CPT del balance al 31-12 anterior)
```

Después: `base - deducciones`, con piso en cero; y si hay `allocatedCapital` (prorrateso entre sucursales del art. 25), ése **sustituye** el resultado anterior. Luego `annualPatent = min(máximo, max(mínimo, base × tasa))`, con `cappedBy` indicando cuál tope actuó.

Riesgo:  **Crítico.** Antes de este módulo la patente se estimaba con `capital enterado × tasa mínima` siempre, para cualquier año; eso sólo se parece a la ley el primer ejercicio, y ni siquiera del todo.

resolveRate({...}) — privada Alto

Precedencia: `municipalRate` explícita → tasa de una municipalidad con `status: 'VERIFIED'` → **el mínimo legal como supuesto declarado.**

Lo no obvio: una tasa sólo cuenta como verificada si viene de la municipalidad **y** no fue sobrescrita a mano. Incluso una tasa explícita dentro del rango legal genera `UNVERIFIED_RATE_WARNING`. El sistema prefiere sobreavisar a dar por acreditado algo que no lo está.

resolveUtm({...}) — privada Alto

La UTM cambia todos los meses y los topes de la patente están en UTM, así que arrastrar la de otro período cambia la cifra **en silencio**. La función siempre declara cuál usó, y añade un `warning` distinto según el caso:

pidieron un mes que no existe, o el mes elegido no pertenece al período.

5 · packages/company-operations/workspace.mjs — clase

CompanyWorkspace

`constructor({ store, mode = 'real' })` . Lanza si `mode` no es `real` ni `sandbox` , o si falta `store` . **No crea el almacén**: lo recibe. Ésa es la razón de que el mismo motor corra en cuatro sitios.

Auditoría

Método	Retorno	Efectos	Riesgo
<code>audit(action, detail)</code>	La fila creada	<code>store.append(KEY.audit, {id, at, mode, action, detail})</code>	 Alto — la llaman los 17 puntos de mutación
<code>listAudit()</code>	Todas las filas	Ninguno	 Bajo

Lo no obvio: no existe ninguna operación de borrado sobre `KEY.audit` , y es deliberado. Si un registro pudiera borrarse sin dejar huella, la bitácora no serviría como evidencia de nada.

Ficha, capital y municipalidad

Método	Propósito	Excepciones	Riesgo
<code>getCompany()</code> / <code>saveCompany(c)</code>	Ficha de empresa; añade <code>updatedAt</code> , <code>createdAt</code> y <code>mode</code>	—	 Medio
<code>getCapitalProfile()</code>	Ficha de capital normalizada, migrando el modelo antiguo en lectura	—	 Alto
<code>saveCapitalProfile(p)</code>	Valida y guarda; sincroniza <code>company.capital</code> con <code>capitalEnterado</code>	Los <code>errors</code> de <code>validateCapitalProfile</code> unidos con espacio	 Alto
<code>listEquityMovements()</code>	El ledger explícito	—	 Bajo
<code>addEquityMovement(m, {registerCashMovement})</code>	Añade y ordena por fecha; opcionalmente crea la operación de caja enlazada	Las de <code>normalizeEquityMovement</code>	 Crítico
<code>deleteEquityMovement(id)</code>	<code>boolean</code>	—	 Alto
<code>effectiveEquityMovements()</code>	Une el ledger explícito con las operaciones de caja antiguas no enlazadas	—	 Crítico
<code>capitalPosition({until})</code>	Las cuatro cifras de capital por separado, nunca fundidas	—	 Crítico
<code>getMunicipalProfile()</code> / <code>saveMunicipalProfile(p)</code>	Ficha municipal normalizada	—	 Medio

El riesgo crítico de `effectiveEquityMovements()`, explicado: une dos orígenes sin duplicar. El ledger explícito manda; las operaciones de caja antiguas se incorporan como movimientos derivados

salvo las que ya están enlazadas por `equityMovementId`. Romper ese filtro contaría cada aporte dos veces al derivar el capital enterado, y el capital enterado alimenta el CPT, que alimenta la patente.

El riesgo crítico de `capitalPosition()`: el capital enterado real es el **mayor** entre lo declarado en la ficha y lo que suman los movimientos. Quedarse con el declarado subestimaría la patente cuando el usuario registró aportes posteriores.

Ejercicio anual

Método	Propósito	Excepciones	Riesgo
<code>periodsOfYear(year)</code>	Períodos <code>YYYY-MM</code> con movimiento de ese año	—	 Bajo
<code>yearSummary(year)</code>	Agregado del año, sumando sus períodos	—	 Alto
<code>estimatedBalance(year)</code>	Balance estimado al cierre, con <code>estimated: true</code> y <code>limitations[]</code>	—	 Alto
<code>taxEquityFor(year, overrides)</code>	CPT del año; <code>overrides</code> reemplaza lo propuesto por lo declarado	Las de <code>calculateTaxEquity</code>	 Crítico
<code>businessStageFor(year)</code>	<code>NEW_BUSINESS</code> o <code>ESTABLISHED_BUSINESS</code>	—	 Crítico
<code>municipalPatentFor(year, overrides)</code>	Patente del período con toda su trazabilidad	Las de <code>calculateMunicipalPatent</code>	 Crítico
<code>listAnnualCloses()</code> / <code>getAnnualClose(year)</code>	Cierres guardados	—	 Bajo
<code>closeFiscalYear(year, {...})</code>	Fotografía inmutable del ejercicio	Año no entero · ejercicio ya cerrado	 Crítico
<code>reopenFiscalYear(year, reason)</code>	Elimina el cierre dejando rastro	Ejercicio no cerrado · motivo vacío	 Alto
<code>capitalHistory()</code>	Historial por año, sin recalculer lo cerrado	—	 Alto

`businessStageFor(year)` , con el detalle que decide la cifra:

```
Number(String(start).slice(0, 4)) >= Number(year) ? 'NEW_BUSINESS' : 'ESTABLISHED_BUSINESS'
```

donde `start` es `fechaInicioActividades || fechaConstitucion` . Sin ninguna de las dos, devuelve `NEW_BUSINESS` . Es empresa nueva **en el año en que inicia actividades**; desde el siguiente, en funcionamiento.

`closeFiscalYear()` — **lo no obvio y valioso**: el snapshot guarda `legalRulesVersion` , una copia de la **versión de las reglas** con que se calculó, no una referencia al archivo. Dentro de tres años `rules/2026.json` puede haberse corregido, y el cierre tiene que seguir explicando con qué normas se calculó ese día. El objeto se devuelve con `Object.freeze()` , y el cierre incluye `municipalPatentBaseForNextPeriod` , que es lo que conecta un ejercicio con el siguiente.

`capitalHistory()` — **lo no obvio**: si falta el archivo de reglas del año pedido, no se rinde. Vuelve a intentar con el último año verificado **anterior** y marca el resultado con `simulatedWithRulesYear` . La distinción es exacta: enseñar el mecanismo sí, fingir que la cifra es la del período no.

Constitución

Método	Propósito	Excepciones	Riesgo
<code>listFormationSteps()</code>	Los 9 pasos del catálogo, fusionados con lo guardado	—	 Medio
<code>updateFormationStep(step)</code>	Actualiza uno	Paso desconocido · <code>status</code> inválido · <code>done</code> sin <code>evidenceRef</code>	 Alto
<code>formationProgress()</code>	<code>{ total, done, percent, ready }</code>	—	 Bajo

Lo no obvio en `listFormationSteps()` : el catálogo vive en el **código**, no en los datos. Cuando se agrega un trámite nuevo, las empresas ya creadas lo ven aparecer en vez de quedarse con una lista congelada del día en que se instalaron. Una prueba lo verifica.

Operaciones

Método	Excepciones	Riesgo
<code>listTransactions()</code>	—	 Bajo
<code>addTransaction(tx)</code>	<code>kind</code> no soportado · <code>date</code> mal formada · período cerrado	 Alto
<code>updateTransaction(id, patch)</code>	No encontrada · período de origen cerrado · período de destino cerrado	 Alto
<code>deleteTransaction(id)</code>	Período cerrado	 Alto

Lo no obvio en `updateTransaction` : comprueba el cierre **dos veces** — el período de la operación original y el de la operación resultante. Sin la segunda, se podría *mover* una operación a un mes ya cerrado, que es la misma violación por la puerta de atrás.

Campos con valor por defecto que conviene conocer: `deductible: tx.deductible !== false` y `vatCreditEligible: tx.vatCreditEligible !== false` . Es decir, **por omisión todo es deducible y toda da crédito**; hay que marcar explícitamente lo contrario.

Obligaciones y períodos

Método	Excepciones	Riesgo
<code>upsertObligation(ob)</code>	Sin <code>type</code> · <code>done</code> sin <code>evidenceRef</code>	 Alto
<code>deleteObligation(id)</code>	—	 Medio
<code>listPeriods()</code>	—	 Bajo
<code>periodSummary(period)</code>	—	 Alto
<code>vatCarryForwardInto(period)</code>	—	 Crítico
<code>isPeriodClosed(period)</code> / <code>listClosedPeriods()</code> / <code>listPeriodCloses()</code>	—	 Bajo
<code>closePeriod(period, checklist)</code>	Formato inválido · ya cerrado	 Alto
<code>reopenPeriod(period, reason)</code>	No cerrado · motivo vacío	 Alto

`periodSummary()` — **lo no obvio**: el crédito fiscal suma **sólo** las filas con `vatCreditEligible`; las demás se acumulan aparte en `rejectedVat`. Y `deductibleExpenses` suma

sólo las filas con `deductible`. Son dos casillas independientes, y esa independencia es exactamente la lección de la Academia: un almuerzo con cliente sale plata igual, pero su IVA no es recuperable y su neto no es gasto deducible.

`vatCarryForwardInto(period)` —  **Crítico y el más fácil de romper**:

```
let carry = 0;
for (const p of this.listPeriods()) {
  if (p >= period) break;
  const s = this.periodSummary(p);
  carry = Math.max(0, (s.creditVat + carry) - s.debitVat);
}
```

Recorre **toda** la historia desde el principio en cada llamada. Sin este arrastre la app le cobraría al usuario un IVA que legalmente no debe. **No aplica reajuste** (art. 27 del D.L. 825), y eso está declarado como limitación en el código y en `ARCHITECTURE.md`. Coste: es $O(n^2)$ sobre el número de períodos — irrelevante con decenas de meses, comentado en [15 · Riesgos](#).

Respaldo y salud

Método	Retorno	Excepciones	Riesgo
<code>exportAll()</code>	Objeto plano con <code>format: 'empresa-operativa-chile/backup'</code> y <code>formatVersion: 2</code>	—	 Alto
<code>importAll(payload, {replace})</code>	<code>{ imported, replace, transactions }</code>	Formato ajeno · <code>formatVersion</code> fuera de <code>[1, 2]</code>	 Crítico
<code>backup()</code>	<code>{ name, location }</code>	—	 Medio
<code>listBackups()</code>	Nombres	—	 Bajo
<code>healthCheck(period)</code>	<code>{ period, level, issues[], formation, summary }</code>	—	 Medio

`importAll()` — **dos cosas no obvias**. Primera: acepta `formatVersion 1 y 2`. Los respaldos v1 se importan igual; los campos nuevos simplemente no vienen y quedan vacíos. Romper los respaldos que la gente ya tiene guardados sería peor que cualquier ventaja de limpiar el formato. Segunda: en modo

fusión, un ejercicio ya cerrado **no se pisa** desde un respaldo. Si se pudiera, el cierre dejaría de ser inmutable por la puerta de atrás. Hay una prueba dedicada a eso.

`healthCheck()` — **la decisión de producto**: deliberadamente **no** dice «todo en orden» cuando faltan evidencias. Emite nueve familias de observación (`company.missing` , `company.rut` , `formation.pending` , `evidence.missing` , `vat.rejected` , `obligations.overdue` , `capital.incomplete` , `capital.pending` , `municipal.missing` / `municipal.rate` , `annual-close.missing`) y el `level` resultante es el peor de todos.

`seedSandboxWorkspace(ws)` — función exportada aparte

Campo	Valor
Excepciones	<code>Error("El sandbox sólo puede sembrarse en modo sandbox")</code>
Idempotencia	Si ya hay transacciones, devuelve <code>ws</code> sin tocar nada
Riesgo	 Alto — sembrar sobre EMPRESA REAL sería una contaminación de datos irreversible. Hay una prueba dedicada

Siembra un ejercicio completo de julio a diciembre de 2026: capital social \$3.000.000 con sólo \$1.000.000 enterado al partir, un notebook aportado en especie, un préstamo del accionista, un retiro y 18 operaciones. Las cifras son distintas **a propósito**: es la situación real más frecuente y la que el modelo anterior, con un solo campo `capital` , no podía representar.

6 • packages/company-operations/capital.mjs

Función	Propósito	Excepciones	Riesgo
<code>equityMovementKind(id)</code>	Metadatos de uno de los 9 tipos, o <code>null</code>	—	 Bajo
<code>normalizeCapitalProfile(company)</code>	Normaliza y migra el modelo antiguo de un solo campo <code>capital</code>	—	 Crítico
<code>capitalPendingToPay(profile)</code>	<code>max(0, suscrito - enterado)</code> o <code>null</code>	—	 Alto
<code>validateCapitalProfile(profile)</code>	<code>{ valid, errors[], warnings[] }</code>	—	 Alto
<code>normalizeEquityMovement(movement)</code>	Valida y normaliza antes de guardar	Tipo no soportado · sin fecha <code>YYYY-MM-DD</code> · monto ≤ 0 · aporte en bienes sin descripción o sin aportante	 Alto
<code>summarizeEquityMovements(movements, {until})</code>	Agrega hasta una fecha de corte	—	 Crítico

`normalizeCapitalProfile()` — la migración que no puede corromper nada. Si la ficha guardada sólo tenía `capital`, esa cifra pasa a `capitalEnterado` —que es lo que el campo rotulaba— y `capitalSocial` y `capitalSuscrito` quedan marcados en `pendingConfirmation`. **No se inventan.** Y la migración ocurre **en lectura**: no toca el almacén, así que si nadie vuelve a guardar, el dato original sigue intacto en disco. Instalar esta versión no puede corromper datos existentes.

`capitalPendingToPay()` devuelve `null` y no cero cuando el capital suscrito no se conoce. Cero significaría «no falta nada», y eso es una afirmación que aquí nadie puede hacer.  **Alto**: quien consuma esta función tiene que distinguir `null` de `0`.

`summarizeEquityMovements()` — lo no obvio: `capitalEnteradoPorMovimientos` sólo suma los tipos con `entersCapital: true` (4 de los 9). Un préstamo del accionista entra por el mismo banco y **no suma un peso** ahí.

7 · packages/company-operations/store.mjs y node-store.mjs

createMemoryStore() Bajo

Todo pasa por `JSON.parse(JSON.stringify(...))`. El clonado no es paranoia: sin él, un test que mutara el objeto devuelto contaminaría el almacén y el siguiente test.

createWebStore({ namespace, storage = globalThis.localStorage }) Alto

Lanza si falta `storage` o `namespace`. Toda clave se prefija `${namespace}:${key}`. Ese prefijo es la **separación EMPRESA REAL / SANDBOX**: no una bandera, un espacio de nombres. Una prueba verifica el aislamiento sobre el mismo origen.

`readRaw` absorbe el error de `JSON.parse` y devuelve el `fallback`. **INFERENCIA**: un valor corrupto en `localStorage` se lee como «vacío» en vez de romper la app — bueno para la robustez, pero también significa que una corrupción **no avisa**. Registrado en [15 · Riesgos](#).

createNodeStore({ rootDir, mode }) Alto

Aspecto	Detalle
Directorio	<code>path.join(path.resolve(rootDir), mode)</code> — la separación por modo es física
Subdirectorios	Crea <code>evidence/</code> y <code>backups/</code> al construirse
<code>write</code>	<code>writeJsonAtomic</code> : escribe a <code>\${file}.\${pid}.tmp</code> y hace <code>rename</code> . Un corte de luz deja intacto el archivo anterior
<code>append</code>	<code>fs.appendFileSync</code> de una línea NDJSON. Nunca reescribe el archivo completo
<code>readAll</code>	Si una línea no parsea, devuelve <code>{ corrupt: true, line }</code> en vez de perderla
<code>saveSnapshot</code>	Copia <code>snapshot.json</code> más los siete archivos individuales

Lo no obvio: **FILES** mapea siete claves a nombres de archivo; las demás (`equity-movements`, `annual-closes`, `municipal-profile`) caen al `fallback ${key}.json`. Consecuencia real: `saveSnapshot` copia sólo las siete de **FILES**, así que un respaldo de archivos individuales **no incluye** los movimientos patrimoniales ni los cierres anuales — aunque `snapshot.json`, que se escribe desde `exportAll()`, sí los lleva completos. Registrado en [15 · Riesgos](#).

8 · packages/company-operations/rut.mjs

Función	Retorno	Excepciones	Riesgo
<code>cleanRut(input)</code>	Texto sin puntos ni espacios, en mayúsculas	—	 Bajo
<code>rutCheckDigit(body)</code>	<code>'0' ... '9'</code> o <code>'K'</code>	<code>Error("El cuerpo del RUT debe tener dígitos")</code>	 Alto
<code>validateRut(input)</code>	<code>{valid: true, formatted, body, dv}</code> o <code>{valid: false, reason}</code>	—	 Alto

Módulo 11 con factor cíclico 2→7. `rest === 11 → '0'`, `rest === 10 → 'K'`. Acepta cuerpos de 7 u 8 dígitos con o sin guion. **Riesgo:** un RUT mal validado no falla el día que se escribe; se propaga a documentos y conciliaciones y reaparece meses después como un descuadre cuyo origen ya nadie recuerda.

9 • packages/shortcuts/index.mjs

`resolveShortcut(event, { typing = false })`  **Medio**

Devuelve el atajo coincidente o `null`. Trata `ctrlKey` y `metaKey` como el mismo modificador (compatibilidad con macOS). El caso `digits` resuelve `Alt + 1...9` devolviendo `{...s, index}`.

La regla que evita el problema clásico de los atajos: cuando `typing` es `true`, sólo pasan los que llevan `alt` o `ctrl`, y `Escape`, que es la salida de emergencia de cualquier diálogo. Escribir «no» en una descripción de gasto no abre nada. Hay una prueba dedicada.

`shortcutsOf`, `shortcutsByGroup`, `keysFor` —  **Bajo.**

10 · `packages/glossary/` y `packages/onboarding/`

Función	Propósito	Riesgo
<code>term(id)</code>	Un término o <code>null</code>	 Bajo
<code>searchTerms(query)</code>	Búsqueda sin acentos ni mayúsculas sobre término, resumen, definición e id	 Bajo
<code>termsByCategory()</code>	Agrupados en el orden de <code>CATEGORIES</code> , saltando las vacías	 Bajo
<code>glossary.danglingReferences()</code>	Ids de <code>related</code> / <code>notToConfuseWith</code> que no existen	 Medio
<code>stage(id)</code> / <code>stagesByPhase()</code>	Etapas de la ruta	 Bajo
<code>onboarding.danglingReferences({views, glossaryIds, formationSteps})</code>	Referencias rotas a fases, vistas, trámites o términos	 Medio

Las dos funciones `danglingReferences()` son la razón de que el glosario y la guía no puedan quedar con enlaces rotos: `build-glossary.mjs` y `build-guide.mjs` **abortan** si devuelven algo, y las pruebas las ejecutan también.  **Medio** — quitarlas no rompe nada visible hoy y deja entrar enlaces rotos mañana.

11 · Interfaz — apps/web/src/lib/

dom.js

Función	Riesgo	Nota
<code>html</code> (plantilla etiquetada)	 Crítico	Escapa toda interpolación por defecto. Los datos los escribe el usuario —descripciones, RUT, motivos de reapertura— y se muestran en tablas y en la bitácora
<code>raw(value)</code>	 Crítico	La única puerta para insertar HTML sin escapar. Cada <code>raw()</code> es una decisión consciente
<code>esc(value)</code>	 Alto	Escapa <code>& < > " ' </code>
<code>attempt(fn, okMessage)</code>	 Medio	Traduce cualquier excepción a un <code>toast</code> . El mensaje del motor es la explicación de la regla
<code>openModal({...})</code>	 Medio	Cierra con Escape y con el fondo; devuelve el <code><form></code> como <code>FormData</code>
<code>confirmAction({...})</code>	 Medio	Confirmación explícita para lo irreversible
<code>fmtCLP</code> , <code>fmtNumber</code> , <code>fmtPercent</code> , <code>fmtPeriod</code> , <code>fmtDate</code> , <code>fmtDateTime</code>	 Bajo	<code>fmtDate</code> parte la cadena ISO en vez de construir un <code>Date</code> , para no depender de la zona horaria del equipo

state.js

Función	Riesgo	Nota
<code>ws(mode = state.mode)</code>	 Alto	Devuelve el <code>CompanyWorkspace</code> del modo, creándolo perezosamente. Los dos modos viven a la vez , con almacenes distintos
<code>ensureSandboxSeeded()</code>	 Alto	Sólo siembra el sandbox. La empresa real nunca se siembra
<code>setMode</code> / <code>setTheme</code> / <code>toggleTheme</code>	 Bajo	Persisten en <code>localStorage</code> bajo la clave <code>...:prefs</code>
<code>selectablePeriods()</code>	 Bajo	Períodos con movimiento + el actual + el seleccionado, del más nuevo al más viejo

platform.js

Función	Riesgo	Nota
<code>PLATFORM</code>	 Medio	<code>'windows'</code> si existe <code>__TAURI__.core</code> ; <code>'android'</code> si <code>Capacitor.isNativePlatform()</code> ; si no, <code>'web'</code>
<code>createPlatformStore(mode)</code>	 Alto	Envuelve <code>createWebStore</code> y añade el espejo. Best effort : si el disco falla, la app sigue con <code>localStorage</code> y avisa por consola
<code>hydrateFromDisk()</code>	 Crítico	Sólo repone lo que falta : si la WebView ya tiene el dato, gana el dato vivo. Sin esa condición, un espejo viejo pisaría una sesión en curso
<code>saveTextFile(filename, contents, mime)</code>	 Medio	En Windows llama al comando Rust; en web y Android crea un <code>Blob</code> y dispara la descarga
<code>pickTextFile(accept)</code>	 Bajo	<code><input type="file"></code> efímero
<code>openExternal(url)</code>	 Medio	En Android usa el plugin Browser de Capacitor para no dejar la app atrapada en la WebView

El espejo, en detalle: `mirror()` es un `setTimeout` de **180 ms** que se reinicia en cada escritura (*debounce*). Vuelca **todas** las claves de `localStorage` que empiecen por el *namespace* del modo y las manda como un único JSON a `save_workspace`.

12 · Backend Rust — apps/contador-desktop/src-tauri/src/lib.rs

Función	Firma	Riesgo	Nota
<code>safe_mode(mode)</code>	<code>&str → Result<&'static str, String></code>	 Crítico	Acepta exactamente <code>"real"</code> y <code>"sandbox"</code> . Sin ella, <code>mode</code> acabaría concatenado en una ruta de disco y <code>../algo</code> sería escritura arbitraria. Dos pruebas Rust la cubren
<code>data_dir(app)</code>	<code>→ Result<PathBuf, String></code>	 Medio	<code>app_data_dir()</code> + <code>create_dir_all</code>
<code>workspace_file(app, mode)</code>	<code>→ Result<PathBuf, String></code>	 Alto	<code>data_dir/{safe_mode(mode)}.json</code>
<code>load_workspace(app, mode)</code>	<code>comando · → Result<Option<String>, String></code>	 Medio	<code>None</code> la primera vez
<code>save_workspace(app, mode, payload)</code>	<code>comando · → Result<(), String></code>	 Crítico	Valida que el payload sea JSON antes de tocar el disco , y después escribe a <code>.tmp</code> + <code>rename</code>
<code>export_file(app, filename, contents)</code>	<code>comando · → Result<String, String></code>	 Alto	Toma sólo <code>file_name()</code> del nombre recibido y rechaza vacío o con punto inicial. Escribe en <code>Documentos/Empresa Operativa Chile</code>
<code>app_info(app)</code>	<code>comando · → Result<Value, String></code>	 Bajo	Versión y directorio de datos

Los cuatro comandos están registrados en `invoke_handler`. Las capacidades declaran `permissions: ["core:default"]` y nada más: **el acceso a disco pasa exclusivamente por estos cuatro comandos, que validan sus argumentos.**

13 · Servidor — apps/empresa-operativa/server.mjs

resolveSafe(urlPath) Crítico

```
const full = path.resolve(distDir, `.${path.posix.normalize(rel)}\`);  
if (full !== distDir && !full.startsWith(distDir + path.sep)) return null;
```

Lo no obvio, y es la razón del comentario en el código: la comprobación usa `distDir + path.sep`, no un `startsWith(distDir)` a secas. Sin ese separador, un directorio hermano llamado `dist-privado` pasaría el filtro.

CI comprueba tres vectores de escape (`/../../package.json`, `/%2e%2e%2f...`, `/..%2f...`) y falla si alguno devuelve 200.

Riesgo:  **Crítico.** El servidor sirve datos contables en la máquina de alguien.

06 · Explicación profunda del código

Este documento recorre el sistema **flujo por flujo**, no archivo por archivo. La referencia alfabética de funciones está en [05](#); aquí se explica **cómo se encadenan** y qué decisión contiene cada eslabón.

Se explica línea a línea sólo donde la literalidad aporta —el arrastre del IVA, el piso del CPT, la comprobación de escape de directorio— y por bloques donde no.

Flujo 0 • El arranque

`apps/web/src/app.js`, función `boot()`, cuatro pasos en un orden que importa:

```
setTheme(state.theme);           // 1. tema antes de pintar, para evitar el destello
await hydrateFromDisk();         // 2. sólo hace algo en Windows
ensureSandboxSeeded();          // 3. siembra el sandbox si está vacío
applyUrlState();                 // 4. #vista, ?modo, ?tema, ?periodo
render();                        // 5. shell + vista
```

El paso 2 es el único `await` de todo el arranque y sólo hace algo bajo Tauri. En navegador y en Android, `hydrateFromDisk()` devuelve `{ hydrated: false }` de inmediato. La razón de esperarlo es que si el disco tiene datos que `localStorage` no tiene, hay que reponerlos **antes** de que `ensureSandboxSeeded()` decida si el sandbox está vacío. Invertir 2 y 3 sembraría el sandbox sobre datos que estaban a punto de llegar del disco.

`applyUrlState()` — **por qué existe**. Tres cosas dependen de ello y ninguna es obvia:

- `#impuestos` abre esa vista directamente, y eso es lo que hace funcionar los accesos directos del manifiesto PWA (mantener pulsado el icono en Android);
- permite enlazar a una pantalla concreta desde la documentación;
- `?modo=sandbox` y `?tema=claro` existen para que ****las capturas del manual se generen de forma reproducible****, sin depender del estado guardado. `capture-screenshots.mjs` los usa.

Después, `installShortcuts()` registra **un solo** oyente en `document`. El comentario del archivo explica el porqué: la app se redibuja entera en cada cambio, así que enganchar teclas por vista dejaría oyentes huérfanos en cada render.

Y por último, el service worker se registra sólo si `location.protocol` empieza por `http`. Bajo Tauri el protocolo es `tauri://`, así que no se intenta — y si falla, el `.catch()` vacío lo absorbe: sin service worker la app sigue funcionando, sólo pierde el modo offline del navegador.

Flujo 1 · Registrar una venta, de principio a fin

Es el flujo más recorrido del producto. Siete eslabones.

1. La vista propone el IVA sin pisar lo escrito

`views/operaciones.js` calcula `neto × tasa` mientras el usuario teclea, pero **no sobrescribe** el campo si ya tiene un valor distinto. La razón está en el dominio: una factura exenta o una nota de crédito tienen un IVA que no es el 19 % del neto, y recalcular sobre el neto le cobraría al usuario un IVA que no debe.

2. `addTransaction()` valida en tres puertas

```
if (!KIND_IDS.includes(tx.kind)) throw new Error(`kind no soportado: ${tx.kind}`);
const date = tx.date || new Date().toISOString().slice(0, 10);
if (!/^\\d{4}-\\d{2}-\\d{2}$/.test(date)) throw new Error('date debe tener formato YYYY-MM-DD');
if (this.isPeriodClosed(date.slice(0, 7))) throw new Error('El período ${date.slice(0, 7)} está cerrado');
```

Las tres validaciones están **en el motor**, no en la vista. Consecuencia práctica: la CLI, las pruebas y las tres plataformas obtienen exactamente la misma protección, y la vista no puede saltársela por descuido.

3. Los valores por defecto que definen el comportamiento

```
deductible: tx.deductible !== false,
vatCreditEligible: tx.vatCreditEligible !== false,
```

Escrito así —y no `tx.deductible ?? true`— para que `undefined`, `null` y omisión den `true`, y sólo un `false` **explícito** dé `false`. Por omisión todo es deducible y todo da crédito; hay que marcar lo contrario a mano. Es la elección correcta para el caso común y obliga a una decisión consciente para el caso raro.

4. El enlace que impide el doble conteo

```
equityMovementId: tx.equityMovementId || null,
```

Un campo de una línea con una consecuencia grande. Cuando `addEquityMovement()` crea una operación de caja asociada, la deja enlazada por aquí. `effectiveEquityMovements()` **excluye** las transacciones que tienen ese enlace, para no contar el mismo aporte dos veces al derivar el capital enterado. Y el capital enterado alimenta el CPT, que alimenta la base de la patente del año siguiente.

5. Escritura y bitácora

```
this.store.write(KEY.transactions, rows);
this.audit('transaction.added', { id, kind, total, period: date.slice(0, 7) });
```

Dos operaciones, no una transacción. **INFERENCIA**: si el proceso muere entre ambas, la operación queda guardada sin línea en la bitácora. No hay mecanismo que lo detecte ni lo repare. Registrado en [15 · Riesgos](#).

6. El espejo de Windows, con retardo

`createPlatformStore()` intercepta `write`, `append` y `saveSnapshot`, y programa `mirror()` con un `setTimeout` de 180 ms que se reinicia en cada escritura. Registrar cinco operaciones seguidas produce **una** escritura a disco, no cinco.

El vuelco recorre `localStorage` entero y selecciona las claves del *namespace* del modo:

```
for (let i = 0; i < localStorage.length; i++) {
  const key = localStorage.key(i);
  if (key.startsWith(`${NS(mode)}:`)) dump[key] = localStorage.getItem(key);
}
```

INFERENCIA sobre el coste: el vuelco es **completo**, no incremental. Con un historial grande, cada escritura serializa todo el espacio de trabajo. A la escala del producto —decenas de operaciones por mes— es irrelevante; con varios años de historia empezaría a notarse.

7. Rust escribe de forma atómica

```
serde_json::from_str::<serde_json::Value>(&payload)
  .map_err(|e| format!("payload inválido: {e}"))?;
let tmp = file.with_extension("json.tmp");
fs::write(&tmp, payload)?;
fs::rename(&tmp, &file)?;
```

Dos protecciones en cinco líneas. La validación **antes** de tocar el disco evita pisar un espejo bueno con basura si la WebView manda algo corrupto. El `rename` hace la sustitución atómica: un corte de luz a mitad deja el archivo anterior intacto en vez de un JSON truncado que no abre.

Flujo 2 · El borrador del F29

El arrastre del remanente

La función más importante del flujo tributario mensual, y la más fácil de romper sin darse cuenta:

```
vatCarryForwardInto(period) {
  let carry = 0;
  for (const p of this.listPeriods()) {
    if (p >= period) break;
    const s = this.periodSummary(p);
    const available = s.creditVat + carry;
    carry = Math.max(0, available - s.debitVat);
  }
  return carry;
}
```

Línea a línea, porque aquí la literalidad sí aporta:

- `listPeriods()` devuelve los períodos **ordenados ascendentemente** (usa `.sort()` sobre cadenas `YYYY-MM`, que ordenan bien lexicográficamente). El recorrido va del más antiguo al más nuevo, y

eso es obligatorio: el remanente se acumula hacia adelante.

- `if (p >= period) break;` corta antes del período pedido. Incluirlo devolvería el remanente *saliente*, no el *entrante*.
- `Math.max(0, ...)` impide un remanente negativo. Si el débito supera al crédito disponible, no hay remanente: hay IVA a pagar, que es otra cosa y la calcula `f29Basic`.

- El resultado **no se reajusta**. El art. 27 del D.L. 825 manda reajustar el remanente, y el sistema no lo hace. Está declarado como limitación en el propio comentario del método y en `docs/ARCHITECTURE.md`.

Coste computacional. `periodSummary(p)` recorre **todas** las transacciones filtrando por prefijo, y esto lo llama una vez por período anterior: es $O(\text{períodos} \times \text{transacciones})$. Con 12 meses y 200 operaciones son 2.400 comparaciones — irrelevante. Con 60 meses y 5.000 operaciones serían

300.000 en cada render del panel, que redibuja la vista entera. Registrado en [15 · Riesgos](#).

Los dos modos de `f29Basic`

```
const usesDeclaredVat = debitVat !== undefined || creditVat !== undefined;
const debit = debitVat === undefined ? derivedDebit : clp(debitVat);
const credit = creditVat === undefined ? derivedCredit : clp(creditVat);
```

La comprobación es `!== undefined`, no *truthy*. Un `debitVat: 0` legítimo —un mes con ventas exentas únicamente— se respeta como cero declarado en vez de caer al derivado. Es la clase de detalle que parece trivial y produce una diferencia con el SII cuando se hace mal.

Y el resultado cambia la lista de advertencias, no sólo el número:

```
if (!usesDeclaredVat) {
  limitations.push('IVA derivado de los netos con la tasa general: no refleja operaciones exentas, notas de crédito ni redondeos del emisor.');
```

La vista muestra esas `limitations` al usuario. El sistema no se limita a calcular distinto: **dice** que calculó distinto.

Los tres vencimientos

`f29DueDates()` construye las fechas en **UTC** (`Date.UTC`) y las serializa con `toISOString().slice(0,10)`. Deliberado: usar la hora local haría que la fecha dependiera de la zona horaria del equipo, y un usuario en otra zona vería un día distinto.

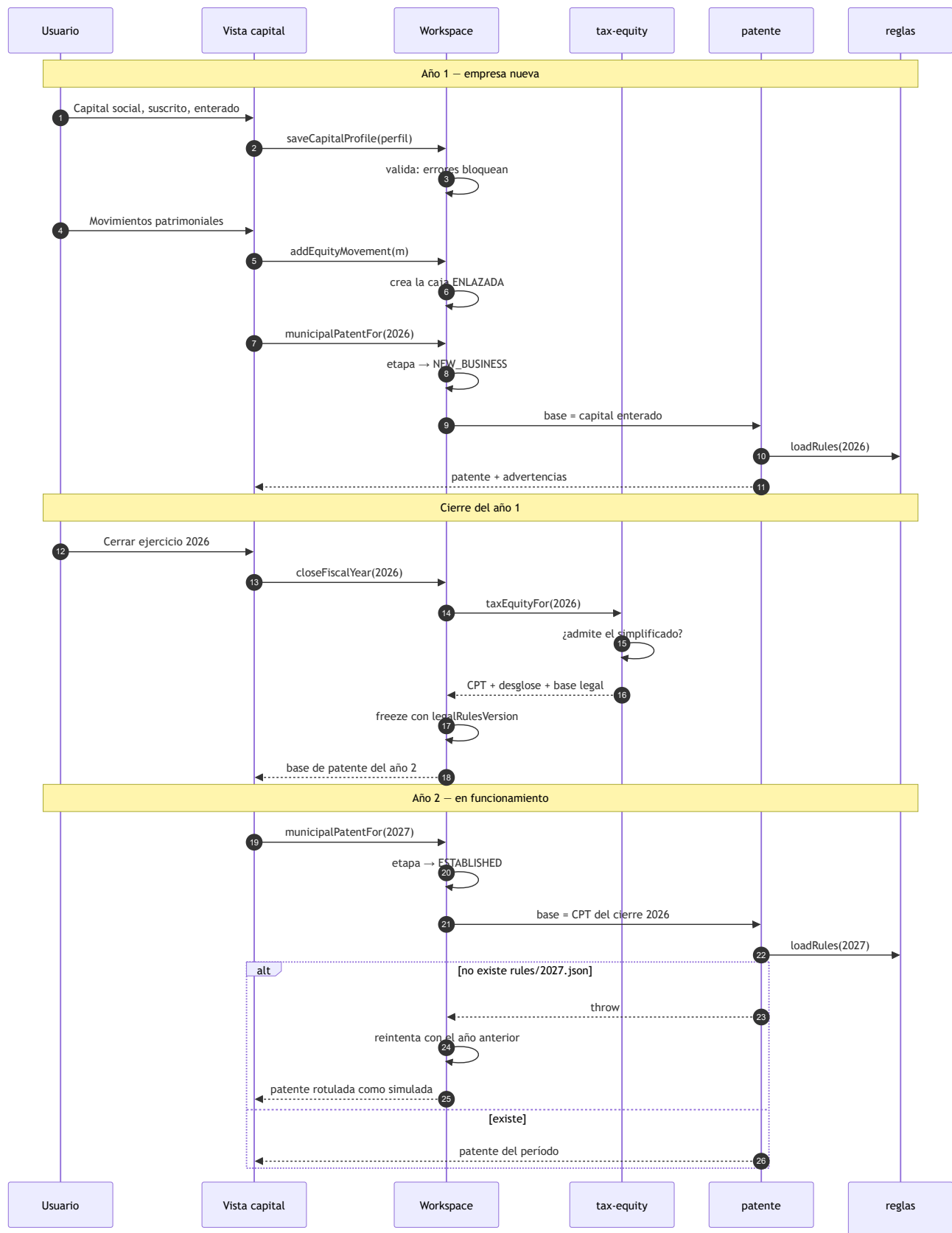
```
while (d.getUTCDay() === 0 || d.getUTCDay() === 6) {
  d.setUTCDate(d.getUTCDate() + 1);
  shifted = true;
}
```

Un `while` y no un `if`, porque el día 12 puede caer sábado y hay que saltar dos días. `shifted` viaja al resultado como `shiftedFromWeekend`, y la vista lo muestra: el usuario ve que la fecha se movió y por qué.

`checkHolidays: true` viaja **siempre**. No es una opción: es la declaración de que los feriados legales chilenos no están modelados.

Flujo 3 · Capital, CPT y patente — la cadena de tres años

Es la lección central del producto y el flujo con más decisiones acumuladas.



Qué muestra la secuencia y qué no. Muestra la cadena completa: el capital enterado del año 1 alimenta la patente del año 1; el cierre del año 1 produce un CPT; ese CPT es la base de la patente del año 2. Muestra

también la degradación honesta cuando faltan las reglas del año pedido.

No muestra que `capitalPosition()` toma el **mayor** entre el capital enterado declarado y el que suman los movimientos, ni que `effectiveEquityMovements()` combina dos orígenes sin duplicar. Ambas cosas están explicadas abajo.

3.1 `capitalPosition()` — por qué el mayor de dos cifras

```
const enteradoDeclarado = money(profile.capitalEnterado);
const enteradoMovimientos = summary.capitalEnteradoPorMovimientos - summary.disminucionesDeCapital;
const capitalEnterado = Math.max(enteradoDeclarado, Math.max(0, enteradoMovimientos));
```

El usuario rellenó la ficha una vez y después registró movimientos durante meses. La ficha se queda corta, y **quedarse corto aquí subestima la patente**, que es un error caro y silencioso. Se toma el mayor. Los dos valores originales siguen expuestos por separado (`capitalEnteradoDeclarado` , `capitalEnteradoPorMovimientos`) para que la vista pueda explicar de dónde salió la cifra.

3.2 `effectiveEquityMovements()` — unir sin duplicar

```
const derived = this.listTransactions()
  .filter(t => legacyKinds[t.kind] && !t.equityMovementId)
  .map(t => ({ id: `tx:${t.id}`, kind: legacyKinds[t.kind], ..., derivedFromTransaction: t.id }));
```

Dos condiciones en el filtro y las dos son necesarias. `legacyKinds[t.kind]` selecciona los cuatro tipos de caja que tienen efecto patrimonial. `!t.equityMovementId` excluye las que **ya** nacieron de un movimiento del ledger. Sin la segunda, cada aporte registrado por la vía nueva se contaría dos veces.

El id derivado lleva prefijo `tx:` para que no pueda colisionar con un id del ledger, y `derivedFromTransaction` conserva la trazabilidad hacia la operación original.

3.3 `calculateTaxEquity()` — la elección de método

```
const simplifiedAllowed = allowsSimplifiedTaxEquity(taxRegime);
const chosen = method ?? (simplifiedAllowed ? 'simplified14D3j' : 'article41');
```

El método se puede forzar con `method` , pero forzarlo **no calla la advertencia**:

```
if (chosen === 'simplified14D3j' && !simplifiedAllowed) {
  warnings.push(`El régimen declarado (${taxRegime || 'sin declarar'}) no corresponde ...`);
}
```

Y `allowsSimplifiedTaxEquity` corta primero por «transparente»:

```
if (/transparente/i.test(value)) return false;
return SIMPLIFIED_REGIMES.some(re => re.test(value));
```

Sin ese corte, «Pro Pyme Transparente (14 D N.º 8)» coincidiría con `/pro\s*pyme\s*general/i` ... no, no coincidiría con «general», pero **sí** con el segundo patrón si alguien escribiera «14 D N.º 3» por error en la ficha.

El corte explícito es la defensa contra el texto libre de un campo de formulario.

3.4 El piso en cero del CPT simplificado

```
const bruto = capitalAportado + bases + participaciones + positive
              - disminuciones - perdidas - art21 - retiros - negative;
const calculated = Math.max(0, bruto);
if (bruto < 0) {
  warnings.push(`El cálculo dio ${bruto.toLocaleString('es-CL')}; la letra (j) manda considerar $0 ...`);
}
```

Tres señales, no una: `calculatedCPT` con el cero legal, `rawCPT` con el negativo real, y `flooredAtZero: true`. Perder cualquiera convertiría un dato en una afirmación falsa: un CPT de \$0 sin más contexto se lee como «esta empresa no tiene capital propio», cuando lo que ocurrió es que la ley truncó una cifra negativa.

Hay una prueba dedicada a que un cero **exacto** no se marque como truncado.

3.5 `closeFiscalYear()` — la fotografía

```
if (this.getAnnualClose(fiscalYear)) {
  throw new Error(`El ejercicio ${fiscalYear} ya está cerrado. Reábrelo indicando el motivo ...`);
}
const rules = loadRulesVersion(fiscalYear);
const close = Object.freeze({ ..., legalRulesVersion: rules });
```

`loadRulesVersion()` no guarda una referencia al archivo de reglas: guarda una **copia** de los valores relevantes —año, `lastVerified`, `schemaVersion`, el bloque completo de patente municipal y el de CPT—. Dentro de tres años `rules/2026.json` puede haberse corregido, y el cierre tiene que seguir explicando con qué normas se calculó ese día.

`Object.freeze()` protege el objeto devuelto en memoria. **No congela lo guardado:** `store.write()` serializa a JSON y el JSON no tiene congelación. La inmutabilidad real la da que ningún método sobrescriba un cierre existente y que `importAll()` en modo fusión no lo pise.

3.6 `capitalHistory()` — la simulación rotulada

```
try {
  patent = this.municipalPatentFor(year);
} catch (error) {
  patentError = error.message;
  const fallback = availableYears().filter(y => y < year).at(-1);
  if (fallback) {
    try { patent = this.municipalPatentFor(year, { rulesYear: fallback }); simulatedWithRulesYear = fallback; }
    catch { /* queda sólo el error, que ya es informativo */ }
  }
}
```

El comentario del código dice exactamente qué se está negociando: «Sin reglas del año no se calcula la patente de ese año: eso no se negocia. Pero sí se puede mostrar qué saldría con las últimas reglas verificadas, rotulado

como simulación. Es la diferencia entre enseñar el mecanismo y fingir que la cifra es la del período.»

El `filter(y => y < year)` sólo mira años **anteriores**. Simular 2028 con reglas de 2026 es razonable; hacerlo con reglas de 2030 no tendría sentido.

Y el `catch` vacío interior no es descuido: si tampoco se puede simular, queda `patentError`, que ya es informativo.

Flujo 4 · Cerrar un período, y por qué eso significa algo

```
closePeriod(period, checklist = {}) {
  if (!/^d{4}-d{2}$/.test(String(period))) throw new Error('period debe tener formato YYYY-MM');
  if (this.isPeriodClosed(period)) throw new Error('El período ya está cerrado');
  const close = { id: uuid(), period, at: isoNow(), summary: this.periodSummary(period),
    carryForwardIn: this.vatCarryForwardInto(period), checklist };
  ...
}
```

El cierre guarda **dos** listas: `periodCloses` con el detalle completo (resumen del período, remanente entrante y la lista de control tal como se declaró) y `closedPeriods` con sólo los identificadores. La segunda existe para que `isPeriodClosed()` sea una comprobación barata que se llama en cada `addTransaction`.

La lista de control se guarda tal cual, con las casillas que el usuario NO marcó. La vista `cierre.js` lo dice: la app deja cerrar aunque falten casillas —no es su trabajo bloquear a alguien que sabe lo que hace— pero deja constancia exacta de qué se declaró revisado y qué no. Meses después, esa constancia es la diferencia entre «creo que lo revisé» y saberlo.

La inmutabilidad se defiende en **tres** métodos, no en uno:

Método	Comprobación
<code>addTransaction</code>	El período de la fecha nueva
<code>updateTransaction</code>	El período de origen y el de destino
<code>deleteTransaction</code>	El período de la operación existente

Falta cualquiera de las cuatro comprobaciones y «cerrar el mes» deja de significar algo.

`reopenPeriod()` existe porque cerrar por error es inevitable, pero exige motivo escrito (`if (!String(reason) || '').trim()`) y deja la razón en la bitácora. La decisión explícita del producto: **la trazabilidad importa más que la inmutabilidad absoluta.**

Flujo 5 · El ciclo de vida de una regla tributaria

build-rules.mjs — de JSON a módulo ESM

```
const entries = years.map(year => {
  const parsed = JSON.parse(fs.readFileSync(path.join(rulesDir, `${year}.json`), 'utf8'));
  if (String(parsed.commercialYear) !== year) throw new Error(`${year}.json declara
commercialYear=${parsed.commercialYear}`);
  if (!parsed.lastVerified) throw new Error(`${year}.json no declara lastVerified`);
  return `  ${year}: ${JSON.stringify(parsed, null, 2).split('\n').join('\n  ')}';
});
```

Dos validaciones que atrapan el error más probable al añadir un año: copiar `2026.json` a `2027.json` y **olvidar cambiar** `commercialYear`. El nombre del archivo y el contenido tienen que coincidir, o el build para.

El `.split('\n').join('\n ')` reindenta el JSON para que el módulo generado se lea como código y no como un volcado. Detalle de presentación con una consecuencia real: hace que un `git diff` sobre `rules.generated.mjs` sea legible.

El modo `--check`

```
if (process.argv.includes('--check')) {
  if (previous !== body) {
    console.error('rules.generated.mjs está desfasado respecto de rules/*.json.');
```

Comparación de **cadenas exactas**, no de objetos. Es más estricto de lo necesario —un cambio de indentación fallaría— y eso es deliberado: el archivo se genera, así que cualquier diferencia significa que alguien lo editó a mano o que el generador cambió.

El comentario del script dice para qué sirve: «*Cl lo ejecuta en ese modo para que nadie pueda editar un JSON y publicar una app que sigue calculando con la tasa anterior.*»

validate-rules.mjs — la red que atrapa el error de dedo

Este script no valida esquema. Valida **valores**:

```
if (r.iva.generalRate !== 0.19) errors.push('IVA 2026 esperado 19%');
if (r.honorarios.retentionRate !== 0.1525) errors.push('Retención honorarios 2026 esperada 15,25%');
if (p.maxUtm !== 8000) errors.push('Máximo legal de patente esperado 8.000 UTM (D.L. 3.063 art. 24)');
```

El comentario sobre el tope de 8.000 UTM registra un error real que existió en este repositorio:

«*el 4.000 que traía este repositorio era el texto anterior a esa reforma*» (Ley 20.280). Es exactamente el fallo que este script existe para que no vuelva.

El bucle final es el más valioso:

```
for (const [name, rule] of Object.entries(r)) {  
  if (!rule || typeof rule !== 'object' || Array.isArray(rule)) continue;  
  if (name === 'taxEquity') continue;  
  if (!rule.source) errors.push(`La regla "${name}" no declara su fuente`);  
  if (!rule.lastVerified) errors.push(`La regla "${name}" no declara su última verificación`);  
}
```

Recorre **todas** las reglas objeto del año, no una lista fija. Una regla nueva que alguien añada mañana queda obligada a citar fuente y fecha sin que nadie tenga que acordarse de actualizar el validador. `taxEquity` se excluye porque su estructura es distinta —las fuentes viven dentro de `methods`, no en el nivel superior— y sus referencias legales se comprueban en las líneas anteriores.

Flujo 6 · El build de la web y el gate `node:*`

`scripts/build-web.mjs`, en orden:

1. `fs.rmSync(distDir, { recursive: true, force: true })` — borra `dist` entero. Un build incremental dejaría archivos de una versión anterior, y esos archivos acabarían dentro del APK.

1. `copyDir(srcDir, distDir)` — copia `apps/web/src` tal cual.

2. **Copia de los 13 módulos de `CORE`** a `dist/core/`, con `throw` si falta alguno.

3. **Manuales:** `EMPEZAR-AQUI.html`, `EMPEZAR-AQUI.pdf` y `MANUAL.html` a `dist/ayuda/`. Si falta uno, el error **dice qué comando lo genera**. El PDF grande del manual (7,7 MB) se queda fuera a propósito: engordar la instalación cuatro veces por un archivo que se descarga una vez no compensa.

1. **Presentación:** tres archivos a `dist/presentacion/`.

2. **El gate:**

```
const offenders = walk(distDir)
  .filter(f => f.endsWith('.mjs') || f.endsWith('.js'))
  .filter(f => /from\s+["']node:/.test(fs.readFileSync(f, 'utf8')));
if (offenders.length) { /* lista los archivos */ process.exit(1); }
```

Se comprueba **aquí, en el build**, y no sólo en un test que quizá nadie corra antes de publicar. El comentario lo dice: «*si eso llega al APK, la pantalla queda en blanco sin un error visible*».

El límite del patrón, verificado: `/from\s+["']node:/` no reconoce `import 'node:fs';` ni `await import('node:fs')`. Registrado en [15 · Riesgos](#).

1. **El `buildId`:**

```
const buildId = crypto.createHash('sha256')
  .update(walk(distDir).sort().map(f => fs.readFileSync(f)).reduce((a, b) => Buffer.concat([a, b]),
    Buffer.alloc(0)))
  .digest('hex').slice(0, 12);
```

El `.sort()` es lo que hace el hash **reproducible**: sin él dependería del orden que devuelva el sistema de archivos, que varía entre plataformas. CI construye dos veces y compara `build-info.json` precisamente para que esa propiedad no se pierda sin que nadie lo note.

1. **Sustitución en el service worker:** `__BUILD_ID__` → el hash. Publicar una versión nueva invalida el caché anterior en vez de dejar a la gente con una app vieja pegada en el navegador.

Flujo 7 · La verificación del APK

`scripts/verify-apk.mjs` no comprueba que el build funcionara: comprueba que el resultado sirva.

Leer el ZIP a mano

Un APK es un ZIP. El script lee el directorio central **sin `unzip` y sin librerías**, para correr igual en Windows, Linux y macOS:

```
const EOCD_SIG = 0x06054b50;
for (let i = buf.length - 22; i >= 0 && i > buf.length - 22 - 65535; i--) {
  if (buf.readUInt32LE(i) === EOCD_SIG) { eocd = i; break; }
}
```

La búsqueda va **hacia atrás** y con un límite de 65.535 bytes porque el «End of Central Directory» está al final, después de un comentario de longitud variable cuyo máximo es precisamente ese.

Después maneja **ZIP64**: si los campos de 32 bits vienen al máximo (`0xffffffff` / `0xffff`), los valores reales están en un registro aparte que hay que localizar por su propia firma. Sin esa rama, un APK grande no se podría verificar.

Las doce comprobaciones

No se comprueba que los archivos existan: se **cuentan**.

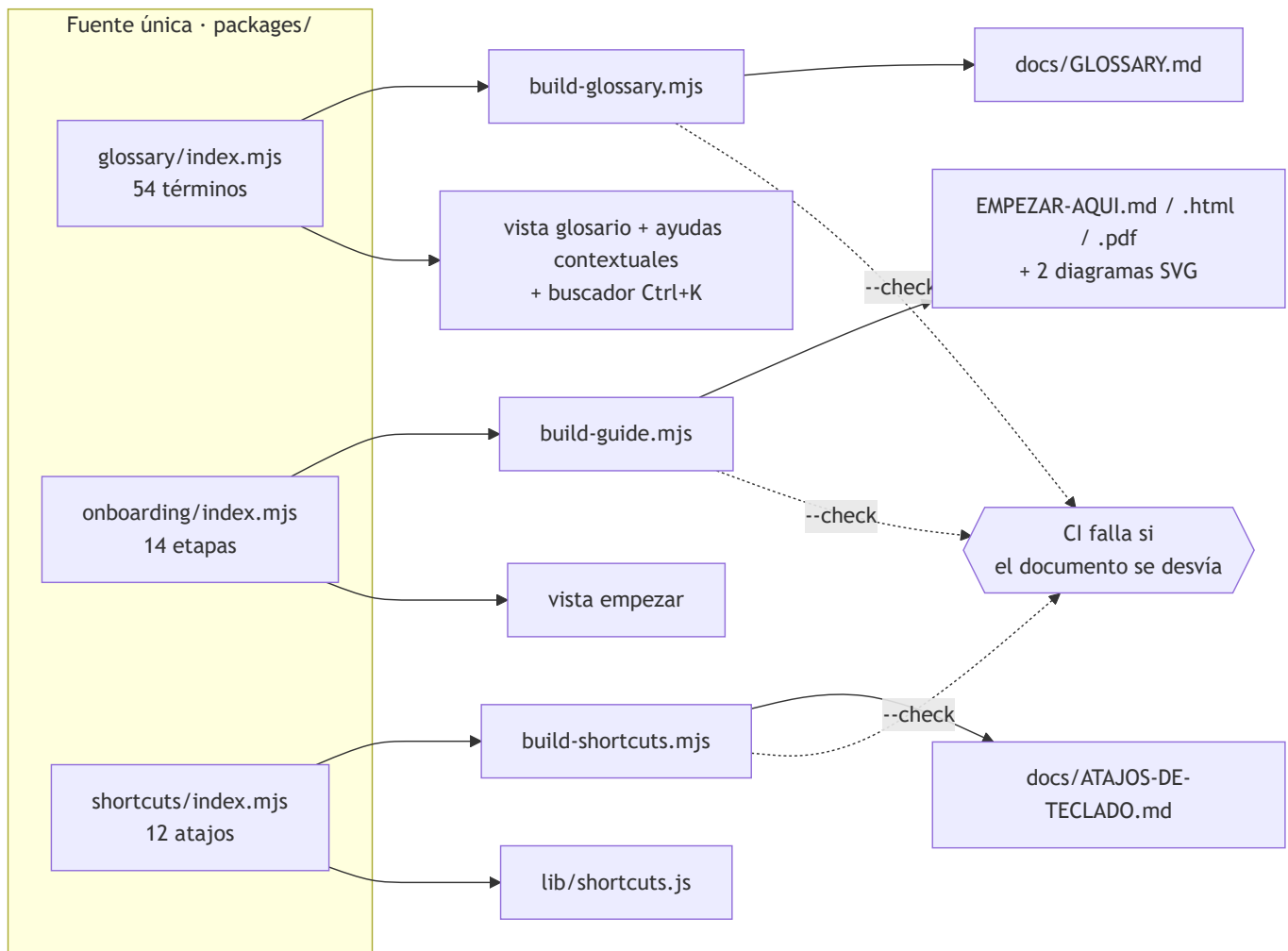
```
[`El APK contiene las 10 vistas (encontradas: ${views})`, views >= 10],
[`El APK contiene el núcleo de cálculo (módulos: ${core})`, core >= 6],
[`Tamaño razonable (${sizeMb.toFixed(1)} MB)`, sizeMb > 1 && sizeMb < 120]
```

Y una última que va más allá del recuento: busca `core/chile-tax-rules/rules.generated.mjs` dentro del ZIP, porque el motor de cálculo tiene que estar de verdad, no sólo los archivos de interfaz.

REQUIERE VALIDACIÓN — el script no se ejecutó en este análisis por no haber APK disponible. La lógica se transcribe del código.

Flujo 8 · Los documentos que se generan desde el código

Tres scripts comparten el mismo patrón, y entenderlo evita el error clásico de editar el documento en vez de la fuente.



Qué muestra el diagrama y qué no. Muestra que cada módulo de contenido tiene **dos o tres** consumidores —la aplicación y el documento— y que CI vigila que no se separen. **No muestra** una asimetría importante: en `build-guide.mjs` sólo el `.md` se compara con `--check`. El `.html` y el `.pdf` se regeneran con `pnpm docs` y **no** en `check`, porque necesitan Chrome y CI no lo tiene garantizado. El propio script lo declara en su cabecera. Consecuencia real: es posible que `EMPEZAR-AQUI.html` —que viaja **dentro** del APK— quede desfasado respecto de `EMPEZAR-AQUI.md` sin que CI lo detecte. Registrado en [15 · Riesgos](#).

El generador de glosario aborta antes de escribir si hay enlaces rotos:

```
const dangling = danglingReferences();
if (dangling.length) { dangling.forEach(d => console.error(` ${d}`)); process.exit(1); }
```

Un `notToConfuseWith` que apunta a un id inexistente produciría un ancla rota en el Markdown, y esa ancla rota viajaría a la vista Glosario y a las ayudas contextuales.

Flujo 9 · La verificación de la presentación

`check-presentation.mjs` es el gate más inusual del repositorio y vale la pena entenderlo, porque es el patrón que esta documentación replicó para sus propios PDF.

Valida tres cosas distintas:

1. **La fuente.** Cuenta las secciones `## N · Título` de `docs/presentacion.md` y suma los minutos declarados en las citas `> Pauta · N min.`.

1. **El artefacto.** Abre los PDF y cuenta objetos `/Type /Page`:

```
const total = (fs.readFileSync(full).toString('latin1').match(/\/Type\s*\/Page[^\s]/g) ?? []).length;
```

El `[^\s]` excluye `/Type /Pages`, que es el nodo del árbol y no una página. Si el PDF no declara ninguna, la generación falló y el archivo quedó inservible — que un archivo exista y pese no prueba nada.

1. **Lo que se anuncia fuera.** Recorre **cinco** archivos (`README.md`, `docs/README.md`, `docs/presentacion.md`, `apps/web/src/views/ayuda.js`, `.github/workflows/release.yml`) buscando `N diapositivas` y `≈N min`, y falla si alguna cifra no coincide con la realidad.

Ese tercer bloque es el que más enseña: «*Nada sujeta las cifras que aparecen en el README, en el índice de docs y en las notas del release salvo esta comprobación: se escriben a mano y envejecen en cuanto alguien añade una lámina.*»

Y ahí está el hallazgo. Ese gate existe para la presentación, y **no existe para el manual, ni para el número de vistas, ni para el número de pruebas**. Es exactamente donde el repositorio tiene hoy cifras desincronizadas. El detalle, con las cinco cifras concretas, está en [15 · Riesgos](#).

Flujo 10 · La interfaz — por qué se redibuja todo

```
export function render(keepScroll = false) {
  const app = document.querySelector('#app');
  app.innerHTML = shell();           // el shell entero
  // ... reenganchar oyentes ...
  renderView(keepScroll);           // la vista entera
}
```

Se reconstruye el DOM completo en cada cambio, y **se vuelven a enganchar todos los oyentes**. A esta escala —decenas de filas por período, no miles— es instantáneo y elimina una clase entera de errores de sincronización entre vista y datos.

Los dos sitios donde sí molesta son los campos de búsqueda, y se resuelven a mano:

```
input?.addEventListener('input', e => {
  filters.query = e.target.value;
  const caret = e.target.selectionStart;
  rerender();
  const next = document.querySelector('[data-q]');
  next?.focus();
  next?.setSelectionRange(caret, caret);
});
```

Guardar la posición del cursor, redibujar, recuperar el foco y restaurar el cursor. Sin esas cuatro líneas, escribir en el buscador de la bitácora perdería el foco en cada tecla.

El otro detalle: `render(true)` conserva `globalThis.scrolling`, para que guardar una fila a mitad de tabla no salte al principio de la página.

El escapado por defecto

```
export const html = (strings, ...values) =>
  strings.reduce((out, chunk, i) => {
    ...
    else if (v && v.__raw !== undefined) rendered = v.__raw;
    else rendered = esc(v);
    return out + rendered + chunk;
  });
```

Toda interpolación se escapa salvo que venga envuelta en `raw()`. Es la decisión correcta para esta aplicación: los datos los escribe el usuario —descripciones de gastos, RUT de proveedores, motivos de reapertura— y después se muestran en tablas y en la bitácora. Una descripción con `<` rompería la vista, o algo peor.

El `raw()` explícito convierte cada excepción en una decisión visible en el código. Hay 169 usos de `raw()` en `apps/web/src`, y prácticamente todos envuelven marcado construido por la propia aplicación, no texto del usuario.

Flujo 11 · El servidor local

```
function resolveSafe(urlPath) {
  const decoded = decodeURIComponent(urlPath.split('?')[0]);
  const rel = decoded === '/' ? '/index.html' : decoded;
  const full = path.resolve(distDir, `.${path.posix.normalize(rel)}`);
  if (full !== distDir && !full.startsWith(distDir + path.sep)) return null;
  return full;
}
```

Cuatro pasos, y cada uno cierra un vector:

1. `decodeURIComponent` — sin esto, `%2e%2e%2f` llegaría sin decodificar y `normalize` no lo vería como `../`.

1. `path.posix.normalize` — resuelve los `..` **antes** de tocar el sistema de archivos.
2. El `.` inicial en ``.${rel}`` **fuerza que la ruta se interprete como relativa a** `distDir` aunque empiece por `/`.

1. `full.startsWith(distDir + path.sep)` — **con el separador**. El comentario del código explica por qué: sin él, un directorio hermano llamado `dist-privado` pasaría el filtro.

Las cabeceras de respuesta son igual de deliberadas. `frame-ancestors 'self'` y no `'none'`, porque la aplicación muestra la guía ilustrada dentro de un marco del **mismo** origen; sigue impidiendo que cualquier sitio externo enmarque la app, que es de lo que protege esa directiva.

Y el detalle de `charset`: sólo los formatos textuales lo llevan. Declarar `charset=utf-8` en un PNG o un PDF es ruido.

Lo que no tiene explicación en el repositorio

Honestidad sobre los límites de este documento:

Elemento	Estado
<code>apps/web/src/app.css</code> (785 líneas)	NO DOCUMENTADO EN EL REPOSITORIO — sin comentarios de sección. Es una hoja de estilos, y su ausencia de documentación no oculta ninguna decisión de negocio
<code>scripts/build-icons.mjs</code> , algoritmo de rasterizado	Documentado en la cabecera del archivo (por qué a mano y no con <code>sharp</code>), pero el supermuestreo y la construcción de los <code>chunks</code> PNG no tienen comentarios internos
<code>docs/assets/diagramas/anatomia.svg</code> , <code>casos-de-uso.svg</code> , <code>seis-estados.svg</code>	NO IDENTIFICADO qué los genera. Dos de los cinco SVG los produce <code>lib/diagrams.mjs</code> ; el origen de los otros tres no consta en el repositorio
Umbral <code>views >= 10</code> y <code>core >= 6</code> repetidos en <code>copiar-www.mjs</code> , <code>verify-apk.mjs</code> , <code>desktop.yml</code> , <code>pages.yml</code> y <code>webapp.test.mjs</code>	Existen y funcionan; no consta por escrito por qué esos números y no el recuento exacto. INFERENCIA : son cotas inferiores tolerantes, para que añadir una vista no rompa cinco sitios

07 · Persistencia

La conclusión, primero

No hay base de datos. No hay motor, no hay servidor de datos, no hay ORM, no hay migraciones SQL, no hay cadena de conexión y no hay credencial que proteger. Se buscó: ni un `.sql`, ni un esquema, ni un driver, ni una dependencia de base de datos en ningún `package.json` del repositorio.

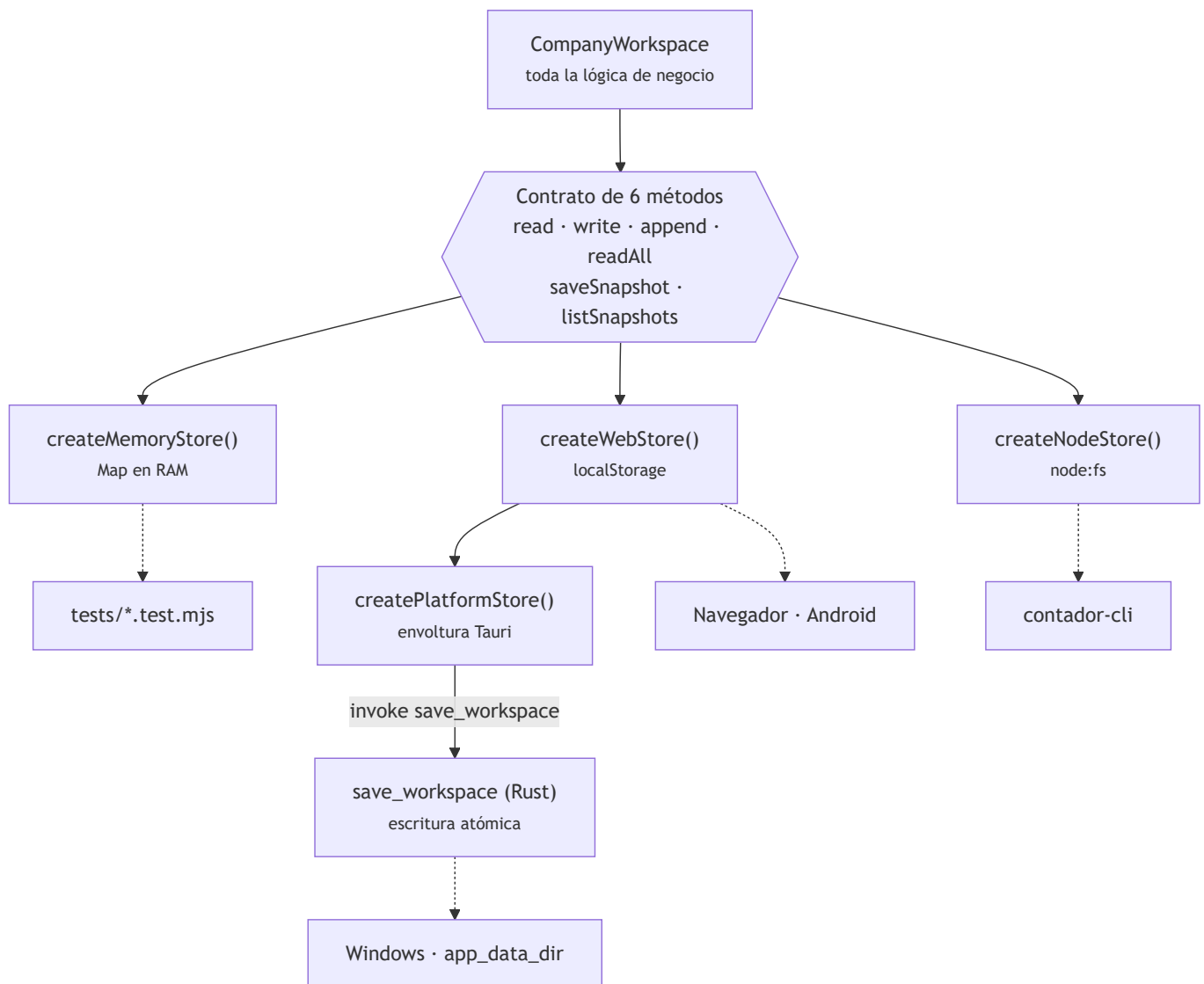
Lo que sí hay es un **contrato de almacén de seis métodos** (`packages/company-operations/store.mjs`) con tres implementaciones. `CompanyWorkspace` —las 1.142 líneas donde vive toda la lógica de negocio— sólo habla ese contrato y no sabe sobre qué está escribiendo.

Este documento cumple lo que se le pide a un capítulo de base de datos: el diccionario de datos está completo, con tipos, claves, relaciones e integridad, aunque la tecnología subyacente sea `localStorage` y archivos JSON en vez de tablas.

Las tres implementaciones del almacén

Implementación	Archivo	kind	Dónde escribe	Quién la usa
Memoria	<code>store.mjs</code> · <code>createMemoryStore()</code>	memory	Tres <code>Map</code> en RAM	Las 13 suites de prueba y las simulaciones desechables
Web	<code>store.mjs</code> · <code>createWebStore()</code>	web	<code>localStorage</code> del navegador	Navegador, WebView de Android, WebView de Windows
Disco	<code>node-store.mjs</code> · <code>createNodeStore()</code>	node	Archivos JSON y NDJSON	La CLI y cualquier proceso Node

Y una cuarta, que es una **envoltura** y no una implementación: `createPlatformStore(mode)` en `apps/web/src/lib/platform.js`. Devuelve el almacén web tal cual en navegador y en Android; bajo Tauri lo envuelve para copiar a disco después de cada escritura.



Qué muestra: que hay un solo punto de bifurcación entre plataformas y está en el almacén, no repartido por la lógica. **Qué no muestra:** que `createNodeStore` y `createPlatformStore` nunca coexisten — son caminos

excluyentes, no capas apiladas.

El contrato, método a método

Método	Firma	Semántica	Qué NO garantiza
<code>read</code>	<code>(key, fallback = null)</code>	Documento JSON completo, o <code>fallback</code>	No distingue «no existe» de «está corrupto»: ambos devuelven <code>fallback</code>
<code>write</code>	<code>(key, value)</code>	Reemplaza el documento entero	Atómico sólo en <code>node</code> y bajo Tauri; en <code>localStorage</code> no hay transacción
<code>append</code>	<code>(key, row)</code>	Agrega una fila a un log	En <code>web</code> lee, empuja y reescribe el arreglo completo — no es un append real
<code>readAll</code>	<code>(key)</code>	Filas del log, en orden de inserción	No pagina ni filtra: devuelve todo
<code>saveSnaps</code> <code>hot</code>	<code>(name, data)</code>	Guarda un respaldo, devuelve su ubicación	El formato de la ubicación cambia por implementación
<code>listSnaps</code> <code>hots</code>	<code>()</code>	Nombres ordenados	No dice cuánto ocupan ni si se pueden leer

La asimetría del `append` importa. En `node-store` un append es una línea nueva en un NDJSON (`fs.appendFileSync`), y el comentario del código explica por qué: reescribir el archivo completo sería la forma más fácil de perder o alterar historial. En `createWebStore` el mismo `append` hace leer, empujar y volver a escribir, es decir ****reescribe todo el arreglo en cada línea de bitácora****. El comportamiento observable es igual; el coste y las propiedades de durabilidad, no. Se recoge como hallazgo en [15 · Riesgos](#).

Dónde acaban los datos, en cada plataforma

Plataforma	Ubicación real	Formato	Sobrevive a...
Navegador / PWA	<code>localStorage</code> del origen	Un <code>string</code> JSON por clave	Cerrar el navegador. No sobrevive a «borrar datos del sitio»
Android (APK)	<code>localStorage</code> de la WebView, dentro del sandbox de la app	Igual	Cerrar la app. No sobrevive a desinstalar ni a «borrar datos»
Windows (Tauri)	<code>localStorage</code> + <code>app_data_dir()/real.json</code> y <code>sandbox.json</code>	Igual + espejo JSON	Reinstalar la app, si no se borra el directorio de datos
CLI / Node	<code><rootDir>/<mode>/*.json</code> + <code>audit.ndjson</code>	Un archivo por entidad	Todo, salvo borrar la carpeta

`app_data_dir()` lo resuelve Tauri a partir del `identifier` de `tauri.conf.json` (`cl.vladimiracuna.empresaoperativa`). La ruta exacta la muestra la propia aplicación en la pestaña

Datos, a través del comando `app_info`. **REQUIERE VALIDACIÓN**: no se ejecutó la app de escritorio en la máquina de análisis, así que la ruta se declara como la que Tauri resuelve, no como una observada.

Cómo se llaman las claves

En `localStorage` cada clave lleva prefijo de aplicación y de modo:

```
empresa-operativa-chile:real:transactions
empresa-operativa-chile:sandbox:transactions
empresa-operativa-chile:real:__snapshots
empresa-operativa-chile:real:snapshot:2026-08-27T12-00-00-000Z
```

El prefijo se arma en `platform.js` y el almacén web antepone `namespace:` a cada clave. **Ese prefijo es toda la separación entre EMPRESA REAL y SANDBOX**: no hay ninguna otra barrera, porque ambos modos comparten origen. Una prueba lo fija (`tests/workspace.test.mjs` , «el almacén web aísla real de sandbox en el mismo origen»).

En disco (`node-store`) cada clave es un archivo, según el mapa `FILES` :

Clave lógica	Archivo	Formato
<code>company</code>	<code>company.json</code>	Objeto
<code>formation</code>	<code>formation.json</code>	Arreglo
<code>transactions</code>	<code>transactions.json</code>	Arreglo
<code>obligations</code>	<code>obligations.json</code>	Arreglo
<code>closed-periods</code>	<code>closed-periods.json</code>	Arreglo de <code>YYYY-MM</code>
<code>period-closes</code>	<code>period-closes.json</code>	Arreglo
<code>audit</code>	<code>audit.ndjson</code>	NDJSON , una línea por hecho
<i>cualquier otra</i>	<code><clave>.json</code>	Objeto o arreglo

Junto al directorio del modo, `createNodeStore` crea siempre `evidence/` y `backups/`.

Hallazgo. `FILES` no incluye `equity-movements`, `annual-closes` ni `municipal-profile`, que son las tres entidades incorporadas en la versión 1.4.0. No se pierden —caen en la rama por defecto y acaban en `equity-movements.json`, `annual-closes.json` y `municipal-profile.json` — pero

`saveSnapshot` sólo copia como archivo suelto lo que está en `FILES`, así que un respaldo hecho con el almacén de disco no incluye esos tres archivos copiados. El `snapshot.json` interior sí lleva los datos, porque sale de `exportAll()`. Ver [15 · Riesgos](#).

Diccionario de datos

Diez entidades. Los nombres de campo se citan **en su forma original** para que buscarlos en el repositorio encuentre exactamente esto. Todos los importes son enteros en pesos chilenos: `money(n) = Math.round(Number(n || 0))`, en `workspace.mjs:27`.

company — ficha de la empresa

Documento único. Lo escribe `saveCompany()`; lo lee casi todo el sistema.

Campo	Tipo	Obligatorio	Notas
<code>legalName</code>	<code>string</code>	No (pero <code>healthCheck</code> lo reclama)	Razón social
<code>rut</code>	<code>string</code>	No (idem)	Validado por <code>packages/company-operations/rut.mjs</code>
<code>commune</code>	<code>string</code>	No	Semilla de la ficha municipal
<code>taxRegime</code>	<code>string</code>	No	Decide si se admite el CPT simplificado
<code>capital</code>	<code>number</code>	No	Campo heredado. Se mantiene sincronizado con <code>capitalProfile.capitalEnterado</code>
<code>capitalProfile</code>	<code>object</code>	No	Ver tabla siguiente
<code>mode</code>	<code>real</code> o <code>sandbox</code>	Lo pone el motor	Se reescribe en cada <code>saveCompany</code>
<code>createdAt</code> / <code>updatedAt</code>	ISO-8601	Los pone el motor	<code>createdAt</code> sólo se fija la primera vez

company.capitalProfile — las magnitudes del capital

No es una entidad aparte: vive **dentro** de `company`. Se normaliza en cada lectura con `normalizeCapitalProfile()` (`capital.mjs:196`), lo que significa que **leer no escribe**: si nadie guarda, el dato original permanece intacto en el almacén.

Campo	Tipo	Valor ausente	Por qué importa
<code>capitalSocial</code>	<code>number</code> o <code>null</code>	<code>null</code>	El del estatuto
<code>capitalSuscrito</code>	<code>number</code> o <code>null</code>	<code>null</code>	Lo que el accionista se comprometió a pagar
<code>capitalEnterado</code>	<code>number</code>	<code>0</code>	Lo efectivamente pagado. Es lo único que es patrimonio
<code>numeroAcciones</code>	<code>number</code> o <code>null</code>	<code>null</code>	
<code>valorNominal</code>	<code>number</code> o <code>null</code>	<code>null</code>	
<code>accionistas</code>	<code>array</code>	<code>[]</code>	<code>name</code> , <code>rut</code> , <code>sharePercent</code> , <code>acciones</code> , <code>capitalSuscrito</code> , <code>capitalEnterado</code>
<code>fechaConstitucion</code>	<code>YYYY-MM-DD</code> o <code>null</code>	<code>null</code>	
<code>fechaInicioActividades</code>	<code>YYYY-MM-DD</code> o <code>null</code>	<code>null</code>	
<code>pendingConfirmation</code>	<code>string[]</code>	<code>[]</code>	Los campos que la app sabe que no sabe
<code>migratedFromLegacyCapital</code>	<code>boolean</code>	<code>false</code>	Marca de la migración del campo <code>capital</code>

`null` no es `0` , y esa distinción es la regla de negocio del documento. `capitalPendingToPay()` devuelve `null` —y no cero— cuando `capitalSuscrito` es desconocido, porque cero significaría «no falta nada», y eso es una afirmación que con ese dato ausente nadie puede hacer.

equity-movements — ledger patrimonial

Arreglo. Lo escriben `addEquityMovement()` y `deleteEquityMovement()` ; lo normaliza `normalizeEquityMovement()` (`capital.mjs:310`), que **lanza** si falta la fecha, si el monto no es mayor que cero, o si un aporte en bienes no describe el bien y a su aportante.

Campo	Tipo	Validación	Notas
<code>id</code>	<code>string</code>	—	<code>crypto.randomUUID()</code> , o <code>id-<base36></code> sin WebCrypto
<code>kind</code>	enum	Debe estar en <code>EQUITY_MOVEMENT_KINDS</code>	Cada tipo trae <code>equityEffect</code> y <code>liabilityEffect</code>
<code>date</code>	<code>YYYY-MM-DD</code>	Formato exacto, obligatorio	
<code>amount</code>	<code>number</code>	Mayor que cero	
<code>description</code>	<code>string</code>	—	
<code>contributedBy</code>	<code>string</code> o <code>null</code>	Obligatorio si el tipo exige bien aportado	
<code>assetType</code> / <code>assetDescription</code>	<code>string</code> o <code>null</code>	<code>assetDescription</code> obligatorio en aportes en bienes	
<code>bookValue</code> / <code>taxValue</code>	<code>number</code> o <code>null</code>	—	Valor contable y tributario del bien
<code>evidenceRef</code>	<code>string</code>	—	
<code>status</code>	enum <code>DATA_STATUSES</code>	Cae a <code>DECLARADO</code>	Distingue lo declarado de lo confirmado
<code>origin</code>	enum <code>DATA_ORIGINS</code>	Cae a <code>usuario</code>	
<code>createdAt</code>	ISO-8601	—	

Un aporte y un préstamo del accionista entran por el mismo banco y son opuestos aquí. Por eso son tipos distintos y la aplicación obliga a elegir al registrar, en vez de adivinar meses después.

`transactions` — operaciones de caja

Arreglo. La entidad más escrita del sistema. `addTransaction()` (`workspace.mjs:688`) la valida en tres puertas: tipo soportado, fecha `YYYY-MM-DD` , y **período no cerrado**.

Campo	Tipo	Valor por defecto	Regla de negocio
<code>id</code>	<code>string</code>	UUID	
<code>date</code>	<code>YYYY-MM-DD</code>	Hoy	Su prefijo <code>YYYY-MM</code> es la clave de período
<code>kind</code>	enum de 9 valores	—	Un valor fuera de <code>TRANSACTION_KINDS</code> lanza
<code>description</code>	<code>string</code>	vacío	
<code>net</code> / <code>vat</code> / <code>total</code>	<code>number</code>	<code>total = net + vat</code>	Enteros
<code>documentType</code> / <code>documentNumber</code>	<code>string</code> o <code>null</code>	<code>null</code>	Su ausencia alimenta <code>withoutEvidence</code>
<code>counterpartyRut</code>	<code>string</code> o <code>null</code>	<code>null</code>	Dato personal de un tercero
<code>paid</code>	<code>boolean</code>	<code>false</code>	
<code>deductible</code>	<code>boolean</code>	<code>true</code>	Sólo un <code>false</code> explícito lo desactiva
<code>vatCreditEligible</code>	<code>boolean</code>	<code>true</code>	Si es <code>false</code> , su IVA va a <code>rejectedVat</code>
<code>source</code>	<code>string</code>	<code>manual</code>	
<code>evidence</code>	<code>array</code>	<code>[]</code>	
<code>equityMovementId</code>	<code>string</code> o <code>null</code>	<code>null</code>	Enlace que impide el doble conteo del capital
<code>createdAt</code> / <code>updatedAt</code>	ISO-8601	—	

Los nueve `kind`, con su efecto sobre el IVA:

id	Etiqueta	flow	affectsVat
sale	Venta	in	debit
purchase	Compra del giro	out	credit
expense	Gasto	out	credit
honorarium	Honorario pagado	out	none
capital	Aporte de capital	in	none
shareholder_loan	Préstamo del accionista	in	none
shareholder_loan_repayment	Devolución de préstamo	out	none
owner_withdrawal	Retiro del accionista	out	none
tax_payment	Pago de impuesto	out	none

formation — pasos de constitución

Arreglo, pero **híbrido**: el catálogo de los 9 pasos vive en el código (`FORMATION_STEPS` , congelado con `Object.freeze`) y en el almacén sólo se guarda el estado. `listFormationSteps()` fusiona ambos en cada lectura, dejando ganar lo guardado sobre los valores por defecto.

Consecuencia de diseño, no accidente: cuando se agrega un trámite nuevo al catálogo, las empresas ya creadas lo ven aparecer, en vez de quedarse con una lista congelada del día en que se instalaron. Una prueba lo fija: «el catálogo de pasos se amplía sin perder lo ya registrado».

Campo	Origen	Notas
id , title , authority , evidenceHint , url	Código	Los 9: design , res , rut , start , activities , address , dte , patent , bank
status	Almacén	pending · in_progress · done · blocked
evidenceRef	Almacén	done sin evidenceRef lanza
updatedAt	Almacén	

obligations — obligaciones con vencimiento

Campo	Tipo	Notas
type	string	Obligatorio; <code>upsertObligation</code> lanza sin él
period	string	YYYY-MM o YYYY, según el tipo
dueDate	YYYY-MM-DD	Vencida y sin comprobante ⇒ <code>healthCheck</code> nivel <code>error</code>
status	enum	Marcarla <code>done</code> exige comprobante
evidenceRef	string	

closed-periods y period-closes — el cierre mensual

Dos claves para el mismo hecho, y la separación es deliberada:

- `closed-periods`: arreglo de `YYYY-MM` **ordenado**. Es el índice barato que consulta

`isPeriodClosed()`, y lo consultan `addTransaction`, `updateTransaction` y `deleteTransaction` en cada llamada.

- `period-closes`: arreglo de fotografías completas — `id`, `period`, `at`, `summary`,

`carryForwardIn` y `checklist` —, donde `summary` es el `periodSummary(period)` congelado el día del cierre.

Reabrir borra el período de `closed-periods` **pero no toca** `period-closes`: la fotografía se queda. Y `reopenPeriod(period, reason)` lanza si el motivo viene vacío.

annual-closes — el cierre del ejercicio

Arreglo de objetos **congelados con** `Object.freeze` en el momento de crearse. La entidad más rica del sistema y la que sostiene el argumento del producto.

Campo	Tipo	Por qué está
<code>id</code> , <code>fiscalYear</code> , <code>closingDate</code> , <code>createdAt</code>	—	<code>closingDate</code> siempre <code><año>-12-31</code>
<code>taxRegime</code>	<code>string</code> o <code>null</code>	Copiado de la ficha ese día
<code>assets</code> , <code>liabilities</code> , <code>accountingEquity</code>	<code>number</code>	
<code>balanceOrigin</code>	texto	<code>estimado por la aplicación</code> o <code>declarado por el usuario</code> . Nunca se presenta una estimación como un dato declarado
<code>taxEquity</code> , <code>CPT</code> , <code>CPTMethod</code>	objeto / <code>number</code> / <code>string</code>	Resultado y método usado
<code>taxAdjustments</code>	objeto	
<code>capital</code> , <code>capitalMovements</code>	objeto / arreglo	Movimientos hasta el 31-12
<code>yearSummary</code>	objeto	Agregado del año
<code>municipalPatentBaseFor</code> <code>NextPeriod</code>	objeto	El puente entre un ejercicio y el siguiente
<code>evidence</code> , <code>notes</code>	arreglo / <code>string</code>	
<code>legalRulesVersion</code>	objeto	Ver abajo

`legalRulesVersion` es la decisión de diseño más importante de la persistencia. No se guarda una referencia al archivo de reglas: se guarda una **huella del contenido** —tasas, referencias legales y fechas de verificación— dentro del propio cierre. El comentario del código lo dice sin rodeos: dentro de tres años `rules/2026.json` puede haberse corregido, y el cierre tiene que seguir explicando con qué normas se calculó ese día.

`closeFiscalYear()` **no sobrescribe nunca**: si el ejercicio ya está cerrado, lanza. Rehacerlo exige `reopenFiscalYear(year, reason)` con motivo, y eso queda en la bitácora junto al CPT anterior.

municipal-profile — ficha municipal

Campo	Tipo	Notas
<code>commune</code>	<code>string</code>	Se siembra desde <code>company.commune</code>
<code>rate</code> , <code>status</code>	<code>number</code> / enum	La tasa de la comuna no está en el repositorio : la fija cada municipalidad por ordenanza
<code>initialOwnCapital</code>	<code>number</code> o <code>null</code>	<code>null</code> no es <code>0</code> , otra vez
<code>deductibleInvestments</code>	<code>number</code>	Inversiones en otros negocios con patente
<code>allocatedCapital</code>	<code>number</code> o <code>null</code>	Prorratio entre sucursales
<code>utmPeriod</code>	<code>YYYY-MM</code> o <code>null</code>	La UTM cambia todos los meses
<code>notes</code> , <code>updatedAt</code>		

audit — bitácora append-only

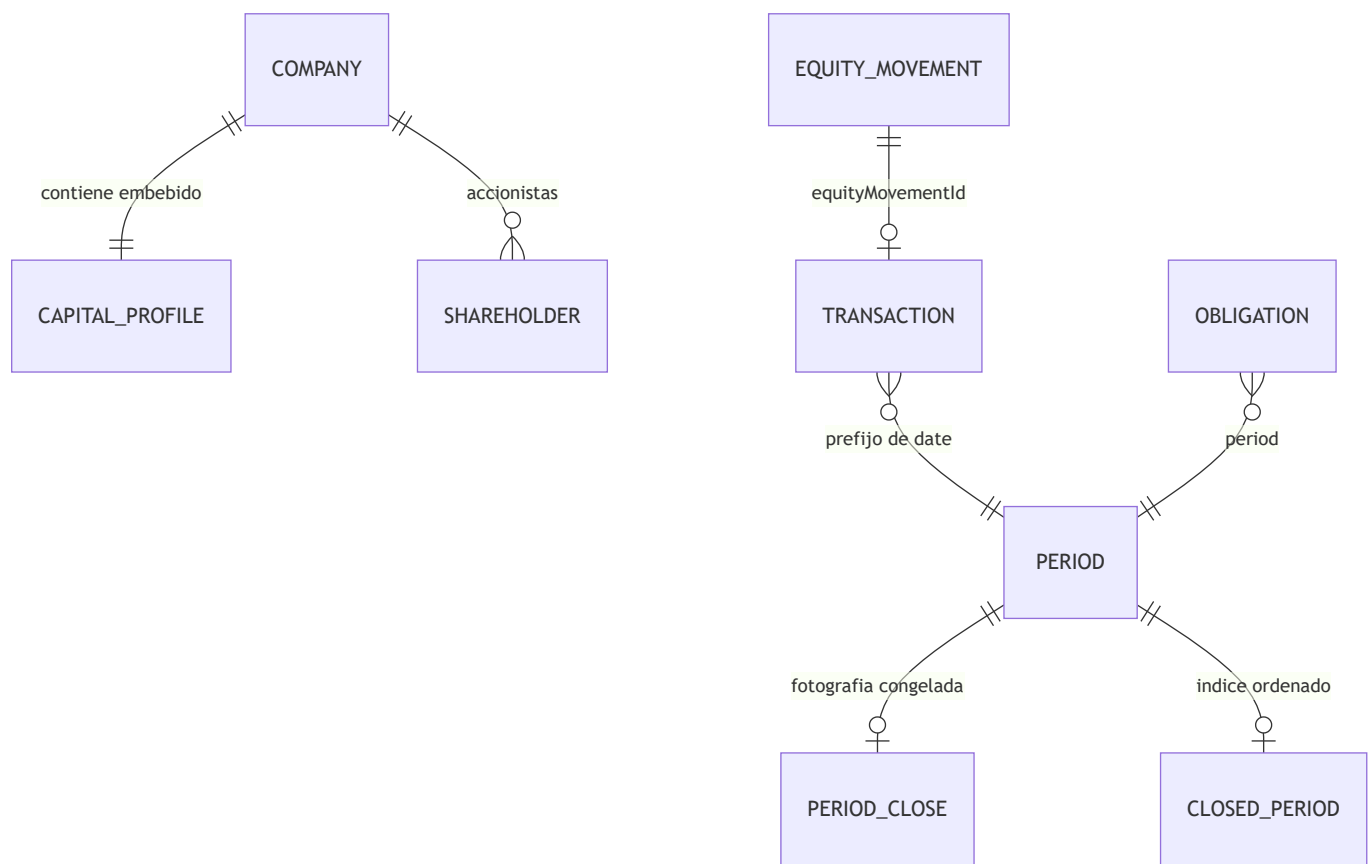
La única entidad que usa `append` y `readAll` en vez de `read` y `write` .

Campo	Tipo	Notas
<code>id</code>	<code>string</code>	UUID
<code>at</code>	ISO-8601	
<code>mode</code>	<code>real</code> o <code>sandbox</code>	
<code>action</code>	<code>string</code>	<code>transaction.added</code> , <code>period.closed</code> , <code>fiscal-year.reopened</code> , ...
<code>detail</code>	objeto	Depende de la acción

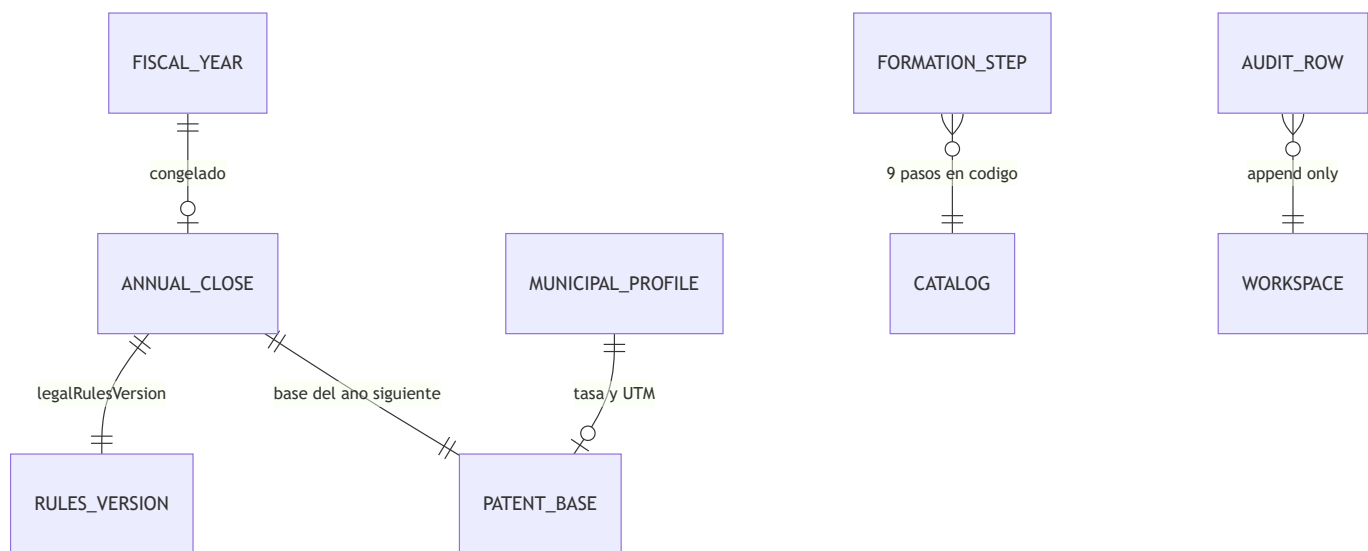
No existe ninguna operación de borrado de la bitácora en toda la clase. No es un olvido: el comentario del código lo declara — si un registro pudiera borrarse sin dejar huella, la bitácora no serviría como evidencia de nada. Una prueba lo fija: «toda mutación queda en la bitácora append-only».

Relaciones e integridad

El día a día — la empresa, su capital y el mes:



El año y la evidencia — lo que se congela y lo que se registra:



Por qué son dos y no uno: en un solo diagrama las trece relaciones salen en una fila de 2.328 píxeles, que impresa en A4 se reduce al 30 % y deja el texto ilegible. El propio generador lo detecta y avisa; partirlo por dominio —el mes por un lado, el ejercicio por otro— es la respuesta a ese aviso.

Qué muestran: que las relaciones existen aunque no haya claves foráneas. **Qué no muestran:** que ninguna de estas relaciones la hace cumplir un motor — todas son código en `workspace.mjs`, y por eso están listadas

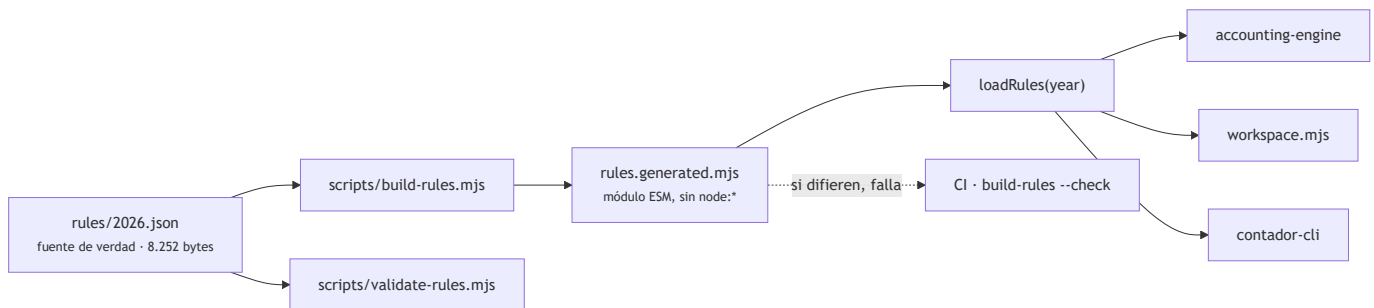
abajo una por una.

#	Regla de integridad	Dónde se hace cumplir	Qué pasa si se viola
1	Un período cerrado no admite alta, edición ni baja	<code>addTransaction</code> , <code>updateTransaction</code> , <code>deleteTransaction</code>	Error
2	Una operación no puede <i>moverse</i> a un período cerrado	<code>updateTransaction</code> , segunda comprobación sobre la fecha ya fusionada	Error
3	Un paso <code>done</code> exige <code>evidenceRef</code> no vacío	<code>updateFormationStep</code>	Error
4	Una obligación <code>done</code> exige comprobante	<code>upsertObligation</code>	Error
5	Reabrir período o ejercicio exige motivo	<code>reopenPeriod</code> , <code>reopenFiscalYear</code>	Error
6	Un ejercicio ya cerrado no se cierra dos veces	<code>closeFiscalYear</code>	Error
7	Un ejercicio cerrado no se pisa al importar en modo fusión	<code>importAll</code>	Se ignora la fila entrante
8	Un aporte enlazado no se cuenta dos veces	<code>equityMovementId</code> + <code>effectiveEquityMovements()</code>	El capital enterado se duplicaría
9	El sandbox no se siembra sobre datos reales	<code>seedSandboxWorkspace</code>	Error
10	El sandbox no se resiembra si ya tiene operaciones	<code>seedSandboxWorkspace</code>	Retorno temprano
11	Un archivo ajeno no se importa	<code>importAll</code> , comprobación de <code>format</code>	Error
12	<code>mode</code> sólo puede ser <code>real</code> o <code>sandbox</code>	Constructor de <code>CompanyWorkspace</code> y <code>safe_mode()</code> en Rust	Error

Lo que ningún mecanismo garantiza: no hay transacciones multi-documento. Cerrar un período escribe `period-closes` y después `closed-periods` en dos operaciones separadas. Si el proceso muere entre ambas —o `localStorage` se llena— queda una fotografía sin su índice. No se observó ocurrir; se declara como límite estructural en [15 · Riesgos](#).

Las reglas tributarias como datos versionados

Es el otro mecanismo de persistencia del sistema, y funciona al revés que el anterior: **de sólo lectura, versionado en Git y compilado durante el build.**



Por qué existe el paso de compilación. Un navegador no puede importar un `.json` de forma portátil, y leerlo con `node:fs` está prohibido por la regla dura de la arquitectura. La salida es un módulo ESM que embebe el objeto, y así el mismo archivo viaja al APK y al ejecutable de Windows.

Por qué un año faltante falla. `loadRules(1999)` lanza `No hay reglas verificadas para el año comercial 1999. Disponibles: 2026`. No degrada al año más cercano. `CONTRIBUTING.md` lo prohíbe por escrito y `tests/rules.test.mjs` lo fija con una prueba. Es la regla de negocio más importante del sistema: un cálculo con la tasa del año equivocado no falla — devuelve un número plausible que aparece meses después como una diferencia con el SII.

Estructura de `rules/<año>.json`

Bloque	Contenido	Campos de procedencia
(raíz)	<code>schemaVersion: 2</code> , <code>country</code> , <code>commercialYear</code> , <code>lastVerified</code> , <code>profile</code> , <code>status</code>	
<code>iva</code>	<code>generalRate: 0.19</code>	<code>source</code> , <code>lastVerified</code> , <code>note</code>
<code>honorarios</code>	<code>retentionRate: 0.1525</code>	idem
<code>ppmProPy</code> <code>me</code>	<code>initialYearRate</code> , <code>upTo50000UfRate</code> , <code>above50000UfRate</code>	idem
<code>idpcProP</code> <code>yme</code>	<code>rate: 0.125</code>	idem
<code>utm</code>	<code>2026-08: 71649</code>	idem
<code>municipalPatent</code>	<code>minUtm</code> , <code>maxUtm</code> , <code>minRate</code> , <code>maxRate</code> , más <code>capitalBasis</code> , <code>deductions</code> , <code>branchAllocation</code> e <code>informationFlow</code>	<code>source</code> , <code>legalReference</code> , <code>effectiveFrom</code> , <code>lastVerified</code> , <code>verificationNote</code>
<code>taxEquity</code>	<code>methods.article41</code> , <code>methods.simplified14D3j</code> , <code>eligibility</code> , <code>initialTaxEquity</code> , <code>warnings</code>	<code>legalReference</code> , <code>source</code> , <code>lastVerified</code> por método
<code>f29</code>	<code>generalDueDay: 12</code> , <code>internetPaidEligibleDueDay: 20</code> , <code>internetNoPaymentDueDay: 28</code>	idem
<code>warnings</code>	6 advertencias de alcance	

Cada tasa lleva su fuente oficial y la fecha en que se verificó. No es decoración: `ruleProvenance(year, ruleName)` la expone a la interfaz, de modo que la aplicación puede decir de dónde salió cada número.

El propio archivo declara su límite más incómodo, y conviene leerlo entero: el `verificationNote` de `municipalPatent` registra que **BCN/LeyChile no respondió el 16-08-2026** y pide reverificar el enlace antes de usar la cifra en una declaración real. Un archivo de reglas que admite lo que no pudo comprobar es exactamente lo que esta documentación pide de sí misma.

Lo que el mecanismo no cubre

- Hay **un solo año**: `availableYears()` devuelve `[2026]`. El sistema no ha vivido todavía un cambio de año, así que la política «un archivo por año» está diseñada y probada, pero **no ejercitada con dos archivos**. **INFERENCIA**: el primer `rules/2027.json` es donde se sabrá si el esquema aguanta.
- `utm` tiene **un solo mes** (`2026-08`). Calcular una patente con la UTM de otro mes exige agregarlo.

- La **tasa de una comuna concreta** no está, y no debe estar: el archivo guarda el rango legal.
-

Datos sensibles

Dato	Dónde vive	Cifrado	Observación
RUT de la empresa	<code>company.rut</code>	No	
RUT de terceros	<code>transactions[].counterpartyRut</code>	No	Dato personal de un tercero
Nombres y RUT de accionistas	<code>capitalProfile.accionistas</code>	No	
Montos, folios y descripciones	<code>transactions</code>	No	Revela relaciones comerciales
Bitácora completa	<code>audit</code>	No	Reconstruye la actividad de la empresa
Respaldos exportados	Archivo JSON elegido por el usuario	No	Legible con cualquier editor

Nada de esto está cifrado en reposo, ni en `localStorage`, ni en el espejo de Windows, ni en los respaldos. `SECURITY.md` lo declara con la misma claridad y lo sitúa en el roadmap. La contrapartida es que tampoco sale del dispositivo: ver [11 · Seguridad](#).

`.gitignore` cubre `.local-data/`, `private-data/`, `real-company-data/`, los `*.sqlite*` y los `empresa-operativa-real-*.json` y `.csv`; el workflow `security.yml` busca en cada push respaldos con la firma `empresa-operativa-chile/backup`, certificados y claves privadas.

Respaldo, restauración y migración

Respaldo. `backup()` nombra el snapshot con la marca de tiempo ISO saneada, guarda `exportAll()` y deja constancia en la bitácora.

Formato portátil. `exportAll()` produce un JSON con `format: empresa-operativa-chile/backup` y `formatVersion: 2`. El mismo archivo se exporta desde Android y se importa en Windows.

Restauración. `importAll(payload, { replace = true })`:

- rechaza cualquier archivo cuyo `format` no sea el propio — prueba: «importar un archivo ajeno se rechaza»;
- acepta `formatVersion 1 y 2`. Los respaldos v1 no traen `equityMovements`, `annualCloses` ni `municipalProfile`, y esos campos quedan vacíos. Romper los respaldos que la gente ya tiene guardados sería peor que limpiar el formato;
- con `replace: false` fusiona por `id` y **respetar los ejercicios ya cerrados**.

Las dos migraciones del sistema, ambas en lectura y sin tocar el almacén:

Migración	Dónde	Comportamiento
Campo <code>capital</code> → <code>capitalProfile</code>	<code>normalizeCapitalProfile()</code>	El valor pasa a <code>capitalEnterado</code> ; <code>capitalSocial</code> y <code>capitalSuscrito</code> quedan en <code>pendingConfirmation</code> . No se inventan
Catálogo de trámites	<code>listFormationSteps()</code>	Los pasos nuevos aparecen sobre el estado guardado

Que las migraciones ocurran **en lectura** tiene una consecuencia que conviene entender: instalar esta versión sobre datos antiguos **no puede corromperlos**, porque hasta que el usuario no guarde, el almacén conserva el dato original.

Lo que no existe, dicho explícitamente

NO IDENTIFICADO — se buscó y no hay evidencia en ningún sentido:

- Motor de base de datos, esquema SQL, migraciones versionadas, procedimientos almacenados, funciones, disparadores, vistas e índices.
 - Pool de conexiones, cadena de conexión, credenciales de base de datos.
 - Caché de datos: **localStorage** es el almacén, no una caché sobre otra cosa. El único caché del sistema es el del service worker, y sólo guarda los archivos de la aplicación.
 - Política de respaldo automática o programada: el respaldo lo dispara el usuario.
 - Retención, purga o archivado de la bitácora. Crece sin techo.
-

08 · Flujo de datos

La conclusión, primero

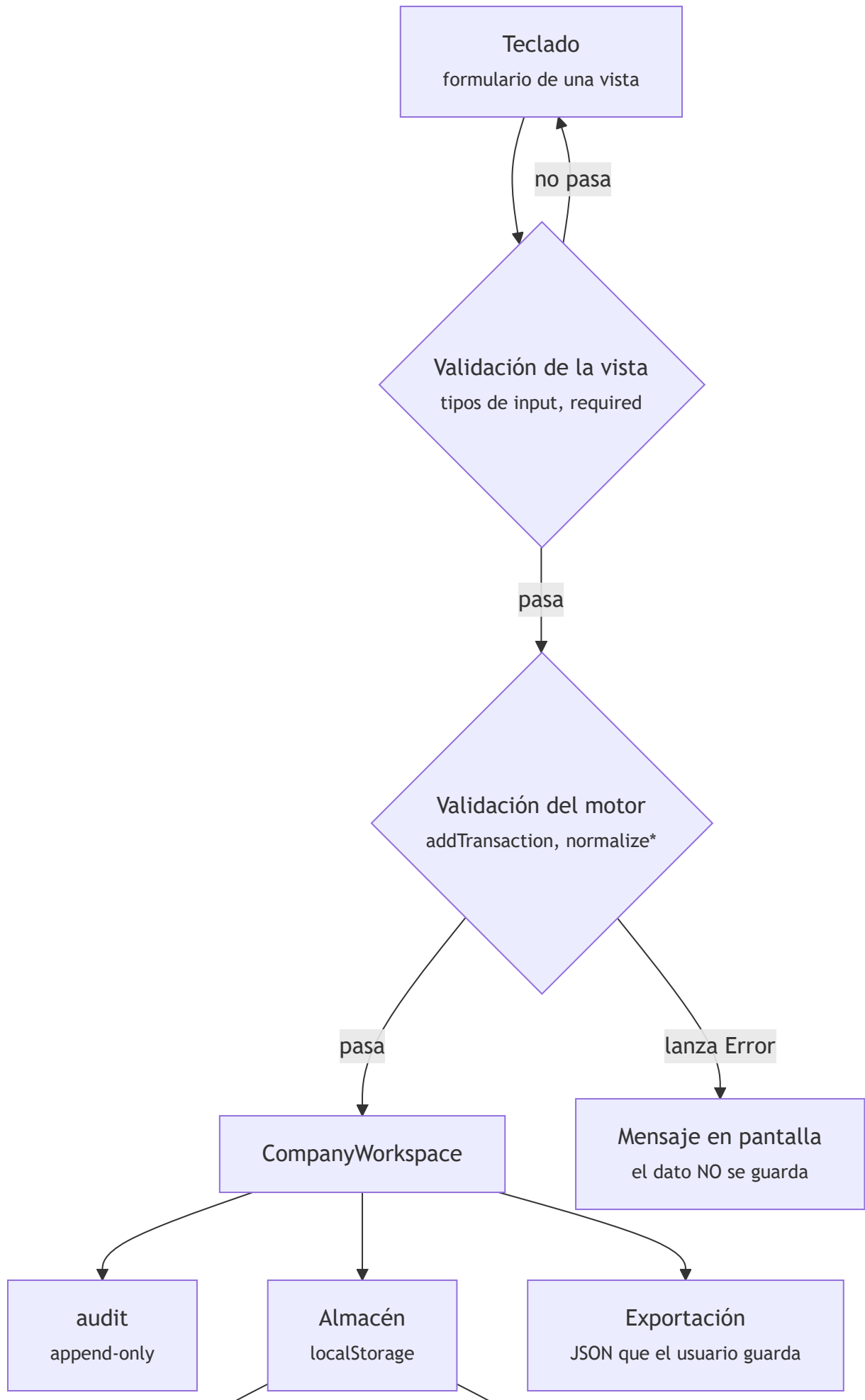
Hay **dos flujos de datos en este sistema y no se tocan**:

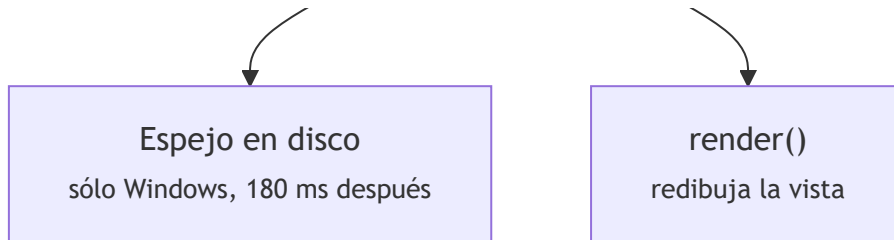
1. **Los datos del usuario** entran por el teclado, se validan, se guardan en el dispositivo y sólo salen si la persona exporta un archivo a propósito. Nunca cruzan la red.

1. **Las reglas tributarias** entran por un *pull request*, se verifican contra una fuente oficial, se compilan al build y viajan **de sólo lectura** dentro del artefacto. Nunca las escribe la app.

El primero es privado y mutable; el segundo es público e inmutable. Que estén separados es lo que permite decir a la vez «tus datos son tuyos» y «las tasas son auditables».

Flujo 1 · Los datos del usuario





Qué muestra: que hay dos capas de validación y que la segunda es la que manda. **Qué no muestra:** que `render()` vuelve a leer del almacén — la vista nunca pinta desde una variable en memoria, siempre relee.

1. De dónde vienen los datos

Origen	Qué aporta	Dónde entra
Formularios de las 14 vistas	Todo lo que el usuario escribe	<code>apps/web/src/views/*.js</code>
Semilla del sandbox	1 empresa, 5 movimientos patrimoniales, 18 operaciones, 4 obligaciones	<code>seedSandboxWorkspace()</code>
Respaldo importado	Un espacio de trabajo completo	<code>importAll()</code>
Escenario JSON de la CLI	Un caso de simulación	<code>data/scenarios/</code>
Espejo del disco (Windows)	Lo que falte en <code>localStorage</code> al arrancar	<code>hydrateFromDisk()</code>
Reglas del año	Tasas, plazos, referencias legales	<code>loadRules(year)</code>
Reloj del sistema	<code>createdAt</code> , <code>updatedAt</code> , <code>at</code> de la bitácora, período por defecto	<code>isoNow()</code>

No hay ninguna otra fuente. No hay importación de RCV, ni de cartola bancaria, ni de DTE, ni lectura de la propuesta del SII. Se declara aquí porque es la pregunta que hace todo el mundo: **NO IDENTIFICADO** — no existe en el repositorio.

2. Cómo se validan

La regla es que **la vista ayuda y el motor decide**. Un formulario puede marcar un campo como requerido, pero la garantía real está en el motor, porque la CLI y las pruebas entran por ahí sin pasar por ninguna vista.

Validación	Dónde	Qué rechaza
Tipo de operación	<code>addTransaction</code>	Un <code>kind</code> fuera de los nueve
Formato de fecha	<code>addTransaction</code> , <code>normalizeEquityMovement</code>	Cualquier cosa que no sea <code>YYYY-MM-DD</code>
Período cerrado	<code>addTransaction</code> , <code>updateTransaction</code> , <code>deleteTransaction</code>	Alta, edición o baja en un mes cerrado
Monto	<code>normalizeEquityMovement</code>	Montos nulos, negativos o no numéricos
Bien aportado	<code>normalizeEquityMovement</code>	Un aporte en bienes sin descripción ni aportante
Coherencia del capital	<code>validateCapitalProfile</code>	Combinaciones imposibles entre social, suscrito y enterado
Evidencia	<code>updateFormationStep</code> , <code>upsertObligation</code>	Dar por hecho algo sin comprobante
Motivo	<code>reopenPeriod</code> , <code>reopenFiscalYear</code>	Reabrir sin explicar por qué
Formato de respaldo	<code>importAll</code>	Un JSON que no es de esta aplicación
RUT	<code>packages/company-operations/rut.mjs</code>	Dígito verificador incorrecto
Modo	Constructor, y otra vez en Rust	Cualquier cosa que no sea <code>real</code> o <code>sandbox</code>

Todas fallan lanzando `Error` , no devolviendo `false` . La consecuencia es que un dato inválido **no llega nunca al almacén**: no hay estado intermedio a medio guardar.

3. Cómo se transforman

Transformación	Función	Qué hace y por qué
Redondeo monetario	<code>money()</code>	Todo importe se guarda como entero de pesos. Sin decimales no hay derivas de centavos al sumar meses
Normalización del capital	<code>normalizeCapitalPosition()</code>	Migra el campo antiguo y marca lo que no se sabe como <code>pendingConfirmation</code>
Normalización del movimiento	<code>normalizeEquityMovement()</code>	Recorta cadenas, fija <code>status</code> y <code>origin</code> por defecto
Derivación del período	<code>date.slice(0, 7)</code>	El período no se guarda : se deriva de la fecha. Una fuente, no dos
Agregación mensual	<code>periodSummary()</code>	18 cifras derivadas de las operaciones del mes
Arrastre del IVA	<code>vatCarryForwardInfo()</code>	Recorre la historia desde el principio acumulando remanente
Unión sin duplicar	<code>effectiveEquityMovements()</code>	Fusiona ledger y operaciones de caja usando <code>equityMovementId</code>
Congelado del cierre	<code>closeFiscalYear()</code>	<code>Object.freeze</code> sobre el resultado, con la huella de las reglas dentro

Nada de esto se guarda calculado. `periodSummary`, `capitalPosition`, `estimatedBalance`, `yearSummary` y `healthCheck` se recalculan en cada lectura desde `transactions` y `equity-movements`. La única excepción es deliberada: **lo que queda dentro de un cierre**, mensual o anual, se congela para siempre.

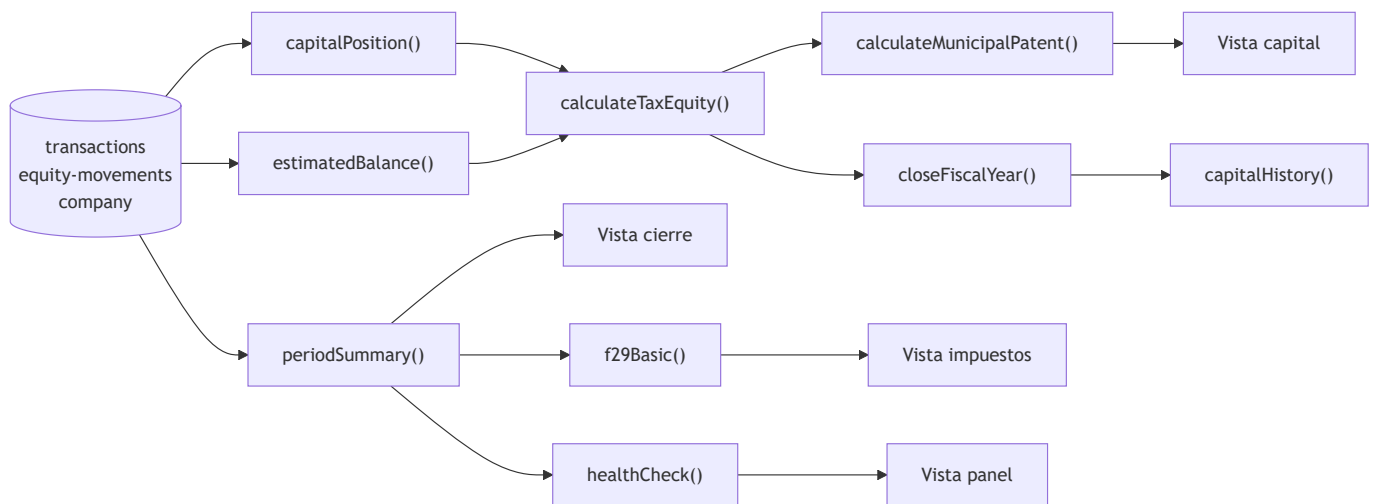
Es el criterio que separa una cifra viva de una cifra histórica. Una vista muestra lo que las operaciones dicen hoy; un cierre muestra lo que decían el día que se cerró. Si todo se recalculara, un cierre no probaría nada.

4. Dónde se almacenan

El detalle completo está en [07 · Persistencia](#). En una línea: `localStorage` con prefijo por modo, más un espejo en JSON bajo Windows, más archivos por entidad cuando el consumidor es Node.

El espejo de Windows llega tarde a propósito. `createPlatformStore` no escribe a disco en cada `write`: agenda un volcado a los **180 ms** y cancela el anterior. Diez escrituras seguidas producen un volcado, no diez. Y si el disco falla, **la aplicación sigue funcionando** con `localStorage` y sólo avisa por consola: perder la sesión por un error de escritura sería peor que perder el espejo.

5. Qué componentes los consumen



Qué muestra: que casi todo el sistema depende de dos colecciones, y que el CPT es el cuello por el que pasa la cadena capital → cierre → patente. **Qué no muestra:** las 14 vistas completas; sólo las que consumen datos derivados.

6. Qué se muestra al usuario

Lo que la interfaz pinta **nunca es el dato crudo**: es el resultado de las derivaciones anteriores. Y hay tres decisiones de presentación que son reglas de negocio disfrazadas:

- **balanceOrigin** viaja hasta la pantalla: la aplicación dice si el balance lo estimó ella o lo declaró el usuario. No presenta una estimación como un dato.
- **pendingConfirmation** se muestra: la app señala qué campos **sabe que no sabe**, en vez de rellenarlos con cero.
- **healthCheck** deliberadamente **no dice «todo en orden»** cuando faltan evidencias. El objetivo es detectar el hueco, no tranquilizar.

7. Qué sale del dispositivo

Nada, salvo que el usuario lo pida. Tres salidas, todas iniciadas por una acción explícita:

Salida	Mecanismo	Destino
Respaldo / exportación	<code>saveTextFile()</code>	En Windows, Documentos/Empresa Operativa Chile ; en web y Android, la descarga del navegador
Enlace a un portal oficial	<code>openExternal()</code>	El navegador del sistema, o el plugin Browser de Capacitor en Android
Nada más	—	—

openExternal merece una nota: abre el sitio del SII o del Registro de Empresas **fuera** de la aplicación, y con **noopener, noreferrer** . La app no enmarca portales oficiales ni les pasa datos: sólo lleva a la persona hasta la puerta.

No hay telemetría, ni analítica, ni informe de errores, ni comprobación de actualizaciones, ni fuentes o iconos remotos. La comprobación está en 09 y en 11.

8. Dónde puede perderse o corromperse un dato

Esta sección existe porque el prompt la pide y porque las respuestas honestas son incómodas.

Punto	Qué puede pasar	Mitigación presente	Mitigación ausente
«Borrar datos del sitio» en el navegador	Pérdida total	Ninguna	La app no puede impedirlo
Desinstalar el APK	Pérdida total	Ninguna	Idem
Cuota de <code>localStorage</code> agotada	<code>setItem</code> lanza y la escritura se pierde	Ninguna: no se captura <code>QuotaExceededError</code>	Ver 15 · Riesgos
Corte durante el espejo de Windows	Espejo a medias	Escritura atómica: temporal + <code>rename</code> en Rust	—
JSON corrupto en <code>localStorage</code>	<code>read</code> devuelve el <code>fallback</code>	El <code>try/catch</code> evita el fallo	Silencioso: no distingue vacío de corrupto
Línea corrupta en <code>audit.ndjson</code>	Se devuelve <code>{ corrupt: true, line }</code>	La fila mala no rompe la lectura	Nadie muestra ese marcador
Cierre a medias	Fotografía sin índice	Ninguna: son dos escrituras	No hay transacción
Dos pestañas del navegador a la vez	La última escritura gana	Ninguna	No hay bloqueo ni fusión
Reloj del sistema mal puesto	Fechas y bitácora incoherentes	Ninguna	No hay fuente de tiempo alternativa

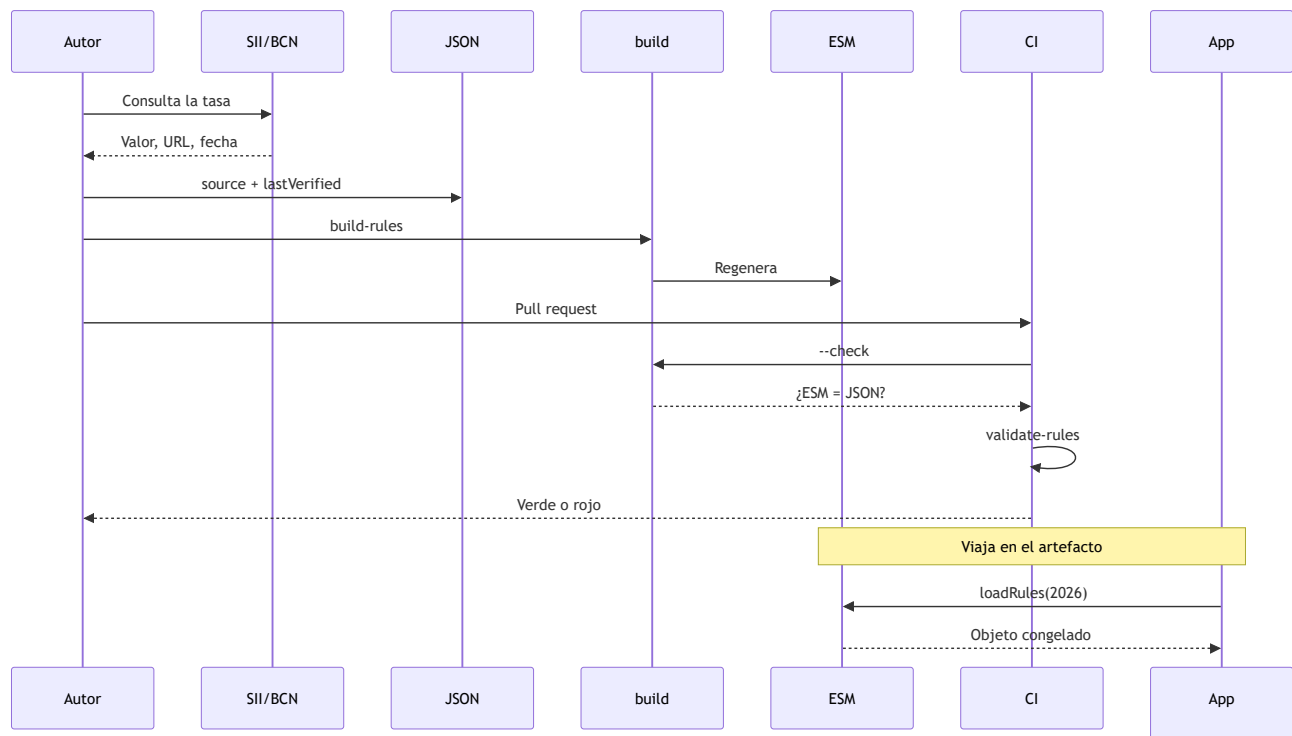
De todos, el único que la aplicación puede resolver por sí sola es el respaldo: `SECURITY.md` insiste en exportar al cerrar cada período, y ese consejo es la contrapartida real de no tener servidor.

9. Qué datos personales se procesan

RUT de la empresa y de terceros, nombres y RUT de accionistas, montos, folios, descripciones de operaciones y comuna. Todos **en claro**, todos **sólo en el dispositivo**, ninguno enviado a nadie. El detalle y sus implicaciones están en 07 y 11.

Flujo 2 · El ciclo de vida de una regla tributaria

El otro flujo, el que no toca al usuario.



Las tres puertas que protegen este flujo, y qué atrapa cada una:

Puerta	Comando	Qué impide
Sincronía	<code>build-rules.mjs --check</code>	Que alguien edite el <code>.json</code> y olvide regenerar, o que edite el generado a mano
Cordura	<code>validate-rules.mjs</code>	Un error de dedo: un 19 donde iba 0.19, una tasa fuera de rango
Existencia	<code>loadRules(year)</code>	Calcular con las tasas del año equivocado

Y la garantía que no está en el flujo, sino en el dato: cada tasa lleva `source` y `lastVerified`. La aplicación no sólo sabe cuánto es el IVA — sabe **de dónde lo sacó y cuándo lo comprobó alguien**. Un cierre anual se lleva esa huella dentro (`legalRulesVersion`), de modo que la trazabilidad sobrevive a que el archivo de reglas cambie después.

Lo que este flujo no tiene: ninguna comprobación automática contra la fuente. Nadie despierta un lunes a preguntarle al SII si la tasa cambió; `lastVerified` es una afirmación humana. **REQUIERE VALIDACIÓN** permanente: las tasas de 2026 hay que reverificarlas antes de la Operación Renta 2027, y el propio archivo lo dice.

Flujo 3 · Los datos que generan la documentación

Un tercer flujo, menor pero real: parte de la documentación del repositorio **no se escribe, se deriva del código**.

Fuente	Script	Salida	Gate en CI
packages/glossary/	build-glossary.mjs	docs/GLOSSARY.md	--check
packages/onboarding/	build-guide.mjs	docs/EMPEZAR-AQUI.md/.html/.pdf	--check
packages/shortcuts/	build-shortcuts.mjs	docs/ATAJOS-DE-TECLADO.md	--check
docs/presentacion.md	build-presentation.mjs	Diapositivas HTML y PDF	--check + check-presentation.mjs
docs/MANUAL.md + capturas	build-manual.mjs	docs/MANUAL.html/.pdf	—

Por qué importa para el flujo de datos: si alguien edita docs/GLOSSARY.md a mano, CI falla. El documento no es la fuente; el paquete lo es. Es el mismo principio que las reglas tributarias aplicado a las palabras: un texto editado en dos sitios acaba diciendo dos cosas.

Un ejemplo completo, de principio a fin

Registrar una venta de \$100.000 netos el 18 de julio de 2026, en Windows:

#	Qué ocurre	Dónde	Dato
1	La vista propone el IVA al escribir el neto	<code>views/operacion.es.js</code>	<code>19000</code>
2	Se envía el formulario	<code>views/operacion.es.js</code>	<code>{ kind: 'sale', date: '2026-07-18', net: 100000, vat: 19000 }</code>
3	Tipo válido, fecha con formato, período <code>2026-07</code> no cerrado	<code>addTransaction</code>	pasa
4	Se completan los valores por defecto	<code>addTransaction</code>	<code>deductible: true , vatCreditEligible: true , total: 119000 , id , createdAt</code>
5	Se escribe la colección completa	<code>createWebStore.write</code>	<code>empresa-operativa-chile:real:transactions</code>
6	Se agrega la línea de bitácora	<code>audit()</code>	<code>transaction.added</code> con id, kind, total y período
7	Se agenda el espejo	<code>createPlatformStore</code>	a los 180 ms
8	Rust valida el JSON y escribe atómicamente	<code>save_workspace</code>	<code>real.json.tmp</code> → <code>real.json</code>
9	La vista se redibuja leyendo del almacén	<code>render()</code>	la operación aparece en la lista
10	El mes cambia sin que nadie lo recalcule	<code>periodSummary('2026-07')</code>	<code>salesNet: 100000 , debitVat: 19000</code>
11	El borrador del F29 cambia con él	<code>f29Basic()</code>	débito fiscal <code>19000</code> , menos el crédito del mes y el remanente que llegue

Once pasos, cero llamadas de red y dos archivos escritos. Ese es el sistema entero.

09 · APIs e integraciones

La conclusión, primero

El sistema no expone ninguna API y no consume ningún servicio externo. No hay endpoints REST, ni GraphQL, ni gRPC, ni webhooks, ni claves de API, ni proveedores, ni reintentos, ni límites de tasa, ni SDK de terceros. No hay integración con el SII, con ninguna municipalidad, con ningún banco ni con ningún emisor de DTE.

Eso no significa que no haya **interfaces**. Hay cinco, y este documento las documenta como lo que son: contratos con consumidores reales, aunque ninguno viaje por la red.

#	Interfaz	Tipo	Consumidor
1	Servidor local de archivos	HTTP <code>GET</code> / <code>HEAD</code> sobre <code>127.0.0.1</code>	El navegador del usuario
2	Comandos Tauri	IPC proceso ↔ WebView	La aplicación de Windows
3	Puente Capacitor	Bridge nativo	La aplicación de Android
4	CLI	Línea de comandos, salida JSON	Personas y scripts
5	Contratos de datos	Formatos de archivo	RespalDOS, reglas y escenarios

La demostración de que no hay red

No es una afirmación de confianza: es comprobable, y estos son los comandos.

```
grep -rn "fetch(\|XMLHttpRequest\|sendBeacon\|WebSocket\|EventSource" apps/web/src/ packages/
```

Resultado real, ejecutado el 27-08-2026: una sola coincidencia, `apps/web/src/sw.js:51` — el `fetch` del service worker, dentro de un manejador que **descarta explícitamente todo lo que no sea del propio origen**:

```
if (url.origin !== self.location.origin) return;
```

Es decir: la única llamada de red del sistema sirve para cachear sus propios archivos, y se niega a tocar nada ajeno.

Y las cuatro barreras que lo sostienen, cada una en una capa distinta:

Barrera	Dónde	Qué impide
CSP <code>default-src 'self'; connect-src 'self'</code>	Cabecera de <code>server.mjs</code>	Que el navegador acepte una conexión saliente
CSP en <code>tauri.conf.json</code>	<code>app.security.csp</code>	Lo mismo dentro de la WebView de Windows
<code>CapacitorHttp.enabled: false</code>	<code>capacitor.config.json</code>	Que el puente nativo de Android haga peticiones
<code>permissions: ["core:default"]</code>	<code>capabilities/default.json</code>	Que Rust exponga red, shell o FS genérico

Un hueco real, y conviene decirlo aquí. La CSP la ponen el servidor local y Tauri. ****La versión publicada en GitHub Pages no tiene ninguna****: Pages no permite cabeceras propias y `apps/web/src/index.html` **no lleva** `<meta http-equiv="Content-Security-Policy">`. Se verificó buscando `http-equiv` en todo `apps/web/src/`: no hay ninguna coincidencia. La app sigue sin hacer llamadas de red, pero ahí la garantía la da sólo el código, no una política que el navegador haga cumplir. Se recoge en [11 · Seguridad](#) y en [15 · Riesgos](#).

1 · Servidor local de archivos

`apps/empresa-operativa/server.mjs` , 101 líneas, `node:http` puro. **No es una API**: no hay rutas, no hay recursos, no hay verbos de escritura. Sirve `apps/web/dist` y nada más.

Aspecto	Valor
Host	<code>127.0.0.1</code> (variable <code>HOST</code>) — nunca <code>0.0.0.0</code>
Puerto	<code>4180</code> (variable <code>PORT</code>)
Métodos	<code>GET</code> y <code>HEAD</code> . Cualquier otro devuelve 405 con cabecera <code>allow</code>
Autenticación	Ninguna. No la necesita: sólo escucha en la interfaz de loopback
Formato	Estático: HTML, JS, MJS, CSS, JSON, webmanifest, SVG, PNG, ICO, PDF
Arranque	Si no existe <code>dist/index.html</code> , imprime la instrucción del build y sale con código 1

Códigos de estado

Código	Cuándo	Cuerpo
<code>200</code>	El archivo existe y es un archivo regular	El archivo
<code>404</code>	No resuelve, no existe, o no es un archivo	<code>No encontrado</code>
<code>405</code>	Método distinto de <code>GET</code> / <code>HEAD</code>	<code>Method Not Allowed</code>

Cabeceras de respuesta

Cabecera	Valor	Por qué
<code>content-type</code>	Según extensión; <code>charset=utf-8</code> sólo en los formatos textuales	Declararlo en un PNG o un PDF es ruido
<code>cache-control</code>	<code>no-cache</code>	La app se actualiza al recargar
<code>content-security-policy</code>	<code>default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; connect-src 'self'; form-action 'none'; base-uri 'none'; frame-ancestors 'self'</code>	Convierte cualquier dependencia externa futura en un error visible
<code>x-content-type-options</code>	<code>nosniff</code>	
<code>referrer-policy</code>	<code>no-referrer</code>	

`frame-ancestors 'self'` y no `'none'` está comentado en el código: la aplicación muestra la guía ilustrada dentro de un marco del **mismo** origen. Sigue impidiendo que un sitio externo la enmarque.

La función que importa: `resolveSafe(urlPath)`

Es el único punto del sistema que traduce entrada de red a una ruta del disco, y por eso es ■ **crítica**. Decodifica, normaliza y **exige el separador** al comparar:

```
if (full !== distDir && !full.startsWith(distDir + path.sep)) return null;
```

El `+ path.sep` no es cosmético: sin él, un directorio hermano llamado `dist-privado` pasaría el filtro. CI lo prueba en cada push con tres vectores — `../../package.json`, la variante `%2e%2e%2f` y la mixta `..%2f` — y falla si alguno devuelve 200.

Ejemplo seguro:

```
curl -sf http://127.0.0.1:4180/core/accounting-engine/index.mjs -o motor.mjs
curl -s -o /dev/null -w '%{http_code}\n' 'http://127.0.0.1:4180/%2e%2e%2f%2e%2e%2fpackage.json'
```

La segunda línea debe imprimir `404`.

2 · Comandos Tauri (IPC)

`apps/contador-desktop/src-tauri/src/lib.rs` . Cuatro comandos y ni uno más; la lista se declara en `invoke_handler` y la capacidad concede sólo `core:default` , así que no hay red, ni shell, ni sistema de archivos genérico.

Se invocan desde la WebView con `globalThis.__TAURI__.core.invoke(nombre, args)` .

`load_workspace(mode) → Option<String>`

Campo	Valor
Parámetros	<code>mode: String</code> — <code>real</code> o <code>sandbox</code>
Retorno	El contenido del espejo, o <code>null</code> la primera vez
Errores	<code>modo desconocido: <x></code> ; error de E/S como texto
Efectos	Crea el directorio de datos si no existe
Quién lo llama	<code>hydrateFromDisk()</code> , una sola vez al arrancar

`save_workspace(mode, payload) → ()`

Campo	Valor
Parámetros	<code>mode: String</code> ; <code>payload: String</code> — JSON serializado del volcado de <code>localStorage</code>
Retorno	Nada
Errores	<code>payload inválido: <detalle></code> si no parsea; <code>modo desconocido</code> ; errores de E/S
Efectos	Escribe <code><app_data_dir>/<mode>.json</code> de forma atómica : temporal + <code>rename</code>
Quién lo llama	La envoltura de <code>createPlatformStore</code> , 180 ms después de cada escritura

Dos decisiones defensivas explícitas en el código: el JSON se valida **antes** de tocar el disco —si la WebView manda basura, mejor fallar que pisar un espejo bueno—, y la escritura es atómica —un corte de luz deja el archivo anterior intacto en vez de un JSON truncado.

`export_file(filename, contents) → String`

Campo	Valor
Parámetros	<code>filename: String</code> ; <code>contents: String</code>
Retorno	Ruta absoluta donde quedó el archivo
Errores	<code>nombre de archivo inválido</code>
Efectos	Escribe en <code>Documentos/Empresa Operativa Chile</code> , creándola si hace falta

El nombre viene de la WebView, así que se sanea: se toma sólo el componente final del path, y se rechaza vacío o empezando por punto. Sin diálogo nativo a propósito — una carpeta fija y predecible es justamente lo que se quiere de un respaldo.

app_info() → JSON

Devuelve { version, dataDir }. Alimenta la pestaña **Datos**, que es donde la aplicación le dice a la persona dónde están sus archivos.

El contrato de seguridad de esta interfaz

Control	Implementación	Prueba
mode no puede ser una ruta	safe_mode() con match sobre dos literales	un_modos_con_ruta_se_rechaza en lib.rs
filename no puede escapar	file_name() + rechazo de vacío y oculto	REQUIERE VALIDACIÓN : sin prueba propia
El payload no corrompe el espejo	serde_json::from_str antes de escribir	REQUIERE VALIDACIÓN : sin prueba propia
Superficie mínima	permissions: ["core:default"]	Declarativo

Las dos pruebas de Rust existentes (solo_se_aceptan_los_dos_modos , un_modos_con_ruta_se_rechaza) no se ejecutaron en esta máquina por falta de toolchain; las corre desktop.yml con cargo test --release .

3 · Puente Capacitor (Android)

`apps/android/capacitor.config.json` . La aplicación **es** la web: no hay código Java ni Kotlin propio.

Clave	Valor	Consecuencia
<code>appId</code>	<code>cl.vladimiracuna.empresaoperativa</code>	El mismo identificador que Windows — hay una prueba que lo verifica
<code>webDir</code>	<code>www</code>	Lo llena <code>copiar-www.mjs</code> desde <code>apps/web/dist</code>
<code>android.allowMixedContent</code>	<code>false</code>	Nada de contenido inseguro
<code>android.captureInput</code>	<code>true</code>	Teclado gestionado por la WebView
<code>android.webContentsDebuggingEnabled</code>	<code>false</code>	La WebView no es inspeccionable en el APK publicado
<code>server.androidScheme</code>	<code>https</code>	El origen es <code>https://localhost</code> , lo que habilita las APIs que exigen contexto seguro
<code>plugins.CapacitorHttp.enabled</code>	<code>false</code>	El puente HTTP nativo está apagado

El **único plugin** que la aplicación usa en tiempo de ejecución es `Browser` , y sólo para una cosa:

```
if (PLATFORM === 'android' && globalThis.Capacitor?.Plugins?.Browser) {  
  globalThis.Capacitor.Plugins.Browser.open({ url });  
}
```

Abrir un portal oficial **fuera** de la app, para no dejar a la persona atrapada dentro de la WebView. No se le pasa ningún dato del usuario: sólo la URL pública.

4 · La CLI como interfaz

`apps/contador-cli/src/index.mjs` . Es la interfaz más parecida a una API que tiene el sistema:

entrada por argumentos, salida JSON por `stdout` , errores por `stderr` , código de salida significativo. Se puede encadenar con `jq` sin ceremonia.

Contrato general

Aspecto	Valor
Salida correcta	<code>JSON.stringify(valor, null, 2)</code> por <code>stdout</code>
Error	<code>error: <mensaje></code> por <code>stderr</code> , código 1
Comando desconocido	Ayuda por <code>stderr</code> , código 1
RUT inválido	JSON con <code>valid: false</code> y <code>exitCode = 1</code> — el único comando que informa por las dos vías
Excepción del motor	Capturada en el <code>try</code> global y convertida en <code>error: <mensaje></code>
Año	<code>--anio</code> , por defecto 2026. Un año sin reglas falla , no degrada

Comandos de cálculo — no tocan el disco

Comando	Argumentos	Devuelve	
venta	--neto	Neto, IVA y total	
compra	--neto	Idem, del lado del crédito	
honorario	--bruto	Bruto, retención y líquido	
f29	--ventas-netas --compras-netas [-remanente] [--honorarios]	Borrador del F29	
vencimientos	--periodo YYYY-MM	Los tres vencimientos	
patente	--capital [--tasa] [--utm]	Estimación rápida sobre una cifra dada	
patente-municipal	`--etapa nueva\	funcionamiento` + capital, CPT, deducciones, tasa, UTM	Patente decidiendo la base según el art. 24
cpt	--regimen + activos, pasivos, movimientos, base imponible...	CPT por el método que corresponda	
idpc	--ingresos --gastos [--ajustes]	IDPC Pro Pyme	
asiento	--tipo --monto	Asiento contable	
escenario	archivo.json	Simulación completa	
rut	76.123.456-7	Validación con dígito verificador	
reglas	[--anio]	El objeto de reglas completo	
glosario	[término]	Términos por categoría, o la búsqueda	

Comandos de espacio de trabajo — escriben en disco

Comando	Argumentos	Efecto
<code>registrar</code>	<code>--fecha --tipo --descripcion --neto [--iva] [--documento] [--rut] [--pagado]</code>	Alta de operación. Si no se da <code>--iva</code> , lo calcula con la tasa del año
<code>operaciones</code>	<code>[--periodo]</code>	Lista
<code>resumen</code>	<code>--periodo</code>	<code>periodSummary</code> + remanente entrante + borrador F29
<code>obligaciones</code>	—	Lista
<code>cerrar</code>	<code>--periodo</code>	Cierra el mes con <code>checklist: { cli: true }</code>
<code>bitacora</code>	<code>[--limite 50]</code>	Últimas líneas de auditoría
<code>exportar</code>	<code>[--salida archivo.json]</code>	Respaldo por <code>stdout</code> o a archivo
<code>importar</code>	<code>archivo.json [--fusionar]</code>	Restaura; sin <code>--fusionar</code> reemplaza

Opciones globales: `--sandbox` (trabaja sobre el entorno de práctica, sembrándolo si hace falta) y `--datos DIR`.

Dónde escribe

Por orden de precedencia: `--datos DIR` → variable de entorno `EMPRESA_OPERATIVA_DATA` → `./.local-data` relativo al directorio actual. Ese último valor por defecto está cubierto por `.gitignore`, lo que no es casualidad: es la ruta por la que se subiría contabilidad real sin darse cuenta.

Ejemplos seguros (cifras inventadas, ningún dato real):

```
node apps/contador-cli/src/index.mjs venta --neto 100000
node apps/contador-cli/src/index.mjs f29 --ventas-netas 1000000 --compras-netas 300000
node apps/contador-cli/src/index.mjs rut 76.000.000-0
node apps/contador-cli/src/index.mjs registrar --sandbox --tipo sale --fecha 2026-07-18 --neto 100000
node apps/contador-cli/src/index.mjs resumen --sandbox --periodo 2026-07
```

Las cuatro primeras familias las ejercita CI en cada push, comprobando valores concretos: `"iva": 19000`, `"valid": true`, `"vatPayable": 133000` y que los vencimientos caigan en `2026-09`.

5 · Contratos de datos

Formatos de archivo con consumidores fuera del proceso que los escribe. Son APIs, aunque el transporte sea un archivo.

Respaldo — `empresa-operativa-chile/backup`

Campo	Valor
<code>format</code>	<code>empresa-operativa-chile/backup</code> — su ausencia rechaza el archivo
<code>formatVersion</code>	1 o 2. Cualquier otro valor lanza
<code>mode</code> , <code>exportedAt</code>	Procedencia
Datos	<code>company</code> , <code>formation</code> , <code>transactions</code> , <code>obligations</code> , <code>closedPeriods</code> , <code>periodCloses</code> , <code>equityMovements</code> , <code>annualCloses</code> , <code>municipalProfile</code> , <code>audit</code>

Es el contrato que hace portables las cuatro superficies. Un respaldo exportado desde Android se importa en Windows, y la CLI lee lo que produjo el teléfono. La compatibilidad hacia atrás con v1 es deliberada: romper los respaldos que la gente ya tiene guardados sería peor que limpiar el formato.

Reglas — `rules/<año>.json`, `schemaVersion: 2`

Documentado en [07 · Persistencia](#). Su contrato con el resto del sistema es `loadRules`, `availableYears` y `ruleProvenance`.

Escenario de simulación — `data/scenarios/*.json`

Entrada de `simulateScenario()` y del comando `escenario`. Hay **un** archivo en el repositorio.

`build-info.json`

Lo genera `build-web.mjs` y lo consume la pestaña **Datos** para mostrar qué versión se está usando. CI lo compara entre dos builds seguidos para verificar que el build es reproducible.

Integraciones externas

Ninguna, en el sentido técnico. Lo que sí existe es una lista de **enlaces salientes** a portales oficiales, que la aplicación abre en el navegador del sistema sin pasarles ningún dato:

Destino	Para qué	Dónde está declarado
<code>sii.cl</code> (10 URL distintas)	Inicio de actividades, regímenes, IVA, UTM, honorarios, F29, factura electrónica, circular 62 de 2020	<code>FORMATION_STEPS</code> , <code>rules/2026.json</code> , glosario
<code>registrodeempresasy sociedades.cl</code>	Constituir la sociedad	<code>FORMATION_STEPS</code>
<code>bcn.cl/leychile</code>	D.L. 3.063 sobre Rentas Municipales	<code>rules/2026.json</code>
14 sitios municipales	Consultar la tasa de la comuna	<code>packages/chile-tax-rules/municipalities.mjs</code>

Todos son destinos de lectura para la persona, no endpoints que el software consulte. La aplicación nunca los llama: los ofrece.

Y una nota que el propio archivo de reglas se encarga de dejar clara: el flujo por el que el SII comunica a las municipalidades el capital propio declarado **está modelado como conocimiento**, no implementado como integración. `rules/2026.json` lo dice literalmente: esta aplicación no está conectada al SII ni a ninguna municipalidad — sólo modela el flujo.

Lo que no existe

NO IDENTIFICADO — se buscó y no hay evidencia en ningún sentido:

- Endpoints REST/GraphQL/gRPC propios, versionado de API, OpenAPI o esquema publicado.
 - Webhooks, colas, reintentos, *backoff*, circuit breakers, límites de tasa.
 - Autenticación de API, tokens, claves, OAuth, firma de peticiones.
 - SDK o cliente de terceros. **Cero dependencias de producción**, verificado en CI en cada push.
 - Integración con SII, municipalidades, bancos, emisores de DTE o pasarelas de pago.
 - Telemetría, analítica, informe de errores o comprobación de actualizaciones.
-

10 · Configuración

La conclusión, primero

No hay archivo de configuración de la aplicación. No hay `.env`, no hay `config.json`, no hay `settings.yml`, no hay banderas de funcionalidad y no hay perfiles de entorno. `.gitignore` cubre `.env` y `.env.*`, pero no existe ningún `.env.example` ni ningún código que lea uno: la cobertura es preventiva, no la señal de un archivo que falte.

Seis variables de entorno existen en todo el repositorio, y ninguna la lee la aplicación en el dispositivo: cuatro son de herramientas de build y dos del servidor local. Ninguna es un secreto y ninguna es obligatoria.

Lo que sí hay son **manifiestos** —de la PWA, de Capacitor, de Tauri y de npm— y ahí es donde vive la configuración real del producto.

Variables de entorno

Se obtuvieron recorriendo `apps/web/src` , `apps/contador-cli` , `apps/empresa-operativa` , `scripts` y `packages` . La lista es exhaustiva.

Variable	Quién la lee	Por defecto	Obligatoria	Para qué
<code>EMPRESA_OPERATIVA_DATA</code>	<code>apps/contador-cli/src/index.mjs</code>	<code>./.local-data</code>	No	Directorio de datos de la CLI
<code>PORT</code>	<code>apps/empresa-operativa/server.mjs</code>	<code>4180</code>	No	Puerto del servidor local
<code>HOST</code>	<code>apps/empresa-operativa/server.mjs</code>	<code>127.0.0.1</code>	No	Interfaz de escucha
<code>CHROME_PATH</code>	<code>scripts/lib/chrome.mjs</code>	Autodetección	No	Ruta a Chrome o Edge para generar PDF y capturas
<code>CAPTURE_URL</code>	<code>scripts/capture-screenshots.mjs</code>	<code>http://127.0.0.1:4180</code>	No	Origen desde el que se capturan las pantallas
<code>EOC_KEEP_HTML</code>	<code>scripts/lib/chrome.mjs</code>	(sin definir)	No	Depuración: conserva el HTML intermedio de un PDF e imprime su ruta

Ninguna contiene un secreto. No hay claves de API, tokens, contraseñas ni cadenas de conexión en todo el repositorio, porque no hay nada a lo que conectarse. `security.yml` lo comprueba en cada push buscando certificados y claves privadas versionados.

⚠ **HOST** es la única variable peligrosa. Su valor por defecto es `127.0.0.1` por una decisión explícita del código: son datos contables de una empresa real, y no tienen por qué quedar expuestos al resto de la red local porque alguien levantó la app en un café. Ponerla en `0.0.0.0` publica la contabilidad a **toda la red**, sin autenticación de ningún tipo, porque el servidor no tiene ninguna. Es la configuración incorrecta más cara de este sistema.

Precedencia del directorio de datos de la CLI, de mayor a menor:

```
--datos DIR → EMPRESA_OPERATIVA_DATA → ./.local-data
```

`./.local-data` está en `.gitignore` a propósito: es la ruta por la que se subiría contabilidad real al repositorio sin darse cuenta.

Manifiestos

package.json (raíz)

Clave	Valor	Consecuencia
version	1.4.0	La leen el generador de PDF y las portadas de los documentos
type	module	Todo el repositorio es ESM. No hay require
private	true	No se publica en npm
engines.node	>=20	CI prueba en 20 y 22
packageManager	pnpm@11.2.2	El gestor del repositorio. pnpm/action-setup instala esa versión exacta
dependencies	ausente	Cero dependencias de producción, verificado en CI
devDependencies	ausente	No hay herramientas instalables: los scripts son Node puro

pnpm no es una preferencia, es el contrato. pnpm-lock.yaml es el lockfile válido y los tres workflows que instalan algo usan pnpm install --frozen-lockfile. Escribir npm install ignoraría el lockfile.

Hallazgo: apps/android/ tiene versionados ** pnpm-lock.yaml y package-lock.json a la vez**. El workflow usa pnpm, así que el package-lock.json no lo consume nadie, pero su presencia contradice la regla del repositorio y puede inducir a alguien a instalar con npm. Se recoge en [15 · Riesgos](#).

apps/web/src/manifest.webmanifest — la PWA

Clave	Valor	Por qué importa
start_url / scope	./index.html / ./	Relativos: la app funciona igual en la raíz de un dominio y en un subdirectorio de Pages
display	standalone	Sin barra de navegador
lang	es-CL	
background_color / theme_color	#0b0f16	Coinciden con el theme-color de index.html
icons	4 entradas: SVG, 192, 512 y maskable 512	Una prueba verifica que existan los archivos que el manifiesto promete
shortcuts	#operaciones y #impuestos	Mantener pulsado el icono en Android; funcionan gracias a applyUrlState()

apps/android/capacitor.config.json

Documentado en [09 · APIs e integraciones](#). Lo esencial: `CapacitorHttp` apagado, depuración de la WebView apagada, contenido mixto prohibido, esquema `https`.

apps/contador-desktop/src-tauri/tauri.conf.json

Clave	Valor	Consecuencia
<code>productName</code>	<code>Empresa Operativa Chile</code>	Nombre del ejecutable y del instalador
<code>version</code>	<code>1.4.0</code>	Duplicado del <code>package.json</code> raíz — ver más abajo
<code>identifier</code>	<code>cl.vladimiracuna.empresaoperativa</code>	Determina <code>app_data_dir()</code> . Cambiarlo deja los datos existentes huérfanos
<code>build.frontendDist</code>	<code>../../web/dist</code>	La interfaz embebida es la misma que la web
<code>app.withGlobalTauri</code>	<code>true</code>	Expone <code>globalThis.__TAURI__</code> , que es como <code>platform.js</code> detecta Windows
<code>app.windows[0]</code>	1320×860, mínimo 940×620, centrada	
<code>app.security.csp</code>	<code>default-src 'self'; ... connect-src 'self' ipc: http://ipc.localhost; form-action 'none'; base-uri 'none'</code>	<code>ipc:</code> es lo único añadido respecto del servidor: sin eso, la WebView no podría hablar con Rust
<code>bundle.targets</code>	<code>msi, nsis</code>	Más el portable, que sale del <code>target/release</code>
<code>bundle.windows.wix.language</code>	<code>es-ES</code>	
<code>bundle.windows.wix.nsis</code>	Español e inglés, <code>installMode: both</code>	Instalación por usuario o por máquina

Diferencia entre las dos CSP, y por qué existe: la del servidor incluye `frame-ancestors 'self'` —necesaria porque la guía ilustrada se muestra en un marco del mismo origen— y la de Tauri incluye `connect-src ipc: http://ipc.localhost`, sin la cual no habría comunicación con el proceso Rust. Ninguna de las dos admite un origen externo.

apps/contador-desktop/src-tauri/capabilities/default.json

Nueve líneas y una decisión: `"permissions": ["core:default"]`. Sin `fs`, sin `shell`, sin `http`, sin `dialog`. Todo el acceso a disco pasa por los cuatro comandos de `lib.rs`, que validan sus argumentos. Es el archivo de configuración más importante del sistema desde el punto de vista de seguridad, y ocupa menos que este párrafo.

Cargo.toml

`rust-version = "1.77"` como mínimo declarado. CI instala el canal `stable`.

.gitattributes y .gitignore

`.gitignore` cubre tres familias, y la tercera es la que importa:

1. **Dependencias:** `node_modules/`, `.pnpm-store/`.

2. **Artefactos de build:** `apps/web/dist/`, `apps/android/www/`, `apps/android/android/`,
`src-tauri/target/`, `src-tauri/gen/`, `dist/`, `artefactos/`.

1. **Datos operativos locales:** `.local-data/`, `private-data/`, `real-company-data/`, `*.sqlite*`,
`empresa-operativa-real-*.json` y `.csv`.

El comentario del propio archivo lo dice: **nunca subir contabilidad real**. Es la misma preocupación que `security.yml` y que `SECURITY.md`.

Configuración de CI

Los seis workflows no leen ningún secreto del repositorio. Usan sólo el `GITHUB_TOKEN` implícito, con permisos declarados **por trabajo** y al mínimo:

Workflow	<code>permissions</code> por defecto	Elevaciones
<code>ci.yml</code>	<code>contents: read</code>	Ninguna
<code>android.yml</code>	<code>contents: read</code>	Ninguna
<code>desktop.yml</code>	<code>contents: read</code>	Ninguna
<code>pages.yml</code>	<code>contents: read</code>	<code>pages: write</code> , <code>id-token: write</code> en el trabajo de publicación
<code>release.yml</code>	<code>contents: read</code>	<code>contents: write</code> sólo en el trabajo <code>publicar</code>
<code>security.yml</code>	<code>contents: read</code>	<code>security-events: write</code> en CodeQL

Todas las acciones están fijadas a un SHA de 40 caracteres, y hay un paso en `ci.yml` que falla si alguien añade una sin fijar. Una etiqueta móvil puede cambiar de contenido bajo los pies; un SHA no.

`package-manager-cache: false` aparece en `ci.yml` y `pages.yml` con su razón escrita al lado: el proyecto no tiene dependencias, y con el caché activo `setup-node` invoca el gestor declarado en `packageManager` sólo para calcular la ruta del store — lo que rompe el trabajo dos veces, porque pnpm 11 no arranca en Node 20 y en Windows el paso de guardado falla porque ese store no existe. Es exactamente el problema que arreglaron los dos últimos commits del repositorio.

Diferencias entre entornos

No hay entornos. No hay `development` , `staging` ni `production` , ni ninguna variable que los distinga. Lo que existe es **una separación de datos dentro de la aplicación**, que es una cosa distinta y más útil:

	EMPRESA REAL	SANDBOX
Prefijo de almacén	<code>empresa-operativa-chile:real</code>	<code>empresa-operativa-chile:sandbox</code>
Se siembra con datos de ejemplo	Nunca — <code>seedSandboxWorkspace</code> lanza si el modo no es sandbox	Sí, si está vacío
Se puede seleccionar por URL	<code>?modo=real</code>	<code>?modo=sandbox</code>
Espejo en Windows	<code>real.json</code>	<code>sandbox.json</code>

No hay ninguna ruta de código que copie una operación de un modo al otro. La separación se consigue dando a cada modo su propio almacén, y es el invariante central del producto.

Lo más parecido a una configuración de entorno son tres parámetros de URL, que existen sobre todo para que las capturas del manual se generen de forma reproducible:

Parámetro	Valores	Efecto
<code>#vista</code>	<code>panel</code> , <code>operaciones</code> , <code>impuestos</code> , ...	Abre esa vista directamente
<code>?modo</code>	<code>real</code> , <code>sandbox</code>	Elige el espacio de trabajo
<code>?tema</code>	<code>claro</code> , <code>oscuro</code>	Fuerza el tema
<code>?periodo</code>	<code>YYYY-MM</code>	Sitúa el período

La versión, y el único punto donde se puede desincronizar

1.4.0 aparece en dos archivos que nadie sincroniza automáticamente:

Archivo	Clave	Quién la usa
package.json	version	Generadores de documentos, portadas de PDF
apps/contador-desktop/src-tauri/tauri.conf.json	version	Instaladores, app_info()

Hoy coinciden — se comprobó. **No hay ninguna comprobación en CI que lo garantice**, así que un bump que olvide el segundo archivo produciría instaladores rotulados con una versión distinta de la que dice la documentación. Se recoge en [15 · Riesgos](#).

REQUIERE VALIDACIÓN: la versión que muestra la aplicación de escritorio sale de `app.package_info().version`, es decir de `tauri.conf.json`, no de `package.json`. No se ejecutó la app para confirmarlo visualmente.

Consecuencias de una configuración incorrecta

Cambio	Consecuencia	Gravedad
<code>HOST=0.0.0.0</code>	La contabilidad queda servida a toda la red local, sin autenticación	 Crítica
Cambiar <code>identifier</code> en <code>tauri.conf.json</code>	<code>app_data_dir()</code> cambia y los datos existentes quedan huérfanos	 Crítica
Editar <code>rules.generated.mjs</code> a mano	CI falla en <code>build-rules.mjs --check</code> . Si se ignorase, el sistema calcularía con tasas que ningún JSON respalda	 Crítica
Añadir un permiso a <code>capabilities/default.json</code>	Amplía la superficie de la app de escritorio en silencio	 Alta
Poner <code>webContentsDebuggingEnabled: true</code>	La WebView del APK publicado queda inspeccionable	 Alta
Añadir una dependencia de producción	<code>security.yml</code> falla. Es un gate, no un aviso	 Media
Cambiar <code>start_url</code> a una ruta absoluta	La PWA deja de funcionar en el subdirectorio de GitHub Pages	 Media
Desincronizar las dos <code>version</code>	Instaladores rotulados con una versión que no es	 Media
<code>PORT</code> ocupado	El servidor no arranca; ver 14 · Troubleshooting	 Baja
<code>CHROME_PATH</code> mal	Fallan los generadores de PDF, no la aplicación	 Baja

Lo que no existe

NO IDENTIFICADO — se buscó y no hay evidencia en ningún sentido:

- `.env`, `.env.example`, `config.json`, `settings.yml` o equivalente.
- Banderas de funcionalidad (*feature flags*), interruptores de despliegue progresivo.
- Configuración remota o descargada en tiempo de ejecución.
- Secretos de repositorio en los workflows, más allá del `GITHUB_TOKEN` implícito.
- Certificado de firma de código para Windows o Android — su ausencia está **declarada** en

`SECURITY.md` y en las notas de cada release, no oculta.

11 · Seguridad

Fuente de verdad de la postura declarada: `SECURITY.md` , que fija el modelo de amenazas, lo que la aplicación garantiza y lo que no, y el canal de reporte. Este documento no lo repite: **verifica en el código** cada garantía que ese documento declara, y añade los controles ausentes que la revisión encontró. Donde ambos hablen de lo mismo, manda `SECURITY.md` .

La conclusión, primero

La superficie de ataque de este sistema es **inusualmente pequeña**, y no por casualidad: no hay servidor, no hay cuentas, no hay red, no hay dependencias de producción y la app de escritorio corre con un solo permiso. Casi todas las categorías de un checklist de seguridad web salen «no aplica», y salen así porque el componente no existe, no porque nadie lo haya mirado.

Lo que queda es un riesgo distinto, y el propio `SECURITY.md` lo ordena bien: **el peligro no es que alguien entre, es que la contabilidad salga**. Subir un respaldo real al repositorio, o publicar un artefacto que calcula mal, hacen más daño que cualquier vulnerabilidad clásica en un sistema sin red.

Lo que esta revisión verificó y confirma: 7 de las 7 garantías de `SECURITY.md` están implementadas como dicen estarlo.

Lo que esta revisión encontró de más: un hueco de CSP en la versión publicada en GitHub Pages, y tres controles ausentes que conviene conocer.

Autenticación, autorización, roles y sesiones

NO IDENTIFICADO , y es una decisión, no un olvido:

Control	Estado	Por qué
Autenticación	No existe	No hay servidor ni cuentas. SECURITY.md : «La aplicación no pide PIN ni contraseña»
Autorización	No existe	No hay múltiples usuarios que separar
Roles y permisos	No existen	Un solo operador por dispositivo
Gestión de sesiones	No existe	No hay sesión: localStorage persiste sin caducidad
Cierre de sesión	No existe	No hay nada que cerrar

La consecuencia, dicha sin adornos: quien tenga el dispositivo desbloqueado tiene la contabilidad. El control de acceso lo aporta el sistema operativo —bloqueo de pantalla, cuenta de usuario, cifrado de disco—, no la aplicación. **SECURITY.md** lo declara en «Qué NO garantiza» y está en el roadmap.

Validación y saneamiento de entradas

Hay dos superficies de entrada y ambas están cubiertas.

Entrada del usuario hacia el motor

Once validaciones, todas lanzando `Error`, listadas en [08 · Flujo de datos](#). Lo relevante desde el punto de vista de seguridad es que **están en el motor y no en la vista**: la CLI y las pruebas entran por ahí sin pasar por ningún formulario, así que la garantía no depende del HTML.

Salida hacia el DOM — escapado por defecto

`apps/web/src/lib/dom.js` define una plantilla `html` que **escapa toda interpolación** — `&`, `<`, `>`, `"` y `'` — salvo que se marque explícitamente con `raw()`.

```
else rendered = esc(v);
```

Por qué importa aquí: los datos de esta app los escribe el usuario —descripciones de gastos, RUT de proveedores, motivos de reapertura— y después se muestran en tablas y en la bitácora. Escapar por defecto y exigir un `raw()` explícito hace que el caso peligroso sea el que hay que teclear a propósito.

Se auditaron los cinco usos de `innerHTML` del repositorio, que son todos los que hay:

Ubicación	Qué asigna	Veredicto
<code>app.js:127</code>	<code>modeBar() + view.render()</code>	Salida de plantillas <code>html</code>
<code>app.js:152</code>	<code>shell()</code>	Idem
<code>lib/dom.js:121</code>	Plantilla <code>html</code>	Idem
<code>lib/shortcuts.js:126</code>	Resultados del buscador de atajos	Datos del propio catálogo, no del usuario
<code>lib/terms.js:90</code>	Tooltip de un término	<code>esc()</code> explícito en los dos campos

No se encontró ninguna asignación de `innerHTML` con datos del usuario sin escapar. **NO IDENTIFICADO**: no hay `eval`, `new Function`, `document.write` ni `insertAdjacentHTML` en el repositorio.

Cifrado

Dato	En reposo	En tránsito
localStorage	Sin cifrar	No hay tránsito
Espejo JSON de Windows	Sin cifrar	No hay tránsito
RespalDOS exportados	Sin cifrar	Lo que el usuario haga con el archivo
Bitácora	Sin cifrar	No hay tránsito

Nada está cifrado, y está declarado. SECURITY.md : «Los datos no están cifrados. Ni en localStorage ni en el espejo de Windows. Quien tenga acceso al dispositivo desbloqueado tiene acceso a la contabilidad.» La contrapartida es que tampoco viaja a ninguna parte, con lo que desaparece toda la clase de riesgos de transporte, de servidor y de proveedor.

INFERENCIA : en Android, localStorage vive dentro del sandbox de la aplicación, así que otra app sin privilegios de root no debería poder leerlo. Eso lo garantiza el sistema operativo, no este código, y no se verificó en dispositivo.

Gestión de secretos

No hay secretos que gestionar. Ni en el código, ni en la configuración, ni en CI —los seis workflows usan sólo el `GITHUB_TOKEN` implícito—, ni en el dispositivo. Es la consecuencia directa de no tener servicios externos.

El repositorio **defiende activamente** esa propiedad. `security.yml`, trabajo `datos`, falla el build si encuentra:

1. un JSON con la firma `"format": "empresa-operativa-chile/backup"` — un respaldo real commiteado;
2. cualquier `.pfx`, `.p12`, `.pem`, `.key`, `.jks` o `.keystore` versionado;
3. archivos bajo `.local-data/`, `private-data/` o `real-company-data/`.

Se ejecutó la misma comprobación sobre el árbol actual en esta revisión: no hay coincidencias. Ningún secreto, certificado, respaldo real ni credencial en el repositorio.

Registro y auditoría

La bitácora `audit` es el control de seguridad más fuerte que tiene el producto:

Propiedad	Estado
Toda mutación queda registrada	✓ Verificado por prueba
Append-only	✓ No existe ninguna operación de borrado en toda la clase
Guarda quién, cuándo, qué y en qué modo	✓ <code>at</code> , <code>mode</code> , <code>action</code> , <code>detail</code>
Reabrir un período o un ejercicio deja el motivo	✓ Y el CPT anterior, en el caso anual
A prueba de manipulación	✗ Es un JSON local: quien controle el dispositivo puede editarlo
Firmada o encadenada por hash	✗ NO IDENTIFICADO
Con retención o rotación	✗ Crece sin techo

La bitácora prueba lo que pasó **frente a un error propio**, que es su caso de uso real, no frente a un adversario con acceso al disco. Conviene no confundir las dos cosas.

NO IDENTIFICADO : no hay registro de eventos de seguridad, ni detección de intrusiones, ni alertas — no hay servidor donde ponerlos.

Superficie de ataque, componente a componente

Componente	Superficie	Controles	Riesgo residual
server.mjs js	HTTP en 127.0.0.1:4180	Sólo GET / HEAD ; resolveSafe ; loopback; CSP; nosniff ; no-referrer	HOST=0.0.0.0 lo expone a la red sin autenticación
App web	Entrada del usuario al DOM	Escapado por defecto; CSP en servidor y Tauri	Sin CSP en GitHub Pages — ver abajo
Service worker	Caché	Descarta todo lo que no sea del propio origen	Ninguno relevante
Comandos Tauri	4 comandos IPC	safe_mode() ; saneo del nombre de archivo; validación del JSON; core:default	Sólo 1 de los 3 saneos tiene prueba
APK	WebView	Depuración apagada; contenido mixto prohibido; CapacitorHttp apagado	Firmado con la clave de debug
CLI	argv y archivos	Las mismas validaciones del motor	Escribe donde le digan; --datos no se restringe
CI	6 workflows	Acciones fijadas a SHA; permisos mínimos por trabajo; sin secretos	Ninguno relevante
Dependencias	Cero en producción	Gate en CI	La superficie de suministro es el propio Node

El hueco: CSP en la versión publicada

Confirmado leyendo el código. La CSP la ponen dos sitios: la cabecera de server.mjs y tauri.conf.json . apps/web/src/index.html no lleva <meta http-equiv="Content-Security-Policy"> — se buscó http-equiv en todo apps/web/src/ y no hay ninguna coincidencia.

GitHub Pages no permite cabeceras propias. Por lo tanto:

Superficie	¿Tiene CSP?
Servidor local (pnpm app)	✅ Cabecera completa
Windows (Tauri)	✅ app.security.csp
Android (Capacitor)	⚠️ REQUIERE VALIDACIÓN — no se comprobó qué política aplica la WebView sin meta
GitHub Pages	❌ Ninguna

Qué significa en la práctica y qué no. La app sigue sin hacer llamadas de red: no hay fetch saliente, no hay recursos de terceros, y el escapado por defecto sigue en pie. La CSP no es lo que impide la fuga; es la red de seguridad que convertiría un futuro descuido en un error visible. En Pages esa red no está.

Mitigación disponible: añadir la política como `<meta http-equiv>` en `index.html` . Se registra como recomendación en [15 · Riesgos](#) y **no se aplicó**: este trabajo documenta, no corrige.

Categorías clásicas

Riesgo	Aplica	Estado
Inyección SQL	No	No hay base de datos ni SQL
Inyección de comandos	Parcial	<code>execFileSync</code> con arreglo de argumentos —nunca <code>exec</code> con cadena— en <code>build-all.mjs</code> y <code>chrome.mjs</code> . Sólo build, nunca la app
XSS	Sí	Escapado por defecto; auditados los 5 <code>innerHTML</code> ; sin <code>eval</code>
Path traversal	Sí	<code>resolveSafe</code> con separador, probado en CI con 3 vectores; <code>safe_mode()</code> y saneo de nombre en Rust
CSRF	No	No hay endpoints con efectos ni cookies. <code>form-action 'none'</code> de todos modos
CORS	No	No hay API que compartir. <code>connect-src 'self'</code>
Clickjacking	Sí	<code>frame-ancestors 'self'</code> en el servidor. No en Pages
Deserialización insegura	Bajo	Sólo <code>JSON.parse</code> con <code>try/catch</code> ; <code>importAll</code> valida <code>format</code> y <code>formatVersion</code>
Carga de archivos	Sí	Un <code><input type="file"></code> que sólo lee texto y lo pasa por <code>importAll</code> . No se ejecuta ni se guarda como archivo
Redirección abierta	No	No hay redirecciones
Exposición de información	Sí	Sin telemetría; <code>no-referrer</code> ; enlaces externos con <code>noopener,noreferrer</code>
Dependencias vulnerables	Bajo	Cero de producción. Las de build (Capacitor, Tauri CLI) sólo corren en el equipo de quien compila
Datos personales	Sí	RUT de terceros y montos, en claro, en el dispositivo. Sin cifrar

Controles presentes — las 7 garantías de **SECURITY.md**, verificadas

Garantía declarada	Implementación verificada			
No hay telemetría	Un solo <code>fetch</code> en todo el código, en <code>sw.js</code> , que descarta lo ajeno al origen	✓		
Los datos no salen del dispositivo	<code>localStorage</code> + espejo local; salida sólo por exportación explícita	✓		
Cero dependencias de producción	<code>package.json</code> sin <code>dependencies</code> ; gate en <code>security.yml</code>	✓		
El servidor no queda expuesto	<code>`host = process.env.HOST \</code>	<code>`</code>	<code>'127.0.0.1'</code>	✓ (salvo que se cambie la variable)
La app de Windows no escribe donde quiera	<code>core:default</code> ; 4 comandos que validan argumentos	✓		
El espejo no se corrompe a medias	Temporal + <code>rename</code> en Rust, y también en <code>node-store</code>	✓		
El texto del usuario no inyecta marcado	Plantilla <code>html</code> con escapado por defecto	✓		

Y cuatro controles adicionales que **SECURITY.md** no menciona y esta revisión encontró:

1. **Acciones de CI fijadas a SHA**, con un paso que falla si alguien añade una sin fijar.
2. **CodeQL** con el conjunto `security-and-quality`, en cada push, cada PR y los lunes a las 06:00 UTC.
1. **Verificación del APK por dentro** (`verify-apk.mjs`): abre el ZIP y cuenta vistas, módulos del motor y reglas antes de publicar. Un APK vacío compila perfectamente.
1. **Build reproducible**: CI construye dos veces y falla si `build-info.json` cambia. Si el artefacto no es reproducible, el que se publica no es el que se probó.

Controles ausentes o no comprobados

Con la misma claridad que la sección anterior.

#	Control ausente	Impacto	Severidad
1	CSP en GitHub Pages	La red de seguridad no está en la superficie más expuesta	 Alta
2	Cifrado en reposo	Dispositivo desbloqueado = contabilidad accesible	 Alta (declarada)
3	Firma de código	SmartScreen y Android avisan; el usuario no puede distinguir un binario alterado sin comparar el <code>SHA256SUMS.txt</code>	 Alta (declarada)
4	APK de depuración	Firmado con la clave de debug; no publicable en Play	 Media (declarada)
5	Bloqueo local (PIN)	Sin barrera ante acceso físico	 Media (declarada)
6	Manejo de <code>QuotaExceededError</code>	Si <code>localStorage</code> se llena, la escritura se pierde en silencio	 Media
7	Pruebas de los saneos de Rust	2 de 3 validaciones de <code>lib.rs</code> sin prueba propia	 Media
8	<code>Subresource Integrity</code>	No aplica: no hay recursos externos	 No aplica
9	Escaneo de dependencias (Dependabot)	No se encontró configuración. Con cero dependencias de producción el impacto es bajo, pero las de build no se vigilan	 Baja
10	Bloqueo entre pestañas	Dos pestañas abiertas: la última escritura gana	 Baja

Los puntos 2, 3, 4 y 5 **están declarados** en `SECURITY.md` y en las notas de cada release: son límites conocidos, no descubrimientos. Los puntos 1, 6, 7, 9 y 10 los añade esta revisión.

Qué se hizo y qué no en esta revisión

Se hizo: lectura completa del código de seguridad relevante; verificación por comando de la ausencia de red, de secretos y de CSP; auditoría de los cinco `innerHTML`; revisión de los seis workflows, de las capacidades de Tauri y de la configuración de Capacitor.

No se hizo, y por qué:

- No se ejecutaron pruebas de penetración, escaneos activos ni ataques contra ningún sistema — el encargo lo prohíbe explícitamente.
 - No se analizaron los binarios publicados: `REQUIERE VALIDACIÓN` con el `SHA256SUMS.txt` del release.
 - No se probó el APK en un dispositivo.
 - No se ejecutó CodeQL localmente; sus resultados viven en la pestaña Security del repositorio y `REQUIEREN VALIDACIÓN` allí.
 - **No se corrigió nada.** Los hallazgos van a [15 · Riesgos](#).
-

Reporte de vulnerabilidades

`SECURITY.md` pide abrir un **Security Advisory privado** en vez de una incidencia pública, e indica qué incluir. Declara además fuera de alcance tres cosas que ya están documentadas: binarios sin firmar, datos locales sin cifrar, y que el borrador F29 no cubra todos los códigos del formulario — esto último es una limitación declarada del producto, no un fallo de seguridad.

12 · Pruebas y calidad

La conclusión, primero

153 pruebas en 13 suites, todas verdes, ejecutadas en esta revisión. Corren con el runner nativo de Node (`node --test`), sin framework, sin *mocks*, sin configuración y en **0,64 segundos**.

Y una segunda capa que importa más de lo que parece: **el repositorio verifica cosas que un test unitario no puede ver**. Que el APK lleve la app dentro. Que el build sea reproducible. Que la documentación no se haya desviado del código. Que ninguna acción de CI esté sin fijar. Esa capa es lo distintivo de este proyecto, y está en `ci.yml` y en los scripts de verificación, no en `tests/`.

El hueco real: la interfaz —las 14 vistas, 4.000 líneas de `apps/web/src/views/` — **no tiene pruebas de comportamiento**. Se verifica su presencia y su contrato, no lo que hace al pulsar un botón.

Comandos ejecutados y su salida real

Todo lo que sigue se ejecutó el **27-08-2026** sobre el commit `2b0e006` , en Windows 11 con Node 22 y pnpm 11.2.2. Ningún número de esta serie de documentos es una estimación.

Suite completa

```
node --test tests/*.test.mjs

i tests 153
i suites 0
i pass 153
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 642.8929
```

Los siete gates de sincronía y validación

Comando	Salida real	r c
<code>node scripts/build-rules.mjs --check</code>	Reglas embebidas sincronizadas (2026).	0
<code>node scripts/build-glossary.mjs --check</code>	Glosario sincronizado (54 términos).	0
<code>node scripts/build-guide.mjs --check</code>	Guía sincronizada (14 etapas, 2 diagramas).	0
<code>node scripts/build-shortcuts.mjs --check</code>	Atajos sincronizados (12).	0
<code>node scripts/build-presentation.mjs --check</code>	Presentación válida: 8 diapositivas, 28 minutos.	0
<code>node scripts/check-presentation.mjs</code>	Presentación: 8 diapositivas (713 KB), ≈28 min, pauta de 5 páginas (726 KB) – coincide con lo anunciado.	0
<code>node scripts/validate-rules.mjs</code>	Reglas 2026 validadas; última verificación 2026-08-09	0

Build y reproducibilidad

```
node scripts/build-all.mjs
```


apps/web/dist listo – 49 archivos, 7256 KB, build 630ed69fe514
Build completo. La aplicación está en apps/web/dist.

Segundo build seguido, comparando `build-info.json` :

BUILD REPRODUCIBLE: 630ed69fe514

```
{ "buildId": "630ed69fe514", "files": 49, "bytes": 7429748, "builtFrom": "apps/web/src + packages/" }
```

La CLI

Comando	Comprobación	Resultado
<code>venta --neto 100000</code>	<code>"iva": 19000</code>	✓
<code>rut 76.000.000-0</code>	<code>"valid": true</code>	✓
<code>f29 --ventas-netas 1000000 --compras-netas 300000</code>	<code>"vatPayable": 133000</code>	✓
<code>vencimientos --periodo 2026-08</code>	Cae en <code>2026-09</code>	✓ <code>2026-09-14</code> , <code>2026-09-21</code> , <code>2026-09-28</code> , dos de ellos con <code>shiftedFromWeekend: true</code>

El servidor local y sus defensas

Arrancado, verificado y detenido en esta revisión:

Comprobación	Resultado
<code>GET /</code> sirve la app	✓ contiene «Empresa Operativa Chile»
<code>GET /core/accounting-engine/index.mjs</code>	✓ contiene <code>f29Basic</code>
<code>GET ../../package.json</code>	404
<code>GET /%2e%2e%2f%2e%2e%2fpackage.json</code>	404
<code>GET /..%2f..%2fpackage.json</code>	404
<code>POST /</code>	405

Los tres vectores de escape son los mismos que ejecuta `ci.yml` ; el `405` lo añade esta revisión.

Las 13 suites

Suite	Pruebas	Qué cubre
<code>municipal-patent.test.mjs</code>	23	Base según etapa del negocio, deducciones, prorrateo, tope y piso en UTM, resolución de tasa y UTM
<code>capital.test.mjs</code>	19	Las seis magnitudes del capital, migración del campo heredado, <code>null</code> frente a <code>0</code> , validación del perfil
<code>tax-equity.test.mjs</code>	17	Elección de método, art. 41 frente a 14 D 3 (j), piso en cero, elegibilidad por régimen
<code>onboarding.test.mjs</code>	17	Las 14 etapas de la ruta, fases, pasos con trámite
<code>ayuda.test.mjs</code>	16	Términos, tooltips y coherencia de la ayuda contextual
<code>workspace.test.mjs</code>	14	Inmutabilidad del período, evidencia obligatoria, aislamiento real/sandbox, bitácora, export/import
<code>glossary.test.mjs</code>	11	Los 54 términos, categorías, búsqueda, «no confundir con»
<code>f29.test.mjs</code>	8	Arrastre del remanente, los dos modos de <code>f29Basic</code> , los tres vencimientos
<code>webapp.test.mjs</code>	8	El índice referencia lo que el build produce; el manifiesto apunta a iconos que existen; mismo <code>appId</code> en Android y Windows
<code>accounting.test.mjs</code>	6	Venta, compra, honorario, PPM, IDPC, asiento
<code>rut.test.mjs</code>	6	Dígito verificador, formatos, casos límite
<code>company-operations.test.mjs</code>	4	Tipos de operación, catálogo de constitución
<code>rules.test.mjs</code>	4	Un año sin reglas lanza ; años disponibles; procedencia
Total	153	

Las pruebas que valen por diez

Estas fijan reglas de negocio, no funciones. Si alguna se pone roja, algo se rompió en el producto y no en el código:

Prueba	Qué protege
<code>LoadRules(1999)</code> lanza	Que el sistema nunca calcule con la tasa del año equivocado
«un período cerrado es inmutable en las dos direcciones»	Que «cerrar el mes» signifique algo
«reabrir un período exige motivo y queda en la bitácora»	Trazabilidad por encima de inmutabilidad absoluta
«el remanente de crédito fiscal viaja entre períodos»	Que no se le cobre al usuario un IVA que no debe
«un paso de constitución no puede darse por hecho sin evidencia»	Que la app no mienta sobre trámites ante terceros
«una obligación no puede marcarse cumplida sin comprobante»	Idem
«el almacén web aísla real de sandbox en el mismo origen»	El invariante central del producto
«sembrar el sandbox sobre una empresa real está prohibido»	Que practicar no contamine lo real
«exportar e importar reproduce el espacio de trabajo completo»	Que los datos sean del usuario
«importar un archivo ajeno se rechaza»	
«toda mutación queda en la bitácora append-only»	Que la bitácora sirva como evidencia
«el diagnóstico marca error cuando hay obligaciones vencidas»	Que el semáforo no tranquilice de más

Cómo están escritas

```
node --test tests/*.test.mjs
```

Sin framework, sin `describe`, sin `mocks` y sin fixtures en disco: `createMemoryStore()` da un espacio de trabajo limpio en cada prueba. La consecuencia práctica es que **no hay nada que instalar para ejecutarlas** — sólo Node ≥ 20 — y que no existe una capa de dobles que pueda diverger del comportamiento real.

`node --test` reporta `suítes 0` porque las pruebas son llamadas planas a `test()`, sin agrupar.

La otra capa: verificar lo que un test unitario no ve

Es lo que distingue a este repositorio, y merece su propia tabla.

Verificación	Script / paso	Qué atrapa que un test no atraparía
El APK lleva la app dentro	<code>verify-apk.mjs</code> , 12 comprobaciones abriendo el ZIP	Un APK vacío compila perfectamente : la WebView arranca, muestra una pantalla en blanco y todo sigue en verde
La interfaz está completa antes de sellarla	<code>desktop.yml</code>	Tauri comprime los recursos dentro del binario: después ya no se pueden contar
El ejecutable arranca y sigue vivo	<code>desktop.yml</code> , 12 segundos	Fallos de arranque por DLL o recursos ausentes
El binario no es sospechosamente pequeño	<code>desktop.yml</code> , umbral de 1 MB	Un empaquetado incompleto
El build es reproducible	<code>ci.yml</code> , dos builds y <code>diff</code>	Que el artefacto publicado no sea el que se probó
El servidor no escapa de <code>dist/</code>	<code>ci.yml</code> , 3 vectores	Una regresión en <code>resolveSafe</code>
La documentación no se desvía del código	5 gates <code>--check</code>	Que el glosario, la guía o los atajos digan una cosa y la app haga otra
Las acciones están fijadas a SHA	<code>ci.yml</code>	Una etiqueta móvil que cambie de contenido
Los workflows son válidos	<code>actionlint</code>	Errores de sintaxis que sólo se verían al fallar
Cero dependencias de producción	<code>security.yml</code>	Que el argumento del proyecto deje de ser cierto por descuido
Sin datos reales en el repositorio	<code>security.yml</code>	El riesgo nº 1 del modelo de amenazas
La presentación tiene lo que anuncia	<code>check-presentation.mjs</code> , abre los PDF y cuenta páginas	Un PDF vacío o desactualizado

Integración continua

Seis workflows. Los tres primeros son los que corren solos:

Workflow	Disparadores	Trabajos	Duración máx.
ci.yml	push y PR a main , manual	pruebas (matriz), servidor , workflows	12 / 8 / 5 min
security.yml	push, PR, lunes 06:00 UTC, manual	codeql , datos	20 / 5 min
pages.yml	push a main , manual	publicar	10 min
android.yml	manual, workflow_call	apk	30 min
desktop.yml	manual, workflow_call	windows	45 min
release.yml	etiqueta v* , manual	android + windows → publicar	10 min

La matriz de ci.yml : Ubuntu × Node 20 y 22, más Windows × Node 22 (Windows con Node 20 está excluido). Windows entra en la matriz porque es la plataforma de escritorio del producto: si las rutas se rompen ahí, se rompe el instalador.

ci.yml no tiene filtros paths , y está comentado por qué: con filtros, un commit que sólo toca documentación no ejecuta nada y la rama queda sin estado, que no es lo mismo que estar en verde.

android.yml y desktop.yml no corren en cada push —tardan 30 y 45 minutos— sino a mano o llamados por release.yml . INFERENCIA : la consecuencia es que una regresión que sólo rompa el empaquetado no se detecta hasta que alguien publica.

Análisis estático, linting y formato

Herramienta	Alcance	Estado
CodeQL	JavaScript/TypeScript, conjunto <code>security-and-quality</code>	✓ Activo en push, PR y semanalmente
actionlint	<code>.github/workflows/</code>	✓ Activo en cada push
Verificación de pinning	<code>.github/workflows/</code>	✓ <code>grep</code> que falla si falta un SHA
ESLint	—	✗ NO IDENTIFICADO : no hay configuración
Prettier	—	✗ NO IDENTIFICADO
TypeScript / <code>checkJs</code>	—	✗ No hay <code>tsconfig.json</code>
Clippy / rustfmt	Rust	✗ NO IDENTIFICADO en los workflows
Cobertura	—	✗ No se mide
Dependabot	—	✗ NO IDENTIFICADO

La ausencia de ESLint y Prettier no ha producido un código inconsistente —el estilo del repositorio es notablemente uniforme—, pero eso hoy lo sostiene una sola persona, no una herramienta.

El tipado sí existe, en JSDoc: `@param {{ store: object, mode?: 'real'|'sandbox' }}` y similares están por todo el código. Nadie los comprueba, porque no hay `checkJs`.

Cobertura observable

No se mide, así que lo que sigue es un mapa cualitativo hecho leyendo qué toca cada suite.

Área	Archivos	Cobertura aparente	Evidencia
chile-tax-rules	index.mjs	Alta	4 pruebas, incluida la que más importa
accounting-engine	index.mjs , tax-equity.mjs , municipal-patent.mjs	Muy alta	6 + 17 + 23 + 8 pruebas
company-operations	workspace.mjs , capital.mjs , rut.mjs , store.mjs	Alta	14 + 19 + 6 + 4
glossary , onboarding , shortcuts		Alta	11 + 17 + 16, más 4 gates <code>--check</code>
Contrato de la web	index.html , manifiesto, iconos	Media	8 pruebas, de presencia y contrato, no de comportamiento
Vistas	apps/web/src/views/*.js (14)	✗ Ninguna	Ninguna prueba las ejecuta
lib/dom.js , state.js , platform.js		✗ Ninguna directa	Ni siquiera el escapado, que es un control de seguridad
server.mjs		Media	Sin prueba unitaria; sí verificación en CI
node-store.mjs		Baja	Se ejercita indirectamente por la CLI en CI
Backend Rust	lib.rs	Baja	2 pruebas para 4 comandos; no ejecutadas en esta máquina
Scripts de build	scripts/*.mjs (13)	✗ Ninguna	Se ejercitan al correr; no hay pruebas propias

Módulos sin pruebas, y qué costaría cubrirlos

Módulo	Riesgo de no cubrirlo	Dificultad
<code>lib/dom.js</code> · <code>esc</code> y <code>html</code>	Alto: es el control anti-XSS del sistema	Trivial: son funciones puras. Media docena de aserciones
<code>lib/platform.js</code> · <code>hydrateFromDisk</code>	Alto: decide si se pisan datos vivos con un espejo viejo	Media: hay que simular <code>__TAURI__</code> y <code>localStorage</code>
<code>store.mjs</code> · <code>createWebStore</code> con cuota llena	Alto: hoy la escritura se pierde en silencio	Media: simular que <code>setItem</code> lanza
Vistas	Medio: un fallo se ve de inmediato al usar la app	Alta: exige un DOM
<code>resolveSafe</code> como unidad	Bajo: ya está probado en CI de punta a punta	Trivial
Saneos de <code>filename</code> en Rust	Medio	Trivial: al lado de las dos que ya existen
Scripts de build	Bajo: fallan ruidosamente	Media

Propuesta priorizada de pruebas faltantes

Ordenada por relación entre lo que protege y lo que cuesta. **Es una recomendación; no se implementó nada.**

#	Prueba	Por qué primero
1	<code>esc()</code> y <code>html</code> escapan <code><</code> , <code>></code> , <code>&</code> , <code>"</code> , <code>'</code> ; <code>raw()</code> no	Control de seguridad, coste trivial
2	<code>createWebStore</code> cuando <code>setItem</code> lanza <code>QuotaExceededError</code>	Hoy se pierde una escritura en silencio
3	Saneos de <code>filename</code> en <code>export_file</code> (Rust)	Al lado de las dos pruebas que ya existen
4	<code>save_workspace</code> rechaza un payload que no es JSON	Idem
5	<code>resolveSafe</code> como prueba unitaria, con los tres vectores	Deja de depender de que el servidor arranque
6	<code>hydrateFromDisk</code> no pisa un dato vivo con uno del espejo	La carrera está comentada en el código, no probada
7	Coherencia entre las dos <code>version</code> (<code>package.json</code> y <code>tauri.conf.json</code>)	Un gate de una línea
8	Al menos una vista: renderizar <code>panel</code> sobre un almacén de memoria	Rompe el cero de cobertura de la interfaz
9	<code>capitalHistory()</code> con dos ejercicios cerrados	La cadena de tres años es la lección del producto
10	Un segundo año de reglas (<code>rules/2027.json</code> de prueba)	El mecanismo nunca se ha ejercitado con dos archivos

Criterios de aceptación

Reconstruidos a partir de lo que CI exige; el repositorio no los enuncia en un documento aparte.

Para que un cambio entre a `main`:

1. Las 153 pruebas pasan en Ubuntu (Node 20 y 22) y Windows (Node 22).
2. Los cinco gates de sincronía documental pasan.
3. `validate-rules.mjs` pasa.
4. El build completo funciona **y es reproducible**.
5. La CLI responde con los valores esperados.
6. El servidor sirve la app y bloquea los tres vectores de escape.
7. `actionlint` pasa y todas las acciones están fijadas a SHA.
8. CodeQL no encuentra nada nuevo.
9. No hay datos reales, certificados ni claves en el repositorio.
10. `dependencies` sigue vacío.

`CONTRIBUTING.md` añade uno que no es automatizable y es el más importante: **una tasa que se toca exige su fuente oficial y su fecha de verificación**.

Lo que no pudo verificarse

Aspecto	Por qué
<code>cargo test --release</code>	Sin toolchain de Rust en la máquina de análisis
Compilación del APK y <code>verify-apk.mjs</code> sobre un APK real	Requiere JDK 21, SDK de Android y Gradle
Compilación de los instaladores de Windows	Requiere Rust y <code>@tauri-apps/cli</code>
Resultados de CodeQL	Viven en la pestaña Security del repositorio
Comportamiento de las vistas	No hay pruebas y no se ejecutó la interfaz en esta revisión
Que CI esté hoy en verde en GitHub	No se consultó el estado remoto

13 · Despliegue y operación

La conclusión, primero

Un solo `apps/web/dist` alimenta las cuatro superficies. GitHub Pages lo publica tal cual, Capacitor lo empaqueta en un APK y Tauri lo embebe en el ejecutable de Windows. No hay tres builds que puedan divergir: hay uno, y tres formas de envolverlo.

Y la regla de operación que gobierna todo lo demás: **un build en verde no prueba que el artefacto sirva**. Un APK vacío compila perfectamente. Por eso cada camino de publicación tiene un paso que

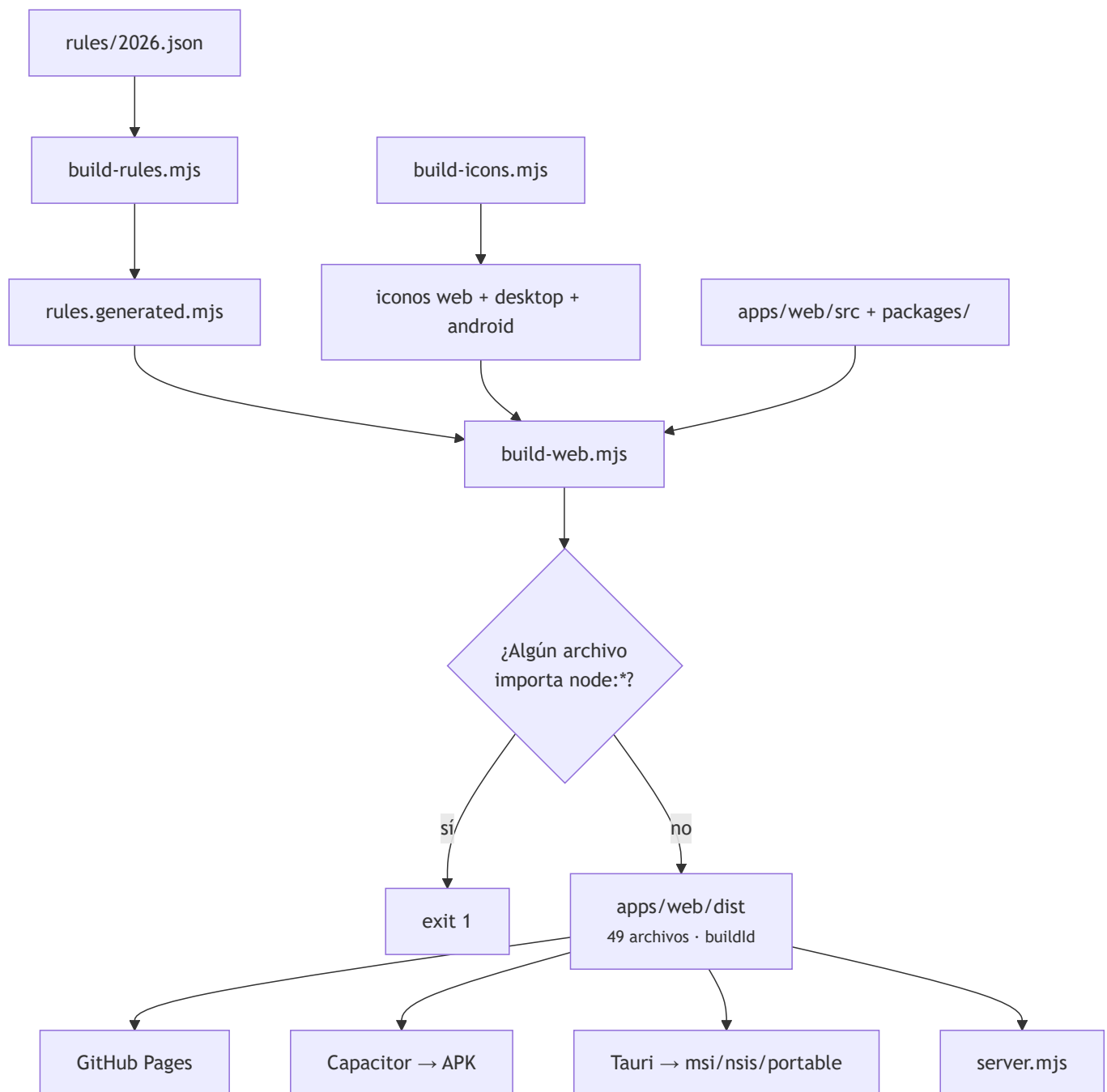
abre el resultado y cuenta lo que hay dentro antes de dejarlo salir.

Entornos

Entorno	Qué es	Cómo se llega
Local	<code>node scripts/build-all.mjs</code> + servidor en <code>127.0.0.1:4180</code>	<code>pnpm start</code>
GitHub Pages	La misma app, pública, instalable como PWA	<code>push</code> a <code>main</code> → <code>pages.yml</code>
Artefacto Android	APK de depuración, retenido 14 días	<code>android.yml</code> a mano, o dentro de un release
Artefacto Windows	<code>.msi</code> , <code>-setup.exe</code> y portable, retenidos 14 días	<code>desktop.yml</code> a mano, o dentro de un release
Release	Todo lo anterior + 3 PDF + <code>SHA256SUMS.txt</code>	Etiqueta <code>vX.Y.Z</code>

No hay staging y no hay entorno de pruebas desplegado. No hay servidor que desplegar: lo que se publica son archivos estáticos y binarios.

El proceso de construcción



El orden importa y está comentado en `build-all.mjs`: la web embebe las reglas y los iconos, así que generar la web con reglas viejas produciría un APK que calcula con la tasa del año pasado **sin que nada se ponga en rojo**.

El gate `node:*` está en el build, no en un test, y la razón está escrita al lado: si eso llega al APK, la pantalla queda en blanco sin un error visible, y un test es algo que quizá nadie corra antes de publicar. El build aborta con `exit 1` y nombra los archivos culpables.

El `buildId` es el SHA-256 de todos los archivos del `dist`, truncado a 12 caracteres. Se sustituye en `sw.js` (`__BUILD_ID__`) para invalidar el caché del service worker: publicar una versión nueva deja de dejar a la gente con una app vieja pegada en el navegador.

Salida real de esta revisión:

```
apps/web/dist listo - 49 archivos, 7256 KB, build 630ed69fe514
```

Publicación en GitHub Pages

`pages.yml` , disparado por cada push a `main` .

Paso	Qué hace
Build	<code>node scripts/build-all.mjs</code>
Verificación	Al menos 10 vistas; <code>index.html</code> contiene el nombre del producto
<code>.nojekyll</code>	Sin ese marcador, Jekyll se comería cualquier ruta que empiece por guion bajo
<code>configure-pages</code> → <code>upload-pages-artifact</code> → <code>deploy-pages</code>	Publicación

Permisos elevados sólo en este trabajo: `pages: write` e `id-token: write` . La concurrencia usa `cancel-in-progress: false` — a diferencia del resto — porque cancelar un despliegue a medias es peor que esperar.

URL: `https://vladimiracunadev-create.github.io/empresa-operativa-chile/` . **REQUIERE VALIDACIÓN :** no se consultó el estado del despliegue en esta revisión.

Recordatorio de 11 · Seguridad: **esta superficie es la única sin CSP**, porque Pages no admite cabeceras propias y el `index.html` no lleva `meta http-equiv` .

Empaquetado de Android

`android.yml` . No hay código de app propio: la app **es** la aplicación web.

Paso	Detalle
Herramientas	pnpm 11.2.2, Node 22, Temurin JDK 21
Build web	<code>node scripts/build-all.mjs</code>
Capacitor	<code>pnpm install --frozen-lockfile</code> → <code>copiar-www.mjs</code> → <code>cap add android</code> → <code>copiar-www.mjs</code> otra vez → <code>cap sync android</code>
Compilación	<code>./gradlew assembleDebug --no-daemon</code>
Verificación	<code>verify-apk.mjs</code> sobre el APK generado
Artefacto	APK + su <code>.sha256</code> , retenidos 14 días

La segunda pasada de `copiar-www.mjs` no es un error de copiar y pegar, y está comentada: `cap add` acaba de crear `android/` , y es entonces cuando se pueden aplicar los iconos propios sobre sus recursos.

`verify-apk.mjs` — 12 comprobaciones abriendo el ZIP

Lee el directorio central del ZIP **a mano**, sin `unzip` ni librerías, para correr igual en Windows, Linux y macOS.

#	Comprobación	Umbral
1–4	<code>index.html</code> , <code>app.js</code> , <code>app.css</code> y el manifiesto PWA están dentro	Presencia
5	Las vistas viajan	≥ 10
6	El núcleo de cálculo viaja	≥ 6 módulos <code>.mjs</code>
7	Los iconos viajan	≥ 3
8–10	<code>AndroidManifest.xml</code> , <code>resources.arsc</code> , <code>classes.dex</code>	Presencia
11	Tamaño razonable	> 1 MB y < 120 MB
12	Las reglas tributarias viajan dentro	<code>core/chile-tax-rules/rules.generated.mjs</code>

Si alguna falla: `este APK NO debe publicarse` y `exit 1` .

El comentario del propio script explica por qué existe, y vale la pena leerlo: no comprueba que el build funcionara, comprueba **que el resultado sirva**. Un APK vacío arranca la WebView, muestra una pantalla en blanco y deja todas las señales del build en verde.

El APK es de depuración, firmado con la clave de debug de Android. Sirve para instalar y usar, no para publicar en Google Play. Está declarado en el resumen del propio workflow y en las notas de cada release.

Empaquetado de Windows

`desktop.yml` , sobre `windows-latest` , hasta 45 minutos.

Paso	Detalle
Herramientas	pnpm 11.2.2, Node 22, Rust <code>stable</code> , caché de Rust (<code>Swatinem/rust-cache</code>)
Build web	<code>node scripts/build-all.mjs</code>
Verificación previa	≥ 10 vistas, ≥ 6 módulos del núcleo, el nombre en <code>index.html</code> , <code>f29Basic</code> en el motor
Pruebas de Rust	<code>cargo test --release</code>
Compilación	<code>pnpm install --frozen-lockfile</code> → <code>pnpm build</code>
Recogida	<code>.msi</code> , <code>-setup.exe</code> y el portable; falla si falta cualquiera
Umbral de tamaño	Falla si un artefacto pesa menos de 1 MB
<code>SHA256SUMS.txt</code>	Calculado en PowerShell
Prueba de arranque	Lanza el portable, espera 12 s y falla si el proceso murió

La verificación va antes de empaquetar, y está comentado por qué: Tauri embebe y comprime los recursos dentro del binario, así que después ya no se pueden contar leyendo el `.exe` . El momento de comprobar que la app está entera es justo antes de sellarla.

Dos detalles de PowerShell que el workflow documenta y que conviene no deshacer:

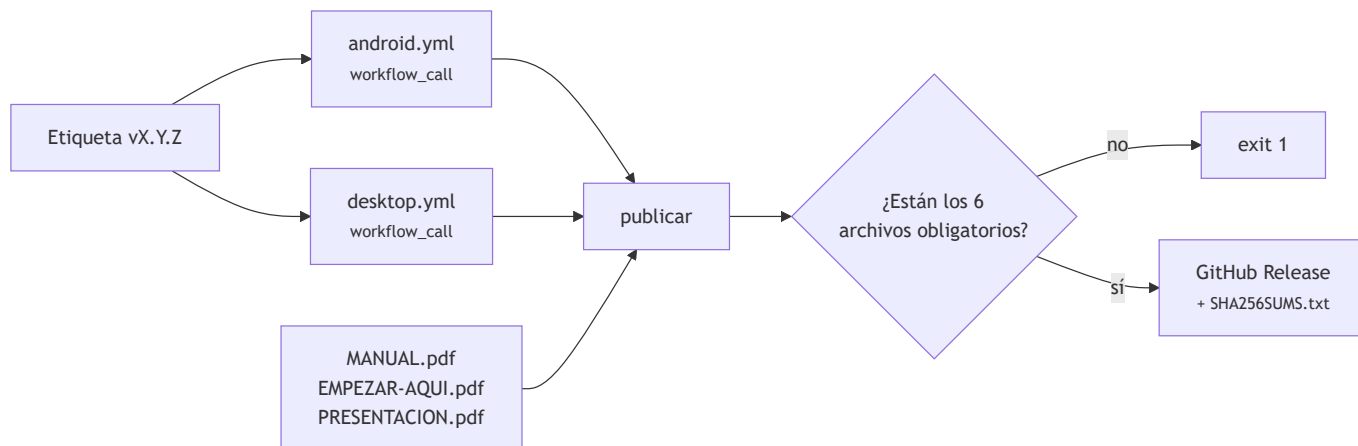
- La lista de archivos se materializa **antes** de escribir `SHA256SUMS.txt` : encadenar `Get-ChildItem | ... | Out-File` en el mismo directorio hace que `Out-File` abra el archivo mientras la enumeración sigue viva, y `Get-FileHash` acaba leyendo el archivo que se está escribiendo.
- No se le puede pedir a una app de ventana que responda por HTTP, pero sí comprobar que el proceso levanta y sigue vivo: eso descarta los fallos de arranque por DLL o recursos ausentes.

Los tres formatos existen porque resuelven situaciones distintas: `.msi` para despliegue corporativo o instalación desatendida, `-setup.exe` (NSIS) para una persona, y el portable para un pendrive o un equipo ajeno.

Los instaladores no están firmados con certificado de código: SmartScreen avisará la primera vez. Declarado en el resumen del workflow.

Release

`release.yml`, disparado por una etiqueta `v*`.



Reutiliza los workflows en vez de duplicar sus pasos: lo que se publica es exactamente lo que esos workflows verificaron.

La documentación viaja con el release —manual, guía «Empezar aquí» y presentación en PDF— porque quien descarga un instalador debería poder llevarse las instrucciones sin volver al repositorio.

Un release incompleto no se publica: hay seis `test -f` que fallan el trabajo si falta el APK, cualquiera de los tres instaladores, el manual o la guía.

Las notas se redactan en el propio workflow e incluyen, además de las descargas, una sección de

advertencias honestas: binarios sin firmar, APK de depuración, la app no presenta ni paga nada ante el SII, y el borrador F29 es una ayuda de control y no una declaración.

Publicar una versión

```
git tag v1.4.1
git push origin v1.4.1
```

Antes conviene haber actualizado `CHANGELOG.md`, la `version` de `package.json` y la de `tauri.conf.json` — nada comprueba que las dos coincidan (ver [10 · Configuración](#)).

Generación de los PDF de esta documentación

`scripts/build-system-docs.mjs`, escrito para esta serie.

```
node scripts/build-system-docs.mjs          # todos + el consolidado
node scripts/build-system-docs.mjs --only 03 # sólo el 03, para iterar rápido
node scripts/build-system-docs.mjs --no-diagrams # sin rasterizar Mermaid
```

Cómo funciona:

1. Lee cada `.md` de `docs/system-documentation/` en orden, con el `README.md` primero.
 2. Quita las barras de navegación entre documentos: son útiles en GitHub y ruido en papel.
 3. **Rasteriza cada bloque Mermaid a SVG** con `mmdc`, cacheado por hash del contenido en `assets/diagramas/`. Regenerar sin tocar un diagrama no vuelve a invocar el rasterizador, que es con diferencia el paso más lento.
1. Convierte a HTML con el mismo `scripts/lib/markdown.mjs` que usan el manual y la guía, embebiendo cada imagen como data URI.
 1. Compone una portada con **datos que no se escriben a mano**: la versión sale de `package.json`, el commit de `git rev-parse --short HEAD` y la fecha de análisis del propio `README.md`.
 1. Imprime a PDF con `scripts/lib/chrome.mjs` y la hoja `PRINT_CSS` compartida.
 2. Genera además `documentacion-completa.pdf` con la serie entera.

Tres decisiones que conviene conocer antes de tocarlo:

- **Si `mmdc` no está instalado, no falla.** Deja los diagramas como bloques de código y lo avisa por consola. Un PDF con un diagrama en texto es peor que uno con el diagrama dibujado, y mucho mejor que ningún PDF.
- **Avisa cuando un diagrama sale ilegible.** Calcula el ancho útil de una A4 (688 px con los márgenes de `PRINT_CSS`) y, si el diagrama se reduce por debajo del 45 %, lo dice con nombre y factor. Es lo que detectó que el diagrama entidad-relación de 07 medía 2.328 px y se imprimía al 30 %; por eso ese diagrama está partido en dos.
- **Pide las imágenes en carga inmediata.** `markdownToHtml` marca las imágenes como `loading="lazy"`, que es correcto para el HTML del manual y un riesgo al imprimir: un diagrama que el navegador considere fuera de vista puede no haberse decodificado al dispararse la impresión, y sale un hueco.

Requisito: Chrome o Edge (autodetectado, o `CHROME_PATH`). Para los diagramas, `@mermaid-js/mermaid-cli` global:

```
pnpm add -g @mermaid-js/mermaid-cli
```

No está integrado en `pnpm check` ni en CI, a diferencia de los generadores del glosario y la guía. **INFERENCIA**: integrarlo exigiría Chrome y `mmdc` en el runner, y ninguno de los dos está hoy en los workflows.

La consecuencia es que **los PDF de esta serie hay que regenerarlos a mano cuando cambie el Markdown;** se recoge en [15 · Riesgos](#).

Otros documentos que se generan

Comando	Qué produce
<code>pnpm docs</code>	Capturas + guía + manual + presentación
<code>pnpm manual</code>	<code>docs/MANUAL.html</code> y <code>.pdf</code>
<code>pnpm guia</code>	<code>docs/EMPEZAR-AQUI.md</code> , <code>.html</code> y <code>.pdf</code>
<code>pnpm presentacion</code>	Diapositivas y pauta, y las verifica
<code>pnpm capturas</code>	Capturas desde la app servida en <code>CAPTURE_URL</code>

Los cinco `--check` correspondientes corren en CI: si alguien edita el documento generado a mano, el build falla.

Operación

Logs, métricas, monitoreo y alertas

`NO IDENTIFICADO` , y es coherente con el diseño: **no hay servidor que observar.**

Aspecto	Estado
Logs de servidor	El servidor local imprime su URL al arrancar y nada más. No escribe archivo de log
Logs de la aplicación	<code>console.warn</code> cuando falla el espejo de disco o la hidratación. Nada más
Métricas	Ninguna. No hay telemetría, por decisión de producto
Monitoreo	Ninguno
Alertas	Las de GitHub: fallo de workflow y hallazgos de CodeQL
Trazas	Ninguna

Lo más parecido a observabilidad que tiene el producto es la bitácora de auditoría, y sirve para que el usuario entienda su propia empresa, no para que nadie observe el sistema.

Respaldo y recuperación

El respaldo es del usuario, y es la contrapartida directa de no tener servidor.

Escenario	Recuperación
Se borran los datos del navegador	Importar el último respaldo exportado. Sin respaldo, no hay recuperación
Se desinstala el APK	Igual
Se corrompe el espejo de Windows	<code>localStorage</code> sigue siendo la fuente; el espejo se reescribe en la siguiente escritura
Se corrompe <code>localStorage</code> en Windows	Al arrancar, <code>hydrateFromDisk()</code> repone lo que falte desde el espejo
Se pierde el dispositivo	Sólo el respaldo exportado fuera de él

`SECURITY.md` da la regla operativa: **exportar desde la pestaña Datos cada vez que se cierra un período**, y guardar el archivo fuera del dispositivo.

Rollback

Qué	Cómo
GitHub Pages	Revertir el commit y empujar: <code>pages.yml</code> republica
Release	Instalar el artefacto de la versión anterior desde su release
Datos	Importar un respaldo anterior. Un ejercicio ya cerrado no se pisa al importar en modo fusión
Reglas tributarias	Revertir el <code>.json</code> y regenerar, o CI falla en <code>--check</code>

No hay migración de datos que revertir, porque no hay migraciones destructivas: las dos que existen ocurren en lectura y no tocan el almacén.

Mantenimiento periódico

Cadencia	Tarea
Anual, antes de que empiece el año comercial	Crear <code>rules/<año>.json</code> , verificar cada tasa contra su fuente oficial , regenerar y publicar
Mensual	Actualizar la UTM del mes en el archivo de reglas si se van a calcular patentes
Semanal	Revisar los hallazgos de CodeQL (el workflow corre los lunes a las 06:00 UTC)
Por release	<code>CHANGELOG.md</code> , las dos <code>version</code> , regenerar documentación
Cuando cambie el Markdown de esta serie	<code>node scripts/build-system-docs.mjs</code>

La tarea anual es la que sostiene el producto. Todo lo demás es higiene; esa es la que evita que la aplicación calcule con la tasa del año pasado. `CONTRIBUTING.md` la formaliza: una tasa que se toca exige su fuente oficial y su fecha de verificación.

Lo que no pudo verificarse

Aspecto	Por qué
Compilación real del APK y del instalador	Faltan JDK/SDK/Gradle y toolchain de Rust en la máquina de análisis
<code>verify-apk.mjs</code> sobre un APK real	No hay APK que verificar
Estado del último despliegue en Pages	No se consultó el repositorio remoto
Que el artefacto publicado coincida con el verificado	<code>sha256sum</code> del archivo descargado contra el <code>SHA256SUMS.txt</code> del release
Comportamiento del instalador NSIS y del MSI	No se ejecutaron

14 · Solución de problemas

Cada entrada sigue el mismo esquema: **síntoma** → **causa probable** → **cómo diagnosticarlo** → **solución** → **archivos implicados** → **riesgo de la solución**. Las causas salen del código, no de la experiencia: donde algo no pudo comprobarse se dice.

Antes de nada, el comando que descarta la mitad de los problemas:

```
node scripts/build-all.mjs && node --test tests/*.test.mjs
```

Instalación y entorno

La aplicación no arranca: «No existe apps/web/dist»

Síntoma	<code>server.mjs</code> sale de inmediato con <code>No existe apps/web/dist. Ejecuta primero:</code>
Causa	<code>apps/web/dist/</code> no está versionado — está en <code>.gitignore</code> . Un clon recién hecho no lo tiene
Diagnóstico	<code>ls apps/web/dist/index.html</code>
Solución	<code>node scripts/build-all.mjs</code>
Archivos	<code>apps/empresa-operativa/server.mjs</code> , <code>.gitignore</code>
Riesgo	Ninguno

Es el tropiezo número uno de quien llega al repositorio, y el servidor lo trata bien: imprime el comando exacto que hay que ejecutar.

`pnpm install` no instala nada

Síntoma	«Already up to date», <code>node_modules</code> vacío o inexistente
Causa	Correcto. El <code>package.json</code> raíz no declara dependencias ni devDependencies
Diagnóstico	<code>node -p "Object.keys(require('./package.json').dependencies ?? {}).length" → 0</code>
Solución	Ninguna. <code>pnpm install</code> sólo hace falta en <code>apps/android/</code> y <code>apps/contador-desktop/</code>
Archivos	<code>package.json</code>
Riesgo	Ninguno

Errores de sintaxis al ejecutar cualquier script

Síntoma	<code>SyntaxError: Unexpected token</code> , <code>Cannot use import statement outside a module</code>
Causa	Node por debajo de 20. El repositorio usa ESM y sintaxis moderna
Diagnóstico	<code>node --version</code>
Solución	Actualizar a Node ≥ 20 ; CI prueba en 20 y 22
Archivos	<code>package.json</code> (<code>engines.node</code>)
Riesgo	Ninguno

npm install deja el proyecto en un estado raro

Síntoma	Versiones distintas de las del lockfile, o node_modules inesperado
Causa	El gestor del repositorio es pnpm, declarado en packageManager. npm ignora pnpm-lock.yaml
Diagnóstico	node -p "require('./package.json').packageManager" → pnpm@11.2.2
Solución	Borrar node_modules, usar pnpm install --frozen-lockfile
Archivos	package.json, pnpm-lock.yaml
Riesgo	Ninguno

⚠ apps/android/ tiene versionados pnpm-lock.yaml y package-lock.json a la vez. El workflow usa pnpm; el package-lock.json no lo consume nadie, pero induce al error. Ver 15 · Riesgos.

Ejecución

El puerto 4180 está ocupado

Síntoma	EADDRINUSE	
Causa	Otro proceso —probablemente otra instancia del propio servidor — escucha ahí	
Diagnóstico	Windows: `netstat -ano` Linux/macOS: `lsof -i :4180`	
Solución	PORT=4181 node apps/empresa-operativa/server.mjs , o cerrar el otro proceso	
Archivos	apps/empresa-operativa/server.mjs	
Riesgo	Si se cambia el puerto, CAPTURE_URL deja de apuntar bien y las capturas fallan	

La app carga en blanco, o se queda en «Cargando...»

Síntoma	Sólo el mensaje de arranque de index.html
Causa probable	Un módulo ES falló al cargar. El caso clásico: **un archivo del bundle importa node:* **
Diagnóstico	Consola del navegador. Y grep -rl "from 'node:" apps/web/dist/
Solución	Rehacer el build: node scripts/build-web.mjs aborta si algún archivo publicado importa node:* y nombra los culpables
Archivos	scripts/build-web.mjs , apps/web/src/index.html
Riesgo	Ninguno

El estado inicial de index.html está pensado justo para esto: si los módulos fallan, lo que queda a la vista es un mensaje, no una página en blanco sin explicación.

El APK se instala pero muestra una pantalla en blanco

Síntoma	La app abre, la WebView arranca, no hay contenido
Causa	El APK se empaquetó vacío. Es el fallo que <code>verify-apk.mjs</code> existe para atrapar
Diagnóstico	<code>node scripts/verify-apk.mjs ruta/al.apk</code>
Solución	<code>node scripts/build-all.mjs && node apps/android/copiar-www.mjs</code> , después <code>cap sync android</code> y recompilar
Archivos	<code>scripts/verify-apk.mjs</code> , <code>apps/android/copiar-www.mjs</code> , <code>.github/workflows/android.yml</code>
Riesgo	Ninguno

Un APK vacío compila perfectamente y deja todas las señales del build en verde. Nunca se publique un APK sin pasar el verificador.

La app de Windows abre pero no ve los datos que había

Síntoma	Espacio de trabajo vacío tras reinstalar
Causa probable	Cambió el <code>identifier</code> de <code>tauri.conf.json</code> , así que <code>app_data_dir()</code> apunta a otro sitio
Diagnóstico	Pestaña Datos de la app, que muestra <code>dataDir</code> vía <code>app_info()</code> . Comparar con el directorio anterior
Solución	Restaurar el <code>identifier</code> original, o importar un respaldo
Archivos	<code>apps/contador-desktop/src-tauri/tauri.conf.json</code> , <code>src/lib.rs</code>
Riesgo	■ Cambiar el identificador deja huérfanos los datos existentes

El espejo en disco no se actualiza (Windows)

Síntoma	<code>real.json</code> no cambia aunque se registren operaciones
Causa probable	El <code>invoke</code> falla; la app sigue funcionando con <code>localStorage</code> y sólo avisa por consola
Diagnóstico	Consola de la WebView: <code>No se pudo escribir el espejo en disco:</code>
Solución	Revisar permisos del directorio de datos; comprobar que el <code>payload</code> es JSON válido
Archivos	<code>apps/web/src/lib/platform.js</code> , <code>src-tauri/src/lib.rs</code>
Riesgo	Ninguno al diagnosticar. Mientras dure, los datos sólo están en <code>localStorage</code>

El espejo es «best effort» por decisión explícita: perder la sesión por un error de escritura sería peor que perder el espejo. La contrapartida es que el fallo es silencioso salvo en la consola.

Los datos desaparecieron

Síntoma	La app arranca vacía
Causas	Se borraron los datos del sitio; se desinstaló el APK; se abrió en otro navegador o perfil; se cambió de modo (real ↔ sandbox)
Diagnóstico	Comprobar el selector de modo. En la consola: <code>Object.keys(localStorage).filter(k => k.startsWith('empresa-operativa-chile'))</code>
Solución	Importar el último respaldo desde la pestaña Datos
Archivos	<code>apps/web/src/lib/platform.js</code> , <code>packages/company-operations/store.mjs</code>
Riesgo	Importar con <code>replace</code> sustituye el contenido. Con fusión, respeta los ejercicios cerrados

Sin respaldo no hay recuperación. No hay servidor que respalde por nadie.

Cálculo y reglas tributarias

«No hay reglas verificadas para el año comercial N»

Síntoma	Error: No hay reglas verificadas para el año comercial 2027. Disponibles: 2026
Causa	Comportamiento correcto y deliberado. No existe rules/2027.json
Diagnóstico	ls packages/chile-tax-rules/rules/
Solución	Crear el archivo del año, verificando cada tasa contra su fuente oficial, y node scripts/build-rules.mjs
Archivos	packages/chile-tax-rules/index.mjs , rules/ , scripts/build-rules.mjs
Riesgo	■ Nunca hacer que degrade al año anterior. CONTRIBUTING.md lo prohíbe y hay una prueba que lo fija

Un cálculo con la tasa del año equivocado **no falla**: devuelve un número plausible que aparece meses después como una diferencia con el SII. Que esto lance es la garantía más importante del sistema.

CI falla: «Las reglas embebidas no están sincronizadas»

Síntoma	build-rules.mjs --check en rojo
Causa	Se editó rules/<año>.json sin regenerar, o se editó rules.generated.mjs a mano
Diagnóstico	node scripts/build-rules.mjs --check
Solución	node scripts/build-rules.mjs y commitear el generado
Archivos	rules/<año>.json , rules.generated.mjs
Riesgo	Editar el generado a mano deja el sistema calculando con tasas que ningún JSON respalda

La patente municipal sale distinta de la que cobra el municipio

Síntoma	Diferencia con el aviso de la municipalidad
Causa	La tasa concreta de una comuna NO está en el repositorio. El archivo guarda el rango legal (0,25 %-0,5 %); cada municipalidad fija la suya por ordenanza. Además, la base cambia según la etapa del negocio
Diagnóstico	Ficha municipal de la app: comuna, tasa y su estado. <code>node apps/contador-cli/src/index.mjs patente-municipal --etapa funcionamiento --cpt N</code>
Solución	Verificar la tasa con la municipalidad y registrarla en la ficha
Archivos	<code>rules/2026.json</code> (<code>municipalPatent</code>), <code>packages/accounting-engine/municipal-patent.mjs</code>
Riesgo	Ninguno

El propio archivo de reglas lo advierte, y también que **BCN/LeyChile no respondió el 16-08-2026**: reverificar el enlace antes de usar la cifra en una declaración real.

El F29 de la app no cuadra con la propuesta del SII

Síntoma	Diferencia con el formulario oficial
Causa	Limitación declarada. <code>f29Basic</code> es una simulación educativa básica: no modela exenciones, proporcionalidad de IVA, activos fijos, importaciones ni regímenes especiales. Tampoco reajusta el remanente (art. 27 del D.L. 825)
Diagnóstico	Comparar contra el RCV y la propuesta oficial. Revisar <code>rejectedVat</code> y las operaciones con <code>vatCreditEligible: false</code>
Solución	Conciliar. Manda el SII
Archivos	<code>packages/accounting-engine/index.mjs</code> , <code>rules/2026.json</code> (<code>warnings</code>)
Riesgo	Ninguno

El CPT sale en cero

Síntoma	Capital propio tributario simplificado = 0
Causa probable	Comportamiento correcto: el método del art. 14 D) N.º 3 (j) tiene piso en cero . Si el resultado es negativo, el CPT se considera \$0
Diagnóstico	<code>node apps/contador-cli/src/index.mjs cpt --regimen "Pro Pyme General (14 D N.º 3)" ...</code> y revisar los componentes
Solución	Ninguna si es correcto. Si no lo es, revisar retiros, pérdidas y disminuciones de capital
Archivos	<code>packages/accounting-engine/tax-equity.mjs</code> , <code>rules/2026.json</code>
Riesgo	Ninguno

El capital enterado no cuadra con lo aportado

Síntoma	Cifras distintas entre la ficha y los movimientos
Causa probable	Un aporte registrado dos veces : como movimiento patrimonial y como operación de caja sin enlazar
Diagnóstico	Buscar operaciones de tipo <code>capital</code> con <code>equityMovementId: null</code>
Solución	Registrar el movimiento con <code>registerCashMovement: true</code> , que crea la operación enlazada
Archivos	<code>packages/company-operations/workspace.mjs</code> (<code>addEquityMovement</code> , <code>effectiveEquityMovements</code>)
Riesgo	Corregir a mano en un período cerrado no se puede: hay que reabrirlo con motivo

«Capital social», «suscrito» y «enterado» aparecen como pendientes

Síntoma	La app marca campos en <code>pendingConfirmation</code>
Causa	Correcto. Se migró desde el campo antiguo <code>capital</code> : ese valor pasó a <code>capitalEnterado</code> y la app no inventa el social ni el suscrito
Diagnóstico	<code>getCapitalProfile().pendingConfirmation</code>
Solución	Confirmar cada cifra contra el estatuto. Confirmarla la saca de la lista
Archivos	<code>packages/company-operations/capital.mjs</code>
Riesgo	Ninguno

Operaciones y cierres

«El período YYYY-MM está cerrado»

Síntoma	Error al agregar, editar o borrar una operación
Causa	Correcto. Un período cerrado es inmutable en las dos direcciones
Diagnóstico	<code>isPeriodClosed(period)</code> , o la vista de cierre
Solución	Reabrir el período indicando el motivo , que queda en la bitácora
Archivos	<code>workspace.mjs</code> (<code>addTransaction</code> , <code>closePeriod</code> , <code>reopenPeriod</code>)
Riesgo	Reabrir queda registrado para siempre. Es lo correcto, pero deja rastro

«Reabrir un período exige indicar el motivo»

Correcto por diseño: la trazabilidad importa más que la inmutabilidad absoluta. Pasar un texto no vacío como segundo argumento.

«Un paso sólo puede marcarse como realizado si registras su evidencia»

Síntoma	Error al marcar un trámite como hecho
Causa	Correcto. La app no da por cumplido un trámite ante un organismo externo porque alguien marcó una casilla
Solución	Registrar folio, certificado o comprobante en <code>evidenceRef</code>
Archivos	<code>workspace.mjs</code> (<code>updateFormationStep</code>)
Riesgo	Ninguno

El semáforo nunca dice «todo en orden»

Síntoma	<code>healthCheck</code> siempre devuelve observaciones
Causa	Deliberado. El objetivo es detectar el hueco, no tranquilizar
Diagnóstico	Revisar los <code>code</code> devueltos: <code>company.missing</code> , <code>formation.pending</code> , <code>evidence.missing</code> , <code>vat.rejected</code> , <code>obligations.overdue</code>
Solución	Resolver cada observación; las de nivel <code>error</code> primero
Archivos	<code>workspace.mjs</code> (<code>healthCheck</code>)
Riesgo	Ninguno

«El archivo no es un respaldo válido de esta aplicación»

Síntoma	Falla la importación
Causa	Falta <code>format: empresa-operativa-chile/backup</code> , o <code>formatVersion</code> no es 1 ni 2
Diagnóstico	Abrir el JSON y mirar las dos primeras claves
Solución	Usar un respaldo exportado por la propia aplicación
Archivos	<code>workspace.mjs</code> (<code>importAll</code> , <code>exportAll</code>)
Riesgo	Ninguno

El sandbox no se siembra

Síntoma	El entorno de práctica sale vacío, o <code>seedSandboxWorkspace</code> lanza
Causas	(a) Se llamó en modo <code>real</code> → lanza, correctamente ; (b) el sandbox ya tiene operaciones → retorno temprano sin sembrar
Diagnóstico	<code>ws.mode</code> y <code>ws.listTransactions().length</code>
Solución	Usar <code>--sandbox</code> ; para resembrar, vaciar antes el espacio de práctica
Archivos	<code>workspace.mjs</code> (<code>seedSandboxWorkspace</code>)
Riesgo	Vaciar el sandbox pierde lo que hubiera en él

Documentación y build

CI falla: «El glosario / la guía / los atajos no están sincronizados»

Síntoma	Uno de los <code>--check</code> en rojo
Causa	Se editó a mano un documento generado . La fuente es el paquete, no el <code>.md</code>
Diagnóstico	El propio comando <code>--check</code> dice cuál
Solución	Editar el paquete (<code>packages/glossary</code> , <code>onboarding</code> , <code>shortcuts</code>) y regenerar
Archivos	<code>scripts/build-glossary.mjs</code> , <code>build-guide.mjs</code> , <code>build-shortcuts.mjs</code>
Riesgo	Regenerar sobrescribe el <code>.md</code> : los cambios hechos a mano se pierden

CI falla: «El build no es reproducible»

Síntoma	El <code>build-info.json</code> cambia entre dos builds seguidos
Causa	Algo depende del reloj o del orden del sistema de archivos
Diagnóstico	El paso imprime el <code>diff</code>
Solución	Localizar la fuente de no determinismo (marcas de tiempo, orden de <code>readdir</code> sin ordenar)
Archivos	<code>scripts/build-web.mjs</code>
Riesgo	 Si no es reproducible, el artefacto que se publica no es el que se probó

CI falla: «Estas acciones no están fijadas a un SHA»

Fijar la acción a un SHA de 40 caracteres, con la etiqueta como comentario al lado. Es el estilo del repositorio y el paso lo exige.

«No se encontró Chrome ni Edge. Define CHROME_PATH.»

Síntoma	Falla la generación de PDF o de capturas
Causa	La autodetección no encontró el navegador
Solución	<code>CHROME_PATH="/ruta/al/chrome" node scripts/build-manual.mjs</code>
Archivos	<code>scripts/lib/chrome.mjs</code>
Riesgo	Ninguno. No afecta a la aplicación

«X.pdf salió vacío o demasiado pequeño»

Síntoma	<code>printPdf</code> lanza
Causa	Las imágenes no se embebieron, o el documento quedó por debajo del suelo de tamaño
Diagnóstico	<code>EOC_KEEP_HTML=1</code> conserva el HTML intermedio e imprime su ruta; abrirlo en el navegador muestra qué falta
Solución	Comprobar que existen las imágenes referenciadas. En documentos legítimamente cortos, bajar <code>minBytes</code> en la llamada
Archivos	<code>scripts/lib/chrome.mjs</code>
Riesgo	Bajar el suelo para todos desactivaría la comprobación donde sí sirve

Los diagramas Mermaid salen como bloques de código en el PDF

Síntoma	El PDF muestra el texto del diagrama, y la consola avisó
Causa	<code>mmdc</code> no está instalado. El generador no falla a propósito
Diagnóstico	<code>mmdc --version</code>
Solución	<code>pnpm add -g @mermaid-js/mermaid-cli</code> y regenerar
Archivos	<code>scripts/build-system-docs.mjs</code>
Riesgo	Ninguno

Un diagrama sale ilegible en el PDF

Síntoma	Aviso: <code>mide N px y en A4 se reduce al X %</code>
Causa	El diagrama es más ancho de lo que cabe legible en una A4
Solución	Partirlo por dominio, o cambiar la orientación a <code>TB</code> . Es lo que se hizo con el diagrama entidad-relación de 07
Archivos	<code>scripts/build-system-docs.mjs</code>
Riesgo	Ninguno

Diagnóstico general

Qué quieres saber	Comando
¿Está todo sano?	<code>node scripts/build-all.mjs && node --test tests/*.test.mjs</code>
¿Coinciden reglas y generado?	<code>node scripts/build-rules.mjs --check</code>
¿Son razonables las tasas?	<code>node scripts/validate-rules.mjs</code>
¿Qué reglas hay y de cuándo?	<code>node apps/contador-cli/src/index.mjs reglas</code>
¿El motor calcula bien?	<code>node apps/contador-cli/src/index.mjs f29 --ventas-netas 1000000 --compras-netas 300000</code>
¿Qué dice mi espacio de trabajo?	<code>node apps/contador-cli/src/index.mjs resumen --periodo 2026-08</code>
¿Qué pasó y cuándo?	<code>node apps/contador-cli/src/index.mjs bitacora --limite 100</code>
¿El APK sirve?	<code>node scripts/verify-apk.mjs ruta/al.apk</code>
¿Hay algo que importe <code>node:*</code> en el bundle?	<code>grep -rL "from 'node:" apps/web/dist/</code>
¿Cuál es el build publicado?	<code>cat apps/web/dist/build-info.json</code>

La bitácora es la primera herramienta de diagnóstico del producto, no la última: registra toda mutación con su fecha, su modo y su detalle, y no se puede borrar.

15 · Riesgos y deuda técnica

Este documento es informativo. No se corrigió nada. El encargo de esta serie es documentar, no arreglar: cada hallazgo lleva su evidencia, su ubicación y una recomendación, y la decisión de aplicarla es de quien mantiene el proyecto.

La conclusión, primero

Ninguno de los 22 hallazgos es un defecto de cálculo. El motor —que es la parte cara de equivocarse— está cubierto por 153 pruebas y sus reglas de negocio están fijadas por pruebas explícitas. Lo que aparece aquí es otra cosa: **puntos ciegos, silencios y afirmaciones que envejecieron.**

Los tres que conviene mirar primero:

1. **Una escritura puede perderse sin que nadie se entere** si `localStorage` se llena.
2. **La única superficie pública no tiene CSP**, porque GitHub Pages no admite cabeceras y el HTML no lleva la política dentro.

1. `docs/ARCHITECTURE.md` **afirma cifras que ya no son ciertas** — dice 10 vistas y 50 pruebas donde hoy hay 14 y 153.

Y la observación que más importa a largo plazo, que no es un hallazgo sino una condición: **el mecanismo de reglas por año nunca se ha ejercitado con dos años.** Todo el diseño apunta a que funcionará; nadie lo ha visto ocurrir.

Cómo leer la clasificación

Campo	Escala
Severidad	■ Crítica · ■ Alta · ■ Media · ■ Baja
Impacto	Qué pasa si se materializa
Probabilidad	Alta / Media / Baja, razonada a partir del código
Prioridad	P1 (antes del próximo release) · P2 (próximo ciclo) · P3 (cuando toque)

Severidad y prioridad no coinciden siempre: algo crítico y ya declarado en `SECURITY.md` es una decisión tomada, no una tarea pendiente.

Hallazgos confirmados

H-01 · Una escritura puede perderse en silencio si `localStorage` se llena

Ubicación	<code>packages/company-operations/store.mjs</code> · <code>createWebStore</code>
Evidencia	<code>write</code> y <code>append</code> llaman a <code>storage.setItem(...)</code> sin <code>try/catch</code> . La lectura sí tiene su <code>try/catch</code> ; la escritura no
Severidad	🔴 Alta · Impacto: la operación no se guarda y la excepción sube sin que nada la traduzca a un mensaje útil · Probabilidad: Baja hoy, creciente con el tiempo
Por qué crece	La bitácora es append-only y no tiene retención ni purga . <code>localStorage</code> ronda los 5–10 MB por origen. Una empresa con años de operaciones y bitácora acabará acercándose
Recomendación	Capturar <code>QuotaExceededError</code> en <code>write</code> / <code>append</code> , avisar en la interfaz e invitar a exportar. Añadir la prueba correspondiente
Prioridad	P1 — es el único hallazgo que puede costar datos del usuario

H-02 · La versión publicada en GitHub Pages no tiene CSP

Ubicación	<code>apps/web/src/index.html</code> , <code>.github/workflows/pages.yml</code>
Evidencia	La CSP la ponen <code>server.mjs</code> (cabecera) y <code>tauri.conf.json</code> . Se buscó <code>http-equiv</code> en todo <code>apps/web/src/</code> : ninguna coincidencia . GitHub Pages no admite cabeceras propias
Severidad	🔴 Alta · Impacto: la superficie más expuesta pierde la red de seguridad · Probabilidad: Baja de materializarse hoy
Matiz honesto	La app sigue sin hacer llamadas de red y el escapado por defecto sigue en pie. La CSP no es lo que impide la fuga; es lo que convertiría un futuro descuido en un error visible
Recomendación	Añadir la política como <code><meta http-equiv="Content-Security-Policy"></code> en <code>index.html</code> . Cubre las cuatro superficies de una vez
Prioridad	P1

H-03 · docs/ARCHITECTURE.md afirma cifras que ya no son ciertas

Ubicación	docs/ARCHITECTURE.md:39 y :47
Evidencia	Dice views/ 10 vistas y tests/ 50 pruebas . Reales, contadas: 14 vistas y 153 pruebas
Severidad	 Media · Impacto: el documento de arquitectura es lo primero que lee alguien nuevo, y empieza desconfiando · Probabilidad: ya ocurrió
Contexto	El README.md raíz sí está al día (153 pruebas, 54 términos, 12 atajos, 14 etapas, 8 diapositivas). El drift está localizado en un archivo
Recomendación	Corregir las dos cifras. Mejor aún: generarlas, como ya se hace con el glosario, la guía y los atajos — el repositorio tiene el patrón resuelto y no lo aplicó aquí
Prioridad	P1 — cuesta dos líneas

H-04 · apps/android/ tiene dos lockfiles contradictorios

Ubicación	apps/android/pnpm-lock.yaml y apps/android/package-lock.json , ambos versionados
Evidencia	git ls-files los muestra a los dos. android.yml usa pnpm install --frozen-lockfile ; el package-lock.json no lo consume nadie
Severidad	 Media · Impacto: alguien instala con npm, obtiene otro árbol y depura un problema que no existe · Probabilidad: Media
Contexto	Contradice una regla explícita del repositorio: packageManager: pnpm@11.2.2 , y los tres últimos commits del proyecto fueron precisamente sobre unificar en pnpm
Recomendación	Borrar apps/android/package-lock.json
Prioridad	P1 — es un git rm

H-05 · Un respaldo del almacén de disco no copia tres entidades como archivo

Ubicación	<code>packages/company-operations/node-store.mjs</code> · <code>FILES</code> y <code>saveSnapshot</code>
Evidencia	<code>FILES</code> mapea 7 claves. Las tres entidades de la versión 1.4.0 — <code>equity-movements</code> , <code>annual-closes</code> , <code>municipal-profile</code> — no están . <code>saveSnapshot</code> itera <code>Object.keys(FILES)</code> para copiar los archivos sueltos
Severidad	 Media · Impacto: el respaldo del almacén de disco queda incompleto como copia de archivos · Probabilidad: Baja — sólo afecta a la CLI
Atenuante importante	El <code>snapshot.json</code> interior sí lleva los datos, porque sale de <code>exportAll()</code> . No hay pérdida real; hay una asimetría que engaña al mirar el directorio
Recomendación	Añadir las tres claves a <code>FILES</code>
Prioridad	P2

H-06 · `append` en el almacén web reescribe la colección completa

Ubicación	<code>store.mjs</code> · <code>createWebStore.append</code>
Evidencia	Hace leer → empujar → <code>setItem</code> del arreglo entero. En <code>node-store</code> , el mismo método es un <code>fs.appendFileSync</code> de una línea
Severidad	 Media · Impacto: coste cuadrático al crecer la bitácora, y multiplica la exposición a H-01 · Probabilidad: Media a largo plazo
Matiz	Es una limitación de <code>localStorage</code> , no un descuido: no ofrece <code>append</code> . Lo que falta no es la técnica, es la mitigación (paginar la bitácora por año, o purgar con aviso)
Recomendación	Particionar la bitácora por año: <code>audit:2026</code> , <code>audit:2027</code>
Prioridad	P2

H-07 · Las 14 vistas no tienen ninguna prueba

Ubicación	<code>apps/web/src/views/*.js</code> — 14 archivos, más de 3.000 líneas
Evidencia	Ninguna de las 13 suites las ejecuta. <code>webapp.test.mjs</code> verifica presencia y contrato, no comportamiento
Severidad	 Media · Impacto: una regresión de interfaz sólo se detecta usando la app · Probabilidad: Alta
Atenuante	La lógica de negocio no está en las vistas , está en el motor, que sí está cubierto. Un fallo de vista se ve de inmediato
Agravante	<code>apps/web/src/views/capital.js</code> tiene 694 líneas , el segundo archivo más grande del repositorio
Recomendación	Empezar por lo barato y valioso: probar <code>lib/dom.js</code> (<code>esc</code> y <code>html</code>), que es un control de seguridad y son funciones puras
Prioridad	P2

H-08 · El escapado anti-XSS no tiene prueba propia

Ubicación	<code>apps/web/src/lib/dom.js</code> · <code>esc</code> y <code>html</code>
Evidencia	Ninguna suite las importa. Son el único control contra inyección de marcado
Severidad	 Media · Impacto: una regresión en el escapado pasaría desapercibida · Probabilidad: Baja
Recomendación	Media docena de aserciones. Es la prueba con mejor relación valor/coste de todo el repositorio
Prioridad	P1 — por lo barata, no por lo grave

H-09 · Dos de las tres validaciones del backend Rust no tienen prueba

Ubicación	<code>apps/contador-desktop/src-tauri/src/lib.rs</code>
Evidencia	Hay dos pruebas, ambas de <code>safe_mode</code> . Sin prueba: el saneo de <code>filename</code> en <code>export_file</code> y la validación del JSON en <code>save_workspace</code>
Severidad	 Media · Impacto: una regresión en el saneo del nombre permitiría escribir fuera de la carpeta prevista · Probabilidad: Baja
Recomendación	Dos pruebas al lado de las que ya existen
Prioridad	P2

H-10 · Nada garantiza que las dos `version` coincidan

Ubicación	<code>package.json</code> y <code>apps/contador-desktop/src-tauri/tauri.conf.json</code>
Evidencia	Hoy ambas dicen <code>1.4.0</code> — comprobado. Ningún workflow menciona <code>tauri.conf</code> : se buscó y no hay coincidencias
Severidad	 Media · Impacto: instaladores rotulados con una versión distinta de la que dice la documentación · Probabilidad: Media en el próximo bump
Recomendación	Un paso de tres líneas en <code>ci.yml</code> que compare ambas
Prioridad	P2

H-11 · Cerrar un período son dos escrituras sin transacción

Ubicación	<code>workspace.mjs</code> · <code>closePeriod</code>
Evidencia	Escribe <code>period-closes</code> y después <code>closed-periods</code> . No hay transacción porque el almacén no la ofrece
Severidad	 Media · Impacto: una fotografía sin su índice — el mes parece abierto y hay un cierre registrado · Probabilidad: Muy baja
Recomendación	Escribir primero el índice, o detectar la incoherencia en <code>healthCheck</code>
Prioridad	P3

H-12 · Un JSON corrupto es indistinguible de un dato ausente

Ubicación	<code>store.mjs</code> · <code>readRaw</code> ; <code>node-store.mjs</code> · <code>read</code>
Evidencia	El <code>catch</code> devuelve el <code>fallback</code> sin distinguir «no existe» de «no parsea»
Severidad	 Media · Impacto: la app arranca vacía, y el usuario concluye que perdió sus datos cuando quizá siguen ahí, corruptos
Recomendación	Registrar la diferencia y exponerla en <code>healthCheck</code>
Prioridad	P3

H-13 · `audit.ndjson` marca las líneas corruptas y nadie las mira

Ubicación	<code>node-store.mjs</code> · <code>readAll</code> devuelve <code>{ corrupt: true, line }</code>
Evidencia	Ningún consumidor busca ese marcador
Severidad	 Baja
Recomendación	Contarlas en el diagnóstico
Prioridad	P3

H-14 · La bitácora crece sin techo

Ubicación	<code>workspace.mjs</code> · <code>audit()</code>
Evidencia	No hay retención, rotación ni purga. Y es deliberado : la ausencia de borrado es lo que la hace servir como evidencia
Severidad	 Media · Impacto : alimenta H-01 y H-06
Tensión de diseño	Purgar la bitácora contradice su propósito. Particionarla por año, no
Prioridad	P2 , junto con H-06

H-15 · `workspace.mjs` concentra 1.142 líneas y toda la lógica de negocio

Evidencia	Es el archivo más grande del repositorio. Contiene ficha, capital, patrimonio, municipalidad, ejercicio anual, constitución, operaciones, obligaciones, períodos, respaldo y salud
Severidad	 Media · Impacto : acoplamiento; cualquier cambio toca el mismo archivo
Atenuante fuerte	Está seccionado con comentarios , la cohesión es real (todo gira sobre el mismo espacio de trabajo) y la clase base ya se separó de la de Node. No es un archivo desordenado: es un archivo grande
Recomendación	Si crece más, extraer el ejercicio anual y la parte municipal a módulos propios. No refactorizar sin una razón concreta
Prioridad	P3

H-16 · `apps/web/src/views/capital.js` tiene 694 líneas

Segundo archivo más grande, y una vista. **Severidad**  Media, **Prioridad P3**. Es también el tema más difícil del producto —seis magnitudes distintas—, así que el tamaño tiene justificación.

H-17 · Los PDF de esta serie no se generan ni se verifican en CI

Ubicación	<code>scripts/build-system-docs.mjs</code> , <code>.github/workflows/ci.yml</code>
Evidencia	El glosario, la guía, los atajos y la presentación tienen su <code>--check</code> en CI. Esta serie no
Severidad	 Media · Impacto: los PDF quedan desfasados del Markdown sin que nada avise · Probabilidad: Alta
Causa	Exigiría Chrome y <code>mmdc</code> en el runner, y hoy no están
Recomendación	O bien instalarlos en un trabajo aparte, o bien un <code>--check</code> ligero que compare fechas de modificación y falle si un <code>.md</code> es más nuevo que su <code>.pdf</code>
Prioridad	P2

H-18 · No hay vigilancia de dependencias de build

Evidencia	No se encontró configuración de Dependabot ni equivalente. Cero dependencias de producción — verificado en CI—, pero <code>apps/android</code> y <code>apps/contador-desktop</code> sí tienen las suyas
Severidad	 Baja · Impacto: acotado al equipo que compila, no al usuario
Prioridad	P3

H-19 · No hay linter ni formateador

Evidencia	Sin ESLint, Prettier ni <code>tsconfig.json</code> . Hay JSDoc por todo el código y nadie lo comprueba
Severidad	 Baja · Impacto: hoy ninguno — el estilo es notablemente uniforme
Matiz	Esa uniformidad la sostiene una persona, no una herramienta. El día que entre alguien más, deja de estar garantizada
Prioridad	P3

H-20 · Dos pestañas abiertas: la última escritura gana

Evidencia	No hay bloqueo, ni evento <code>storage</code> , ni fusión
Severidad	 Baja · Probabilidad: Baja — es una app de un solo operador
Prioridad	P3

H-21 · `android.yml` y `desktop.yml` no corren en cada push

Evidencia	Sólo <code>workflow_dispatch</code> y <code>workflow_call</code> . Tardan 30 y 45 minutos
Severidad	 Media · Impacto: una regresión que sólo rompa el empaquetado aparece al publicar
Matiz	La decisión es razonable: 75 minutos por push es caro. Una ejecución nocturna sería un término medio
Prioridad	P3

H-22 · `docs/` tiene 74 archivos y ningún índice único

Evidencia	<code>docs/README.md</code> existe, pero conviven manual, guía, runbooks, políticas contables, árbol de decisión, presentación, glosario y ahora esta serie
Severidad	 Baja
Mitigación ya aplicada	El índice de esta serie declara qué documento es la fuente de verdad de cada tema, precisamente para no añadir confusión
Prioridad	P3

Riesgos que no son defectos

Cosas que están bien decididas y conviene entender antes de «arreglarlas».

#	Observación	Por qué no es un defecto
R-1	El mecanismo de reglas por año nunca se ha ejercitado con dos años	Está diseñado, probado y documentado. Pero <code>availableYears()</code> devuelve <code>[2026]</code> : el primer <code>rules/2027.json</code> es donde se sabrá si el esquema aguanta. REQUIERE VALIDACIÓN en su momento
R-2	Los datos no están cifrados	Declarado en <code>SECURITY.md</code> , con su contrapartida: tampoco salen del dispositivo
R-3	Los binarios no están firmados	Declarado en <code>SECURITY.md</code> y en cada release. Un certificado cuesta dinero y renovación
R-4	El APK es de depuración	Declarado. Publicar en Play exige firma de release
R-5	No hay autenticación local	Declarado. El control de acceso lo aporta el sistema operativo
R-6	El F29 no cubre todos los códigos	Limitación declarada del producto , no un fallo. <code>SECURITY.md</code> lo pone explícitamente fuera de alcance
R-7	El remanente de IVA no se reajusta (art. 27, D.L. 825)	Declarado en el código y en las advertencias de las reglas
R-8	La tasa de cada comuna no está en el repositorio	Correcto : el archivo guarda el rango legal. Publicar tasas que nadie verificó sería el error
R-9	<code>HOST</code> puede exponer el servidor a la red	El valor por defecto es seguro y la razón está comentada. El riesgo lo introduce quien lo cambia
R-10	Todo es síncrono y de un solo hilo	Decisión de arquitectura consciente. Cambiarla obligaría a rehacer las catorce vistas

Decisiones que requieren validación humana

Ninguna de éstas la puede resolver una revisión de código.

#	Decisión	Quién la toma
V-1	¿Siguen vigentes las tasas de 2026? El propio archivo advierte que BCN/LeyChile no respondió el 16-08-2026	Quien mantiene las reglas, contra las fuentes oficiales
V-2	¿Se firma el código y el APK?	Depende de presupuesto
V-3	¿Se cifran los datos en reposo? Cambia el modelo de recuperación: sin contraseña no hay respaldo legible	Producto
V-4	¿Se purga o se particiona la bitácora? Tiene implicaciones sobre su valor como evidencia	Producto
V-5	¿Entra el empaquetado en CI nocturno?	Coste frente a detección temprana
V-6	¿Se generan las cifras de <code>ARCHITECTURE.md</code> o se corrigen a mano?	Mantenimiento

Resumen por prioridad

Prioridad	Hallazgos	Esfuerzo estimado
P1	H-01, H-02, H-03, H-04, H-08	Bajo. Cuatro son de minutos; sólo H-01 exige pensar la interfaz del aviso
P2	H-05, H-06, H-07, H-09, H-10, H-14, H-17	Medio
P3	H-11, H-12, H-13, H-15, H-16, H-18, H-19, H-20, H-21, H-22	Variable

INFERENCIA : las cinco P1 son, en conjunto, menos de una jornada de trabajo, y cierran el único riesgo de pérdida de datos y el único hueco de seguridad no declarado.

Lo que esta revisión NO buscó

Para que la ausencia de hallazgos no se lea como una garantía que nadie dio:

- No se hicieron pruebas de penetración ni escaneos activos.
- No se auditó la corrección **tributaria** de las fórmulas contra la normativa. Se verificó que cada

tasa lleva su fuente y que el sistema falla en vez de degradar; **no que el resultado sea el que el SII aceptaría.**

- No se midió cobertura real de pruebas: el mapa de 12 es cualitativo.
- No se ejecutaron el APK, los instaladores ni las pruebas de Rust.
- No se revisaron los 74 documentos previos de docs/ en busca de más drift; sólo se comprobaron las

cifras de estado en los principales.

16 • Glosario

Fuente de verdad de los términos del dominio: `docs/GLOSSARY.md` , **54

términos generados desde `packages/glossary/` **, con su definición larga, sus «no confundir con» y su base legal. CI falla si ese documento se desvía del paquete.

Este glosario **no lo duplica**. Cubre lo que aquel no cubre: los términos **técnicos** del repositorio, los nombres propios del sistema y las abreviaturas de esta serie. Cuando un término aparece en ambos, aquí se explica lo que significa **en el código**, y se remite allá para lo que significa en la ley.

Las definiciones están escritas para que las entienda alguien que no programa. Cuando hace falta un tecnicismo, se explica.

Términos del dominio — dónde están

Los 54 términos contables, tributarios y municipales —capital social, capital enterado, CPT, F29, remanente de crédito fiscal, patente municipal, UTM, DTE, RCV...— están en `docs/GLOSSARY.md` y dentro de la propia aplicación, en la pestaña **Glosario**, con ayuda al pasar el cursor sobre cualquier término.

Los seis que más se confunden, y que son la razón de ser del producto, se distinguen así:

Término	En una frase
Capital social	La cifra que dice el estatuto
Capital suscrito	Lo que el accionista se comprometió a pagar
Capital enterado	Lo que efectivamente pagó. Lo único que es patrimonio
Patrimonio contable	Lo que dice el balance: activo menos pasivo
Capital propio tributario	Lo que dice la ley tributaria, con sus propias reglas
Capital base de patente	Lo que la municipalidad usa para cobrar, que depende de la etapa del negocio

Que sean seis cosas distintas y no un número es, literalmente, el argumento del sistema.

Nombres propios del sistema

Cosas que sólo existen en este repositorio y que hay que conocer para leer el código.

Término	Qué es
Empresa Operativa Chile	La aplicación. El mismo producto en navegador, Android, Windows y línea de comandos
EMPRESA REAL	El espacio de trabajo de la empresa de verdad. En el código, <code>mode: 'real'</code>
SANDBOX	El espacio de práctica, con datos de ejemplo. <code>mode: 'sandbox'</code> . Nunca hay una ruta de código que copie una operación de un espacio al otro
Espacio de trabajo (workspace)	Todos los datos de una empresa en un modo: ficha, operaciones, obligaciones, cierres y bitácora. En el código, <code>CompanyWorkspace</code>
Motor	<code>packages/</code> — el cálculo y la operación, sin interfaz. Viaja completo a las tres plataformas
Superficie	Cada una de las cuatro formas de usar el producto: navegador, Android, Windows y CLI
**El gate <code>node:*</code> **	La comprobación que aborta el build si algún archivo que va a viajar al dispositivo importa un módulo de Node
Reglas del año	<code>packages/chile-tax-rules/rules/<año>.json</code> : las tasas, con su fuente oficial y su fecha de verificación
Bitácora	El registro de auditoría. Se puede añadir, nunca borrar
Espejo (mirror)	La copia en archivos JSON que la versión de Windows mantiene en el disco
Semáforo	<code>healthCheck()</code> : el diagnóstico que no dice «todo en orden» cuando faltan evidencias
Ruta / Empezar aquí	Las 14 etapas de <code>packages/onboarding/</code> , desde antes de que la empresa exista
Período	Un mes, escrito <code>YYYY-MM</code> . No se guarda: se deriva de la fecha de cada operación
Ejercicio	Un año comercial. Se cierra una vez y queda congelado

Términos técnicos

Ordenados por lo que cuesta entenderlos sin explicación.

Almacenamiento

Término	Qué significa aquí
<code>localStorage</code>	Un almacén de texto que el navegador guarda por sitio web, en el propio dispositivo. Sobrevive a cerrar el navegador; no sobrevive a «borrar datos del sitio»
Almacén (<i>store</i>)	El contrato de seis funciones —leer, escribir, añadir, leer todo, respaldar, listar respaldos— que <code>CompanyWorkspace</code> usa sin saber qué hay debajo
Adaptador	Cada implementación de ese contrato: memoria, <code>localStorage</code> o archivos
Espacio de nombres (<i>namespace</i>)	El prefijo <code>empresa-operativa-chile:real:</code> que separa un modo de otro. Es toda la barrera entre lo real y la práctica
Append-only	«Sólo se añade». Se pueden agregar líneas, nunca borrarlas ni cambiarlas
NDJSON	Un archivo de texto con un dato JSON por línea. Permite añadir sin reescribir lo anterior
Escritura atómica	Escribir en un archivo temporal y después renombrarlo. Un corte de luz deja intacto el archivo anterior en vez de uno a medio escribir
Snapshot / fotografía	Una copia congelada de un momento. Aquí: el respaldo, el cierre mensual y el cierre anual
Cuota	El límite de espacio que el navegador da a <code>localStorage</code> , del orden de 5 a 10 MB

Arquitectura

Término	Qué significa aquí
Local-first	El producto funciona entero en el dispositivo. No hay servidor que guarde nada y no hace falta conexión
PWA	<i>Progressive Web App</i> : una web que se instala como aplicación y funciona sin conexión
Service worker	Un pequeño programa del navegador que guarda los archivos de la app para que abra sin conexión. No toca los datos del usuario
WebView	Un navegador incrustado dentro de otra aplicación. Es lo que muestra la app en Android y en Windows
Capacitor	La herramienta que envuelve la web en un APK de Android
Tauri	La que la envuelve en un ejecutable de Windows, con un proceso en Rust al lado
IPC	<i>Inter-Process Communication</i> : cómo la parte web habla con la parte Rust
ESM	<i>ECMAScript Modules</i> : la forma moderna y estándar de dividir JavaScript en módulos. Todo el repositorio la usa
** node: * **	Los módulos que sólo existen en Node —leer archivos, red, rutas—. Un navegador no los tiene , y de ahí la regla dura
Bundler	Una herramienta que junta y comprime el código antes de publicarlo. Este proyecto no usa ninguna
Síncrono	Que ocurre de inmediato, sin esperar. Todo el motor lo es, y por eso <code>localStorage</code> encaja
Puro (función pura)	Una función que con las mismas entradas da siempre la misma salida y no toca nada más
Congelado (<code>Object.freeze</code>)	Un objeto que ya no puede modificarse. Se usa en los cierres anuales y en los catálogos

Datos y cálculo

Término	Qué significa aquí
Derivar	Calcular algo cada vez a partir de los datos, en vez de guardarlo. El período se deriva de la fecha; el resumen del mes, de las operaciones
Normalizar	Dejar un dato en su forma canónica antes de guardarlo: recortar espacios, redondear importes, poner valores por defecto
Migración en lectura	Adaptar un dato antiguo al leerlo sin reescribirlo . Por eso instalar esta versión sobre datos viejos no puede corromperlos
PENDING_CONFIRMATION	La marca con la que el sistema declara que un dato no lo sabe , en vez de poner un cero
Invariante	Algo que siempre debe ser cierto. El del producto: real y sandbox no se mezclan
Precondición	Lo que debe cumplirse antes de llamar a una función. Ejemplo: el período no puede estar cerrado
Idempotente	Que repetirlo no cambia el resultado. <code>seedSandboxWorkspace</code> lo es: si ya hay operaciones, no hace nada
Enlace <code>equityMovementId</code>	El campo que conecta un aporte con su movimiento de caja para que el mismo peso no se cuente dos veces

Seguridad

Término	Qué significa aquí
CSP	<i>Content-Security-Policy</i> : una regla que le dice al navegador de dónde puede cargar cosas. Aquí, sólo del propio sitio
XSS	Inyección de marcado: que un texto escrito por alguien acabe ejecutándose como código de la página
Escapar	Convertir los caracteres peligrosos de un texto en inofensivos antes de mostrarlo
Path traversal	Usar <code>..</code> en una ruta para salir del directorio permitido y leer archivos que no se debían
Superficie de ataque	Todo aquello por donde alguien podría entrar. La de este sistema es pequeña porque no hay servidor ni red
Fijar a un SHA (<i>pinning</i>)	Anclar una herramienta a una versión exacta e inmutable, en vez de a una etiqueta que puede cambiar
Telemetría	Datos de uso que una aplicación envía a su fabricante. Aquí no hay
Firma de código	Un certificado que prueba quién construyó un programa. Estos binarios no la tienen , y está declarado

Desarrollo y publicación

Término	Qué significa aquí
CI	<i>Integración continua</i> : comprobaciones automáticas en cada cambio
Workflow	Cada uno de esos procesos automáticos. Hay seis
Gate	Una comprobación que detiene el proceso si falla. No es un aviso
Build reproducible	Que construir dos veces produzca exactamente lo mismo. Si no, lo que se publica no es lo que se probó
<code>buildId</code>	La huella de todos los archivos publicados. Cambiarla invalida la copia guardada en el navegador
Artefacto	El archivo que produce una construcción: el APK, el instalador, el PDF
Release	Una publicación con nombre, notas y archivos descargables
<code>SHA256SUMS.txt</code>	La lista de huellas de cada archivo, para comprobar que lo descargado es lo publicado
Lockfile	El archivo que fija las versiones exactas de las dependencias. Aquí, <code>pnpm-lock.yaml</code>
pnpm	El gestor de paquetes de este repositorio. Escribir <code>npm</code> es un error
Monorepo	Un solo repositorio con varios paquetes y aplicaciones dentro

Abreviaturas y siglas

Sigla	Significado	Contexto
SII	Servicio de Impuestos Internos	La autoridad tributaria de Chile. Cuando la app y el SII no coincidan, manda el SII
BCN	Biblioteca del Congreso Nacional	Fuente del texto legal (LeyChile)
LIR	Ley sobre Impuesto a la Renta	
D.L. 3.063	Decreto Ley sobre Rentas Municipales, de 1979	Su art. 24 rige la patente municipal
CPT	Capital Propio Tributario	Ver GLOSSARY.md
IDPC	Impuesto de Primera Categoría	
PPM	Pago Provisional Mensual	
IVA	Impuesto al Valor Agregado	
UTM	Unidad Tributaria Mensual	Cambia todos los meses
UF	Unidad de Fomento	
DTE	Documento Tributario Electrónico	
RCV	Registro de Compras y Ventas	
RES	Registro de Empresas y Sociedades	Donde se constituye la sociedad
SpA	Sociedad por Acciones	El tipo societario que modela el sistema
APK	El archivo instalable de Android	
MSI / NSIS	Los dos formatos de instalador de Windows	
CLI	<i>Command-Line Interface</i>	La versión de línea de comandos
ESM	<i>ECMAScript Modules</i>	
IPC	<i>Inter-Process Communication</i>	
CSP	<i>Content-Security-Policy</i>	
PWA	<i>Progressive Web App</i>	
SVG / PNG	Formatos de imagen: vectorial y de mapa de bits	
NDJSON	<i>Newline-Delimited JSON</i>	

Convenciones de esta serie

Marcador	Significado
(sin marcador)	Hecho verificado leyendo el repositorio, con archivo y símbolo citados
INFERENCIA	Conclusión razonada a partir del código, no afirmación literal del repositorio
REQUIERE VALIDACIÓN	Depende de una ejecución real, de un servicio externo o de una decisión humana
NO DOCUMENTADO EN EL REPOSITORIO	Existe en el código y nadie lo explica
NO IDENTIFICADO	Se buscó y no hay evidencia en ningún sentido
	Crítico · Alto · Medio · Bajo
H-nn	Hallazgo de 15 · Riesgos
R-n / V-n	Riesgo asumido / Decisión que requiere validación humana
P1 · P2 · P3	Prioridad: antes del próximo release · próximo ciclo · cuando toque

Estados que aparecen en los datos

Estado	Dónde	Qué significa
<code>pending</code> · <code>in_progress</code> · <code>done</code> · <code>blocked</code>	Pasos de constitución y obligaciones	<code>done</code> exige evidencia
<code>real</code> · <code>sandbox</code>	Modo del espacio de trabajo	
<code>DECLARADO</code> · resto de <code>DATA_STATUS</code>	Movimientos patrimoniales	Distingue lo declarado de lo confirmado
<code>usuario</code> · resto de <code>DATA_ORIGIN</code>	Movimientos patrimoniales	De dónde vino el dato
<code>estimado por la aplicación</code> · <code>declarado por el usuario</code>	<code>balanceOrigin</code> en un cierre anual	Nunca se presenta una estimación como un dato declarado
<code>ESTIMADO</code>	Base de patente del año siguiente	Es una proyección, no una liquidación
<code>baseline</code>	<code>status</code> del archivo de reglas	Línea base operativa y educativa
<code>info</code> · <code>warn</code> · <code>error</code>	Observaciones del semáforo	

Nombres históricos

Nombre	Estado
company.capital	Campo heredado. Hoy se mantiene sincronizado con capitalProfile.capitalEnterado para no romper vistas ni respaldos antiguos
formatVersion: 1	Formato de respaldo anterior. Se sigue importando: romper los respaldos que la gente ya tiene guardados sería peor que limpiar el formato
contador-cli , contador-desktop	Nombres de directorio anteriores al nombre actual del producto. Siguen así porque cambiarlos rompería rutas, workflows y enlaces

17 · Resumen ejecutivo

Para dirección, clientes, evaluadores y posibles colaboradores. Sin detalle técnico innecesario.

Qué es

Empresa Operativa Chile es una aplicación que acompaña a una sociedad chilena (SpA) a lo largo del tiempo: desde antes de existir —cuando alguien decide crearla— hasta el cierre de cada mes y de cada año.

No es un software contable que registra asientos. Es una herramienta de **control y evidencia**: calcula lo que hay que pagar, avisa de lo que falta, exige el comprobante antes de dar algo por hecho, y deja registro de todo lo que cambió.

Corre en cuatro sitios con un solo motor de cálculo: navegador, teléfono Android, Windows y línea de comandos. Los datos nunca salen del dispositivo.

Qué necesidad cubre

Llevar una empresa pequeña en Chile no falla por no saber sumar. Falla por perder el hilo: una factura sin respaldo, un remanente de IVA que no se arrastró, un trámite que se dio por hecho sin guardar el comprobante, un mes cerrado sin conciliar.

Su aporte diferencial es **distinguir seis magnitudes que casi todas las herramientas funden en un solo número**: capital social, capital suscrito, capital enterado, patrimonio contable, capital propio tributario y capital base de patente municipal. Confundirlas es lo que produce una patente mal pagada o una declaración que no cuadra — y son cosas distintas, con momentos y bases legales distintas.

Quién lo utiliza

Perfil	Para qué
Quien va a crear una SpA	La ruta de 14 etapas: qué decidir, qué trámite sigue, qué documento le queda
Quien ya tiene una SpA pequeña	Operaciones del mes, borrador del F29, obligaciones, cierre
Quien acompaña a varios emprendedores	El entorno de práctica para enseñar sin tocar datos reales
Quien estudia contabilidad y tributación chilena	Cada número lleva su norma y su fuente oficial

Capacidades principales

Capacidad	Qué resuelve
Constitución con evidencia	Nueve trámites con su organismo. No se puede marcar uno como hecho sin registrar su comprobante
Operaciones	Ventas, compras, gastos, honorarios, aportes, préstamos del accionista, retiros e impuestos
IVA con remanente arrastrado	El crédito que sobra un mes está disponible al siguiente, como corresponde
Borrador del F29	Ayuda de control, con los tres vencimientos calculados
Capital y patrimonio	Las seis magnitudes, cada una con su momento y su evidencia
Capital propio tributario	Por el método que corresponde al régimen, con su base legal
Patente municipal	Con la base correcta según la etapa del negocio
Cierre mensual y anual	Inmutables. Reabrir exige motivo y queda registrado
Bitácora de auditoría	Toda mutación queda. Nada se puede borrar
Respaldo portátil	Se exporta desde el teléfono y se importa en Windows
Entorno de práctica	Separado del real. Ninguna ruta de código los mezcla
Manual y glosario dentro de la app	54 términos con ayuda al pasar el cursor, sin conexión

Lo que deliberadamente NO hace

Está declarado por el propio producto, y esa franqueza es parte de su propuesta:

- **No presenta ni paga nada ante el SII.** Calcula, controla y guarda evidencia; la presentación ocurre en los sistemas oficiales.
- **No es asesoría tributaria.** Cuando la aplicación y el SII no coincidan, **manda el SII.**
- No importa datos del SII, ni cartolas bancarias, ni documentos electrónicos.
- El borrador F29 no cubre todos los casos: no modela exenciones, proporcionalidad de IVA, activos fijos ni importaciones.
- No publica la tasa de patente de ninguna comuna: guarda el rango legal y remite a la municipalidad.

Tecnologías

Aspecto	Elección
Lenguaje	JavaScript moderno (ESM) y Rust para el proceso de escritorio
Dependencias de producción	Cero. Comprobado automáticamente en cada cambio
Interfaz	Sin framework, sin bundler
Almacenamiento	En el propio dispositivo. Sin base de datos y sin servidor
Empaquetado	Capacitor (Android) y Tauri 2 (Windows)
Pruebas	Runner nativo de Node — 153 pruebas, sin framework externo
Automatización	6 procesos de integración continua

Cero dependencias de producción no es una anécdota: significa que no hay una cadena de terceros que pueda romperse, quedar sin mantenimiento o introducir una vulnerabilidad. Es el argumento de sostenibilidad del proyecto.

Arquitectura, en un párrafo

Un solo motor de cálculo en `packages/`, y tres envoltorios que lo llevan al navegador, a Android y a Windows. La restricción que lo sostiene está automatizada: **nada de lo que viaja al dispositivo puede depender de módulos de servidor, y el proceso de construcción se detiene si alguien lo intenta**. Las tasas y plazos tributarios no están en el código: viven en un archivo por año, cada cifra con su fuente oficial y su fecha de verificación, y **pedir un año que no existe produce un error en vez de un cálculo con las tasas del año pasado**.

Estado actual

Indicador	Valor · verificado el 27-08-2026
Versión	1.4.0
Archivos versionados	262
Código propio	13.251 líneas
Pruebas	153, todas en verde
Duración de la suite	0,64 segundos
Superficies funcionando	4
Dependencias de producción	0
Procesos automáticos	6
Documentos de producto previos	74 archivos en docs/
Defectos de cálculo encontrados en esta revisión	Ninguno

Fortalezas

1. **El producto se niega a mentir.** No marca un trámite como hecho sin evidencia, no modifica un período cerrado, no dice «todo en orden» si faltan respaldos, y no publica tasas que nadie verificó. Es poco común y es difícil de imitar.

1. **Cada número tiene procedencia.** Fuente oficial y fecha de verificación, y un cierre anual se lleva dentro la huella de las reglas con que se calculó — de modo que dentro de tres años seguirá explicando con qué normas se hizo.

1. **Verifica el resultado, no sólo el proceso.** Antes de publicar, abre el instalable y **cuenta lo que hay dentro**. Un paquete vacío se construye sin errores; esa comprobación es la que evita publicarlo.

1. **Cero dependencias de producción,** con comprobación automática que impide que deje de ser cierto por descuido.

1. **Un solo motor para cuatro superficies,** con la regla que lo garantiza automatizada.

2. **Documentación generada desde el propio sistema:** glosario, guía y atajos se producen desde el código, y el proceso falla si el documento se desvía.

1. **Privacidad por arquitectura, no por promesa.** No hay servidor al que enviar nada. Se verificó: en todo el código hay **una sola llamada de red**, y sirve para guardar los propios archivos de la aplicación.

Riesgos

Del análisis salieron 22 hallazgos. **Ninguno es un defecto de cálculo.** Los que importan para una decisión:

Riesgo	Impacto	Estado
Si el almacenamiento del navegador se llena, una operación puede no guardarse sin aviso	Pérdida de un dato del usuario	Corregible en horas
La versión web pública no tiene una de las protecciones que sí tienen las otras tres	Menor margen ante un descuido futuro	Corregible en minutos
Un documento de arquitectura afirma cifras que ya no son ciertas	Erosiona la confianza de quien llega	Corregible en minutos
Los datos no están cifrados en el dispositivo	Quien acceda al equipo desbloqueado ve la contabilidad	Declarado por el proyecto; en su hoja de ruta
Los instaladores no están firmados	El sistema operativo mostrará un aviso	Declarado ; requiere presupuesto
Depende de una sola persona	Continuidad	Estructural
El mecanismo de tasas por año nunca se ha usado con dos años	Se sabrá en el primer cambio de año	A validar en su momento

Los cinco arreglos prioritarios suman menos de una jornada de trabajo y cierran el único riesgo de pérdida de datos y el único hueco de seguridad no declarado.

La dependencia que no es técnica

La única obligación recurrente del producto es **humana y anual**: antes de cada año comercial hay que crear el archivo de tasas de ese año verificando **cada cifra contra su fuente oficial**. El sistema está construido para fallar ruidosamente si no se hace —no calcula con las tasas del año anterior—, así que el riesgo no es un cálculo equivocado: es que la aplicación deje de servir hasta que alguien haga ese trabajo.

El propio archivo de 2026 registra que una de sus fuentes legales **no respondió el 16 de agosto de 2026** y pide reverificarla antes de usar la cifra en una declaración real. Que el sistema documente lo que no pudo comprobar dice bastante sobre cómo está hecho.

Oportunidades de mejora

Oportunidad	Por qué
Cifrado local y bloqueo por PIN	Cierra la brecha declarada más visible
Firma de código y APK de publicación	Elimina las advertencias del sistema operativo y abre Google Play
Segundo año de tasas	Convierte una promesa de diseño en un hecho comprobado
Pruebas de interfaz	La única parte sin cobertura
Importar el registro de compras y ventas del SII	El paso que más trabajo manual ahorraría
Un segundo mantenedor	Es hoy el mayor riesgo de continuidad

El propio `docs/ROADMAP.md` ya ordena las tres primeras, y cierra con una sección que conviene leer: **las reglas que ninguna versión romperá**.

Próximos pasos recomendados

#	Paso	Plazo	Esfuerzo
1	Aplicar los cinco arreglos prioritarios de 15 · Riesgos	Antes del próximo release	< 1 jornada
2	Reverificar las tasas de 2026 contra sus fuentes oficiales	Antes de la Operación Renta 2027	1 jornada
3	Preparar el archivo de tasas de 2027	Antes de enero de 2027	1–2 jornadas
4	Decidir sobre cifrado local y firma de código	Próximo ciclo	Decisión + presupuesto
5	Primeras pruebas de interfaz, empezando por el escapado de texto	Próximo ciclo	2–3 jornadas
6	Integrar la generación de esta documentación en la automatización	Cuando haya margen	1 jornada

El paso 2 es el único que no puede esperar por razones ajenas al equipo: las fechas tributarias las pone el calendario, no el proyecto.

Conclusión

Un producto pequeño, cuidadosamente construido y honesto sobre sus límites. Su calidad no está en la cantidad de funcionalidad, sino en el rigor: **cada cifra tiene procedencia, cada afirmación tiene evidencia, y el sistema prefiere fallar antes que dar un número plausible y equivocado.**

Los riesgos abiertos son conocidos, están acotados y la mayoría ya estaban declarados por el propio proyecto antes de esta revisión. El que más pesa no es técnico: es que todo esto lo sostiene una persona.

18 · Guía del nuevo desarrollador

Fuente de verdad del proceso de contribución: `CONTRIBUTING.md` . Esta

guía no lo repite: añade el **itinerario de lectura** y el mapa de dónde tocar cada cosa.

Lo primero, en cinco minutos

```
git clone https://github.com/vladimiracunadev-create/empresa-operativa-chile.git
cd empresa-operativa-chile
node scripts/build-all.mjs
node apps/empresa-operativa/server.mjs
```

Abre `http://127.0.0.1:4180`, pulsa **SANDBOX** y trastea. Hay una empresa de ejemplo con seis meses de operaciones.

No hace falta `pnpm install`. El proyecto no tiene dependencias de producción ni de desarrollo: sólo Node $\geq$ 20. `pnpm install` sólo se necesita para empaquetar Android o Windows.

Comprueba que todo está sano:

```
node --test tests/*.test.mjs
```

Deberías ver `pass 153`, en menos de un segundo.

Las cinco cosas que hay que saber antes de tocar nada

Si sólo lees una sección de esta guía, que sea ésta.

1. `pnpm`, nunca `npm`

Está declarado en `package.json` como `packageManager: pnpm@11.2.2`. `pnpm-lock.yaml` es el lockfile válido y `npm install` lo ignoraría.

2. Nada que viaje al dispositivo puede importar `node:*`

Es la regla dura de la arquitectura. Si un archivo que acaba en `apps/web/dist` importa `node:fs`, `node:path` o cualquier otro, la app queda en blanco en el APK sin un error visible.

`scripts/build-web.mjs` aborta el build y nombra los culpables. No es un aviso.

Cuando necesites algo de Node, sepáralo: es exactamente por lo que `workspace.mjs` y `store.mjs` son navegables y `node-store.mjs` vive aparte.

3. Un año sin reglas debe fallar, nunca degradar

`loadRules(2027)` lanza si no existe `rules/2027.json`. Jamás lo hagas caer al año anterior.

Un cálculo con la tasa del año equivocado no falla: devuelve un número plausible que aparece meses después como una diferencia con el SII. `CONTRIBUTING.md` lo prohíbe por escrito y hay una prueba que lo fija.

4. Una tasa que se toca exige su fuente y su fecha

Cada cifra de `rules/<año>.json` lleva `source` (URL oficial) y `lastVerified`. Cambiar un número sin actualizarlos no es un descuido de estilo: rompe la trazabilidad, que es el argumento del producto.

Y después: `node scripts/build-rules.mjs`, o CI falla.

5. La documentación generada no se edita a mano

`docs/GLOSSARY.md`, `docs/EMPEZAR-AQUI.*` y `docs/ATAJOS-DE-TECLADO.md` se generan desde `packages/glossary`, `packages/onboarding` y `packages/shortcuts`. Editarlos directamente pone CI en rojo y tu cambio se perderá en la siguiente regeneración.

Edita el paquete y regenera.

Cómo está organizado el repositorio

```
packages/           El motor. Sin interfaz, sin node:*, es lo que viaja a las 3 plataformas
chile-tax-rules/    Las tasas por año, con su fuente
accounting-engine/  El cálculo: IVA, F29, CPT, patente, PPM, IDPC
company-operations/ La operación: CompanyWorkspace, capital, RUT, almacenes
glossary/ onboarding/ shortcuts/  Contenido, fuente de la documentación generada

apps/
web/               LA aplicación. 14 vistas. Lo demás son envoltorios
empresa-operativa/ Servidor local de archivos estáticos
android/           Envoltorio Capacitor (sin código propio)
contador-desktop/  Envoltorio Tauri + 4 comandos en Rust
contador-cli/      Línea de comandos

scripts/           13 scripts de build y verificación + 4 librerías compartidas
tests/             13 suites, 153 pruebas
docs/              Documentación de producto + esta serie
```

La forma más rápida de orientarse: casi todo lo importante está en dos archivos. `packages/company-operations/workspace.mjs` (1.142 líneas) tiene toda la lógica de negocio, y `packages/accounting-engine/index.mjs` todo el cálculo. Las cuatro superficies son envoltorios delgados sobre eso.

Itinerario de incorporación

Día 1 — entender el producto antes que el código

1. Usa la aplicación en **SANDBOX** durante media hora. Registra una venta, mira cómo cambia el borrador del F29, cierra un mes e intenta editar una operación de ese mes.

1. Lee `docs/EMPEZAR-AQUI.md`. Son 14 etapas y explica **para qué existe** el producto.

1. Lee [01 · Visión general](#), sobre todo «Qué NO hace».

2. Hojea `docs/GLOSSARY.md`. No hace falta memorizarlo, pero ****sí** entender por qué capital social, suscrito y enterado son tres cosas^{**}: es el corazón del producto.

Al final del día 1 deberías poder responder: ¿por qué la app se niega a marcar un trámite como hecho?

Día 2 — la arquitectura y sus reglas duras

1. `docs/ARCHITECTURE.md` — las siete decisiones de diseño.

⚠ Sus cifras de vistas y pruebas están desactualizadas ([H-03](#)); las decisiones, no.

1. [03 · Arquitectura](#) de esta serie: los diagramas y cómo se hacen cumplir las reglas.

1. Lee `scripts/build-web.mjs`, en concreto el gate `node:*`. Son quince líneas y explican media arquitectura.

1. Lee `packages/chile-tax-rules/index.mjs`. Son 1.599 bytes y contiene la regla más importante del sistema.

1. Ejecuta `node scripts/build-all.mjs` y mira qué produce.

Día 3 — el motor

1. `packages/accounting-engine/index.mjs` — empieza por `f29Basic`.

2. `tests/f29.test.mjs` en paralelo: las pruebas explican el comportamiento esperado mejor que cualquier comentario.

1. `packages/company-operations/workspace.mjs`, por secciones. Está dividido con comentarios; no lo leas de corrido.

1. [06 · Explicación profunda](#), flujos 1 y 2.

Ejercicio: sigue una venta desde el formulario hasta que aparece en el borrador del F29. Está descrito paso a paso en [08 · Flujo de datos](#); compruébalo tú.

Día 4 — la interfaz y las plataformas

1. `apps/web/src/app.js` (249 líneas) — el arranque y el ciclo de render.
2. `apps/web/src/lib/dom.js` — la plantilla `html`. **Entiende por qué escapa por defecto.**
3. `apps/web/src/lib/platform.js` — el único sitio donde se nota en qué plataforma corres.
4. Una vista sencilla, por ejemplo `views/panel.js`.
5. `apps/contador-desktop/src-tauri/src/lib.rs` — 135 líneas, los cuatro comandos.

Día 5 — cómo se publica y qué se verifica

1. `.github/workflows/ci.yml`. Lee los comentarios: explican por qué cada paso está ahí.
 2. `scripts/verify-apk.mjs`. **Es la pieza que mejor resume la cultura del proyecto.**
 3. [13 · Despliegue](#).
 4. [15 · Riesgos](#) — para saber dónde están las minas.
-

Dónde tocar cada cosa

Quiero...	Toca	Cuidado con
Cambiar una tasa o un plazo	<code>packages/chile-tax-rules/rules/<año>.json</code>	Actualizar <code>source</code> y <code>lastVerified</code> , y regenerar
Agregar un año tributario	Crear <code>rules/<año>.json</code> + regenerar	Verificar cada cifra contra la fuente oficial
Agregar un cálculo	<code>packages/accounting-engine/</code>	Que reciba el año y use <code>loadRules</code>
Agregar un tipo de operación	<code>TRANSACTION_KINDS</code> en <code>workspace.mjs</code>	Su <code>affectsVat</code> y su efecto en <code>periodSummary</code>
Agregar un trámite de constitución	<code>FORMATION_STEPS</code> en <code>workspace.mjs</code>	Aparecerá en las empresas existentes — es lo deseado
Agregar un campo a una entidad	<code>workspace.mjs</code> y <code>exportAll</code> / <code>importAll</code>	La compatibilidad con respaldos v1 y v2
Agregar una vista	<code>apps/web/src/views/</code> + registro en <code>app.js</code>	Nada de <code>node:*</code> . Escapa todo lo que venga del usuario
Agregar un término al glosario	<code>packages/glossary/</code>	No <code>docs/GLOSSARY.md</code> ; regenera
Agregar una etapa a la ruta	<code>packages/onboarding/</code>	Idem con <code>docs/EMPEZAR-AQUI.md</code>
Agregar un atajo	<code>packages/shortcuts/</code>	Idem con <code>docs/ATAJOS-DE-TECLADO.md</code>
Cambiar la persistencia	<code>store.mjs</code> / <code>node-store.mjs</code>	Respetar el contrato de seis métodos
Agregar un comando de escritorio	<code>src-tauri/src/lib.rs</code> + <code>invoke_handler</code>	Validar los argumentos ; no ampliar <code>capabilities</code> sin pensarlo
Cambiar la interfaz de la CLI	<code>apps/contador-cli/src/index.mjs</code>	Actualizar el texto de <code>AYUDA</code>
Tocar un workflow	<code>.github/workflows/</code>	Fijar cualquier acción nueva a un SHA de 40 caracteres

Cómo escribir una prueba

Sin framework, sin `describe`, sin *mocks*:

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { CompanyWorkspace, createMemoryStore } from '../packages/company-operations/index.mjs';

test('descripción en español, en presente, describiendo la REGLA', () => {
  const ws = new CompanyWorkspace({ store: createMemoryStore(), mode: 'real' });
  // ...
  assert.throws(() => ws.addTransaction({ kind: 'sale', date: '2026-07-01' }), /cerrado/);
});
```

El estilo de los nombres importa. Fíjate en los que ya hay: «un período cerrado es inmutable en las dos direcciones», «reabrir un período exige motivo y queda en la bitácora». Describen la **regla de negocio**, no la función. Una prueba que se rompe debería decirle a quien la rompió qué garantía del producto acaba de perder.

`createMemoryStore()` da un espacio limpio en cada prueba: no hay fixtures que limpiar.

Convenciones que hay que respetar

Convención	Detalle
Idioma	Todo en español: comentarios, nombres de prueba, mensajes de error, documentación
Nombres de código	En inglés (<code>addTransaction</code> , <code>capitalEnterado</code> mezcla ambos por ser término legal chileno)
Comentarios	Explican por qué , no qué. Mira cualquier archivo: casi todos los comentarios justifican una decisión
Mensajes de error	En español y dirigidos a la persona, no al programador. Ejemplo: «Un paso sólo puede marcarse como realizado si registras su evidencia»
Módulos	ESM. Nada de <code>require</code>
Importes	Enteros en pesos. Se redondean con <code>money()</code>
Fechas	<code>YYYY-MM-DD</code> para días, <code>YYYY-MM</code> para períodos, ISO-8601 para marcas de tiempo
Sin dependencias	Antes de añadir una, comprueba si el repositorio ya resolvió el problema en <code>scripts/lib/</code>
Acciones de CI	Siempre fijadas a SHA, con la etiqueta como comentario
Commits	Mira <code>git log</code> : mensajes en español que explican la consecuencia , no el cambio

Sobre el último punto, un ejemplo real del historial: «fix: la presentación se publica como presentación, no como un `.html` en el repo». Ese es el registro.

Antes de abrir un pull request

```
pnpm check
```

Ejecuta los cinco gates de sincronía, la validación de reglas y las 153 pruebas. Es lo mismo que correrá CI.

Si tocaste la aplicación web:

```
node scripts/build-all.mjs
```

Si tocaste esta documentación:

```
node scripts/build-system-docs.mjs
```

Qué requiere especial cuidado

Zona	Por qué	Antes de tocar
<code>loadRules</code>	■ Es la única puerta a las tasas	Lee <code>CONTRIBUTING.md</code> y <code>tests/rules.test.mjs</code>
<code>f29Basic</code>	■ Un error produce una cifra plausible y equivocada	Lee <code>tests/f29.test.mjs</code> entero
<code>calculateTaxEquity</code>	■ El piso en cero y la elección de método son la ley, no una preferencia	<code>docs/tax/CAPITAL-PROPIO-TRIBUTARIO.md</code>
<code>calculateMunicipalPatent</code>	■ La base cambia según la etapa del negocio	<code>docs/municipal/PATENTE-MUNICIPAL.md</code>
<code>closeFiscalYear</code>	■ Congela una fotografía que debe seguir siendo válida en años	Entiende <code>legalRulesVersion</code>
El escapado en <code>dom.js</code>	■ Es el control anti-XSS del sistema	No introduces <code>raw()</code> con datos del usuario
<code>resolveSafe</code> en <code>server.mjs</code>	■ Traduce entrada de red a rutas de disco	El <code>+ path.sep</code> no es cosmético
<code>safe_mode</code> en <code>lib.rs</code>	■ Sin él, <code>mode</code> sería una escritura arbitraria	Las dos pruebas de al lado
Las claves del almacén	■ Cambiar una deja datos huérfanos	Piensa la migración primero
<code>identifier</code> de Tauri	■ Cambiarlo deja huérfanos los datos de escritorio	No lo cambies

Primeras tareas apropiadas para alguien junior

Ordenadas de menos a más. Todas salen de [15 · Riesgos](#), así que además de aprender, cierran deuda real.

#	Tarea	Qué se aprende	Hallazgo
1	Corregir «10 vistas» y «50 pruebas» en <code>docs/ARCHITECTURE.md</code>	Cómo se cuentan las cosas aquí	H-03
2	Borrar <code>apps/android/package-lock.json</code>	Por qué el gestor importa	H-04
3	Escribir las pruebas de <code>esc()</code> y <code>html</code>	El control de seguridad de la interfaz, con funciones puras	H-08
4	Añadir las tres claves que faltan en <code>FILES</code> de <code>node-store.mjs</code>	Cómo funciona el almacén de disco	H-05
5	Añadir un paso en CI que compare las dos <code>version</code>	Cómo se escribe un gate	H-10
6	Probar el saneo de <code>filename</code> en <code>lib.rs</code>	Rust, y el modelo de seguridad de escritorio	H-09
7	Capturar <code>QuotaExceededError</code> en <code>createWebStore</code> y avisar	El riesgo real de perder datos	H-01
8	Añadir la CSP como <code>meta</code> en <code>index.html</code>	Qué protege y qué no	H-02

Empieza por la 3. Es pequeña, es una función pura, y protege algo que hoy no tiene red.

Dónde preguntar

Duda	Documento
¿Por qué está hecho así?	<code>docs/ARCHITECTURE.md</code> → 03
¿Qué hace esta función?	05 · Referencia técnica
¿Cómo se encadena todo?	06 · Explicación profunda
¿Dónde se guarda esto?	07 · Persistencia
¿Qué significa esta palabra?	<code>docs/GLOSSARY.md</code> → 16
Me falla algo	14 · Troubleshooting
¿Qué está roto o pendiente?	15 · Riesgos
¿Qué viene después?	<code>docs/ROADMAP.md</code>
¿Cómo se opera la empresa?	<code>docs/RUNBOOK-MENSUAL.md</code> y <code>RUNBOOK-ANUAL.md</code>

Lo que no debes romper

`docs/ROADMAP.md` cierra con una sección titulada «**Reglas que ninguna versión romperá**». Léela antes de proponer un cambio grande. En resumen, y con lo verificado en esta revisión:

1. Las tasas nunca degradan a otro año.
 2. Un período cerrado no se modifica sin dejar rastro.
 3. La bitácora no se borra.
 4. EMPRESA REAL y SANDBOX no se mezclan.
 5. Nada se marca como hecho sin evidencia.
 6. Los datos no salen del dispositivo.
 7. Cuando la app y el SII no coincidan, manda el SII.
-

19 · Matriz de trazabilidad

Una fila por funcionalidad, para seguirla **desde la pantalla hasta donde se guarda y hasta la prueba que la protege**. Es la respuesta a «si cambio esto, ¿qué se rompe?» y a «esta regla, ¿está realmente implementada?».

Cómo leer el estado de validación:

Estado	Significado
 Verificado	Se leyó el código y existe una prueba que lo fija
 Verificado sin prueba	Se leyó el código y hace lo que dice; ninguna prueba lo protege
 Verificado en CI	No hay prueba unitaria, pero un paso de CI lo comprueba de punta a punta
 Requiere validación	Depende de una ejecución que no se pudo hacer en esta revisión

Las columnas de persistencia usan las claves reales del almacén, documentadas en [07](#).

1 · Constitución de la empresa

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Ver los trámites	9 pasos con su organismo y su evidencia esperada	Vista company-constitution	company-operations	listFormationSteps() · FORMATION_STEPS	formation + catálogo en código	company-operations.test.mjs	06	✓
Marcar un trámite hecho	done exige evidenceReal no vacío	Vista company-constitution	company-operations	updateFormationStep()	formation	workspace.test.mjs «un paso de constitución no puede darse por hecho sin evidencia»	07	✓
Los trámites nuevos aparecen	El catálogo vive en el código, no en los datos	Vista company-constitution	company-operations	listFormationSteps()	Fusión en lectura	workspace.test.mjs «el catálogo de pasos se amplía sin perder lo ya registrado»	07	✓
Progreso de habilitación	Porcentaje contado sobre evidencia real	Vista company-panel	company-operations	formationProgress()	Derivado	Indirecta	05	✓
La ruta completa	14 etapas en 5 fases	Vista company-onboarding	company-onboarding	PHASES , STAGES	Ninguna (contenido)	onboarding.test.mjs (17)	04	✓

2 · Ficha de la empresa y RUT

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Guardar la ficha	<code>mode</code> se reescribe siempre; <code>createdAt</code> sólo la primera vez	Vista <code>empresa</code>	<code>company</code> - <code>operations</code>	<code>saveCompany()</code>	<code>company</code>	Indirecta	07	✓
Validar RUT	Dígito verificador chileno	Vista <code>empresa</code> , CLI <code>rut</code>	<code>company</code> - <code>operations</code>	<code>validateRut()</code>	—	<code>rut.test.mjs</code> (6)	05	✓
Régimen tributario	Decide si se admite el CPT simplificado	Vista <code>empresa</code>	<code>accounting-engine</code>	<code>allowsSimplifiedTaxEquity()</code>	<code>company.taxRegister</code>	<code>tax-equity.test.mjs</code>	06	✓

3 · Capital y patrimonio

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Las seis magnitudes	Social, suscrito, enterado, patrimonio, CPT y base de patente son distintos	Vista capital	company-operations	normalizeCapitalProfile()	company.capitalProfile	capital.test.mjs (19)	07	✓
Capital por enterar	Devuelve null , no 0 , cuando el suscrito es desconocido	Vista capital	company-operations	capitalPendingToPay()	Derivado	capital.test.mjs	07	✓
Migrar el campo antiguo	El valor pasa a capitalEnterado ; social y suscrito quedan PENDING_CONFIRMATION . No se inventan	Vista capital	company-operations	normalizeCapitalProfile()	En lectura, sin escribir	capital.test.mjs	07	✓
Registrar un movimiento	Fecha obligatoria, monto > 0, aporte en bienes exige bien y aportante	Vista capital	company-operations	normalizeEquityMovement()	equity-movements	capital.test.mjs	07	✓
Aporte ≠ préstamo	Entran por el mismo banco y son opuestos en patrimonio y pasivo	Vista capital	company-operations	EQUITY_MOVEMENT_KINDS	equity-movements	capital.test.mjs	16	✓
No contar dos veces	El enlace equityMovementId evita duplicar el capital enterado	Vista capital	company-operations	effectiveEquityMovements()	transactions.equityMovementId	capital.test.mjs	06	✓
Posición de capital	El mayor entre lo declarado y lo derivado de movimientos	Vista capital	company-operations	capitalPosition()	Derivado	capital.test.mjs	06	✓

4 · Operaciones

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Registrar	Tres puertas: tipo válido, fecha <code>YYYY-MM-DD</code> , período no cerrado	Vista <code>operaciones</code> , CLI <code>registrar</code>	<code>company-operations</code>	<code>addTransaction()</code>	<code>transactions</code>	<code>workspace.test.mjs</code> , «los tipos de operación no soportados se rechazan»	06	✓
Nueve tipos	Cada uno con su <code>flow</code> y su efecto sobre el IVA	Vista <code>operaciones</code>	<code>company-operations</code>	<code>TRANSACTION_KINDS</code>	<code>transactions.kind</code>	<code>company-operations.test.mjs</code>	07	✓
Deducible y con derecho a crédito	Por defecto <code>true</code> : sólo un <code>false</code> explícito los desactiva	Vista <code>operaciones</code>	<code>company-operations</code>	<code>addTransaction()</code>	<code>transactions</code>	<code>workspace.test.mjs</code> · «el sandbox sembrado deja un remanente visible y un IVA no recuperable»	07	✓
Editar y borrar	No se puede en un período cerrado, ni mover una operación a uno cerrado	Vista <code>operaciones</code>	<code>company-operations</code>	<code>updateTransaction()</code> , <code>deleteTransaction()</code>	<code>transactions</code>	<code>workspace.test.mjs</code> · «un período cerrado es inmutable en las dos direcciones»	07	✓
Resumen del mes	18 cifras derivadas, nunca guardadas	Vista <code>cierre</code> , CLI <code>resumen</code>	<code>company-operations</code>	<code>periodSummary()</code>	Derivado de <code>transactions</code>	<code>f29.test.mjs</code> , <code>workspace.test.mjs</code>	08	✓

5 · IVA, F29 y otros cálculos

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Venta / compra desde el neto	IVA 19 % del año	Vista <code>operaciones</code> , CLI <code>venta / compra</code>	<code>accounting-engine</code>	<code>saleFromNet()</code> , <code>purchaseFromNet()</code>	—	<code>accounting.test.mjs</code>	05	✓ + ⚙️
Honorario	Retención 15,25 % (Ley 21.133)	CLI <code>honorario</code>	<code>accounting-engine</code>	<code>honorariumFromGross()</code>	—	<code>accounting.test.mjs</code>	05	✓
Borrador F29	Débito – crédito – remanente	Vista <code>impuestos</code> , CLI <code>f29</code>	<code>accounting-engine</code>	<code>f29Basic()</code> 	Derivado	<code>f29.test.mjs</code> (8)	06	✓ + ⚙️
Remanente arrastra	El crédito que sobra un mes está disponible al siguiente	Vista <code>impuestos</code>	<code>company-operations</code>	<code>vatCarryForwardInto()</code>	Derivado de la historia completa	<code>workspace.test.mjs</code> · «el remanente de crédito fiscal viaja entre períodos»	06	✓
Sin reajuste del remanente	Limitación declarada (art. 27, D.L. 825)	—	<code>company-operations</code>	<code>vatCarryForwardInto()</code>	—	—	14 · R-7	☑️ declarada
Vencimientos del F29	Días 12, 20 y 28, con traslado por día inhábil	Vista <code>impuestos</code> , CLI <code>vencimientos</code>	<code>accounting-engine</code>	<code>f29DueDates()</code>	<code>rules.f29</code>	<code>f29.test.mjs</code>	05	✓ + ⚙️
PPM Pro Pyme	Tasas por tramo de UF	Vista <code>impuestos</code>	<code>accounting-engine</code>	<code>ppmFromSalesNet()</code>	<code>rules.ppmProPyme</code>	<code>accounting.test.mjs</code>	05	✓
IDPC Pro Pyme	Tasa 12,5 %	CLI <code>idpc</code>	<code>accounting-engine</code>	<code>idpcProPyme()</code>	<code>rules.idpcProPyme</code>	<code>accounting.test.mjs</code>	05	✓

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Asiento contable	Un asiento por tipo de operación	CLI asiento	accounting-engine	journal() ■	—	accounting.test.mjs	05	✓

6 · Capital propio tributario

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Elegir el método	Art. 41 N.º 1, o simplificado del art. 14 D) N.º 3 (j) según régimen	Vista <code>capital</code>	<code>accounting-engine</code>	<code>calculateTaxEquity()</code>	<code>rules.taxEquity</code>	<code>tax-equity.test.mjs</code> (17)	06	✓
Elegibilidad del simplificado	Sólo Pro Pyme General (14 D N.º 3)	Vista <code>capital</code>	<code>accounting-engine</code>	<code>allowsSimplifiedTaxEquity()</code>	<code>rules.taxEquity.eligibility</code>	<code>tax-equity.test.mjs</code>	05	✓
Piso en cero	Si el simplificado da negativo, el CPT es \$0	Vista <code>capital</code> , CLI <code>cpt</code>	<code>accounting-engine</code>	<code>simplified()</code>	<code>rules.floorZero: true</code>	<code>tax-equity.test.mjs</code>	06	✓
El CPT es una estimación	El que rige es el declarado en el F22 y aceptado por el SII	Vista <code>capital</code>	—	Advertencia en <code>rules</code>	<code>rules.taxEquity.warnings</code>	—	01	✓ declarada

7 · Patente municipal

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Base según etapa	Negocio nuevo → capital propio inicial. En funcionamiento → capital del balance al 31-12 anterior (art. 24, D.L. 3.063)	Vista capital , CLI patente-municipal	accounting-engine	calculateMunicipalPatent() 	rules.municipalPatent.Basis	municipal-patent.test.mjs (23)	06	
Rango legal, no tasa	0,25 %–0,5 %. La tasa de la comuna NO está en el repositorio	Vista capital	chile-tax-rules	resolveRate() 	municipal-profile.rate	municipal-patent.test.mjs	07	
Tope y piso en UTM	Entre 1 y 8.000 UTM	Vista capital	accounting-engine	calculateMunicipalPatent()	rules.municipalPatent	municipal-patent.test.mjs	05	
Deducciones	Inversiones en otros negocios con patente, con certificado municipal	Vista capital	accounting-engine	calculateMunicipalPatent()	municipal-profile deductibleInvestments	municipal-patent.test.mjs	07	
Prorateo entre sucursales	Por número de trabajadores (art. 25)	Vista capital	accounting-engine	calculateMunicipalPatent()	municipal-profile.allocatedCapital	municipal-patent.test.mjs	07	
UTM del mes	Cambia todos los meses; hoy sólo hay 2026-08	Vista capital	chile-tax-rules	resolveUtm() 	rules.utm	municipal-patent.test.mjs	07	
Catálogo de comunas	14 comunas con su sitio oficial para verificar la tasa	Vista capital	chile-tax-rules	normalizeMunicipality()	municipal-profile commune	Indirecta	09	

8 · Obligaciones y cierres

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Registrar obligación	<code>type</code> obligatorio	Vista <code>obligaciones</code> , CLI <code>obligaciones</code>	<code>comp</code> <code>any-operation</code> s	<code>upsertObligation()</code>	<code>obligations</code>	<code>workspace.test.mjs</code>	07	✓
Cumplida exige comprobante	No se marca <code>done</code> sin evidencia	Vista <code>obligaciones</code>	<code>comp</code> <code>any-operation</code> s	<code>upsertObligation()</code>	<code>obligations</code>	<code>workspace.test.mjs</code> · «una obligación no puede marcarse cumplida sin comprobante»	07	✓
Cierre mensual	Congela el resumen y el remanente entrante	Vista <code>cierre</code> , CLI <code>cerrar</code>	<code>comp</code> <code>any-operation</code> s	<code>closePeriod()</code>	<code>period-closes</code> + <code>closed-periods</code>	<code>workspace.test.mjs</code>	06	✓
Reabrir exige motivo	Trazabilidad por encima de inmutabilidad absoluta	Vista <code>cierre</code>	<code>comp</code> <code>any-operation</code> s	<code>reopenPeriod()</code>	<code>closed-periods</code> + <code>audit</code>	<code>workspace.test.mjs</code> · «reabrir un período exige motivo y queda en la bitácora»	06	✓
Cierre anual	Congelado con <code>Object.freeze</code> . No se cierra dos veces	Vista <code>capital</code>	<code>comp</code> <code>any-operation</code> s	<code>closeFiscalYear()</code>	<code>annual-closes</code>	<code>capital.test.mjs</code>	07	✓
Huella de las reglas	El cierre guarda con qué normas se calculó, no una referencia al archivo	Vista <code>capital</code>	<code>comp</code> <code>any-operation</code> s	<code>loadRulesVersion()</code>	<code>annual-closes.legalRulesVersion</code>	<code>capital.test.mjs</code>	07	✓
Base de patente del año siguiente	El puente entre un ejercicio y el siguiente	Vista <code>capital</code>	<code>comp</code> <code>any-operation</code> s	<code>closeFiscalYear()</code>	<code>annual-closes.municipalPatentBaseForNextPeriod</code>	<code>capital.test.mjs</code>	06	✓

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Historial por año	Nunca se recalcula un ejercicio cerrado	Vista capital	comp any- opera tion s	capi talHi story ()	annual- closes	Parcial	06	☒

9 · Auditoría, respaldo y salud

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Bitácora append-only	Toda mutación queda. No existe operación de borrado	Vista <code>auditoria</code> , CLI <code>bitacora</code>	<code>compa</code> <code>ny-</code> <code>operations</code>	<code>audit()</code> , <code>listAudit()</code>	<code>audit</code> (NDJSO N en disco)	<code>workspace.test.mjs</code> · «toda mutación queda en la bitácora append-only»	07	✓
Exportar	Formato portátil <code>v2</code> , compatible entre las 4 superficies	Vista <code>datos</code> , CLI <code>exportar</code>	<code>compa</code> <code>ny-</code> <code>operations</code>	<code>exportAll()</code>	Todas las claves	<code>workspace.test.mjs</code> · «exportar e importar reproduce el espacio de trabajo completo»	09	✓
Importar	Rechaza archivos ajenos; acepta v1 y v2; no pisa ejercicios cerrados	Vista <code>datos</code>	<code>compa</code> <code>ny-</code> <code>operations</code>	<code>importAll()</code>	Todas las claves	<code>workspace.test.mjs</code> · «importar un archivo ajeno se rechaza»	07	✓
Respaldo con nombre	Marca de tiempo ISO saneada	Vista <code>datos</code>	<code>compa</code> <code>ny-</code> <code>operations</code>	<code>backup()</code>	<code>__snapshots / backup</code> <code>s/</code>	Indirecta	13	✓
Semáforo	No dice «todo en orden» si faltan evidencias	Vista <code>panel</code>	<code>compa</code> <code>ny-</code> <code>operations</code>	<code>healthCheck()</code>	Derivado	<code>workspace.test.mjs</code> · «el diagnóstico marca error cuando hay obligaciones vencidas»	08	✓

10 · Aislamiento real / sandbox

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Dos espacios separados	Ninguna ruta de código copia una operación de uno al otro	Select or de modo	company-operations	createWebStore({ namespace })	Prefijo ...:real: / ...:sandbox:	workspace.test.mjs · «el almacén web aísla real de sandbox en el mismo origen»	07	✓
Sembrar el sandbox	Prohibido sobre una empresa real; idempotente si ya tiene datos	Automático	company-operations	seedSandboxWorkspace()	Todas las claves del modo sandbox	workspace.test.mjs · «sembrar el sandbox sobre una empresa real está prohibido»	07	✓
Modo por URL	?modo=sandbox — hace reproducibles las capturas	URL	apps/web	applyUrlState()	—	—	10	✓

11 · Reglas tributarias

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Un año sin reglas lanza	Nunca degrada al año anterior	Todas	chile-tax-rules	loadRules()	rules/<año>.json	rules.test.mjs	07	✓
Años disponibles	Devuelve una copia del arreglo	Vista empresa , CLI reglas	chile-tax-rules	availableYears()	—	rules.test.mjs	05	✓
Procedencia de una tasa	Fuente oficial + fecha de verificación	Vista academia	chile-tax-rules	ruleProvenance()	rules.<bloque>.source	rules.test.mjs	08	✓
JSON y generado sincronizados	Editar uno sin el otro falla el build	—	scripts	build-rules.mjs -check	—	Paso de ci.yml	08	⚙
Las tasas son razonables	Atrapa el error de dedo	—	scripts	validate-rules.mjs	—	Paso de ci.yml	12	⚙

12 · Plataformas y persistencia física

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Detectar plataforma	Tauri → Windows; Capacitor → Android; si no, navegador	—	<code>apps/web/lib</code>	<code>PLATFORM</code>	—	—	05	✓
Espejo en disco	«Best effort» con retardo de 180 ms; si falla, la app sigue	Automático	<code>apps/web/lib</code> + Rust	<code>createPlatformStore() → save_workspace</code>	<code><app_data_dir>/<mode>.json</code>	<code>lib.rs</code> (parcial)	07	⚠
Escritura atómica	Temporal + <code>rename</code> . Un corte deja intacto el archivo anterior	—	Rust · <code>node-store</code>	<code>save_workspace</code> , <code>writeJsonAtomic</code>	—	—	09	⚠
Hidratar al arrancar	El disco sólo repone lo que falta: gana el dato vivo	Automático	<code>apps/web/lib</code>	<code>hydrateFromDisk()</code>	<code>localStorage</code>	—	06	⚠ H-06
<code>mode</code> no puede ser una ruta	Sin la validación sería escritura arbitraria	—	Rust	<code>safe_mode()</code> 	—	<code>lib.rs</code> · <code>un_modo_corta_ruta_se_rechaza</code>	11	⚠ no ejecutada
Exportar archivo	Nombre saneado; carpeta fija y predecible	Vista <code>datos</code>	Rust	<code>export_file</code>	<code>Documentos/Empresa Operativa Chile</code>	Ninguna · H-09	09	⚠
Ubicación de los datos	La app dice dónde están sus archivos	Vista <code>datos</code>	Rust	<code>app_info</code>	—	—	09	⚠

13 · Interfaz, contenido y seguridad de presentación

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Escapado por defecto	Toda interpolación se escapa salvo <code>raw()</code> explícito	Todas	<code>apps/web/lib</code>	<code>html</code> , <code>esc</code>	—	Ninguna · H-08	11	✓
Glosario	54 términos, fuente única	Vista <code>glosario</code>	<code>glossary</code>	<code>termsByCategory()</code> , <code>searchTerms()</code>	—	<code>glossary.test.mjs</code> (11)	16	✓
Ayuda contextual	Definición al pasar el cursor	Todas	<code>apps/web/lib</code>	<code>terms.js</code>	—	<code>ayuda.test.mjs</code> (16)	04	✓
Atajos de teclado	12, con buscador y ayuda	Todas	<code>shortcuts</code>	<code>resolveShortcut()</code>	—	<code>ayuda.test.mjs</code>	05	✓
Contrato de la web	El índice referencia lo que el build produce	—	<code>apps/web</code>	<code>index.html</code> , manifiesto	—	<code>webapp.test.mjs</code> (8)	12	✓
Mismo <code>appId</code>	Android y Windows comparten identificador	—	Configuración	—	—	<code>webapp.test.mjs</code>	10	✓
Funciona sin conexión	Cachea sólo los archivos de la app	—	<code>apps/web</code>	<code>sw.js</code>	Cache API	—	09	✓

14 · Servidor local

Funcionali dad	Regla de negocio	Interfaz	Módulo	Función	Persis tencia	Prueba	Docu ment o	Esta do
Servir la app	Sólo GET / HEAD ; el resto → 405	http://127.0.0.1:4180	empresa- operativ a	server.m js	—	Trabajo servidor de ci.yml	09	⚙
No escapar de dist/	Comparación con separador	—	empresa- operativ a	resolveS afe() ■	—	ci.yml , 3 vectores	11	⚙
Sólo loopback	127.0.0.1 , nunca 0.0.0.0	—	empresa- operativ a	server.m js	—	—	10	✓
Cabeceras de seguridad	CSP, nosniff , no-referrer	—	empresa- operativ a	server.m js	—	—	11	✓

15 · Integridad del artefacto

Funcionalidad	Regla de negocio	Interfaz	Módulo	Función	Persistencia	Prueba	Documento	Estado
Nada con <code>node:*</code> viaja	El build aborta	—	<code>scripts</code>	<code>build-web.mjs</code>	—	Gate del build	03	⚙️
El APK lleva la app dentro	12 comprobaciones abriendo el ZIP	—	<code>scripts</code>	<code>verify-apk.mjs</code>	—	Paso de <code>android.yml</code>	13	⚠️ sin APK que probar
La interfaz está completa	Se cuenta antes de sellarla en el binario	—	CI	<code>desktop.yml</code>	—	Paso del workflow	13	⚠️
El ejecutable arranca	Vive más de 12 segundos	—	CI	<code>desktop.yml</code>	—	Paso del workflow	13	⚠️
Build reproducible	Dos builds seguidos deben coincidir	—	CI	<code>ci.yml</code>	<code>build-info.json</code>	Paso del workflow	12	⚙️ reproducido en esta revisión
Cero dependencias	Es un gate, no un aviso	—	CI	<code>security.yml</code>	<code>package.json</code>	Paso del workflow	11	⚙️
Sin datos reales	El riesgo nº 1 del modelo de amenazas	—	CI	<code>security.yml</code>	—	Paso del workflow	11	⚙️ comprobado en esta revisión
Acciones fijadas a SHA	Una etiqueta móvil puede cambiar bajo los pies	—	CI	<code>ci.yml</code>	—	Paso del workflow	12	⚙️
Documentación sincronizada	5 gates <code>--check</code>	—	<code>scripts</code>	<code>build-*mjs --check</code>	—	Pasos de <code>ci.yml</code>	12	⚙️ ejecutados en esta revisión

Cobertura de la matriz

88 filas de funcionalidad, con 91 marcas de estado: tres filas llevan dos, porque además de su prueba las cubre un paso de CI.

Estado	Marcas	Lectura
✔ Verificado con prueba	56	La lógica de negocio y de cálculo está protegida
⚙ Verificado en CI	13	Integridad del artefacto, reglas y servidor
✔ Verificado sin prueba	13	Presentación, plataforma y utilidades
⚠ Requiere validación	9	Todo lo que exige compilar, empaquetar o ejecutar en un dispositivo

Cifras contadas sobre este mismo archivo, no estimadas.

Dónde se concentran los ⚠: en el backend de escritorio y en el empaquetado — exactamente lo que no se pudo ejecutar en la máquina de análisis por falta de Rust, JDK y SDK de Android. No es un vacío de pruebas: es un límite de esta revisión, y los workflows que lo cubren existen.

Dónde se concentran los ✔: en la interfaz. Coincide con [H-07](#) y [H-08](#), y confirma desde otro ángulo que ahí está el hueco de cobertura.

Cómo usar esta matriz

Pregunta	Cómo responderla
«Voy a cambiar X, ¿qué se rompe?»	Busca la función en la columna Función ; su fila da la vista, el almacenamiento y la prueba
«¿Esta regla está implementada?»	Búscala en Regla de negocio ; si tiene  , hay una prueba que la fija
«¿Qué protege esta prueba?»	Busca el nombre en Prueba
«¿Dónde acaba este dato?»	Columna Persistencia , y después 07
«¿Qué no está probado?»	Filtra por  y 
«¿Dónde se explica esto?»	Columna Documento